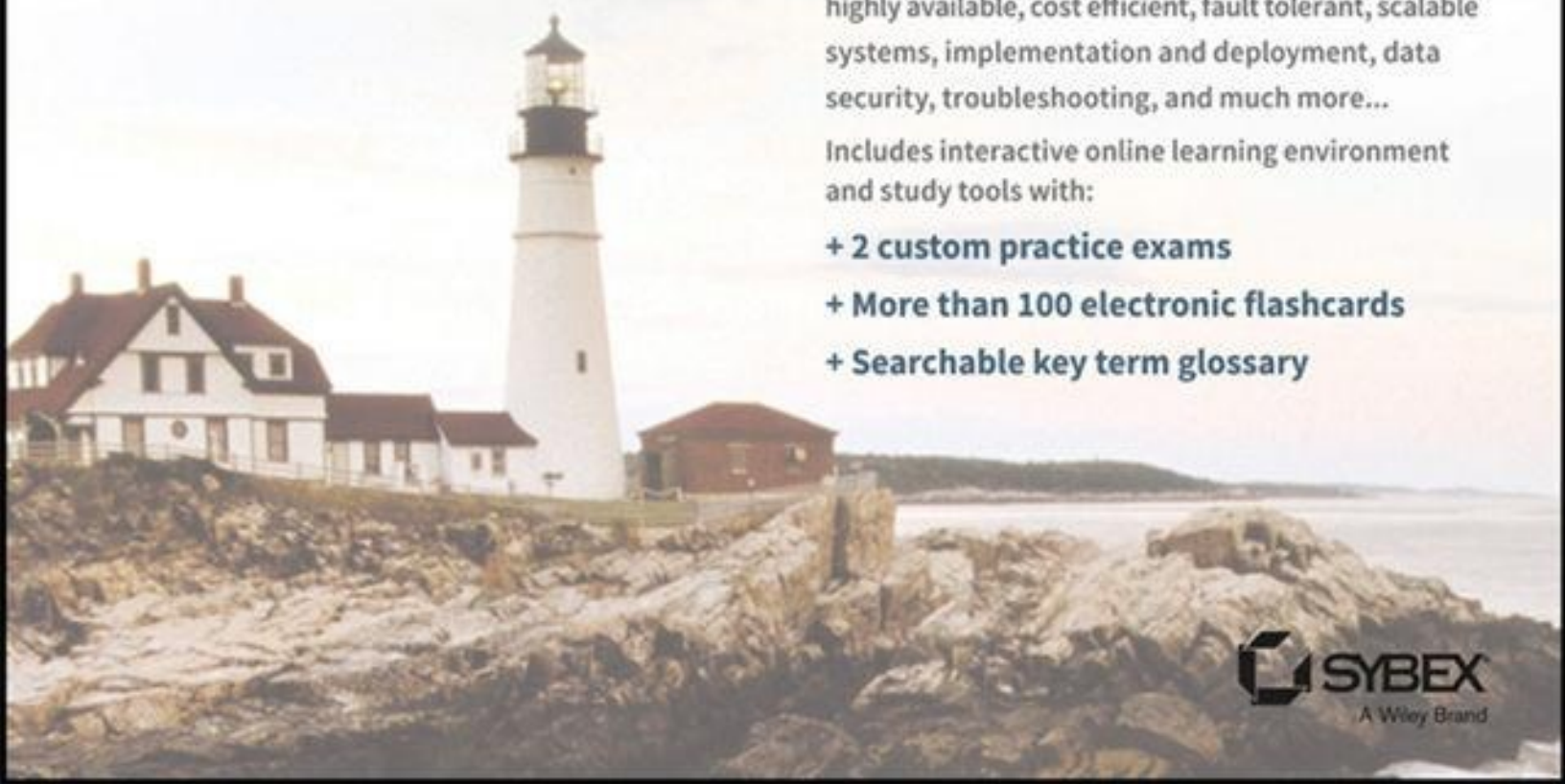
amazon web services

Joe Baron, Hisham Baz, Tim Bixler, Biff Gaut,
Kevin E. Kelly, Sean Senior, John Stamper

AWS Certified Solutions Architect

OFFICIAL STUDY GUIDE

ASSOCIATE EXAM



Covers exam objectives, including designing highly available, cost efficient, fault tolerant, scalable systems, implementation and deployment, data security, troubleshooting, and much more...

Includes interactive online learning environment and study tools with:

+ 2 custom practice exams
+ More than 100 electronic flashcards
+ Searchable key term glossary

SYBEX
A Wiley Brand

AWS®

Certified Solutions Architect Official

Study Guide - Associate Exam



Joe Baron, Hisham Baz, Tim Bixler, Biff Gaut, Kevin E. Kelly, Sean Senior, John Stamper

SYBEX
A Wiley Brand

Senior Acquisitions Editor: Kenyon Brown
Project Editor: Gary Schwartz
Production Editor: Dassi Zeidel
Copy Editor: Kezia Endsley
Editorial Manager: Mary Beth Wakefield
Production Manager: Kathleen Wisor
Executive Editor: Jim Minatel
Book Designers: Judy Fung and Bill Gibson
Proofreader: Nancy Carrasco
Indexer: Johnna vanHoose Dinse
Project Coordinator, Cover: Brent Savage
Cover Designer: Wiley
Cover Image: ©Getty Images, Inc./Jeremy Woodhouse Copyright © 2017 by AWS

Published by John Wiley & Sons, Inc. Indianapolis, Indiana Published simultaneously in Canada

ISBN: 978-1-119-13855-6

ISBN: 978-1-119-13955-3 (ebk.)

ISBN: 978-1-119-13954-6 (ebk.)

Manufactured in the United States of America No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under Sections 107 or 108 of the 1976 United States Copyright Act, without either the prior written permission of the Publisher, or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923, (978) 750-8400, fax (978) 646-8600. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, or online at http://www.wiley.com/go/permissions.

Limit of Liability/Disclaimer of Warranty: The publisher and the author make no representations or warranties with respect to the accuracy or completeness of the contents of this work and specifically disclaim all warranties, including without limitation warranties of fitness for a particular purpose. No warranty may be created or extended by sales or promotional materials. The advice and strategies contained herein may not be suitable for every situation. This work is sold with the understanding that the publisher is not engaged in rendering legal, accounting, or other professional services. If professional assistance is required, the services of a competent professional person should be sought. Neither the publisher nor the author shall be liable for damages arising herefrom. The fact that an organization or Web site is referred to in this work as a citation and/or a potential source of further information does not mean that the author or the publisher endorses the information the organization or Web site may provide or recommendations it may make. Further, readers should be aware that Internet Web sites listed in this work may have changed or disappeared between when this work was written and when it is read.

For general information on our other products and services or to obtain technical support, please contact our Customer Care Department within the U.S. at (877) 762-2974, outside the U.S. at (317) 572-3993 or fax (317) 572-4002.

Wiley publishes in a variety of print and electronic formats and by print-on-demand. Some material included with standard print versions of this book may not be included in e-books or in print-on-demand. If this book refers to media such as a CD or DVD that is not included in the version you purchased, you may download this material at http://booksupport.wiley.com. For more information about Wiley products, visit www.wiley.com.

Library of Congress Control Number: 2016949703

TRADEMARKS: Wiley, the Wiley logo, and the Sybex logo are trademarks or registered trademarks of John Wiley & Sons, Inc. and/or its affiliates, in the United States and other countries, and may not be used without written permission. AWS is a registered trademark of Amazon Technologies, Inc. All other trademarks are the property of their respective owners. John Wiley & Sons, Inc. is not associated with any product or vendor mentioned in this book.

For the original AWS instructor, Mike Culver, who taught us how to teach, lead, and inspire with tenacity and kindness.

CONTENTS

- Acknowledgments
- About the Authors
- Foreword
- Introduction
- Assessment Test
- Answers to Assessment Test
- Chapter 1 Introduction to AWS
  - What Is Cloud Computing?
  - AWS Fundamentals
  - AWS Cloud Computing Platform
  - Summary
  - Exam Essentials
  - Review Questions
- Chapter 2 Amazon Simple Storage Service (Amazon S3) and Amazon Glacier Storage
  - Introduction
  - Object Storage versus Traditional Block and File Storage
  - Amazon Simple Storage Service (Amazon S3) Basics
  - Buckets
  - Amazon S3 Advanced Features
  - Amazon Glacier
  - Summary
  - Exam Essentials
  - Exercises
  - Review Questions
- Chapter 3 Amazon Elastic Compute Cloud (Amazon EC2) and Amazon Elastic Block Store (Amazon EBS)
  - Introduction
  - Amazon Elastic Compute Cloud (Amazon EC2)
  - Amazon Elastic Block Store (Amazon EBS)
  - Summary
  - Exam Essentials
  - Exercises
  - Review Questions
- Chapter 4 Amazon Virtual Private Cloud (Amazon VPC)

- Introduction
- Amazon Virtual Private Cloud (Amazon VPC)
- Subnets
- Route Tables
- Internet Gateways
- Dynamic Host Configuration Protocol (DHCP) Option Sets
- Elastic IP Addresses (EIPs)
- Elastic Network Interfaces (ENIs)
- Endpoints
- Peering
- Security Groups
- Network Access Control Lists (ACLs)
- Network Address Translation (NAT) Instances and NAT Gateways
- Virtual Private Gateways (VPGs), Customer Gateways (CGWs), and Virtual Private Networks (VPNs)
- Summary
- Exam Essentials
- Exercises
- Review Questions

Chapter 5 Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling

- Introduction
- Elastic Load Balancing
- Amazon CloudWatch
- Auto Scaling
- Summary
- Exam Essentials
- Exercises
- Review Questions

Chapter 6 AWS Identity and Access Management (IAM)

- Principals
- Authentication
- Authorization
- Other Key Features
- Summary
- Exam Essentials
- Exercises
- Review Questions

Chapter 7 Databases and AWS
- Database Primer
- Amazon Relational Database Service (Amazon RDS)
- Amazon Redshift
- Amazon DynamoDB
- Summary
- Exam Essentials
- Exercises
- Review Questions

Chapter 8 SQS, SWF, and SNS
- Amazon Simple Queue Service (Amazon SQS)
- Amazon Simple Workflow Service (Amazon SWF)
- Amazon Simple Notification Service (Amazon SNS)
- Summary
- Exam Essentials
- Exercises
- Review Questions

Chapter 9 Domain Name System (DNS) and Amazon Route 53
- Domain Name System (DNS)
- Amazon Route 53 Overview
- Summary
- Exam Essentials
- Exercises
- Review Questions

Chapter 10 Amazon ElastiCache
- Introduction
- In-Memory Caching
- Amazon ElastiCache
- Summary
- Exam Essentials
- Exercises
- Review Questions

Chapter 11 Additional Key Services
- Introduction
- Storage and Content Delivery
- Security
- Analytics

DevOps
Summary
Exam Essentials
Review Questions
Chapter 12 Security on AWS
Introduction
Shared Responsibility Model
AWS Compliance Program
AWS Global Infrastructure Security
AWS Account Security Features
AWS Cloud Service-Specific Security
Summary
Exam Essentials
Exercises
Review Questions
Chapter 13 AWS Risk and Compliance
Introduction
Overview of Compliance in AWS
Evaluating and Integrating AWS Controls
AWS Risk and Compliance Program
AWS Reports, Certifications, and Third-Party Attestations
Summary
Exam Essentials
Review Questions
Chapter 14 Architecture Best Practices
Introduction
Design for Failure and Nothing Fails
Implement Elasticity
Leverage Different Storage Options
Build Security in Every Layer
Think Parallel
Loose Coupling Sets You Free
Don’t Fear Constraints
Summary
Exam Essentials
Exercises
Review Questions

Appendix A Answers to Review Questions
- Chapter 1: Introduction to AWS
- Chapter 2: Amazon Simple Storage Service (Amazon S3) and Amazon Glacier Storage
- Chapter 3: Amazon Elastic Compute Cloud (Amazon EC2) and Amazon Elastic Block Store (Amazon EBS)
- Chapter 4: Amazon Virtual Private Cloud (Amazon VPC)
- Chapter 5: Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling
- Chapter 6: AWS Identity and Access Management (IAM)
- Chapter 7: Databases and AWS
- Chapter 8: SQS, SWF, and SNS
- Chapter 9: Domain Name System (DNS) and Amazon Route 53
- Chapter 10: Amazon ElastiCache
- Chapter 11: Additional Key Services
- Chapter 12: Security on AWS
- Chapter 13: AWS Risk and Compliance
- Chapter 14: Architecture Best Practices

Advert

EULA

List of Tables

- Chapter 3
  - TABLE 3.1
  - TABLE 3.2
  - TABLE 3.3
  - TABLE 3.4
  - TABLE 3.5
  - TABLE 3.6
- Chapter 4
  - TABLE 4.1
  - TABLE 4.2
  - TABLE 4.3
  - TABLE 4.4
  - TABLE 4.5
- Chapter 6
  - TABLE 6.1
  - TABLE 6.2
  - TABLE 6.3
- Chapter 7
  - TABLE 7.1
  - TABLE 7.2
  - TABLE 7.3
  - TABLE 7.4
  - TABLE 7.5
- Chapter 12
  - TABLE 12.1
- Chapter 14
  - TABLE 14.1

List of Illustrations

Chapter 1

FIGURE 1.1 Six advantages of cloud computing

FIGURE 1.2 AWS Cloud computing platform

FIGURE 1.3 Auto scaling capacity

FIGURE 1.4 AWS CloudFormation workflow summary

Chapter 3

FIGURE 3.1 Memory and vCPUs for the m4 instance family

FIGURE 3.2 A workload using a mix of On-Demand and Reserved Instances

Chapter 4

FIGURE 4.1 VPC, subnets, and a route table

FIGURE 4.2 VPC, subnet, route table, and an Internet gateway

FIGURE 4.3 VPC peering connections do not support transitive routing

FIGURE 4.4 VPC with VPN connection to a customer network

Chapter 5

FIGURE 5.1 Auto Scaling group behind an Elastic Load Balancing load balancer

FIGURE 5.2 Auto Scaling group with policy

FIGURE 5.3 Amazon CloudWatch alarm triggering scaling out

Chapter 6

FIGURE 6.1 Different identities authenticating with AWS

FIGURE 6.2 Associating IAM users with policies

Chapter 7

FIGURE 7.1 Multi-AZ Amazon RDS architecture

FIGURE 7.2 Amazon Redshift cluster architecture

FIGURE 7.3 Table, items, attributes relationship

FIGURE 7.4 Table partitioning

Chapter 8

FIGURE 8.1 Message lifecycle

FIGURE 8.2 Diagram of visibility timeout

FIGURE 8.3 Amazon SWF workflow illustration

FIGURE 8.4 Diagram of topic delivery

FIGURE 8.5 Diagram of fanout scenario

Chapter 9

FIGURE 9.1 FQDN components

Chapter 10

FIGURE 10.1 Common caching architecture

FIGURE 10.2 Redis replication group

Chapter 11

FIGURE 11.1 Delivering static and dynamic content

FIGURE 11.2 High availability CloudHSM architecture

FIGURE 11.3 Amazon Kinesis Firehose

FIGURE 11.4 Amazon Kinesis Streams

FIGURE 11.5 Example pipeline

FIGURE 11.6 Simple application server stack

FIGURE 11.7 Simple application server stack with AWS OpsWorks

FIGURE 11.8 Creating a stack workflow

FIGURE 11.9 Updating a stack workflow

FIGURE 11.10 AWS Trusted Advisor Console dashboard

Chapter 12

FIGURE 12.1 The shared responsibility model

FIGURE 12.2 Amazon Web Services regions

FIGURE 12.3 Amazon EC2 multiple layers of security

FIGURE 12.4 Amazon EC2 security group firewall

FIGURE 12.5 Amazon VPC network architecture

FIGURE 12.6 Flexible network architectures

Chapter 13

FIGURE 13.1 Shared responsibility model

Chapter 14

FIGURE 14.1 Simple web application architecture

FIGURE 14.2 Updated web application architecture with redundancy

FIGURE 14.3 Updated web application architecture with auto scaling

FIGURE 14.4 Updated web application architecture with Amazon S3 and Amazon CloudFront

FIGURE 14.5 Updated web application architecture with Amazon ElastiCache and Amazon DynamoDB

FIGURE 14.6 Tight and loose coupling

FIGURE 14.7 Sample web application for chapter exercises

Acknowledgments

The authors would like to thank a few people who helped us develop and write this *AWS Certified Solutions Architect Official Study Guide: Associate Exam.*

First, thanks to all our families who put up with us spending weekends and evenings creating content, writing questions, and reviewing each other's chapters. Their patience and support made this book possible.

Niamh O'Byrne, AWS Certification Manager, who introduced all of the authors and many more solutions architects at AWS to certification testing and got this book started by challenging some of us to extend our reach and help more cloud practitioners get certified.

Nathan Bower and Victoria Steidel, amazing technical writers at AWS who reviewed and edited all the content and every question and gently made us better writers and communicators. They were tireless in reviewing and helping us hone and focus our content.

Patrick Shumate, a fellow AWS solutions architect who contributed test questions right when we needed the help to get us over the finish line.

We could not have written this book without the help of our friends at Wiley. Kenyon Brown, Senior Acquisitions Editor, corralled us and focused us on the end goal. Additionally, we were guided by Gary Schwartz, Project Editor; Kezia Endsley, Copyeditor; and Dassi Zeidel, Production Editor who took output from different authors and turned it into a cohesive and complete finished product.

Lastly, we want to thank all the solutions architects at AWS who participated in certification blueprint development, question writing, and review sessions, and the development of a world-class certification program for cloud practitioners that is setting the standard for our industry.

About the Authors



Joe Baron, Principal Solutions Architect for AWS, is currently working with customers in the Southeastern United States. Joe joined AWS in 2009 as one of the first solutions architects, and in the years since he has helped customers of all sizes, from small startups to some of the largest enterprises in the world, to architect their infrastructures and migrate their applications to the cloud. He was also an early contributor to the AWS Associate and Professional Certified Solutions Architect programs. Joe holds a BS degree in engineering physics from Cornell University and is proud to be an "expert generalist." Prior to joining AWS, Joe had 25 years of experience in technology, with roles in data center automation, virtualization, life sciences, high-performance computing, 3D visualization, hardware and software development, and Independent Software Vendor (ISV) program management. He is also a dedicated husband to Carol and father of two children, Matt and Jessie. When not helping customers migrate all the things to the cloud, Joe is an amateur classical pianist and collector of traditional woodworking tools. He lives in the Raleigh, NC area.



Hisham Baz is a passionate software engineer and systems architect with expertise building distributed applications and high-performance, mission-critical systems. Since 2013, Hisham has been a solutions architect with AWS working with customers like Pinterest, Airbnb, and General Electric to build resilient architectures in the cloud with a focus on big data and analytics. Prior to Amazon, Hisham founded two early-stage startups, modernized the communications network connecting critical transportation infrastructure, and improved cellular networks with large-scale data analytics. Hisham is based in San Francisco, CA and lives with his wife, Suki. They can often be found hiking the redwoods.



Tim Bixler, Commercial Americas Southeast Area Solutions Architecture Leader for AWS, leads teams of solutions architects who provide AWS technical enablement, evangelism, and knowledge transfer to customers like Capital One, The Coca-Cola Company, AOL, Koch Industries, Cox Automotive, NASCAR, Emdeon, and Neustar. Tim has over 20 years of experience in improving systems and operational performance, productivity, and customer satisfaction for private and public global corporations as well as government agencies. He is also a public speaker for Amazon and enjoys helping customers adopt innovative solutions on AWS. But if you ask his 7-year-old son TJ what he does, he might say that daddy is a builder and a fixer. When not otherwise tasked, you can find him burrowed in his lab building robots driven by microcontrollers or at the local BrickFair admiring the creations that he has no time to build.



Biff Gaut started writing programs for a living on CP/M on the Osborne 1. Since those early days, he obtained a BS in engineering from Virginia Tech while writing C code on MS-DOS, married his wife, Holly, while writing his first GUI apps, and raised two children while transitioning from COM objects in C++ to web apps in .NET. Along the way, he led development teams from 1 to 50 members for companies including NASDAQ, Thomson Reuters, Verizon, Microsoft, FINRA, and Marriott. He has collaborated on two books and spoken at countless conferences, including Windows World and the Microsoft PDC. Biff is currently a solutions architect at AWS, helping customers across the country realize the benefits of the cloud by deploying secure, available, efficient workloads on AWS. And yes, that’s his real name.



Kevin E. Kelly, Solutions Architecture Manager and early contributor to the AWS Solutions Architecture Certification exams, has been at AWS for over seven years helping companies architect their infrastructures and migrate their applications to the cloud. Kevin has a BS in computer science from Mercer University and a Master of Information Systems in business from the University of Montana. Before joining Amazon, Kevin was an Air Force officer, a programmer—including embedded programming—and a technical presales leader. Kevin has been the chairman of the Worldwide Web Consortium (W3C) Compound Document Format Working Group and led that open-standards working group in developing the Web Interactive Compound Document (WICD) profile for mobile and desktop devices. He has also served as the W3C Advisory Council Representative for Health Level 7 (HL7). Kevin lives in Virginia with his wife, Laurie, and their two daughters, Caroline and Amelia. Kevin is an amateur violin and mandolin player and a zymurgist.



Sean Senior is a solutions architect at AWS. Sean is a builder at heart and thrives in a fast-paced environment with continuous challenges. Sean has a BS in computer information and sciences from the University of Maryland University College. Sean is a devoted husband and father of a beautiful girl. He is a U.S. Navy veteran, avid sports fan, and gym rat. He loathes talking about himself in the third person, but can be persuaded to do so for a good reason.



John Stamper, Principal Solutions Architect at AWS, is a co-inventor for multiple AWS patents and is particularly fond of distributed systems at scale. John holds a BS in mathematics from James Madison University (94) and an MS in Information Systems from George Mason University (04). In addition to building systems on the cloud and helping customers reimagine their businesses, John is a dedicated husband and father of three children. He is a CrossFit athlete, youth sports coach, and vocal supporter of the arts.

Foreword

This *AWS Certified Solutions Architect Official Study Guide: Associate Exam* has been written to help you prepare for the AWS Certified Solutions Architect – Associate exam. This certification is becoming an increasingly important credential that every information technology professional and cloud practitioner who plans, designs, and builds application architectures for deployment on AWS should obtain. Passing the AWS Certified Solutions Architect – Associate exam demonstrates to your colleagues, employers, and the industry at large that you know how to build and deploy AWS solutions that are highly available, secure, performant, and cost effective.

This study guide was written by AWS solutions architects who wrote and reviewed exam questions for the AWS Certified Solutions Architect exams. Although nothing replaces hands-on experience building and deploying a variety of cloud applications and controls on AWS, this study guide, and the questions and exercises in each chapter, provide you with coverage of the basic AWS Cloud services combined with architectural recommendations and best practices that will help prepare you for the exam. Combining this study guide with production application deployment experience and taking the practice exams online will prepare you well and allow you to take the exam with confidence. Adding the AWS Certified Solutions Architect—Associate certification to your credentials will establish you as an industry-recognized solutions architect for the AWS platform!

—Kevin E. Kelly Americas Solutions Architecture Lead AWS Certified Solutions Architect – Associate AWS Certified Solutions Architect – Professional Herndon, VA

Introduction

Studying for any certification exam can seem daunting. This *AWS Certified Solutions Architect Official Study Guide: Associate Exam* was designed and developed with relevant topics, questions, and exercises to enable a cloud practitioner to focus their precious study time and effort on the germane set of topics targeted at the right level of abstraction so they can confidently take the AWS Certified Solutions Architect – Associate exam.

This study guide presents a set of topics needed to round out a cloud practitioner's hands-on experiences with AWS by covering the basic AWS Cloud services and concepts within the scope of the AWS Certified Solutions Architect – Associate exam. This study guide begins with an introduction to AWS, which is then followed by chapters on specific AWS Cloud services. In addition to the services chapters, the topics of security, risk and compliance, and architecture best practices are covered, providing the reader with a solid base for understanding how to build and deploy applications on the AWS platform. Furthermore, the AWS architectural best practices and principles are reinforced in every chapter and reflected in the self-study questions and examples to highlight the development and deployment of applications for AWS that are secure, highly available, performant, and cost effective. Each chapter includes specific information on the service or topic covered, followed by an Exam Essentials section that contains key information needed in your exam preparation. The Exam Essentials section is followed by an Exercise section with exercises designed to help reinforce the topic of the chapter with hands-on learning. Next, each chapter contains sample questions to get you accustomed to answering questions about AWS Cloud services and architecture topics. The book also contains a self-assessment exam with 25 questions, two practice exams, with 50 questions each to help you gauge your readiness to take the exam, and flashcards to help you learn and retain key facts needed to prepare for the exam.

If you are looking for a targeted book written by solutions architects who wrote, reviewed, and developed the AWS Certified Solutions Architect – Associate exam, then this is the book for you.

What Does This Book Cover?

This book covers topics you need to know to prepare for the Amazon Web Services (AWS) Certified Solutions Architect – Associate exam:

Chapter 1: Introduction to AWS This chapter provides an introduction to the AWS Cloud computing platform. It discusses the advantages of cloud computing and the fundamentals of AWS. It provides an overview of the AWS Cloud services that are fundamentally important for the exam.

Chapter 2: Amazon Simple Storage Service (Amazon S3) and Amazon Glacier Storage This chapter provides you with a basic understanding of the core object storage services available on AWS: Amazon Simple Storage Service (Amazon S3) and Amazon Glacier. These services are used to store objects on AWS.

Chapter 3: Amazon Elastic Compute Cloud (Amazon EC2) and Amazon Elastic Block Store (Amazon EBS) In this chapter, you will learn how Amazon Elastic Compute Cloud (Amazon EC2) and Amazon Elastic Block Store (Amazon EBS) provide the basic elements of compute and block-level storage to run your workloads on AWS.

Chapter 4: Amazon Virtual Private Cloud (Amazon VPC) This chapter describes Amazon Virtual Private Cloud (Amazon VPC), which is a custom-defined virtual network within AWS. You will learn how to design secure architectures using Amazon VPC to provision your own logically isolated section of AWS.

Chapter 5: Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling In this chapter, you will learn how Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling work independently and together to help you efficiently and cost-effectively deploy highly available and optimized workloads on AWS.

Chapter 6: AWS Identity and Access Management (IAM) This chapter covers AWS Identity and Access Management (IAM), which is used to secure transactions with the AWS resources in your AWS account.

Chapter 7: Databases and AWS This chapter covers essential database concepts and introduces three of AWS managed database services: Amazon Relational Database Service (Amazon RDS), Amazon DynamoDB, and Amazon Redshift. These managed services simplify the setup and operation of relational databases, NoSQL databases, and data warehouses.

Chapter 8: SQS, SWF, and SNS This chapter focuses on application services in AWS, specifically Amazon Simple Queue Service (Amazon SQS), Amazon Simple Workflow Service (SWF), and Amazon Simple Notification Service (Amazon SNS). It also covers architectural guidance on using these services and the use of Amazon SNS in mobile applications.

Chapter 9: Domain Name System (DNS) and Amazon Route 53 In this chapter, you will learn about Domain Name System (DNS) and the Amazon Route 53 service, which is designed to help users find your website or application over the Internet.

Chapter 10: Amazon ElastiCache This chapter focuses on building high-performance applications using in-memory caching technologies and Amazon ElastiCache.

Chapter 11: Additional Key Services Additional services not covered in other chapters are covered in this chapter. Topics include Amazon CloudFront, AWS Storage Gateway, AWS Directory Service, AWS

Key Management Service (KMS), AWS CloudHSM, AWS CloudTrail, Amazon Kinesis, Amazon Elastic Map Reduce (Amazon EMR), AWS Data Pipeline, AWS Import/Export, AWS OpsWorks, AWS CloudFormation, AWS Elastic Beanstalk, AWS Trusted Advisor, and AWS Config.

Chapter 12: Security on AWS This chapter covers the relevant security topics that are within scope for the AWS Certified Solutions Architect – Associate exam.

Chapter 13: AWS Risk and Compliance This chapter covers topics associated with risk and compliance, risk mitigation, and the shared responsibility model of using AWS.

Chapter 14: Architecture Best Practices The final chapter covers the AWS-recommended design principles and best practices for architecting systems and applications for the Cloud.

Interactive Online Learning Environment and Test Bank

The authors have worked hard to provide some really great tools to help you with your certification process. The interactive online learning environment that accompanies the *AWS Certified Solutions Architect Official Study Guide: Associate Exam* provides a test bank with study tools to help you prepare for the certification exam—and increase your chances of passing it the first time! The test bank includes the following:

Sample Tests All the questions in this book are provided, including the assessment test at the end of this Introduction and the chapter tests that include the review questions at the end of each chapter. In addition, there are two practice exams with 50 questions each. Use these questions to test your knowledge of the study guide material. The online test bank runs on multiple devices.

Flashcards The online text banks include 100 flashcards specifically written to hit you hard, so don't get discouraged if you don't ace your way through them at first. They're there to ensure that you're really ready for the exam. And no worries—armed with the review questions, practice exams, and flashcards, you'll be more than prepared when exam day comes. Questions are provided in digital flashcard format (a question followed by a single correct answer). You can use the flashcards to reinforce your learning and provide last-minute test prep before the exam.

Glossary A glossary of key terms from this book is available as a fully searchable PDF.



Go to http://www.wiley.com/go/sybextestprep to register and gain access to this interactive online learning environment and test bank with study tools.

Exam Objectives

The AWS Certified Solutions Architect—Associate exam is intended for people who have experience in designing distributed applications and systems on the AWS platform. Here are some of the key exam topics that you should understand for this exam:

- Designing and deploying scalable, highly available, and fault-tolerant systems on AWS
- Migrating existing on-premises applications to AWS
- Ingress and egress of data to and from AWS
- Selecting the appropriate AWS service based on data, compute, database, or security requirements
- Identifying appropriate use of AWS architectural best practices
- Estimating AWS costs and identifying cost control mechanisms

In general, candidates should have the following:

- One or more years of hands-on experience designing highly available, cost efficient, secure, fault tolerant, and scalable distributed systems on AWS
- In-depth knowledge of at least one high-level programming language
- Ability to identify and define requirements for an AWS-based application
- Experience with deploying hybrid systems with on-premises and AWS components
- Capability to provide best practices for building secure and reliable applications on the AWS platform

The exam covers four different domains, with each domain broken down into objectives and subobjectives.

Objective Map

The following table lists each domain and its weighting in the exam, along with the chapters in the book where that domain's objectives and subobjectives are covered.

Domain	**Percentage of Exam**	**Chapter**
1 Domain 1.0: Designing highly available, cost-efficient, fault-tolerant, scalable systems	60%	
1.1 Identify and recognize cloud architecture considerations, such as fundamental components and effective designs.		1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 14
Content may include the following:		
How to design cloud services		1, 2, 3, 4, 8, 9,

		11, 14
Planning and design		1, 2, 3, 4, 7, 8, 9, 10, 11, 14
Monitoring and logging		2, 3, 8, 9, 11
Familiarity with:		
Best practices for AWS architecture		1, 2, 4, 7, 8, 9, 10, 14
Developing to client specifications, including pricing/cost (e.g., on Demand vs. Reserved vs. Spot; RTO and RPO DR Design)		2, 7, 9
Architectural trade-off decisions (e.g., high availability vs. cost, Amazon Relational Database Service (RDS) vs. installing your own database on Amazon Elastic Compute Cloud (EC2))		2, 4, 7, 8, 9, 10
Hybrid IT architectures (e.g., Direct Connect, Storage Gateway, VPC, Directory Services)		1, 2, 4, 14
Elasticity and scalability (e.g., Auto Scaling, SQS, ELB, CloudFront)		1, 2, 5, 7, 8, 9, 10, 14
2 Domain 2.0: Implementation/Deployment	10%	
2.1 Identify the appropriate techniques and methods using Amazon EC2, Amazon S3, AWS Elastic Beanstalk, AWS CloudFormation, AWS OpsWorks, Amazon Virtual Private Cloud (VPC), and AWS Identity and Access Management (IAM) to code and implement a cloud solution.		1, 2, 3, 4, 5, 6, 8, 11, 13
Content may include the following:		
Configure an Amazon Machine Image (AMI).		2, 3, 11
Operate and extend service management in a hybrid IT architecture.		1, 4
Configure services to support compliance requirements in the cloud.		2, 3, 4, 11, 13
Launch instances across the AWS global infrastructure.		1, 2, 3, 5, 8, 11
Configure IAM policies and best practices.		2, 6
3 Domain 3.0: Data Security	20%	
3.1 Recognize and implement secure practices for optimum cloud deployment and maintenance.		2, 4, 10, 12, 13
Content may include the following:		
AWS shared responsibility model		12, 13
AWS platform compliance		11, 12,

		13
AWS security attributes (customer workloads down to physical layer)		4, 11, 12, 13
AWS administration and security services		7, 10, 11, 12
AWS Identity and Access Management (IAM)		6, 12
Amazon Virtual Private Cloud (VPC)		4, 12
AWS CloudTrail		11, 12
Ingress vs. egress filtering, and which AWS services and features fit		11, 12
“Core” Amazon EC2 and S3 security feature sets		2, 4, 12
Incorporating common conventional security products (Firewall, VPN)		4, 12
Design patterns		7, 13
DDoS mitigation		12
Encryption solutions (e.g., key services)		2, 11, 12
Complex access controls (building sophisticated security groups, ACLs, etc.)		2, 12
Amazon CloudWatch for the security architect		5
Trusted Advisor		11
CloudWatch Logs		5
3.2 Recognize critical disaster recovery techniques and their implementation.		3, 7, 9, 10
Content may include the following:		
Disaster recovery		3
Recovery time objective		7
Recovery point objective		7
Amazon Elastic Block Store		3
AWS Import/Export		11
AWS Storage Gateway		11
Amazon Route53		9
Validation of data recovery method		3
4 Domain 4.0: Troubleshooting	10%	
Content may include the following:		
General troubleshooting information and questions		5, 8

Assessment Test

1. Under a single AWS account, you have set up an Auto Scaling group with a maximum capacity of 50 Amazon Elastic Compute Cloud (Amazon EC2) instances in us-west-2. When you scale out, however, it only increases to 20 Amazon EC2 instances. What is the likely cause?
   A. Auto Scaling has a hard limit of 20 Amazon EC2 instances.
   B. If not specified, the Auto Scaling group maximum capacity defaults to 20 Amazon EC2 instances.
   C. The Auto Scaling group desired capacity is set to 20, so Auto Scaling stopped at 20 Amazon EC2 instances.
   D. You have exceeded the default Amazon EC2 instance limit of 20 per region.
2. Elastic Load Balancing allows you to distribute traffic across which of the following?
   A. Only within a single Availability Zone
   B. Multiple Availability Zones within a region
   C. Multiple Availability Zones within and between regions
   D. Multiple Availability Zones within and between regions and on-premises virtualized instances running OpenStack
3. Amazon CloudWatch offers which types of monitoring plans? (Choose 2 answers)
   A. Basic
   B. Detailed
   C. Diagnostic
   D. Precognitive
   E. Retroactive
4. An Amazon Elastic Compute Cloud (Amazon EC2) instance in an Amazon Virtual Private Cloud (Amazon VPC) subnet can send and receive traffic from the Internet when which of the following conditions are met? (Choose 3 answers)
   A. Network Access Control Lists (ACLs) and security group rules disallow all traffic except relevant Internet traffic.
   B. Network ACLs and security group rules allow relevant Internet traffic.
   C. Attach an Internet Gateway (IGW) to the Amazon VPC and create a subnet route table to send all non-local traffic to that IGW.
   D. Attach a Virtual Private Gateway (VPG) to the Amazon VPC and create subnet routes to send all non-local traffic to that VPG.
   E. The Amazon EC2 instance has a public IP address or Elastic IP (EIP) address.
   F. The Amazon EC2 instance does not need a public IP or Elastic IP when using Amazon VPC.

5. If you launch five Amazon Elastic Compute Cloud (Amazon EC2) instances in an Amazon Virtual Private Cloud (Amazon VPC) without specifying a security group, the instances will be launched into a default security group that provides which of the following? (Choose 3 answers)
   A. The five Amazon EC2 instances can communicate with each other.
   B. The five Amazon EC2 instances cannot communicate with each other.
   C. All inbound traffic will be allowed to the five Amazon EC2 instances.
   D. No inbound traffic will be allowed to the five Amazon EC2 instances.
   E. All outbound traffic will be allowed from the five Amazon EC2 instances.
   F. No outbound traffic will be allowed from the five Amazon EC2 instances.
6. Your company wants to host its secure web application in AWS. The internal security policies consider any connections to or from the web server as insecure and require application data protection. What approaches should you use to protect data in transit for the application? (Choose 2 answers)
   A. Use BitLocker to encrypt data.
   B. Use HTTPS with server certificate authentication.
   C. Use an AWS Identity and Access Management (IAM) role.
   D. Use Secure Sockets Layer (SSL)/Transport Layer Security (TLS) for database connection.
   E. Use XML for data transfer from client to server.
7. You have an application that will run on an Amazon Elastic Compute Cloud (Amazon EC2) instance. The application will make requests to Amazon Simple Storage Service (Amazon S3) and Amazon DynamoDB. Using best practices, what type of AWS Identity and Access Management (IAM) identity should you create for your application to access the identified services?
   A. IAM role
   B. IAM user
   C. IAM group
   D. IAM directory
8. When a request is made to an AWS Cloud service, the request is evaluated to decide whether it should be allowed or denied. The evaluation logic follows which of the following rules? (Choose 3 answers)
   A. An explicit allow overrides any denies.
   B. By default, all requests are denied.
   C. An explicit allow overrides the default.
   D. An explicit deny overrides any allows.
   E. By default, all requests are allowed.
9. What is the data processing engine behind Amazon Elastic MapReduce (Amazon EMR)?
   A. Apache Hadoop

B. Apache Hive

C. Apache Pig

D. Apache HBase

10. What type of AWS Elastic Beanstalk environment tier provisions resources to support a web application that handles background processing tasks?

A. Web server environment tier

B. Worker environment tier

C. Database environment tier

D. Batch environment tier

11. What Amazon Relational Database Service (Amazon RDS) feature provides the high availability for your database?

A. Regular maintenance windows

B. Security groups

C. Automated backups

D. Multi-AZ deployment

12. What administrative tasks are handled by AWS for Amazon Relational Database Service (Amazon RDS) databases? (Choose 3 answers)

A. Regular backups of the database

B. Deploying virtual infrastructure

C. Deploying the schema (for example, tables and stored procedures)

D. Patching the operating system and database software

E. Setting up non-admin database accounts and privileges

13. Which of the following use cases is well suited for Amazon Redshift?

A. A 500TB data warehouse used for market analytics

B. A NoSQL, unstructured database workload

C. A high traffic, e-commerce web application

D. An in-memory cache

14. Which of the following statements about Amazon DynamoDB secondary indexes is true?

A. There can be many per table, and they can be created at any time.

B. There can only be one per table, and it must be created when the table is created.

C. There can be many per table, and they can be created at any time.

D. There can only be one per table, and it must be created when the table is created.

15. What is the primary use case of Amazon Kinesis Firehose?

A. Ingest huge streams of data and allow custom processing of data in flight.

B. Ingest huge streams of data and store it to Amazon Simple Storage Service (Amazon S3), Amazon Redshift, or Amazon Elasticsearch Service.

C. Generate a huge stream of data from an Amazon S3 bucket.

D. Generate a huge stream of data from Amazon DynamoDB.

16. Your company has 17TB of financial trading records that need to be stored for seven years by law. Experience has shown that any record more than a year old is unlikely to be accessed. Which of the following storage plans meets these needs in the most cost-efficient manner?

A. Store the data on Amazon Elastic Block Store (Amazon EBS) volume attached to t2.large instances.

B. Store the data on Amazon Simple Storage Service (Amazon S3) with lifecycle policies that change the storage class to Amazon Glacier after one year, and delete the object after seven years.

C. Store the data in Amazon DynamoDB, and delete data older than seven years.

D. Store the data in an Amazon Glacier Vault Lock.

17. What must you do to create a record of who accessed your Amazon Simple Storage Service (Amazon S3) data and from where?

A. Enable Amazon CloudWatch logs.

B. Enable versioning on the bucket.

C. Enable website hosting on the bucket.

D. Enable server access logs on the bucket.

E. Create an AWS Identity and Access Management (IAM) bucket policy.

18. Amazon Simple Storage Service (Amazon S3) is an eventually consistent storage system. For what kinds of operations is it possible to get stale data as a result of eventual consistency?

A. GET after PUT of a new object

B. GET or LIST after a DELETE

C. GET after overwrite PUT (PUT to an existing key)

D. DELETE after GET of new object

19. How is data stored in Amazon Simple Storage Service (Amazon S3) for high durability?

A. Data is automatically replicated to other regions.

B. Data is automatically replicated to different Availability Zones within a region.

C. Data is replicated only if versioning is enabled on the bucket.

D. Data is automatically backed up on tape and restored if needed.

20. Your company needs to provide streaming access to videos to authenticated users around the world. What is a good way to accomplish this?

A. Use Amazon Simple Storage Service (Amazon S3) buckets in each region with website hosting enabled.

B. Store the videos on Amazon Elastic Block Store (Amazon EBS) volumes.

C. Enable Amazon CloudFront with geolocation and signed URLs.

D. Run a fleet of Amazon Elastic Compute Cloud (Amazon EC2) instances to host the videos.

21. Which of the following are true about the AWS shared responsibility model? (Choose 3 answers)

A. AWS is responsible for all infrastructure components (that is, AWS Cloud services) that support customer deployments.

B. The customer is responsible for the components from the guest operating system upward (including updates, security patches, and antivirus software).

C. The customer may rely on AWS to manage the security of their workloads deployed on AWS.

D. While AWS manages security of the cloud, security in the cloud is the responsibility of the customer.

E. The customer must audit the AWS data centers personally to confirm the compliance of AWS systems and services.

22. Which process in an Amazon Simple Workflow Service (Amazon SWF) workflow implements a task?

A. Decider

B. Activity worker

C. Workflow starter

D. Business rule

23. Which of the following is true if you stop an Amazon Elastic Compute Cloud (Amazon EC2) instance with an Elastic IP address in an Amazon Virtual Private Cloud (Amazon VPC)?

A. The instance is disassociated from its Elastic IP address and must be re-attached when the instance is restarted.

B. The instance remains associated with its Elastic IP address.

C. The Elastic IP address is released from your account.

D. The instance is disassociated from the Elastic IP address temporarily while you restart the instance.

24. Which Amazon Elastic Compute Cloud (Amazon EC2) pricing model allows you to pay a set hourly price for compute, giving you full control over when the instance launches and terminates?

A. Spot instances

B. Reserved instance

C. On Demand instances

D. Dedicated instances

25. Under what circumstances will Amazon Elastic Compute Cloud (Amazon EC2) instance store data not be preserved?

A. The associated security groups are changed.

B. The instance is stopped or rebooted.

C. The instance is rebooted or terminated.

D. The instance is stopped or terminated.

E. None of the above

Answers to Assessment Test

1. D. Auto Scaling may cause you to reach limits of other services, such as the default number of Amazon EC2 instances you can currently launch within a region, which is 20.
2. B. The Elastic Load Balancing service allows you to distribute traffic across a group of Amazon Elastic Compute Cloud (Amazon EC2) instances in one or more Availability Zones within a region.
3. A and B. Amazon CloudWatch has two plans: basic and detailed. There are no diagnostic, precognitive, or retroactive monitoring plans for Amazon CloudWatch.
4. B, C, and E. You must do the following to create a public subnet with Internet access: Attach an IGW to your Amazon VPC.

   Create a subnet route table rule to send all non-local traffic (for example, 0.0.0.0/0) to the IGW.

   Configure your network ACLs and security group rules to allow relevant traffic to flow to and from your instance.

   You must do the following to enable an Amazon EC2 instance to send and receive traffic from the Internet: Assign a public IP address or EIP address.
5. A, D, and E. If a security group is not specified at launch, then an Amazon EC2 instance will be launched into the default security group for the Amazon VPC. The default security group allows communication between all resources within the security group, allows all outbound traffic, and denies all other traffic.
6. B and D. To protect data in transit from the clients to the web application, HTTPS with server certificate authentication should be used. To protect data in transit from the web application to the database, SSL/TLS for database connection should be used.
7. A. Don't create an IAM user (or an IAM group) and pass the user's credentials to the application or embed the credentials in the application. Instead, create an IAM role that you attach to the Amazon EC2 instance to give applications running on the instance temporary security credentials. The credentials have the permissions specified in the policies attached to the role. A directory is not an identity object in IAM.
8. B, C, and D. When a request is made, the AWS service decides whether a given request should be allowed or denied. The evaluation logic follows these rules: 1) By default, all requests are denied (in general, requests made using the account credentials for resources in the account are always allowed).

   2) An explicit allow overrides this default.

   3) An explicit deny overrides any allows.
9. A. Amazon EMR uses Apache Hadoop as its distributed data processing engine. Hadoop is an open source, Java software framework that supports data-intensive distributed applications running on large clusters of commodity hardware. Hive, Pig, and HBase are packages that run on top of Hadoop.
10. B. An environment tier whose web application runs background jobs is known as a worker tier. An

environment tier whose web application processes web requests is known as a web server tier. Database and batch are not valid environment tiers.

11. D. Multi-AZ deployment uses synchronous replication to a different Availability Zone so that operations can continue on the replica if the master database stops responding for any reason. Automated backups provide disaster recovery, not high availability. Security groups, while important, have no effect on availability. Maintenance windows are actually times when the database may not be available.

12. A, B, and D. Amazon RDS will launch Amazon Elastic Compute Cloud (Amazon EC2) instances, install the database software, handle all patching, and perform regular backups. Anything within the database software (schema, user accounts, and so on) is the responsibility of the customer.

13. A. Amazon Redshift is a petabyte-scale data warehouse. It is not well suited for unstructured NoSQL data or highly dynamic transactional data. It is in no way a cache.

14. D. There can be one secondary index per table, and it must be created when the table is created.

15. B. The Amazon Kinesis family of services provides functionality to ingest large streams of data. Amazon Kinesis Firehose is specifically designed to ingest a stream and save it to any of the three storage services listed in Response B.

16. B. Amazon S3 and Amazon Glacier are the most cost-effective storage services. After a year, when the objects are unlikely to be accessed, you can save costs by transferring the objects to Amazon Glacier where the retrieval time is three to five hours.

17. D. Server access logs provide a record of any access to an object in Amazon S3.

18. C. Amazon S3 provides read-after-write consistency for PUTs to new objects (new key), but eventual consistency for GETs and DELETEs of existing objects (existing key). Response C changes the existing object so that a subsequent GET may fetch the previous and inconsistent object.

19. B. AWS will never transfer data between regions unless directed to by you. Durability in Amazon S3 is achieved by replicating your data geographically to different Availability Zones regardless of the versioning configuration. AWS doesn't use tapes.

20. C. Amazon CloudFront provides the best user experience by delivering the data from a geographically advantageous edge location. Signed URLs allow you to control access to authenticated users.

21. A, B, and D. In the AWS shared responsibility model, customers retain control of what security they choose to implement to protect their own content, platform, applications, systems, and networks, no differently than they would for applications in an on-site data center.

22. B. An activity worker is a process or thread that performs the activity tasks that are part of your workflow. Each activity worker polls Amazon SWF for new tasks that are appropriate for that activity worker to perform; certain tasks can be performed only by certain activity workers. After receiving a task, the activity worker processes the task to completion and then reports to Amazon SWF that the task was completed and provides the result. The activity task represents one of the tasks that you identified in your application.

23. B. In an Amazon VPC, an instance's Elastic IP address remains associated with an instance when the instance is stopped.

24. C. You pay a set hourly price for an On Demand instance from when you launch it until you explicitly

stop or terminate it. Spot instances can be terminated when the spot price goes above your bid price. Reserved instances involve paying for an instance over a one- or three-year term. Dedicated instances run on hardware dedicated to your account and are not a pricing model.

25. D. The data in an instance store persists only during the lifetime of its associated instance. If an instance is stopped or terminated, then the instance store does not persist. Rebooting an instance does not shut down the instance; if an instance reboots (intentionally or unintentionally), data on the instance store persists. Security groups have nothing to do with the lifetime of an instance and have no effect here.

Chapter 1
Introduction to AWS

THE AWS CERTIFIED SOLUTIONS ARCHITECT ASSOCIATE EXAM OBJECTIVES COVERED IN THIS CHAPTER MAY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

Domain 1.0: Designing highly available, cost-efficient, fault-tolerant, scalable systems

✓**1.1 Identify and recognize cloud architecture considerations, such as fundamental components and effective designs.**

Content may include the following:

- How to design cloud services
- Planning and design
- Familiarity with:
  - Best practices for AWS architecture
  - Hybrid IT architectures (e.g., AWS Direct Connect, AWS Storage Gateway, Amazon Virtual Private Cloud [Amazon VPC], AWS Directory Service)
  - Elasticity and scalability (e.g., Auto Scaling, Amazon Simple Queue Service [Amazon SQS], Elastic Load Balancing, Amazon CloudFront)

Domain 2.0: Implementation/Deployment

✓ **2.1 Identify the appropriate techniques and methods using Amazon Elastic Compute Cloud (Amazon EC2), Amazon Simple Storage Service (Amazon S3), AWS Elastic Beanstalk, AWS CloudFormation, AWS OpsWorks, Amazon VPC, and AWS Identity and Access Management (IAM) to code and implement a cloud solution.**

Content may include the following:

- Operate and extend service management in a hybrid IT architecture.
- Configure services to support compliance requirements in the cloud.
- Launch instances across the AWS global infrastructure.



In 2006, Amazon Web Services, Inc. (AWS) began offering IT infrastructure services

to businesses in the form of web services, now commonly known as *cloud computing*. One of the key benefits of cloud computing is the opportunity to replace up-front capital infrastructure expenses with low variable costs that scale with your business. With the cloud, businesses no longer need to plan for and procure servers and other IT infrastructure weeks or months in advance. Instead, they can instantly spin up hundreds or thousands of servers in minutes and deliver results faster.

Today, AWS provides a highly reliable, scalable, and low-cost infrastructure platform in the cloud that powers hundreds of thousands of businesses in more than 190 countries around the world.

This chapter provides an introduction to the AWS Cloud computing platform. It discusses the advantages of cloud computing and the fundamentals of AWS. It provides an overview of the AWS Cloud services that are fundamentally important for the exam.

What Is Cloud Computing?

Cloud computing is the on-demand delivery of IT resources and applications via the Internet with pay-as-you-go pricing. Whether you run applications that share photos to millions of mobile users or deliver services that support the critical operations of your business, the cloud provides rapid access to flexible and low-cost IT resources. With cloud computing, you don't need to make large up-front investments in hardware and spend a lot of time managing that hardware. Instead, you can provision exactly the right type and size of computing resources you need to power your newest bright idea or operate your IT department. With cloud computing, you can access as many resources as you need, almost instantly, and only pay for what you use.

In its simplest form, cloud computing provides an easy way to access servers, storage, databases, and a broad set of application services over the Internet. Cloud computing providers such as AWS own and maintain the network-connected hardware required for these application services, while you provision and use what you need for your workloads.

Advantages of Cloud Computing

Cloud computing introduces a revolutionary shift in how technology is obtained, used, and managed, and in how organizations budget and pay for technology services. With the ability to reconfigure the computing environment quickly to adapt to changing business requirements, organizations can optimize spending. Capacity can be automatically scaled up or down to meet fluctuating usage patterns. Services can be temporarily taken offline or shut down permanently as business demands dictate. In addition, with pay-per-use billing, AWS Cloud services become an operational expense instead of a capital expense.

While each organization experiences a unique journey to the cloud with numerous benefits, six advantages become apparent time and time again, as illustrated in Figure 1.1.



FIGURE 1.1 Six advantages of cloud computing

Variable vs. Capital Expense

Let's begin with the ability to *trade capital expense for variable operational expense*. Instead of having

to invest heavily in data centers and servers before knowing how you're going to use them, you can pay only when you consume computing resources and pay only for how much you consume.

Economies of Scale

Another advantage of cloud computing is that *organizations benefit from massive economies of scale*. By using cloud computing, you can achieve a lower variable cost than you would get on your own. Because usage from hundreds of thousands of customers is aggregated in the cloud, providers such as AWS can achieve higher economies of scale, which translates into lower prices.

Stop Guessing Capacity

When you make a capacity decision prior to deploying an application, you often end up either sitting on expensive idle resources or dealing with limited capacity. With cloud computing, organizations can *stop guessing about capacity requirements* for the infrastructure necessary to meet their business needs. They can access as much or as little as they need and scale up or down as required with only a few minutes' notice.

Increase Speed and Agility

In a cloud computing environment, new IT resources are one click away, which allows organizations to reduce the time it takes to make those resources available to developers from weeks to just minutes. This results in a dramatic *increase in speed and agility* for the organization, because the cost and time it takes to experiment and develop is significantly lower.

Focus on Business Differentiators

Cloud computing allows organizations to focus on their business priorities, instead of on the heavy lifting of racking, stacking, and powering servers. By embracing this paradigm shift, organizations can *stop spending money on running and maintaining data centers*. This allows organizations to focus on projects that differentiate their businesses, such as analyzing petabytes of data, delivering video content, building great mobile applications, or even exploring Mars.

Go Global in Minutes

Another advantage of cloud computing is the ability to *go global in minutes*. Organizations can easily deploy their applications to multiple locations around the world with just a few clicks. This allows organizations to provide redundancy across the globe and to deliver lower latency and better experiences to their customers at minimal cost. Going global used to be something only the largest enterprises could afford to do, but cloud computing democratizes this ability, making it possible for any organization.

While specific questions on these advantages of cloud computing are unlikely to be on the exam, having exposure to these benefits can help rationalize the appropriate answers.

Cloud Computing Deployment Models

The two primary cloud computing deployment models that the exam focuses on are "all-in" cloud-based deployments and hybrid deployments. It is important to understand how each strategy applies to architectural options and decisions.

An *all-in cloud-based application* is fully deployed in the cloud, with all components of the application running in the cloud. Applications in the cloud have either been created in the cloud or have been

migrated from an existing infrastructure to take advantage of the benefits of cloud computing. Cloud-based applications can be built on low-level infrastructure pieces or can use higher-level services that provide abstraction from the management, architecting, and scaling requirements of core infrastructure.

A *hybrid deployment* is a common approach taken by many enterprises that connects infrastructure and applications between cloud-based resources and existing resources, typically in an existing data center. The most common method of hybrid deployment is between the cloud and existing on-premises infrastructure to extend and grow an organization's infrastructure while connecting cloud resources to internal systems. Choosing between an existing investment in infrastructure and moving to the cloud does not need to be a binary decision. Leveraging dedicated connectivity, identity federation, and integrated tools allows organizations to run hybrid applications across on-premises and cloud services.

AWS Fundamentals

At its core, AWS provides on-demand delivery of IT resources via the Internet on a secure cloud services platform, offering compute power, storage, databases, content delivery, and other functionality to help businesses scale and grow. Using AWS resources instead of your own is like purchasing electricity from a power company instead of running your own generator, and it provides the key advantages of cloud computing: Capacity exactly matches your need, you pay only for what you use, economies of scale result in lower costs, and the service is provided by a vendor experienced in running large-scale networks.

AWS global infrastructure and AWS approach to security and compliance are key foundational concepts to understand as you prepare for the exam.

Global Infrastructure

AWS serves over one million active customers in more than 190 countries, and it continues to expand its global infrastructure steadily to help organizations achieve lower latency and higher throughput for their business needs.

AWS provides a highly available technology infrastructure platform with multiple locations worldwide. These locations are composed of regions and Availability Zones. Each *region* is a separate geographic area. Each region has multiple, isolated locations known as *Availability Zones*. AWS enables the placement of resources and data in multiple locations. Resources aren't replicated across regions unless organizations choose to do so.

Each region is completely independent and is designed to be completely isolated from the other regions. This achieves the greatest possible fault tolerance and stability. Each Availability Zone is also isolated, but the Availability Zones in a region are connected through low-latency links. Availability Zones are physically separated within a typical metropolitan region and are located in lower-risk flood plains (specific flood zone categorization varies by region). In addition to using a discrete uninterruptable power supply (UPS) and on-site backup generators, they are each fed via different grids from independent utilities (when available) to reduce single points of failure further. Availability Zones are all redundantly connected to multiple tier-1 transit providers. By placing resources in separate Availability Zones, you can protect your website or application from a service disruption impacting a single location.



You can achieve high availability by deploying your application across multiple Availability Zones. Redundant instances for each tier (for example, web, application, and database) of an application should be placed in distinct Availability Zones, thereby creating a multisite solution. At a minimum, the goal is to have an independent copy of each application stack in two or more Availability Zones.

Security and Compliance

Whether on-premises or on AWS, information security is of paramount importance to organizations running critical workloads. Security is a core functional requirement that protects mission-critical information from accidental or deliberate theft, leakage, integrity compromise, and deletion. Helping to

protect the confidentiality, integrity, and availability of systems and data is of the utmost importance to AWS, as is maintaining your trust and confidence.

This section is intended to provide a very brief introduction to AWS approach to security and compliance. Chapter 12, "Security on AWS," and Chapter 13, "AWS Risk and Compliance," will address these topics in greater detail, including the importance of each on the exam.

Security

Cloud security at AWS is the number one priority. All AWS customers benefit from data center and network architectures built to satisfy the requirements of the most security-sensitive organizations. AWS and its partners offer hundreds of tools and features to help organizations meet their security objectives for visibility, auditability, controllability, and agility. This means that organizations can have the security they need, but without the capital outlay and with much lower operational overhead than in an on-premises environment.

Organizations leveraging AWS inherit all the best practices of AWS policies, architecture, and operational processes built to satisfy the requirements of the most security-sensitive customers. The AWS infrastructure has been designed to provide the highest availability while putting strong safeguards in place regarding customer privacy and segregation. When deploying systems on the AWS Cloud computing platform, AWS helps by sharing the security responsibilities with the organization. AWS manages the underlying infrastructure, and the organization can secure anything it deploys on AWS. This affords each organization the flexibility and agility they need in security controls.

This infrastructure is built and managed not only according to security best practices and standards, but also with the unique needs of the cloud in mind. AWS uses redundant and layered controls, continuous validation and testing, and a substantial amount of automation to ensure that the underlying infrastructure is monitored and protected 24/7. AWS ensures that these controls are consistently applied in every new data center or service.

Compliance

When customers move their production workloads to the AWS Cloud, both parties become responsible for managing the IT environment. Customers are responsible for setting up their environment in a secure and controlled manner. Customers also need to maintain adequate governance over their entire IT control environment. By tying together governance-focused, audit-friendly service features with applicable compliance or audit standards, AWS enables customers to build on traditional compliance programs. This helps organizations establish and operate in an AWS security control environment.



Organizations retain complete control and ownership over the region in which their data is physically located, allowing them to meet regional compliance and data residency requirements.

The IT infrastructure that AWS provides to organizations is designed and managed in alignment with security best practices and a variety of IT security standards. The following is a partial list of the many certifications and standards with which AWS complies:

- Service Organization Controls (SOC) 1/International Standard on Assurance Engagements (ISAE) 3402, SOC 2, and SOC 3

- Federal Information Security Management Act (FISMA), Department of Defense Information Assurance Certification and Accreditation Process (DIACAP), and Federal Risk and Authorization Management Program (FedRAMP)
- Payment Card Industry Data Security Standard (PCI DSS) Level 1
- International Organization for Standardization (ISO) 9001, ISO 27001, and ISO 27018

AWS provides a wide range of information regarding its IT control environment to help organizations achieve regulatory commitments in the form of reports, certifications, accreditations, and other third-party attestations.

AWS Cloud Computing Platform

AWS provides many cloud services that you can combine to meet business or organizational needs (see Figure 1.2). While being knowledgeable about all the platform services will allow you to be a well-rounded solutions architect, understanding the services and fundamental concepts outlined in this book will help prepare you for the AWS Certified Solutions Architect – Associate exam.

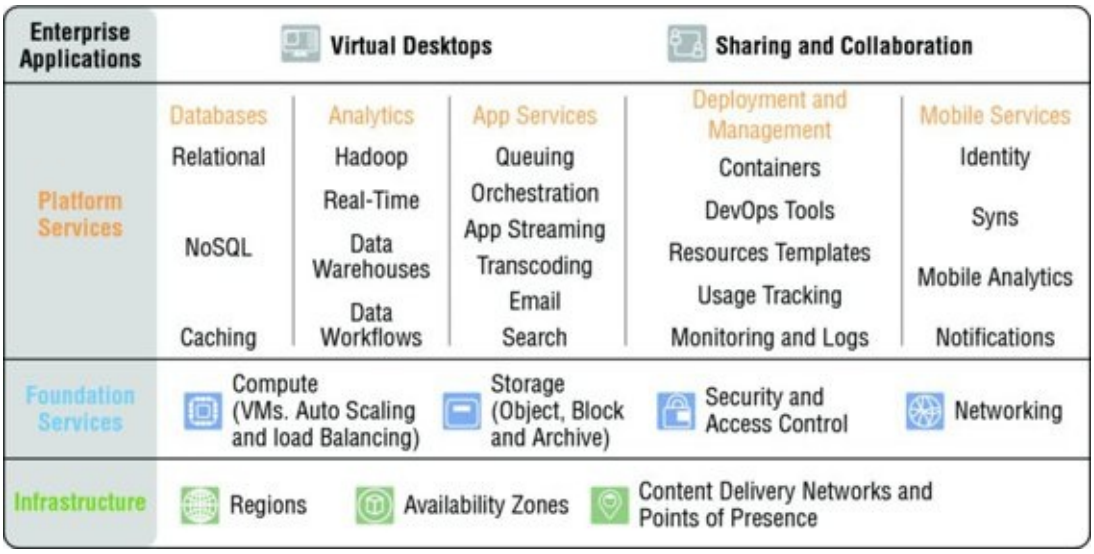


FIGURE 1.2 AWS Cloud computing platform

This section introduces the major AWS Cloud services by category. Subsequent chapters provide a deeper view of the services pertinent to the exam.

Accessing the Platform

To access AWS Cloud services, you can use the AWS Management Console, the AWS Command Line Interface (CLI), or the AWS Software Development Kits (SDKs).

The *AWS Management Console* is a web application for managing AWS Cloud services. The console provides an intuitive user interface for performing many tasks. Each service has its own console, which can be accessed from the AWS Management Console. The console also provides information about the account and billing.

The *AWS Command Line Interface (CLI)* is a unified tool used to manage AWS Cloud services. With just one tool to download and configure, you can control multiple services from the command line and automate them through scripts.

The *AWS Software Development Kits (SDKs)* provide an application programming interface (API) that interacts with the web services that fundamentally make up the AWS platform. The SDKs provide support for many different programming languages and platforms to allow you to work with your preferred language. While you can certainly make HTTP calls directly to the web service endpoints, using the SDKs can take the complexity out of coding by providing programmatic access for many of the services.

Compute and Networking Services

AWS provides a variety of compute and networking services to deliver core functionality for businesses to develop and run their workloads. These compute and networking services can be leveraged with the

storage, database, and application services to provide a complete solution for computing, query processing, and storage across a wide range of applications. This section offers a high-level description of the core computing and networking services.

Amazon Elastic Compute Cloud (Amazon EC2)

Amazon Elastic Compute Cloud (Amazon EC2) is a web service that provides resizable compute capacity in the cloud. It allows organizations to obtain and configure virtual servers in Amazon's data centers and to harness those resources to build and host software systems. Organizations can select from a variety of operating systems and resource configurations (memory, CPU, storage, and so on) that are optimal for the application profile of each workload. Amazon EC2 presents a true virtual computing environment, allowing organizations to launch compute resources with a variety of operating systems, load them with custom applications, and manage network access permissions while maintaining complete control.

AWS Lambda

AWS Lambda is a zero-administration compute platform for back-end web developers that runs your code for you on the AWS Cloud and provides you with a fine-grained pricing structure. AWS Lambda runs your back-end code on its own AWS compute fleet of Amazon EC2 instances across multiple Availability Zones in a region, which provides the high availability, security, performance, and scalability of the AWS infrastructure.

Auto Scaling

Auto Scaling allows organizations to scale Amazon EC2 capacity up or down automatically according to conditions defined for the particular workload (see Figure 1.3). Not only can it be used to help maintain application availability and ensure that the desired number of Amazon EC2 instances are running, but it also allows resources to scale in and out to match the demands of dynamic workloads. Instead of provisioning for peak load, organizations can optimize costs and use only the capacity that is actually needed.

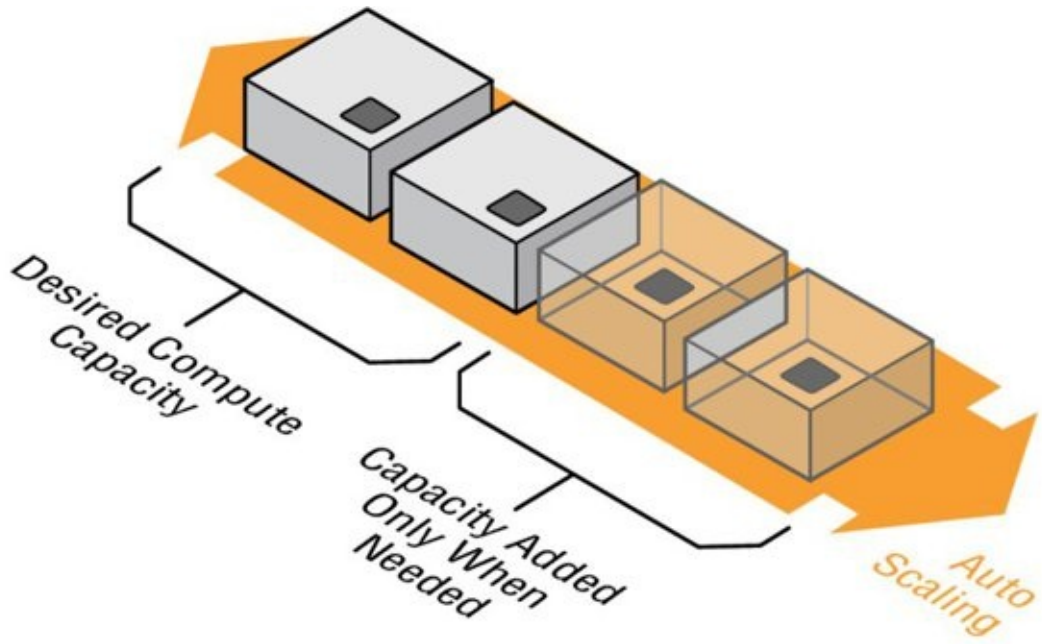


FIGURE 1.3 Auto scaling capacity

Auto Scaling is well suited both to applications that have stable demand patterns and to applications that experience hourly, daily, or weekly variability in usage.

Elastic Load Balancing

Elastic Load Balancing automatically distributes incoming application traffic across multiple Amazon EC2 instances in the cloud. It enables organizations to achieve greater levels of fault tolerance in their applications, seamlessly providing the required amount of load balancing capacity needed to distribute application traffic.

AWS Elastic Beanstalk

AWS Elastic Beanstalk is the fastest and simplest way to get a web application up and running on AWS. Developers can simply upload their application code, and the service automatically handles all the details, such as resource provisioning, load balancing, Auto Scaling, and monitoring. It provides support for a variety of platforms, including PHP, Java, Python, Ruby, Node.js, .NET, and Go. With AWS Elastic Beanstalk, organizations retain full control over the AWS resources powering the application and can access the underlying resources at any time.

Amazon Virtual Private Cloud (Amazon VPC)

Amazon Virtual Private Cloud (Amazon VPC) lets organizations provision a logically isolated section of the AWS Cloud where they can launch AWS resources in a virtual network that they define. Organizations have complete control over the virtual environment, including selection of the IP address range, creation of subnets, and configuration of route tables and network gateways. In addition, organizations can extend their corporate data center networks to AWS by using hardware or software *virtual private network (VPN)* connections or dedicated circuits by using AWS Direct Connect.

AWS Direct Connect

AWS Direct Connect allows organizations to establish a dedicated network connection from their data center to AWS. Using AWS Direct Connect, organizations can establish private connectivity between AWS and their data center, office, or colocation environment, which in many cases can reduce network costs, increase bandwidth throughput, and provide a more consistent network experience than Internet-based VPN connections.

Amazon Route 53

Amazon Route 53 is a highly available and scalable Domain Name System (DNS) web service. It is designed to give developers and businesses an extremely reliable and cost-effective way to route end users to Internet applications by translating human readable names, such as `www.example.com`, into the numeric IP addresses, such as 192.0.2.1, that computers use to connect to each other. Amazon Route 53 also serves as domain registrar, allowing you to purchase and manage domains directly from AWS.

Storage and Content Delivery

AWS provides a variety of services to meet your storage needs, such as Amazon Simple Storage Service, Amazon CloudFront, and Amazon Elastic Block Store. This section provides an overview of the storage and content delivery services.

Amazon Simple Storage Service (Amazon S3)

Amazon Simple Storage Service (Amazon S3) provides developers and IT teams with highly durable and scalable object storage that handles virtually unlimited amounts of data and large numbers of concurrent users. Organizations can store any number of objects of any type, such as HTML pages, source code files, image files, and encrypted data, and access them using HTTP-based protocols. Amazon S3 provides cost-effective object storage for a wide variety of use cases, including backup and recovery, nearline archive, big data analytics, disaster recovery, cloud applications, and content distribution.

Amazon Glacier

Amazon Glacier is a secure, durable, and extremely low-cost storage service for data archiving and long-term backup. Organizations can reliably store large or small amounts of data for a very low cost per gigabyte per month. To keep costs low for customers, Amazon Glacier is optimized for infrequently accessed data where a retrieval time of several hours is suitable. Amazon S3 integrates closely with Amazon Glacier to allow organizations to choose the right storage tier for their workloads.

Amazon Elastic Block Store (Amazon EBS)

Amazon Elastic Block Store (Amazon EBS) provides persistent block-level storage volumes for use with Amazon EC2 instances. Each Amazon EBS volume is automatically replicated within its Availability Zone to protect organizations from component failure, offering high availability and durability. By delivering consistent and low-latency performance, Amazon EBS provides the disk storage needed to run a wide variety of workloads.

AWS Storage Gateway

AWS Storage Gateway is a service connecting an on-premises software appliance with cloud-based storage to provide seamless and secure integration between an organization's on-premises IT environment and the AWS storage infrastructure. The service supports industry-standard storage protocols that work with existing applications. It provides low-latency performance by maintaining a cache of frequently accessed data on-premises while securely storing all of your data encrypted in Amazon S3 or Amazon Glacier.

Amazon CloudFront

Amazon CloudFront is a content delivery web service. It integrates with other AWS Cloud services to give developers and businesses an easy way to distribute content to users across the world with low latency, high data transfer speeds, and no minimum usage commitments. Amazon CloudFront can be used to deliver your entire website, including dynamic, static, streaming, and interactive content, using a global network of edge locations. Requests for content are automatically routed to the nearest edge location, so content is delivered with the best possible performance to end users around the globe.

Database Services

AWS provides fully managed relational and NoSQL database services, and in-memory caching as a service and a petabyte-scale data warehouse solution. This section provides an overview of the products that the database services comprise.

Amazon Relational Database Service (Amazon RDS)

Amazon Relational Database Service (Amazon RDS) provides a fully managed relational database with

support for many popular open source and commercial database engines. It's a cost-efficient service that allows organizations to launch secure, highly available, fault-tolerant, production-ready databases in minutes. Because Amazon RDS manages time-consuming administration tasks, including backups, software patching, monitoring, scaling, and replication, organizational resources can focus on revenue-generating applications and business instead of mundane operational tasks.

Amazon DynamoDB

Amazon DynamoDB is a fast and flexible NoSQL database service for all applications that need consistent, single-digit millisecond latency at any scale. It is a fully managed database and supports both document and key/value data models. Its flexible data model and reliable performance make it a great fit for mobile, web, gaming, ad-tech, Internet of Things, and many other applications.

Amazon Redshift

Amazon Redshift is a fast, fully managed, petabyte-scale data warehouse service that makes it simple and cost effective to analyze structured data. Amazon Redshift provides a standard SQL interface that lets organizations use existing business intelligence tools. By leveraging columnar storage technology that improves I/O efficiency and parallelizing queries across multiple nodes, Amazon Redshift is able to deliver fast query performance. The Amazon Redshift architecture allows organizations to automate most of the common administrative tasks associated with provisioning, configuring, and monitoring a cloud data warehouse.

Amazon ElastiCache

Amazon ElastiCache is a web service that simplifies deployment, operation, and scaling of an in-memory cache in the cloud. The service improves the performance of web applications by allowing organizations to retrieve information from fast, managed, in-memory caches, instead of relying entirely on slower, disk-based databases. As of this writing, Amazon ElastiCache supports Memcached and Redis cache engines.

Management Tools

AWS provides a variety of tools that help organizations manage your AWS resources. This section provides an overview of the management tools that AWS provides to organizations.

Amazon CloudWatch

Amazon CloudWatch is a monitoring service for AWS Cloud resources and the applications running on AWS. It allows organizations to collect and track metrics, collect and monitor log files, and set alarms. By leveraging Amazon CloudWatch, organizations can gain system-wide visibility into resource utilization, application performance, and operational health. By using these insights, organizations can react, as necessary, to keep applications running smoothly.

AWS CloudFormation

AWS CloudFormation gives developers and systems administrators an effective way to create and manage a collection of related AWS resources, provisioning and updating them in an orderly and predictable fashion. AWS CloudFormation defines a JSON-based templating language that can be used to describe all the AWS resources that are necessary for a workload. Templates can be submitted to AWS CloudFormation and the service will take care of provisioning and configuring those resources in

appropriate order (see Figure 1.4).

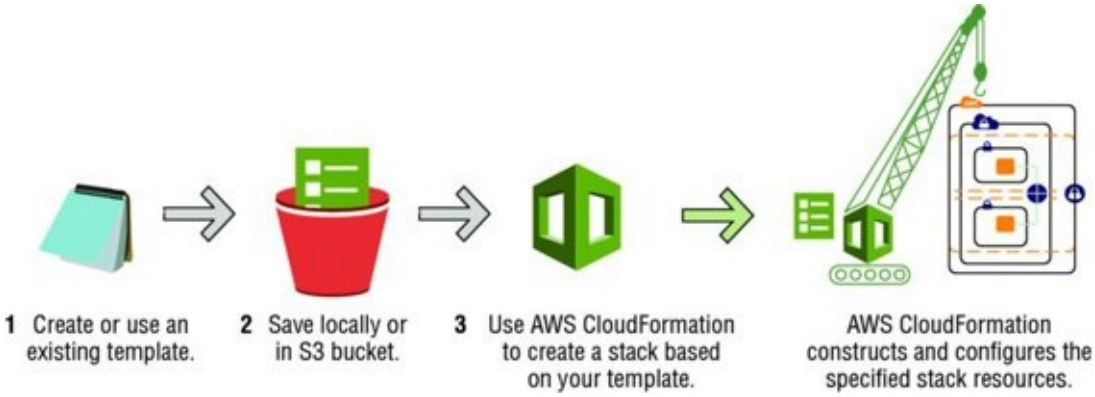


FIGURE 1.4 AWS CloudFormation workflow summary

AWS CloudTrail

AWS CloudTrail is a web service that records AWS API calls for an account and delivers log files for audit and review. The recorded information includes the identity of the API caller, the time of the API call, the source IP address of the API caller, the request parameters, and the response elements returned by the service.

AWS Config

AWS Config is a fully managed service that provides organizations with an AWS resource inventory, configuration history, and configuration change notifications to enable security and governance. With AWS Config, organizations can discover existing AWS resources, export an inventory of their AWS resources with all configuration details, and determine how a resource was configured at any point in time. These capabilities enable compliance auditing, security analysis, resource change tracking, and troubleshooting.

Security and Identity

AWS provides security and identity services that help organizations secure their data and systems on the cloud. The following section explores these services at a high level.

AWS Identity and Access Management (IAM)

AWS Identity and Access Management (IAM) enables organizations to securely control access to AWS Cloud services and resources for their users. Using IAM, organizations can create and manage AWS users and groups and use permissions to allow and deny their access to AWS resources.

AWS Key Management Service (KMS)

AWS Key Management Service (KMS) is a managed service that makes it easy for organizations to create and control the encryption keys used to encrypt their data and uses Hardware Security Modules (HSMs) to protect the security of your keys. AWS KMS is integrated with several other AWS Cloud services to help protect data stored with these services.

AWS Directory Service

AWS Directory Service allows organizations to set up and run Microsoft Active Directory on the AWS Cloud or connect their AWS resources with an existing on-premises Microsoft Active Directory.

Organizations can use it to manage users and groups, provide single sign-on to applications and services, create and apply Group Policies, domain join Amazon EC2 instances, and simplify the deployment and management of cloud-based Linux and Microsoft Windows workloads.

AWS Certificate Manager

AWS Certificate Manager is a service that lets organizations easily provision, manage, and deploy Secure Sockets Layer/Transport Layer Security (SSL/TLS) certificates for use with AWS Cloud services. It removes the time-consuming manual process of purchasing, uploading, and renewing SSL/TLS certificates. With AWS Certificate Manager, organizations can quickly request a certificate, deploy it on AWS resources such as Elastic Load Balancing or Amazon CloudFront distributions, and let AWS Certificate Manager handle certificate renewals.

AWS Web Application Firewall (WAF)

AWS Web Application Firewall (WAF) helps protect web applications from common attacks and exploits that could affect application availability, compromise security, or consume excessive resources. AWS WAF gives organizations control over which traffic to allow or block to their web applications by defining customizable web security rules.

Application Services

AWS provides a variety of managed services to use with applications. The following section explores the application services at a high level.

Amazon API Gateway

Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. Organizations can create an API that acts as a "front door" for applications to access data, business logic, or functionality from back-end services, such as workloads running on Amazon EC2, code running on AWS Lambda, or any web application. Amazon API Gateway handles all the tasks involved in accepting and processing up to hundreds of thousands of concurrent API calls, including traffic management, authorization and access control, monitoring, and API version management.

Amazon Elastic Transcoder

Amazon Elastic Transcoder is media transcoding in the cloud. It is designed to be a highly scalable and cost-effective way for developers and businesses to convert (or transcode) media files from their source formats into versions that will play back on devices like smartphones, tablets, and PCs.

Amazon Simple Notification Service (Amazon SNS)

Amazon Simple Notification Service (Amazon SNS) is a web service that coordinates and manages the delivery or sending of messages to recipients. In Amazon SNS, there are two types of clients—publishers and subscribers—also referred to as producers and consumers. Publishers communicate asynchronously with subscribers by producing and sending a message to a topic, which is a logical access point and communication channel. Subscribers consume or receive the message or notification over one of the supported protocols when they are subscribed to the topic.

Amazon Simple Email Service (Amazon SES)

Amazon Simple Email Service (Amazon SES) is a cost-effective email service that organizations can use to send transactional email, marketing messages, or any other type of content to their customers. Amazon SES can also be used to receive messages and deliver them to an Amazon S3 bucket, call custom code via an AWS Lambda function, or publish notifications to Amazon SNS.

Amazon Simple Workflow Service (Amazon SWF)

Amazon Simple Workflow Service (Amazon SWF) helps developers build, run, and scale background jobs that have parallel or sequential steps. Amazon SWF can be thought of as a fully managed state tracker and task coordinator on the cloud. In common architectural patterns, if your application's steps take more than 500 milliseconds to complete, it is vitally important to track the state of processing and to provide the ability to recover or retry if a task fails. Amazon SWF helps organizations achieve this reliability.

Amazon Simple Queue Service (Amazon SQS)

Amazon Simple Queue Service (Amazon SQS) is a fast, reliable, scalable, fully managed message queuing service. Amazon SQS makes it simple and cost effective to decouple the components of a cloud application. With Amazon SQS, organizations can transmit any volume of data, at any level of throughput, without losing messages or requiring other services to be always available.

Summary

The term “cloud computing” refers to the on-demand delivery of IT resources via the Internet with pay-as-you-go pricing. Instead of buying, owning, and maintaining data centers and servers, organizations can acquire technology such as compute power, storage, databases, and other services on an as-needed basis. With cloud computing, AWS manages and maintains the technology infrastructure in a secure environment and businesses access these resources via the Internet to develop and run their applications. Capacity can grow or shrink instantly and businesses pay only for what they use.

Cloud computing introduces a revolutionary shift in how technology is obtained, used, and managed, and how organizations budget and pay for technology services. While each organization experiences a unique journey to the cloud with numerous benefits, six advantages become apparent time and time again. Understanding these advantages allows architects to shape solutions that deliver continuous benefits to organizations.

AWS provides a highly available technology infrastructure platform with multiple locations worldwide. These locations are composed of regions and Availability Zones. This enables organizations to place resources and data in multiple locations around the globe. Helping to protect the confidentiality, integrity, and availability of systems and data is of the utmost importance to AWS, as is maintaining the trust and confidence of organizations around the world.

AWS offers a broad set of global compute, storage, database, analytics, application, and deployment services that help organizations move faster, lower IT costs, and scale applications. Having a broad understanding of these services allows solutions architects to design effective distributed applications and systems on the AWS platform.

Exam Essentials

Understand the global infrastructure. AWS provides a highly available technology infrastructure platform with multiple locations worldwide. These locations are composed of regions and Availability Zones. Each region is located in a separate geographic area and has multiple, isolated locations known as Availability Zones.

Understand regions. An AWS region is a physical geographic location that consists of a cluster of data centers. AWS regions enable the placement of resources and data in multiple locations around the globe. Each region is completely independent and is designed to be completely isolated from the other regions. This achieves the greatest possible fault tolerance and stability. Resources aren't replicated across regions unless organizations choose to do so.

Understand Availability Zones. An Availability Zone is one or more data centers within a region that are designed to be isolated from failures in other Availability Zones. Availability Zones provide inexpensive, low-latency network connectivity to other zones in the same region. By placing resources in separate Availability Zones, organizations can protect their website or application from a service disruption impacting a single location.

Understand the hybrid deployment model. A hybrid deployment model is an architectural pattern providing connectivity for infrastructure and applications between cloud-based resources and existing resources that are not located in the cloud.

Review Questions

1. Which of the following describes a physical location around the world where AWS clusters data centers?
   A. Endpoint
   B. Collection
   C. Fleet
   D. Region
2. Each AWS region is composed of two or more locations that offer organizations the ability to operate production systems that are more highly available, fault tolerant, and scalable than would be possible using a single data center. What are these locations called?
   A. Availability Zones
   B. Replication areas
   C. Geographic districts
   D. Compute centers
3. What is the deployment term for an environment that extends an existing on-premises infrastructure into the cloud to connect cloud resources to internal systems?
   A. All-in deployment
   B. Hybrid deployment
   C. On-premises deployment
   D. Scatter deployment
4. Which AWS Cloud service allows organizations to gain system-wide visibility into resource utilization, application performance, and operational health?
   A. AWS Identity and Access Management (IAM)
   B. Amazon Simple Notification Service (Amazon SNS)
   C. Amazon CloudWatch
   D. AWS CloudFormation
5. Which of the following AWS Cloud services is a fully managed NoSQL database service?
   A. Amazon Simple Queue Service (Amazon SQS)
   B. Amazon DynamoDB
   C. Amazon ElastiCache
   D. Amazon Relational Database Service (Amazon RDS)
6. Your company experiences fluctuations in traffic patterns to their e-commerce website based on flash sales. What service can help your company dynamically match the required compute capacity to the

spike in traffic during flash sales?

A. Auto Scaling

B. Amazon Glacier

C. Amazon Simple Notification Service (Amazon SNS)

D. Amazon Virtual Private Cloud (Amazon VPC)

7. Your company provides an online photo sharing service. The development team is looking for ways to deliver image files with the lowest latency to end users so the website content is delivered with the best possible performance. What service can help speed up distribution of these image files to end users around the world?

A. Amazon Elastic Compute Cloud (Amazon EC2)

B. Amazon Route 53

C. AWS Storage Gateway

D. Amazon CloudFront

8. Your company runs an Amazon Elastic Compute Cloud (Amazon EC2) instance periodically to perform a batch processing job on a large and growing filesystem. At the end of the batch job, you shut down the Amazon EC2 instance to save money but need to persist the filesystem on the Amazon EC2 instance from the previous batch runs. What AWS Cloud service can you leverage to meet these requirements?

A. Amazon Elastic Block Store (Amazon EBS)

B. Amazon DynamoDB

C. Amazon Glacier

D. AWS CloudFormation

9. What AWS Cloud service provides a logically isolated section of the AWS Cloud where organizations can launch AWS resources in a virtual network that they define?

A. Amazon Simple Workflow Service (Amazon SWF)

B. Amazon Route 53

C. Amazon Virtual Private Cloud (Amazon VPC)

D. AWS CloudFormation

10. Your company provides a mobile voting application for a popular TV show, and 5 to 25 million viewers all vote in a 15-second timespan. What mechanism can you use to decouple the voting application from your back-end services that tally the votes?

A. AWS CloudTrail

B. Amazon Simple Queue Service (Amazon SQS)

C. Amazon Redshift

D. Amazon Simple Notification Service (Amazon SNS)

Chapter 2
Amazon Simple Storage Service (Amazon S3) and Amazon Glacier Storage

THE AWS CERTIFIED SOLUTIONS ARCHITECT ASSOCIATE EXAM OBJECTIVES COVERED IN THIS CHAPTER MAY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

Domain 1.0: Designing highly available, cost-efficient, fault-tolerant, scalable systems

✓ **1.1 Identify and recognize cloud architecture considerations, such as fundamental components and effective designs.**

Content may include the following:

- How to design cloud services
- Planning and design
- Monitoring and logging
- Familiarity with:
  - Best practices for AWS architecture
  - Developing to client specifications, including pricing/cost (e.g., On Demand vs. Reserved vs. Spot; Recovery Time Objective [RTO] and Recovery Point Objective [RPO] disaster recovery design)
  - Architectural trade-off decisions (e.g., high availability vs. cost)
  - Hybrid IT architectures
  - Elasticity and scalability

Domain 2.0: Implementation/Deployment

✓ **2.1 Identify the appropriate techniques and methods using Amazon Simple Storage Service (Amazon S3) to code and implement a cloud solution.**

Content may include the following:

- Configure services to support compliance requirements in the cloud.
- Launch instances across the AWS global infrastructure.
- Configure AWS Identity and Access Management (IAM) policies and best practices.

Domain 3.0: Data Security

✓ **3.1 Recognize and implement secure practices for optimum cloud deployment and**

maintenance

Content may include the following:

- Security Architecture with AWS
  - “Core” Amazon S3 security feature sets
  - Encryption solutions (e.g., key services)
  - Complex access controls (building sophisticated security groups, Access Control Lists [ACLs], etc.)



Introduction

This chapter is intended to provide you with a basic understanding of the core object storage services available on AWS: Amazon Simple Storage Service (Amazon S3) and Amazon Glacier.

Amazon S3 provides developers and IT teams with secure, durable, and highly-scalable cloud storage. Amazon S3 is easy-to-use *object storage* with a simple web service interface that you can use to store and retrieve any amount of data from anywhere on the web. Amazon S3 also allows you to pay only for the storage you actually use, which eliminates the capacity planning and capacity constraints associated with traditional storage.

Amazon S3 is one of first services introduced by AWS, and it serves as one of the foundational web services—nearly any application running in AWS uses Amazon S3, either directly or indirectly. Amazon S3 can be used alone or in conjunction with other AWS services, and it offers a very high level of integration with many other AWS cloud services. For example, Amazon S3 serves as the durable target storage for Amazon Kinesis and Amazon Elastic MapReduce (Amazon EMR), it is used as the storage for Amazon Elastic Block Store (Amazon EBS) and Amazon Relational Database Service (Amazon RDS) snapshots, and it is used as a data staging or loading storage mechanism for Amazon Redshift and Amazon DynamoDB, among many other functions. Because Amazon S3 is so flexible, so highly integrated, and so commonly used, it is important to understand this service in detail.

Common use cases for Amazon S3 storage include:

- Backup and archive for on-premises or cloud data
- Content, media, and software storage and distribution
- Big data analytics
- Static website hosting
- Cloud-native mobile and Internet application hosting
- Disaster recovery

To support these use cases and many more, Amazon S3 offers a range of *storage classes* designed for various generic use cases: general purpose, infrequent access, and archive. To help manage data through its lifecycle, Amazon S3 offers configurable lifecycle policies. By using lifecycle policies, you can have your data automatically migrate to the most appropriate storage class, without modifying your application code. In order to control who has access to your data, Amazon S3 provides a rich set of permissions, access controls, and encryption options.

Amazon Glacier is another cloud storage service related to Amazon S3, but optimized for data archiving and long-term backup at extremely low cost. Amazon Glacier is suitable for “cold data,” which is data that is rarely accessed and for which a retrieval time of three to five hours is acceptable. Amazon Glacier can be used both as a storage class of Amazon S3 (see Storage Classes and Object Lifecycle Management topics in the Amazon S3 Advanced Features section), and as an independent archival storage service (see the Amazon Glacier section).

Object Storage versus Traditional Block and File Storage

In traditional IT environments, two kinds of storage dominate: *block storage* and *file storage*. Block storage operates at a lower level—the raw storage device level—and manages data as a set of numbered, fixed-size blocks. File storage operates at a higher level—the operating system level—and manages data as a named hierarchy of files and folders. Block and file storage are often accessed over a network in the form of a Storage Area Network (SAN) for block storage, using protocols such as iSCSI or Fibre Channel, or as a Network Attached Storage (NAS) file server or "filer" for file storage, using protocols such as Common Internet File System (CIFS) or Network File System (NFS). Whether directly-attached or network-attached, block or file, this kind of storage is very closely associated with the server and the operating system that is using the storage.

Amazon S3 object storage is something quite different. Amazon S3 is cloud *object storage*. Instead of being closely associated with a server, Amazon S3 storage is independent of a server and is accessed over the Internet. Instead of managing data as blocks or files using SCSI, CIFS, or NFS protocols, data is managed as objects using an Application Program Interface (API) built on standard HTTP verbs.

Each Amazon S3 object contains both data and metadata. Objects reside in containers called *buckets*, and each object is identified by a unique user-specified key (filename). Buckets are a simple flat folder with no file system hierarchy. That is, you can have multiple buckets, but you can't have a sub-bucket within a bucket. Each bucket can hold an unlimited number of objects.

It is easy to think of an Amazon S3 object (or the data portion of an object) as a file, and the key as the filename. However, keep in mind that Amazon S3 is not a traditional file system and differs in significant ways. In Amazon S3, you `GET` an object or `PUT` an object, operating on the whole object at once, instead of incrementally updating portions of the object as you would with a file. You can't "mount" a bucket, "open" an object, install an operating system on Amazon S3, or run a database on it.

Instead of a file system, Amazon S3 is highly-durable and highly-scalable object storage that is optimized for reads and is built with an intentionally minimalistic feature set. It provides a simple and robust abstraction for file storage that frees you from many underlying details that you normally do have to deal with in traditional storage. For example, with Amazon S3 you don't have to worry about device or file system storage limits and capacity planning—a single bucket can store an unlimited number of files. You also don't need to worry about data durability or replication across availability zones—Amazon S3 objects are automatically replicated on multiple devices in multiple facilities within a region. The same with scalability—if your request rate grows steadily, Amazon S3 automatically partitions buckets to support very high request rates and simultaneous access by many clients.



If you need traditional block or file storage in addition to Amazon S3 storage, AWS provides options. The Amazon EBS service provides block level storage for Amazon Elastic Compute Cloud (Amazon EC2) instances. Amazon Elastic File System (AWS EFS) provides network-attached shared file storage (NAS storage) using the NFS v4 protocol.

Amazon Simple Storage Service (Amazon S3) Basics

Now that you have an understanding of some of the key differences between traditional block and file storage versus cloud object storage, we can explore the basics of Amazon S3 in more detail.

Buckets

A *bucket* is a container (web folder) for objects (files) stored in Amazon S3. Every Amazon S3 object is contained in a bucket. Buckets form the top-level namespace for Amazon S3, and bucket names are global. This means that your bucket names must be unique across all AWS accounts, much like Domain Name System (DNS) domain names, not just within your own account. Bucket names can contain up to 63 lowercase letters, numbers, hyphens, and periods. You can create and use multiple buckets; you can have up to 100 per account by default.

 It is a best practice to use bucket names that contain your domain name and conform to the rules for DNS names. This ensures that your bucket names are your own, can be used in all regions, and can host static websites.

AWS Regions

Even though the namespace for Amazon S3 buckets is global, each Amazon S3 bucket is created in a specific region that you choose. This lets you control where your data is stored. You can create and use buckets that are located close to a particular set of end users or customers in order to minimize latency, or located in a particular region to satisfy data locality and sovereignty concerns, or located far away from your primary facilities in order to satisfy disaster recovery and compliance needs. You control the location of your data; data in an Amazon S3 bucket is stored in that region unless you explicitly copy it to another bucket located in a different region.

Objects

Objects are the entities or files stored in Amazon S3 buckets. An object can store virtually any kind of data in any format. Objects can range in size from 0 bytes up to 5TB, and a single bucket can store an unlimited number of objects. This means that Amazon S3 can store a virtually unlimited amount of data.

Each object consists of data (the file itself) and *metadata* (data about the file). The data portion of an Amazon S3 object is opaque to Amazon S3. This means that an object's data is treated as simply a stream of bytes—Amazon S3 doesn't know or care what type of data you are storing, and the service doesn't act differently for text data versus binary data.

The metadata associated with an Amazon S3 object is a set of name/value pairs that describe the object. There are two types of metadata: system metadata and user metadata. System metadata is created and used by Amazon S3 itself, and it includes things like the date last modified, object size, MD5 digest, and HTTP Content-Type. User metadata is optional, and it can only be specified at the time an object is created. You can use custom metadata to tag your data with attributes that are meaningful to you.

Keys

Every object stored in an S3 bucket is identified by a unique identifier called a *key*. You can think of the key as a filename. A key can be up to 1024 bytes of Unicode UTF-8 characters, including embedded slashes, backslashes, dots, and dashes.

Keys must be unique within a single bucket, but different buckets can contain objects with the same key. The combination of bucket, key, and optional version ID uniquely identifies an Amazon S3 object.

Object URL

Amazon S3 is storage for the Internet, and every Amazon S3 object can be addressed by a unique URL formed using the web services endpoint, the bucket name, and the object key. For example, with the URL:

`http://mybucket.s3.amazonaws.com/jack.doc`

`mybucket` is the S3 bucket name, and `jack.doc` is the key or filename. If another object is created, for instance:

`http://mybucket.s3.amazonaws.com/fee/fi/fo/fum/jack.doc`

then the bucket name is still `mybucket`, but now the key or filename is the string `fee/fi/fo/fum/jack.doc`. A key may contain delimiter characters like slashes or backslashes to help you name and logically organize your Amazon S3 objects, but to Amazon S3 it is simply a long key name in a flat namespace. There is no actual file and folder hierarchy. See the topic “Prefixes and Delimiters” in the “Amazon S3 Advanced Features” section that follows for more information.

 For convenience, the Amazon S3 console and the Prefix and Delimiter feature allow you to navigate within an Amazon S3 bucket as if there were a folder hierarchy. However, remember that a bucket is a single flat namespace of keys with no structure.

Amazon S3 Operations

The Amazon S3 API is intentionally simple, with only a handful of common operations. They include:

- Create/delete a bucket
- Write an object
- Read an object
- Delete an object
- List keys in a bucket

REST Interface

The native interface for Amazon S3 is a *REST (Representational State Transfer)* API. With the REST interface, you use standard HTTP or HTTPS requests to create and delete buckets, list keys, and read and write objects. REST maps standard HTTP “verbs” (HTTP methods) to the familiar CRUD (Create, Read, Update, Delete) operations. Create is `HTTP PUT` (and sometimes `POST`); read is `HTTP GET`; delete is `HTTP DELETE`; and update is `HTTP POST` (or sometimes `PUT`).

 Always use HTTPS for Amazon S3 API requests to ensure that your requests and data are secure.

In most cases, users do not use the REST interface directly, but instead interact with Amazon S3 using one of the higher-level interfaces available. These include the AWS Software Development Kits (SDKs) (wrapper libraries) for iOS, Android, JavaScript, Java, .NET, Node.js, PHP, Python, Ruby, Go, and C++, the AWS Command Line Interface (CLI), and the AWS Management Console.

 Amazon S3 originally supported a SOAP (Simple Object Access Protocol) API in addition to the REST API, but you should use the REST API. The legacy HTTPS endpoint is still available, but new features are not supported.

Durability and Availability

Data *durability* and *availability* are related but slightly different concepts. Durability addresses the question, “Will my data still be there in the future?” Availability addresses the question, “Can I access my data right now?” Amazon S3 is designed to provide both very high durability and very high availability for your data.

Amazon S3 standard storage is designed for 99.999999999% durability and 99.99% availability of objects over a given year. For example, if you store 10,000 objects with Amazon S3, you can on average expect to incur a loss of a single object once every 10,000,000 years. Amazon S3 achieves high durability by automatically storing data redundantly on multiple devices in multiple facilities within a region. It is designed to sustain the concurrent loss of data in two facilities without loss of user data. Amazon S3 provides a highly durable storage infrastructure designed for mission-critical and primary data storage.

If you need to store non-critical or easily reproducible derived data (such as image thumbnails) that doesn’t require this high level of durability, you can choose to use Reduced Redundancy Storage (RRS) at a lower cost. RRS offers 99.99% durability with a lower cost of storage than traditional Amazon S3 storage.

 Even though Amazon S3 storage offers very high durability at the infrastructure level, it is still a best practice to protect against user-level accidental deletion or overwriting of data by using additional features such as versioning, cross-region replication, and MFA Delete.

Data Consistency

Amazon S3 is an *eventually consistent* system. Because your data is automatically replicated across multiple servers and locations within a region, changes in your data may take some time to propagate to all locations. As a result, there are some situations where information that you read immediately after an update may return stale data.

For PUTs to new objects, this is not a concern—in this case, Amazon S3 provides read-after-write consistency. However, for PUTs to existing objects (object overwrite to an existing key) and for object DELETEs, Amazon S3 provides *eventual consistency*.

Eventual consistency means that if you PUT new data to an existing key, a subsequent GET might return the old data. Similarly, if you DELETE an object, a subsequent GET for that object might still read the deleted object. In all cases, updates to a single key are atomic—for eventually-consistent reads, you will get the new data or the old data, but never an inconsistent mix of data.

Access Control

Amazon S3 is secure by default; when you create a bucket or object in Amazon S3, only you have access. To allow you to give controlled access to others, Amazon S3 provides both coarse-grained access controls (Amazon S3 Access Control Lists [ACLs]), and fine-grained access controls (Amazon S3 bucket policies, AWS Identity and Access Management [IAM] policies, and query-string authentication).

Amazon S3 ACLs allow you to grant certain coarse-grained permissions: READ, WRITE, or FULL-CONTROL at the object or bucket level. ACLs are a legacy access control mechanism, created before IAM existed. ACLs are best used today for a limited set of use cases, such as enabling bucket logging or making a bucket that hosts a static website be world-readable.

Amazon S3 bucket policies are the recommended access control mechanism for Amazon S3 and provide much finer-grained control. Amazon S3 bucket policies are very similar to IAM policies, which were discussed in Chapter 6, “AWS Identity and Access Management (IAM),” but are subtly different in that:

- They are associated with the bucket resource instead of an IAM principal.
- They include an explicit reference to the IAM principal in the policy. This principal can be associated with a different AWS account, so Amazon S3 bucket policies allow you to assign cross-account access to Amazon S3 resources.

Using an Amazon S3 bucket policy, you can specify who can access the bucket, from where (by Classless Inter-Domain Routing [CIDR] block or IP address), and during what time of day.

Finally, IAM policies may be associated directly with IAM principals that grant access to an Amazon S3 bucket, just as it can grant access to any AWS service and resource. Obviously, you can only assign IAM policies to principals in AWS accounts that you control.

Static Website Hosting

A very common use case for Amazon S3 storage is *static website* hosting. Many websites, particularly micro-sites, don’t need the services of a full web server. A static website means that all of the pages of the website contain only static content and do not require server-side processing such as PHP, ASP.NET, or JSP. (Note that this does not mean that the website cannot be interactive and dynamic; this can be accomplished with client-side scripts, such as JavaScript embedded in static HTML webpages.) Static websites have many advantages: they are very fast, very scalable, and can be more secure than a typical dynamic website. If you host a static website on Amazon S3, you can also leverage the security, durability, availability, and scalability of Amazon S3.

Because every Amazon S3 object has a URL, it is relatively straightforward to turn a bucket into a website. To host a static website, you simply configure a bucket for website hosting and then upload the content of the static website to the bucket.

To configure an Amazon S3 bucket for static website hosting:

1. Create a bucket with the same name as the desired website hostname.
2. Upload the static files to the bucket.
3. Make all the files public (world readable).
4. Enable static website hosting for the bucket. This includes specifying an Index document and an Error document.
5. The website will now be available at the S3 website URL:

   `<bucket-name>.s3-website-<AWS-region>.amazonaws.com`.
6. Create a friendly DNS name in your own domain for the website using a DNS CNAME, or an Amazon Route 53 alias that resolves to the Amazon S3 website URL.
7. The website will now be available at your website domain name.

Amazon S3 Advanced Features

Beyond the basics, there are some advanced features of Amazon S3 that you should also be familiar with.

Prefixes and Delimiters

While Amazon S3 uses a flat structure in a bucket, it supports the use of *prefix* and *delimiter* parameters when listing key names. This feature lets you organize, browse, and retrieve the objects within a bucket hierarchically. Typically, you would use a slash (`/`) or backslash (`\`) as a delimiter and then use key names with embedded delimiters to emulate a file and folder hierarchy within the flat object key namespace of a bucket.

For example, you might want to store a series of server logs by server name (such as server42), but organized by year and month, like so:

```
logs/2016/January/server42.log
logs/2016/February/server42.log
logs/2016/March/server42.log
```

The REST API, wrapper SDKs, AWS CLI, and the Amazon Management Console all support the use of delimiters and prefixes. This feature lets you logically organize new data and easily maintain the hierarchical folder-and-file structure of existing data uploaded or backed up from traditional file systems. Used together with IAM or Amazon S3 bucket policies, prefixes and delimiters also allow you to create the equivalent of departmental “subdirectories” or user “home directories” within a single bucket, restricting or sharing access to these “subdirectories” (defined by prefixes) as needed.

 Use delimiters and object prefixes to hierarchically organize the objects in your Amazon S3 buckets, but always remember that Amazon S3 is not really a file system.

Storage Classes

Amazon S3 offers a range of *storage classes* suitable for various use cases.

Amazon S3 Standard offers high durability, high availability, low latency, and high performance object storage for general purpose use. Because it delivers low first-byte latency and high throughput, Standard is well-suited for short-term or long-term storage of frequently accessed data. For most general purpose use cases, Amazon S3 Standard is the place to start.

Amazon S3 Standard – Infrequent Access (Standard-IA) offers the same durability, low latency, and high throughput as Amazon S3 Standard, but is designed for long-lived, less frequently accessed data. Standard-IA has a lower per GB-month storage cost than Standard, but the price model also includes a minimum object size (128KB), minimum duration (30 days), and per-GB retrieval costs, so it is best suited for infrequently accessed data that is stored for longer than 30 days.

Amazon S3 Reduced Redundancy Storage (RRS) offers slightly lower durability (4 nines) than Standard or Standard-IA at a reduced cost. It is most appropriate for derived data that can be easily reproduced, such as image thumbnails.

Finally, the *Amazon Glacier* storage class offers secure, durable, and extremely low-cost cloud storage for data that does not require real-time access, such as archives and long-term backups. To keep costs low, Amazon Glacier is optimized for infrequently accessed data where a retrieval time of several hours is suitable. To retrieve an Amazon Glacier object, you issue a restore command using one of the Amazon S3 APIs; three to five hours later, the Amazon Glacier object is copied to Amazon S3 RRS. Note that the restore simply creates a copy in Amazon S3 RRS; the original data object remains in Amazon Glacier until explicitly deleted. Also be aware that Amazon Glacier allows you to retrieve up to 5% of the Amazon S3 data stored in Amazon Glacier for free each month; restores beyond the daily restore allowance incur a restore fee. Refer to the Amazon Glacier pricing page on the AWS website for full details.

In addition to acting as a storage tier in Amazon S3, Amazon Glacier is also a standalone storage service with a separate API and some unique characteristics. However, when you use Amazon Glacier as a storage class of Amazon S3, you always interact with the data via the Amazon S3 APIs. Refer to the Amazon Glacier section for more details.

 Set a data retrieval policy to limit restores to the free tier or to a maximum GB-per-hour limit to avoid or minimize Amazon Glacier restore fees.

Object Lifecycle Management

Amazon S3 *Object Lifecycle Management* is roughly equivalent to automated *storage tiering* in traditional IT storage infrastructures. In many cases, data has a natural lifecycle, starting out as “hot” (frequently accessed) data, moving to “warm” (less frequently accessed) data as it ages, and ending its life as “cold” (long-term backup or archive) data before eventual deletion.

For example, many business documents are frequently accessed when they are created, then become much less frequently accessed over time. In many cases, however, compliance rules require business documents to be archived and kept accessible for years. Similarly, studies show that file, operating system, and database backups are most frequently accessed in the first few days after they are created, usually to restore after an inadvertent error. After a week or two, these backups remain a critical asset, but they are much less likely to be accessed for a restore. In many cases, compliance rules require that a certain number of backups be kept for several years.

Using Amazon S3 lifecycle configuration rules, you can significantly reduce your storage costs by automatically transitioning data from one storage class to another or even automatically deleting data after a period of time. For example, the lifecycle rules for backup data might be:

- Store backup data initially in Amazon S3 Standard.
- After 30 days, transition to Amazon Standard-IA.
- After 90 days, transition to Amazon Glacier.
- After 3 years, delete.

Lifecycle configurations are attached to the bucket and can apply to all objects in the bucket or only to objects specified by a prefix.

Encryption

It is strongly recommended that all sensitive data stored in Amazon S3 be encrypted, both in flight and at rest.

To encrypt your Amazon S3 data in flight, you can use the Amazon S3 Secure Sockets Layer (SSL) API endpoints. This ensures that all data sent to and from Amazon S3 is encrypted while in transit using the HTTPS protocol.

To encrypt your Amazon S3 data at rest, you can use several variations of *Server-Side Encryption (SSE)*. Amazon S3 encrypts your data at the object level as it writes it to disks in its data centers and decrypts it for you when you access it. All SSE performed by Amazon S3 and AWS Key Management Service (Amazon KMS) uses the 256-bit Advanced Encryption Standard (AES). You can also encrypt your Amazon S3 data at rest using *Client-Side Encryption*, encrypting your data on the client before sending it to Amazon S3.

SSE-S3 (AWS-Managed Keys)

This is a fully integrated "check-box-style" encryption solution where AWS handles the key management and key protection for Amazon S3. Every object is encrypted with a unique key. The actual object key itself is then further encrypted by a separate master key. A new master key is issued at least monthly, with AWS rotating the keys. Encrypted data, encryption keys, and master keys are all stored separately on secure hosts, further enhancing protection.

SSE-KMS (AWS KMS Keys)

This is a fully integrated solution where Amazon handles your key management and protection for Amazon S3, but where you manage the keys. SSE-KMS offers several additional benefits compared to SSE-S3. Using SSE-KMS, there are separate permissions for using the master key, which provide protection against unauthorized access to your objects stored in Amazon S3 and an additional layer of control. AWS KMS also provides auditing, so you can see who used your key to access which object and when they tried to access this object. AWS KMS also allows you to view any failed attempts to access data from users who did not have permission to decrypt the data.

SSE-C (Customer-Provided Keys)

This is used when you want to maintain your own encryption keys but don't want to manage or implement your own client-side encryption library. With SSE-C, AWS will do the encryption/decryption of your objects while you maintain full control of the keys used to encrypt/decrypt the objects in Amazon S3.

Client-Side Encryption

Client-side encryption refers to encrypting data on the client side of your application before sending it to Amazon S3. You have the following two options for using data encryption keys:

- Use an AWS KMS-managed customer master key.
- Use a client-side master key.

When using client-side encryption, you retain end-to-end control of the encryption process, including management of the encryption keys.

 For maximum simplicity and ease of use, use server-side encryption with AWS-managed keys (SSE-S3 or SSE-KMS).

Versioning

Amazon S3 versioning helps protects your data against accidental or malicious deletion by keeping multiple versions of each object in the bucket, identified by a unique version ID. Versioning allows you to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. If a user makes an accidental change or even maliciously deletes an object in your S3 bucket, you can restore the object to its original state simply by referencing the version ID in addition to the bucket and object key. Versioning is turned on at the bucket level. Once enabled, versioning cannot be removed from a bucket; it can only be suspended.

MFA Delete

MFA Delete adds another layer of data protection on top of bucket versioning. MFA Delete requires additional authentication in order to permanently delete an object version or change the versioning state of a bucket. In addition to your normal security credentials, MFA Delete requires an authentication code (a temporary, one-time password) generated by a hardware or virtual Multi-Factor Authentication (MFA) device. Note that MFA Delete can only be enabled by the root account.

Pre-Signed URLs

All Amazon S3 objects by default are private, meaning that only the owner has access. However, the object owner can optionally share objects with others by creating a *pre-signed URL*, using their own security credentials to grant time-limited permission to download the objects. When you create a pre-signed URL for your object, you must provide your security credentials and specify a bucket name, an object key, the HTTP method (`GET` to download the object), and an expiration date and time. The pre-signed URLs are valid only for the specified duration. This is particularly useful to protect against "content scraping" of web content such as media files stored in Amazon S3.

Multipart Upload

To better support uploading or copying of large objects, Amazon S3 provides the Multipart Upload API. This allows you to upload large objects as a set of parts, which generally gives better network utilization (through parallel transfers), the ability to pause and resume, and the ability to upload objects where the size is initially unknown.

Multipart upload is a three-step process: initiation, uploading the parts, and completion (or abort). Parts can be uploaded independently in arbitrary order, with retransmission if needed. After all of the parts are uploaded, Amazon S3 assembles the parts in order to create an object.

In general, you *should* use multipart upload for objects larger than 100 Mbytes, and you *must* use multipart upload for objects larger than 5GB. When using the low-level APIs, you must break the file to be uploaded into parts and keep track of the parts. When using the high-level APIs and the high-level Amazon S3 commands in the AWS CLI (`aws s3 cp`, `aws s3 mv`, and `aws s3 sync`), multipart upload is

automatically performed for large objects.

 You can set an object lifecycle policy on a bucket to abort incomplete multipart uploads after a specified number of days. This will minimize the storage costs associated with multipart uploads that were not completed.

Range GETs

It is possible to download (`GET`) only a portion of an object in both Amazon S3 and Amazon Glacier by using something called a *Range `GET`*. Using the Range HTTP header in the `GET` request or equivalent parameters in one of the SDK wrapper libraries, you specify a range of bytes of the object. This can be useful in dealing with large objects when you have poor connectivity or to download only a known portion of a large Amazon Glacier backup.

Cross-Region Replication

Cross-region replication is a feature of Amazon S3 that allows you to asynchronously replicate all new objects in the source bucket in one AWS region to a target bucket in another region. Any metadata and ACLs associated with the object are also part of the replication. After you set up cross-region replication on your source bucket, any changes to the data, metadata, or ACLs on an object trigger a new replication to the destination bucket. To enable cross-region replication, versioning must be turned on for both source and destination buckets, and you must use an IAM policy to give Amazon S3 permission to replicate objects on your behalf.

Cross-region replication is commonly used to reduce the latency required to access objects in Amazon S3 by placing objects closer to a set of users or to meet requirements to store backup data at a certain distance from the original source data.

 If turned on in an existing bucket, cross-region replication will only replicate new objects. Existing objects will not be replicated and must be copied to the new bucket via a separate command.

Logging

In order to track requests to your Amazon S3 bucket, you can enable Amazon S3 server access logs. Logging is off by default, but it can easily be enabled. When you enable logging for a bucket (the source bucket), you must choose where the logs will be stored (the target bucket). You can store access logs in the same bucket or in a different bucket. Either way, it is optional (but a best practice) to specify a prefix, such as `logs/` or `yourbucketname/logs/`, so that you can more easily identify your logs.

Once enabled, logs are delivered on a best-effort basis with a slight delay. Logs include information such as:

- Requestor account and IP address

- Bucket name
- Request time
- Action (`GET`, `PUT`, `LIST`, and so forth)
- Response status or error code

Event Notifications

Amazon S3 *event notifications* can be sent in response to actions taken on objects uploaded or stored in Amazon S3. Event notifications enable you to run workflows, send alerts, or perform other actions in response to changes in your objects stored in Amazon S3. You can use Amazon S3 event notifications to set up triggers to perform actions, such as transcoding media files when they are uploaded, processing data files when they become available, and synchronizing Amazon S3 objects with other data stores.

Amazon S3 event notifications are set up at the bucket level, and you can configure them through the Amazon S3 console, through the REST API, or by using an AWS SDK. Amazon S3 can publish notifications when new objects are created (by a `PUT`, `POST`, `COPY`, or multipart upload completion), when objects are removed (by a `DELETE`), or when Amazon S3 detects that an RRS object was lost. You can also set up event notifications based on object name prefixes and suffixes. Notification messages can be sent through either Amazon Simple Notification Service (Amazon SNS) or Amazon Simple Queue Service (Amazon SQS) or delivered directly to AWS Lambda to invoke AWS Lambda functions.

Best Practices, Patterns, and Performance

It is a common pattern to use Amazon S3 storage in hybrid IT environments and applications. For example, data in on-premises file systems, databases, and compliance archives can easily be backed up over the Internet to Amazon S3 or Amazon Glacier, while the primary application or database storage remains on-premises.

Another common pattern is to use Amazon S3 as bulk "blob" storage for data, while keeping an index to that data in another service, such as Amazon DynamoDB or Amazon RDS. This allows quick searches and complex queries on key names without listing keys continually.

Amazon S3 will scale automatically to support very high request rates, automatically re-partitioning your buckets as needed. If you need request rates higher than 100 requests per second, you may want to review the Amazon S3 best practices guidelines in the Developer Guide. To support higher request rates, it is best to ensure some level of random distribution of keys, for example by including a hash as a prefix to key names.



If you are using Amazon S3 in a `GET`-intensive mode, such as a static website hosting, for best performance you should consider using an Amazon CloudFront distribution as a caching layer in front of your Amazon S3 bucket.

Amazon Glacier

Amazon Glacier is an extremely low-cost storage service that provides durable, secure, and flexible storage for data archiving and online backup. To keep costs low, Amazon Glacier is designed for infrequently accessed data where a retrieval time of three to five hours is acceptable.

Amazon Glacier can store an unlimited amount of virtually any kind of data, in any format. Common use cases for Amazon Glacier include replacement of traditional tape solutions for long-term backup and archive and storage of data required for compliance purposes. In most cases, the data stored in Amazon Glacier consists of large TAR (Tape Archive) or ZIP files.

Like Amazon S3, Amazon Glacier is extremely durable, storing data on multiple devices across multiple facilities in a region. Amazon Glacier is designed for 99.999999999% durability of objects over a given year.

Archives

In Amazon Glacier, data is stored in *archives*. An archive can contain up to 40TB of data, and you can have an unlimited number of archives. Each archive is assigned a unique archive ID at the time of creation. (Unlike an Amazon S3 object key, you cannot specify a user-friendly archive name.) All archives are automatically encrypted, and archives are immutable—after an archive is created, it cannot be modified.

Vaults

Vaults are containers for archives. Each AWS account can have up to 1,000 vaults. You can control access to your vaults and the actions allowed using IAM policies or vault access policies.

Vaults Locks

You can easily deploy and enforce compliance controls for individual Amazon Glacier vaults with a *vault lock* policy. You can specify controls such as Write Once Read Many (WORM) in a vault lock policy and lock the policy from future edits. Once locked, the policy can no longer be changed.

Data Retrieval

You can retrieve up to 5% of your data stored in Amazon Glacier for free each month, calculated on a daily prorated basis. If you retrieve more than 5%, you will incur retrieval fees based on your maximum retrieval rate. To eliminate or minimize those fees, you can set a data retrieval policy on a vault to limit your retrievals to the free tier or to a specified data rate.

Amazon Glacier versus Amazon Simple Storage Service (Amazon S3)

Amazon Glacier is similar to Amazon S3, but it differs in several key aspects. Amazon Glacier supports 40TB archives versus 5TB objects in Amazon S3. Archives in Amazon Glacier are identified by system-generated archive IDs, while Amazon S3 lets you use “friendly” key names. Amazon Glacier archives are

automatically encrypted, while encryption at rest is optional in Amazon S3. However, by using Amazon Glacier as an Amazon S3 storage class together with object lifecycle policies, you can use the Amazon S3 interface to get most of the benefits of Amazon Glacier without learning a new interface.

Summary

Amazon S3 is the core object storage service on AWS, allowing you to store an unlimited amount of data with very high durability.

Common Amazon S3 use cases include backup and archive, web content, big data analytics, static website hosting, mobile and cloud-native application hosting, and disaster recovery.

Amazon S3 is integrated with many other AWS cloud services, including AWS IAM, AWS KMS, Amazon EC2, Amazon EBS, Amazon EMR, Amazon DynamoDB, Amazon Redshift, Amazon SQS, AWS Lambda, and Amazon CloudFront.

Object storage differs from traditional block and file storage. Block storage manages data at a device level as addressable blocks, while file storage manages data at the operating system level as files and folders. Object storage manages data as objects that contain both data and metadata, manipulated by an API.

Amazon S3 buckets are containers for objects stored in Amazon S3. Bucket names must be globally unique. Each bucket is created in a specific region, and data does not leave the region unless explicitly copied by the user.

Amazon S3 objects are files stored in buckets. Objects can be up to 5TB and can contain any kind of data. Objects contain both data and metadata and are identified by keys. Each Amazon S3 object can be addressed by a unique URL formed by the web services endpoint, the bucket name, and the object key.

Amazon S3 has a minimalistic API—create/delete a bucket, read/write/delete objects, list keys in a bucket—and uses a REST interface based on standard HTTP verbs—`GET`, `PUT`, `POST`, and `DELETE`. You can also use SDK wrapper libraries, the AWS CLI, and the AWS Management Console to work with Amazon S3.

Amazon S3 is highly durable and highly available, designed for 11 nines of durability of objects in a given year and four nines of availability.

Amazon S3 is eventually consistent, but offers read-after-write consistency for new object `PUT`s.

Amazon S3 objects are private by default, accessible only to the owner. Objects can be marked public readable to make them accessible on the web. Controlled access may be provided to others using ACLs and AWS IAM and Amazon S3 bucket policies.

Static websites can be hosted in an Amazon S3 bucket.

Prefixes and delimiters may be used in key names to organize and navigate data hierarchically much like a traditional file system.

Amazon S3 offers several storage classes suited to different use cases: Standard is designed for general-purpose data needing high performance and low latency. Standard-IA is for less frequently accessed data. RRS offers lower redundancy at lower cost for easily reproduced data. Amazon Glacier offers low-cost durable storage for archive and long-term backups that can are rarely accessed and can accept a three- to five-hour retrieval time.

Object lifecycle management policies can be used to automatically move data between storage classes based on time.

Amazon S3 data can be encrypted using server-side or client-side encryption, and encryption keys can be managed with Amazon KMS.

Versioning and MFA Delete can be used to protect against accidental deletion.

Cross-region replication can be used to automatically copy new objects from a source bucket in one region to a target bucket in another region.

Pre-signed URLs grant time-limited permission to download objects and can be used to protect media and other web content from unauthorized "web scraping."

Multipart upload can be used to upload large objects, and Range `GET`s can be used to download portions of an Amazon S3 object or Amazon Glacier archive.

Server access logs can be enabled on a bucket to track requestor, object, action, and response.

Amazon S3 event notifications can be used to send an Amazon SQS or Amazon SNS message or to trigger an AWS Lambda function when an object is created or deleted.

Amazon Glacier can be used as a standalone service or as a storage class in Amazon S3.

Amazon Glacier stores data in archives, which are contained in vaults. You can have up to 1,000 vaults, and each vault can store an unlimited number of archives.

Amazon Glacier vaults can be locked for compliance purposes.

Exam Essentials

Know what amazon s3 is and what it is commonly used for. Amazon S3 is secure, durable, and highly scalable cloud storage that can be used to store an unlimited amount of data in almost any format using a simple web services interface. Common use cases include backup and archive, content storage and distribution, big data analytics, static website hosting, cloud-native application hosting, and disaster recovery.

Understand how object storage differs from block and file storage. Amazon S3 cloud object storage manages data at the application level as objects using a REST API built on HTTP. Block storage manages data at the operating system level as numbered addressable blocks using protocols such as SCSI or Fibre Channel. File storage manages data as shared files at the operating system level using a protocol such as CIFS or NFS.

Understand the basics of Amazon S3. Amazon S3 stores data in objects that contain data and metadata. Objects are identified by a user-defined key and are stored in a simple flat folder called a bucket. Interfaces include a native REST interface, SDKs for many languages, an AWS CLI, and the AWS Management Console.

Know how to create a bucket; how to upload, download, and delete objects; how to make objects public; and how to open an object URL.

Understand the durability, availability, and data consistency model of Amazon S3. Amazon S3 standard storage is designed for 11 nines durability and four nines availability of objects over a year. Other storage classes differ. Amazon S3 is eventually consistent, but offers read-after-write consistency for `PUT`s to new objects.

Know how to enable static website hosting on Amazon S3. To create a static website on Amazon S3, you must create a bucket with the website hostname, upload your static content and make it public, enable static website hosting on the bucket, and indicate the index and error page objects.

Know how to protect your data on Amazon S3. Encrypt data in flight using HTTPS and at rest using SSE or client-side encryption. Enable versioning to keep multiple versions of an object in a bucket. Enable MFA Delete to protect against accidental deletion. Use ACLs Amazon S3 bucket policies and AWS IAM policies for access control. Use pre-signed URLs for time-limited download access. Use cross-region replication to automatically replicate data to another region.

Know the use case for each of the Amazon S3 storage classes. Standard is for general purpose data that needs high durability, high performance, and low latency access. Standard-IA is for data that is less frequently accessed, but that needs the same performance and availability when accessed. RRS offers lower durability at lower cost for easily replicated data. Amazon Glacier is for storing rarely accessed archival data at lowest cost, when three- to five-hour retrieval time is acceptable.

Know how to use lifecycle configuration rules. Lifecycle rules can be configured in the AWS Management Console or the APIs. Lifecycle configuration rules define actions to transition objects from one storage class to another based on time.

Know how to use Amazon S3 event notifications. Event notifications are set at the bucket level and can trigger a message in Amazon SNS or Amazon SQS or an action in AWS Lambda in response to an upload or a delete of an object.

Know the basics of amazon glacier as a standalone service. Data is stored in encrypted archives that can be as large as 40TB. Archives typically contain TAR or ZIP files. Vaults are containers for archives, and vaults can be locked for compliance.

Exercises

For assistance in completing the following exercises, reference the following documentation:

- Getting started with Amazon S3: http://docs.aws.amazon.com/AmazonS3/latest/gsg/GetStartedWithS3.html
- Setting up a static website: http://docs.aws.amazon.com/AmazonS3/latest/dev/HostingWebsiteOnS3Setup.html
- Using versioning: http://docs.aws.amazon.com/AmazonS3/latest/dev/Versioning.html
- Object Lifecycle Management: http://docs.aws.amazon.com/AmazonS3/latest/dev/object-lifecycle-mgmt.html

EXERCISE 2.1

Create an Amazon Simple Storage Service (Amazon S3) Bucket

In this exercise, you will create a new Amazon S3 bucket in your selected region. You will use this bucket in the following exercises.

1. Log in to the AWS Management Console.
2. Choose an appropriate region, such as US West (Oregon).
3. Navigate to the Amazon S3 console. Notice that the region indicator now says Global. Remember that Amazon S3 buckets form a global namespace, even though each bucket is created in a specific region.
4. Start the create bucket process.
5. When prompted for Bucket Name, use **`mynewbucket`**.
6. Choose a region, such as US West (Oregon).
7. Try to create the bucket. You almost surely will get a message that the requested bucket name is not available. Remember that a bucket name must be unique globally.
8. Try again using your surname followed by a hyphen and then today's date in a six-digit format as the bucket name (a bucket name that is not likely to exist already).

You should now have a new Amazon S3 bucket.

EXERCISE 2.2

Upload, Make Public, Rename, and Delete Objects in Your Bucket

In this exercise, you will upload a new object to your bucket. You will then make this object public and view the object in your browser. You will then rename the object and finally delete it from the bucket.

Upload an Object

1. Load your new bucket in the Amazon S3 console.
2. Select Upload, then Add Files.
3. Locate a file on your PC that you are okay with uploading to Amazon S3 and making public to the Internet. (We suggest using a non-personal image file for the purposes of this exercise.)
4. Select a suitable file, then Start Upload. You will see the status of your file in the Transfers section.
5. After your file is uploaded, the status should change to Done.

The file you uploaded is now stored as an Amazon S3 object and should be now listed in the contents of your bucket.

Open the Amazon S3 URL

6. Now open the properties for the object. The properties should include bucket, name, and link.
7. Copy the Amazon S3 URL for the object.
8. Paste the URL in the address bar of a new browser window or tab.

You should get a message with an XML error code `AccessDenied`. Even though the object has a URL, it is private by default, so it cannot be accessed by a web browser.

Make the Object Public

9. Go back to the Amazon S3 Console and select Make Public. (Equivalently, you can change the object's permissions and add grantee Everyone and permissions Open/Download.)
10. Copy the Amazon S3 URL again and try to open it in a browser or tab. Your public image file should now display in the browser or browser tab.

Rename Object

11. In the Amazon S3 console, select Rename.
12. Rename the object, but keep the same file extension.
13. Copy the new Amazon S3 URL and try to open it in a browser or tab. You should see the same image file.

Delete the Object

14. In the Amazon S3 console, select Delete. Select OK when prompted if you want to delete the object.
15. The object has now been deleted.
16. To verify, try to reload the deleted object's Amazon S3 URL.

You should once again get the XML `AccessDenied` error message.

EXERCISE 2.3

Enable Version Control

In this exercise, you will enable version control on your newly created bucket.

Enable Versioning

1. In the Amazon S3 console, load the properties of your bucket. Don't open the bucket.
2. Enable versioning in the properties and select OK to verify. Your bucket now has versioning enabled. (Note that versioning can be suspended, but not turned off.)

Create Multiple Versions of an Object

3. Create a text file named `foo.txt` on your computer and write the word **`blue`** in the text file.
4. Save the text file to a location of your choosing.
5. Upload the text file to your bucket. This will be version 1.
6. After you have uploaded the text file to your bucket, open the copy on your local computer and change the word **`blue`** to **`red`**. Save the text file with the original filename.
7. Upload the modified file to your bucket.
8. Select Show Versions on the uploaded object.

You will now see two different versions of the object with different Version IDs and possibly different sizes. Note that when you select Show Version, the Amazon S3 URL now includes the version ID in the query string after the object name.

EXERCISE 2.4

Delete an Object and Then Restore It

In this exercise, you will delete an object in your Amazon S3 bucket and then restore it.

Delete an Object

1. Open the bucket containing the text file for which you now have two versions.
2. Select Hide Versions.
3. Select Delete, and then select OK to verify.
4. Your object will now be deleted, and you can no longer see the object.
5. Select Show Versions.

Both versions of the object now show their version IDs.

Restore an Object

6. Open your bucket.
7. Select Show Versions.
8. Select the oldest version and download the object. Note that the filename is simply `foo.txt` with no version indicator.
9. Upload `foo.txt` to the same bucket.
10. Select Hide Versions, and the file `foo.txt` should re-appear.



To restore a version, you copy the desired version into the same bucket. In the Amazon S3 console, this requires a download then re-upload of the object. Using APIs, SDKs, or AWS CLI, you can copy a version directly without downloading and re-uploading.

EXERCISE 2.5

Lifecycle Management

In this exercise, you will explore the various options for lifecycle management.

1. Select your bucket in the Amazon S3 console.
2. Under Properties, add a Lifecycle Rule.
3. Explore the various options to add lifecycle rules to objects in this bucket. It is recommended that you do not implement any of these options, as you may incur additional costs. After you have finished, click the Cancel button.

 Most lifecycle rules require some number of days to expire before the transition takes effect. For example, it takes a minimum of 30 days to transition from Amazon S3 Standard to Amazon S3 Standard-IA. This makes it impractical to create a lifecycle rule and see the actual result in an exercise.

EXERCISE 2.6

Enable Static Hosting on Your Bucket

In this exercise, you will enable static hosting on your newly created bucket.

1. Select your bucket in the Amazon S3 console.
2. In the Properties section, select Enable Website Hosting.
3. For the index document name, enter **`index.txt`**, and for the error document name, enter **`error.txt`**.
4. Use a text editor to create two text files and save them as **`index.txt`** and **`error.txt`**. In the `index.txt` file, write the phrase "**`Hello World`**," and in the `error.txt` file, write the phrase "**`Error Page`**." Save both text files and upload them to your bucket.
5. Make the two objects public.
6. Copy the Endpoint: link under Static Website Hosting and paste it in a browser window or tab. You should now see the phrase `"Hello World"` displayed.
7. In the address bar in your browser, try adding a forward slash followed by a made-up filename (for example, `/test.html`). You should now see the phrase `"Error Page"` displayed.
8. To clean up, delete all of the objects in your bucket and then delete the bucket itself.

Review Questions

1. In what ways does Amazon Simple Storage Service (Amazon S3) object storage differ from block and file storage? (Choose 2 answers)
   A. Amazon S3 stores data in fixed size blocks.
   B. Objects are identified by a numbered address.
   C. Objects can be any size.
   D. Objects contain both data and metadata.
   E. Objects are stored in buckets.
2. Which of the following are not appropriates use cases for Amazon Simple Storage Service (Amazon S3)? (Choose 2 answers)
   A. Storing web content
   B. Storing a file system mounted to an Amazon Elastic Compute Cloud (Amazon EC2) instance
   C. Storing backups for a relational database
   D. Primary storage for a database
   E. Storing logs for analytics
3. What are some of the key characteristics of Amazon Simple Storage Service (Amazon S3)? (Choose 3 answers)
   A. All objects have a URL.
   B. Amazon S3 can store unlimited amounts of data.
   C. Objects are world-readable by default.
   D. Amazon S3 uses a REST (Representational State Transfer) Application Program Interface (API).
   E. You must pre-allocate the storage in a bucket.
4. Which features can be used to restrict access to Amazon Simple Storage Service (Amazon S3) data? (Choose 3 answers)
   A. Enable static website hosting on the bucket.
   B. Create a pre-signed URL for an object.
   C. Use an Amazon S3 Access Control List (ACL) on a bucket or object.
   D. Use a lifecycle policy.
   E. Use an Amazon S3 bucket policy.
5. Your application stores critical data in Amazon Simple Storage Service (Amazon S3), which must be protected against inadvertent or intentional deletion. How can this data be protected? (Choose 2 answers)

A. Use cross-region replication to copy data to another bucket automatically.

B. Set a vault lock.

C. Enable versioning on the bucket.

D. Use a lifecycle policy to migrate data to Amazon Glacier.

E. Enable MFA Delete on the bucket.

6. Your company stores documents in Amazon Simple Storage Service (Amazon S3), but it wants to minimize cost. Most documents are used actively for only about a month, then much less frequently. However, all data needs to be available within minutes when requested. How can you meet these requirements?

   A. Migrate the data to Amazon S3 Reduced Redundancy Storage (RRS) after 30 days.

   B. Migrate the data to Amazon Glacier after 30 days.

   C. Migrate the data to Amazon S3 Standard – Infrequent Access (IA) after 30 days.

   D. Turn on versioning, then migrate the older version to Amazon Glacier.

7. How is data stored in Amazon Simple Storage Service (Amazon S3) for high durability?

   A. Data is automatically replicated to other regions.

   B. Data is automatically replicated within a region.

   C. Data is replicated only if versioning is enabled on the bucket.

   D. Data is automatically backed up on tape and restored if needed.

8. Based on the following Amazon Simple Storage Service (Amazon S3) URL, which one of the following statements is correct?

   https://bucket1.abc.com.s3.amazonaws.com/folderx/myfile.doc

   A. The object “myfile.doc” is stored in the folder “folderx” in the bucket “bucket1.abc.com.”

   B. The object “myfile.doc” is stored in the bucket “bucket1.abc.com.”

   C. The object “folderx/myfile.doc” is stored in the bucket “bucket1.abc.com.”

   D. The object “myfile.doc” is stored in the bucket “bucket1.”

9. To have a record of who accessed your Amazon Simple Storage Service (Amazon S3) data and from where, you should do what?

   A. Enable versioning on the bucket.

   B. Enable website hosting on the bucket.

   C. Enable server access logs on the bucket.

   D. Create an AWS Identity and Access Management (IAM) bucket policy.

   E. Enable Amazon CloudWatch logs.

10. What are some reasons to enable cross-region replication on an Amazon Simple Storage Service (Amazon S3) bucket? (Choose 2 answers)

    A. You want a backup of your data in case of accidental deletion.

B. You have a set of users or customers who can access the second bucket with lower latency.

C. For compliance reasons, you need to store data in a location at least 300 miles away from the first region.

D. Your data needs at least five nines of durability.

11. Your company requires that all data sent to external storage be encrypted before being sent. Which Amazon Simple Storage Service (Amazon S3) encryption solution will meet this requirement?

A. Server-Side Encryption (SSE) with AWS-managed keys (SSE-S3)

B. SSE with customer-provided keys (SSE-C)

C. Client-side encryption with customer-managed keys

D. Server-side encryption with AWS Key Management Service (AWS KMS) keys (SSE-KMS)

12. You have a popular web application that accesses data stored in an Amazon Simple Storage Service (Amazon S3) bucket. You expect the access to be very read-intensive, with expected request rates of up to 500 GETs per second from many clients. How can you increase the performance and scalability of Amazon S3 in this case?

A. Turn on cross-region replication to ensure that data is served from multiple locations.

B. Ensure randomness in the namespace by including a hash prefix to key names.

C. Turn on server access logging.

D. Ensure that key names are sequential to enable pre-fetch.

13. What is needed before you can enable cross-region replication on an Amazon Simple Storage Service (Amazon S3) bucket? (Choose 2 answers)

A. Enable versioning on the bucket.

B. Enable a lifecycle rule to migrate data to the second region.

C. Enable static website hosting.

D. Create an AWS Identity and Access Management (IAM) policy to allow Amazon S3 to replicate objects on your behalf.

14. Your company has 100TB of financial records that need to be stored for seven years by law. Experience has shown that any record more than one-year old is unlikely to be accessed. Which of the following storage plans meets these needs in the most cost efficient manner?

A. Store the data on Amazon Elastic Block Store (Amazon EBS) volumes attached to t2.micro instances.

B. Store the data on Amazon Simple Storage Service (Amazon S3) with lifecycle policies that change the storage class to Amazon Glacier after one year and delete the object after seven years.

C. Store the data in Amazon DynamoDB and run daily script to delete data older than seven years.

D. Store the data in Amazon Elastic MapReduce (Amazon EMR).

15. Amazon Simple Storage Service (S3) bucket policies can restrict access to an Amazon S3 bucket and objects by which of the following? (Choose 3 answers)

A. Company name

B. IP address range

C. AWS account

D. Country of origin

E. Objects with a specific prefix

16. Amazon Simple Storage Service (Amazon S3) is an eventually consistent storage system. For what kinds of operations is it possible to get stale data as a result of eventual consistency? (Choose 2 answers)

A. GET after PUT of a new object

B. GET or LIST after a DELETE

C. GET after overwrite PUT (PUT to an existing key)

D. DELETE after PUT of new object

17. What must be done to host a static website in an Amazon Simple Storage Service (Amazon S3) bucket? (Choose 3 answers)

A. Configure the bucket for static hosting and specify an index and error document.

B. Create a bucket with the same name as the website.

C. Enable File Transfer Protocol (FTP) on the bucket.

D. Make the objects in the bucket world-readable.

E. Enable HTTP on the bucket.

18. You have valuable media files hosted on AWS and want them to be served only to authenticated users of your web application. You are concerned that your content could be stolen and distributed for free. How can you protect your content?

A. Use static web hosting.

B. Generate pre-signed URLs for content in the web application.

C. Use AWS Identity and Access Management (IAM) policies to restrict access.

D. Use logging to track your content.

19. Amazon Glacier is well-suited to data that is which of the following? (Choose 2 answers)

A. Is infrequently or rarely accessed

B. Must be immediately available when needed

C. Is available after a three- to five-hour restore period

D. Is frequently erased within 30 days

20. Which statements about Amazon Glacier are true? (Choose 3 answers)

A. Amazon Glacier stores data in objects that live in archives.

B. Amazon Glacier archives are identified by user-specified key names.

C. Amazon Glacier archives take three to five hours to restore.

D. Amazon Glacier vaults can be locked.

E. Amazon Glacier can be used as a standalone service and as an Amazon S3 storage class.

Chapter 3
Amazon Elastic Compute Cloud (Amazon EC2) and Amazon Elastic Block Store (Amazon EBS)

THE AWS CERTIFIED SOLUTIONS ARCHITECT ASSOCIATE EXAM OBJECTIVES COVERED IN THIS CHAPTER MAY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

Domain 1.0: Designing highly available, cost-efficient, fault-tolerant, scalable systems

✓**1.1 Identify and recognize cloud architecture considerations, such as fundamental components and effective designs.**

Content may include the following:

- How to design cloud services
- Planning and design
- Monitoring and logging

Domain 2.0: Implementation/Deployment

✓**2.1 Identify the appropriate techniques and methods using Amazon EC2, Amazon Simple Storage Service (Amazon S3), AWS Elastic Beanstalk, AWS CloudFormation, AWS OpsWorks, Amazon Virtual Private Cloud (Amazon VPC), and AWS Identity and Access Management (IAM) to code and implement a cloud solution.**

Content may include the following:

- Configure an Amazon Machine Image (AMI)
- Configure services to support compliance requirements in the cloud
- Launch instances across the AWS global infrastructure

Domain 3.0: Data Security

✓**3.2 Recognize critical disaster recovery techniques and their implementation.**

Content may include the following:

- Disaster recovery
- Amazon EB



Introduction

In this chapter, you learn how Amazon Elastic Compute Cloud (Amazon EC2) and Amazon Elastic Block Store (Amazon EBS) provide the basic elements of compute and block-level storage to run your workloads on AWS. It focuses on key topics you need to understand for the exam, including:

- How instance types and Amazon Machine Images (AMIs) define the capabilities of instances you launch on the cloud
- How to securely access your instances running on the cloud
- How to protect your instances with virtual firewalls called security groups
- How to have your instances configure themselves for unattended launch
- How to monitor and manage your instances on the cloud
- How to change the capabilities of an existing instance
- The payment options available for the best mix of affordability and flexibility
- How tenancy options and placement groups provide options to optimize compliance and performance
- How instance stores differ from Amazon EBS volumes and when they are effective
- What types of volumes are available through Amazon EBS
- How to protect your data on Amazon EBS

Amazon Elastic Compute Cloud (Amazon EC2)

Amazon EC2 is AWS primary web service that provides resizable compute capacity in the cloud.

Compute Basics

Compute refers to the amount of computational power required to fulfill your workload. If your workload is very small, such as a website that receives few visitors, then your compute needs are very small. A large workload, such as screening ten million compounds against a common cancer target, might require a great deal of compute. The amount of compute you need might change drastically over time.

Amazon EC2 allows you to acquire compute through the launching of virtual servers called *instances*. When you launch an instance, you can make use of the compute as you wish, just as you would with an on-premises server. Because you are paying for the computing power of the instance, you are charged per hour while the instance is running. When you stop the instance, you are no longer charged.

There are two concepts that are key to launching instances on AWS: (1) the amount of virtual hardware dedicated to the instance and (2) the software loaded on the instance. These two dimensions of new instances are controlled, respectively, by the instance type and the AMI.

Instance Types

The instance type defines the virtual hardware supporting an Amazon EC2 instance. There are dozens of instance types available, varying in the following dimensions:

- Virtual CPUs (vCPUs)
- Memory
- Storage (size and type)
- Network performance

Instance types are grouped into families based on the ratio of these values to each other. For instance, the m4 family provides a balance of compute, memory, and network resources, and it is a good choice for many applications. Within each family there are several choices that scale up linearly in size. Figure 3.1 shows the four instance sizes in the m4 family. Note that the ratio of vCPUs to memory is constant as the sizes scale linearly. The hourly price for each size scales linearly as well. For example, an m4.xlarge instance costs twice as much as the m4.large instance.

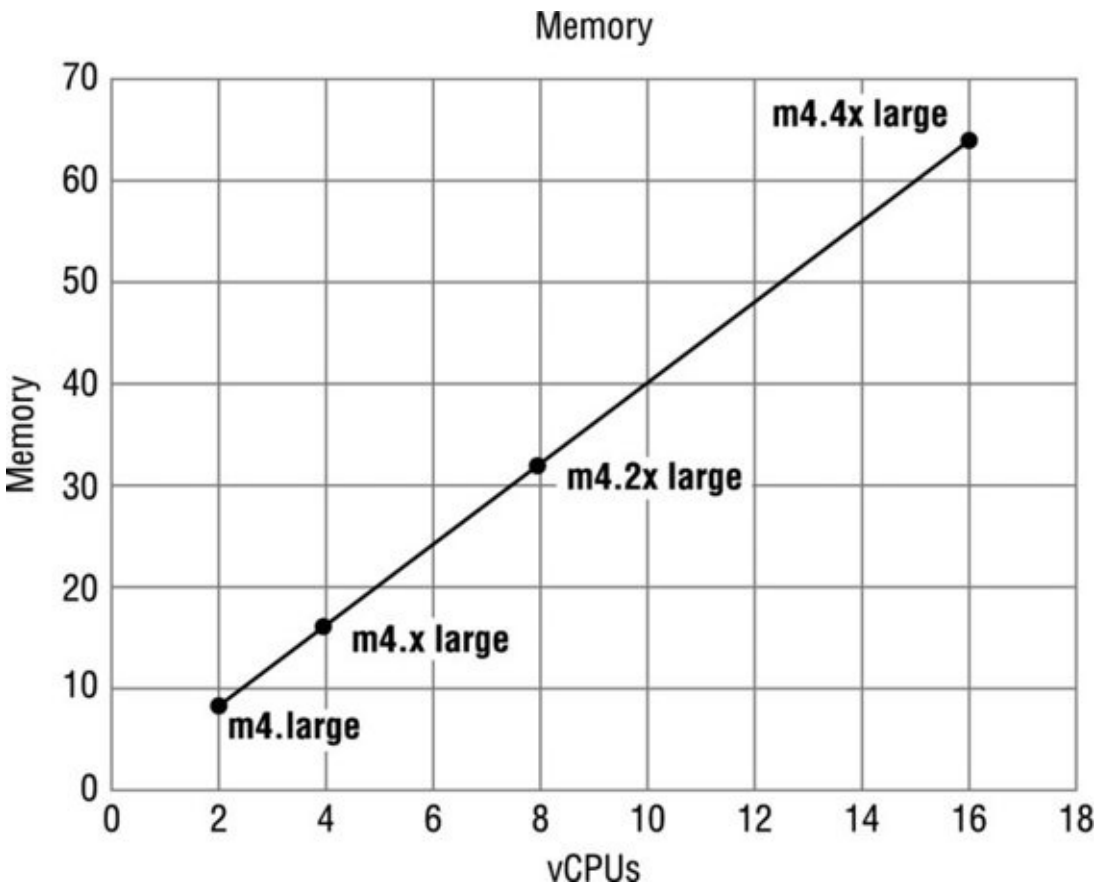


FIGURE 3.1 Memory and vCPUs for the m4 instance family

Different instance type families tilt the ratio to accommodate different types of workloads, but they all exhibit this linear scale up behavior within the family. Table 3.1 lists some of the families available.

TABLE 3.1 Sample Instance Type Families

Family	
c4	***Compute optimized***—For workloads requiring significant processing
r3	***Memory optimized***—For memory-intensive workloads
i2	***Storage optimized***—For workloads requiring high amounts of fast SSD storage
g2	***GPU-based instances***—Intended for graphics and general-purpose GPU compute workloads

In response to customer demand and to take advantage of new processor technology, AWS occasionally introduces new instance families. Check the AWS website for the current list.

Another variable to consider when choosing an instance type is network performance. For most instance types, AWS publishes a relative measure of network performance: low, moderate, or high. Some instance types specify a network performance of 10 Gbps. The network performance increases within a family as the instance type grows.

For workloads requiring greater network performance, many instance types support *enhanced networking*. Enhanced networking reduces the impact of virtualization on network performance by enabling a capability called Single Root I/O Virtualization (SR-IOV). This results in more Packets Per Second (PPS), lower latency, and less jitter. At the time of this writing, there are instance types that support enhanced networking in the C3, C4, D2, I2, M4, and R3 families (consult the AWS documentation for a current list). Enabling enhanced networking on an instance involves ensuring the correct drivers are installed and modifying an instance attribute. Enhanced networking is available only for instances

launched in an Amazon Virtual Private Cloud (Amazon VPC), which is discussed in Chapter 4, "Amazon Virtual Private Cloud (Amazon VPC)."

Amazon Machine Images (AMIs)

The *Amazon Machine Image (AMI)* defines the initial software that will be on an instance when it is launched. An AMI defines every aspect of the software state at instance launch, including:

- The Operating System (OS) and its configuration
- The initial state of any patches
- Application or system software

All AMIs are based on x86 OSs, either Linux or Windows.

There are four sources of AMIs:

- ***Published by AWS***—AWS publishes AMIs with versions of many different OSs, both Linux and Windows. These include multiple distributions of Linux (including Ubuntu, Red Hat, and Amazon's own distribution) and Windows 2008 and Windows 2012. Launching an instance based on one of these AMIs will result in the default OS settings, similar to installing an OS from the standard OS ISO image. As with any OS installation, you should immediately apply all appropriate patches upon launch.
- ***The AWS Marketplace***—AWS Marketplace is an online store that helps customers find, buy, and immediately start using the software and services that run on Amazon EC2. Many AWS partners have made their software available in the AWS Marketplace. This provides two benefits: the customer does not need to install the software, and the license agreement is appropriate for the cloud. Instances launched from an AWS Marketplace AMI incur the standard hourly cost of the instance type plus an additional per-hour charge for the additional software (some open-source AWS Marketplace packages have no additional software charge).
- ***Generated from Existing Instances***—An AMI can be created from an existing Amazon EC2 instance. This is a very common source of AMIs. Customers launch an instance from a published AMI, and then the instance is configured to meet all the customer's corporate standards for updates, management, security, and so on. An AMI is then generated from the configured instance and used to generate all instances of that OS. In this way, all new instances follow the corporate standard and it is more difficult for individual projects to launch non-conforming instances.
- ***Uploaded Virtual Servers***—Using AWS VM Import/Export service, customers can create images from various virtualization formats, including raw, VHD, VMDK, and OVA. The current list of supported OSs (Linux and Windows) can be found in the AWS documentation. It is incumbent on the customers to remain compliant with the licensing terms of their OS vendor.

Securely Using an Instance

Once launched, instances can be managed over the Internet. AWS has several services and features to ensure that this management can be done simply and securely.

Addressing an Instance

There are several ways that an instance may be addressed over the web upon creation:

- ***Public Domain Name System (DNS) Name***—When you launch an instance, AWS creates a DNS name that can be used to access the instance. This DNS name is generated automatically and cannot be specified by the customer. The name can be found in the Description tab of the AWS Management Console or via the Command Line Interface (CLI) or Application Programming Interface (API). This DNS name persists only while the instance is running and cannot be transferred to another instance.
- ***Public IP***—A launched instance may also have a public IP address assigned. This IP address is assigned from the addresses reserved by AWS and cannot be specified. This IP address is unique on the Internet, persists only while the instance is running, and cannot be transferred to another instance.
- ***Elastic IP***—An elastic IP address is an address unique on the Internet that you reserve independently and associate with an Amazon EC2 instance. While similar to a public IP, there are some key differences. This IP address persists until the customer releases it and is not tied to the lifetime or state of an individual instance. Because it can be transferred to a replacement instance in the event of an instance failure, it is a public address that can be shared externally without coupling clients to a particular instance.

 Private IP addresses and Elastic Network Interfaces (ENIs) are additional methods of addressing instances that are available in the context of an Amazon VPC. These are discussed in Chapter 4.

Initial Access

Amazon EC2 uses public-key cryptography to encrypt and decrypt login information. Public-key cryptography uses a public key to encrypt a piece of data and an associated private key to decrypt the data. These two keys together are called a *key pair*. Key pairs can be created through the AWS Management Console, CLI, or API, or customers can upload their own key pairs. AWS stores the public key, and the private key is kept by the customer. The private key is essential to acquiring secure access to an instance for the first time.

 Store your private keys securely. When Amazon EC2 launches a Linux instance, the public key is stored in the `~/.ssh/authorized_keys` file on the instance and an initial user is created. The initial user can vary depending on the OS. For example, the Amazon Linux distribution initial user is ec2-user. Initial access to the instance is obtained by using the ec2-user and the private key to log in via SSH. At this point, you can configure other users and enroll in a directory such as LDAP.

When launching a Windows instance, Amazon EC2 generates a random password for the local administrator account and encrypts the password using the public key. Initial access to the instance is obtained by decrypting the password with the private key, either in the console or through the API. The decrypted password can be used to log in to the instance with the local administrator account via RDP. At this point, you can create other local users and/or connect to an Active Directory domain.

 It is a best practice to change the initial local administrator password.

Virtual Firewall Protection

AWS allows you to control traffic in and out of your instances through virtual firewalls called *security groups*. Security groups allow you to control traffic based on port, protocol, and source/destination. Security groups have different capabilities depending on whether they are associated with an Amazon VPC or Amazon EC2-Classic. Table 3.2 compares these different capabilities (Amazon VPC is discussed in Chapter 4).

TABLE 3.2 Different Security Groups

Type of Security Group	Capabilities
EC2-Classic Security Groups	Control outgoing instance traffic
VPC Security Groups	Control outgoing and incoming instance traffic

Security groups are associated with instances when they are launched. Every instance must have at least one security group but can have more.

A security group is default deny; that is, it does not allow any traffic that is not explicitly allowed by a security group rule. A rule is defined by the three attributes in Table 3.3. When an instance is associated with multiple security groups, the rules are aggregated and all traffic allowed by each of the individual groups is allowed. For example, if security group A allows RDP traffic from 72.58.0.0/16 and security group B allows HTTP and HTTPS traffic from 0.0.0.0/0 and your instance is associated with both groups, then both the RDP and HTTP/S traffic will be allowed in to your instance.

TABLE 3.3 Security Group Rule Attributes

Attribute	Meaning
Port	The port number affected by this rule. For instance, port 80 for HTTP traffic.
Protocol	The communications standard for the traffic affected by this rule.
Source/Destination	Identifies the other end of the communication, the source for incoming traffic rules, or the destination for outgoing traffic rules. The source/destination can be defined in two ways: ***CIDR block***—An x.x.x.x/x style definition that defines a specific range of IP addresses. ***Security group***—Includes any instance that is associated with the given security group. This helps prevent coupling security group rules with specific IP addresses.

A security group is a stateful firewall; that is, an outgoing message is remembered so that the response is allowed through the security group without an explicit inbound rule being required.

Security groups are applied at the instance level, as opposed to a traditional on-premises firewall that protects at the perimeter. The effect of this is that instead of having to breach a single perimeter to access all the instances in your security group, an attacker would have to breach the security group repeatedly for each individual instance.

The Lifecycle of Instances

Amazon EC2 has several features and services that facilitate the management of Amazon EC2 instances over their entire lifecycle.

Launching

There are several additional services that are useful when launching new Amazon EC2 instances.

Bootstrapping A great benefit of the cloud is the ability to script virtual hardware management in a manner that is not possible with on-premises hardware. In order to realize the value of this, there has to be some way to configure instances and install applications programmatically when an instance is launched. The process of providing code to be run on an instance at launch is called *bootstrapping*.

One of the parameters when an instance is launched is a string value called *UserData*. This string is passed to the operating system to be executed as part of the launch process the first time the instance is booted. On Linux instances this can be shell script, and on Windows instances this can be a batch style script or a PowerShell script. The script can perform tasks such as:

- Applying patches and updates to the OS
- Enrolling in a directory service
- Installing application software
- Copying a longer script or program from storage to be run on the instance
- Installing Chef or Puppet and assigning the instance a role so the configuration management software can configure the instance



UserData is stored with the instance and is not encrypted, so it is important to not include any secrets such as passwords or keys in the UserData.

VM Import/Export In addition to importing virtual instances as AMIs, VM Import/Export enables you to easily import Virtual Machines (VMs) from your existing environment as an Amazon EC2 instance and export them back to your on-premises environment. You can only export previously imported Amazon EC2 instances. Instances launched within AWS from AMIs cannot be exported.

Instance Metadata *Instance metadata* is data about your instance that you can use to configure or manage the running instance. This is unique in that it is a mechanism to obtain AWS properties of the instance from within the OS without making a call to the AWS API. An HTTP call to `http://169.254.169.254/latest/meta-data/` will return the top node of the instance metadata tree. Instance metadata includes a wide variety of attributes, including:

- The associated security groups
- The instance ID
- The instance type
- The AMI used to launch the instance

This only begins to scratch the surface of the information available in the metadata. Consult the AWS documentation for a full list.

Managing Instances

When the number of instances in your account starts to climb, it can become difficult to keep track of them. Tags can help you manage not just your Amazon EC2 instances, but also many of your AWS Cloud

services. Tags are key/value pairs you can associate with your instance or other service. Tags can be used to identify attributes of an instance like project, environment (dev, test, and so on), billable department, and so forth. You can apply up to 10 tags per instance. Table 3.4 shows some tag suggestions.

TABLE 3.4 Sample Tags

Key	Value
Project	TimeEntry
Environment	Production
BillingCode	4004

Monitoring Instances

AWS offers a service called Amazon CloudWatch that provides monitoring and alerting for Amazon EC2 instances, and also other AWS infrastructure. Amazon CloudWatch is discussed in detail in Chapter 5, "Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling."

Modifying an Instance

There are several aspects of an instance that can be modified after launch.

Instance Type The ability to change the instance type of an instance contributes greatly to the agility of running workloads in the cloud. Instead of committing to a certain hardware configuration months before a workload is launched, the workload can be launched using a best estimate for the instance type. If the compute needs prove to be higher or lower than expected, the instances can be changed to a different size more appropriate to the workload.

Instances can be resized using the AWS Management Console, CLI, or API. To resize an instance, set the state to Stopped. Choose the "Change Instance Type" function in the tool of your choice (the instance type is listed as an Instance Setting in the console and an Instance Attribute in the CLI) and select the desired instance type. Restart the instance and the process is complete.

Security Groups If an instance is running in an Amazon VPC (discussed in Chapter 4), you can change which security groups are associated with an instance while the instance is running. For instances outside of an Amazon VPC (called EC2-Classic), the association of the security groups cannot be changed after launch.

Termination Protection

When an Amazon EC2 instance is no longer needed, the state can be set to Terminated and the instance will be shut down and removed from the AWS infrastructure. In order to prevent termination via the AWS Management Console, CLI, or API, *termination protection* can be enabled for an instance. While enabled, calls to terminate the instance will fail until termination protection is disabled. This helps to prevent accidental termination through human error.

Note that this just protects from termination calls from the AWS Management Console, CLI, or API. It does not prevent termination triggered by an OS shutdown command, termination from an Auto Scaling group (discussed in Chapter 5), or termination of a Spot Instance due to Spot price changes (discussed in the next section).

Options

There are several additional options available in Amazon EC2 to improve cost optimization, security, and performance that are important to know for the exam.

Pricing Options

You are charged for Amazon EC2 instances for each hour that they are in a running state, but the amount you are charged per hour can vary based on three pricing options: On-Demand Instances, Reserved Instances, and Spot Instances.

On-Demand Instances The price per hour for each instance type published on the AWS website represents the price for *On-Demand Instances*. This is the most flexible pricing option, as it requires no up-front commitment, and the customer has control over when the instance is launched and when it is terminated. It is the least cost effective of the three pricing options per compute hour, but its flexibility allows customers to save by provisioning a variable level of compute for unpredictable workloads.

Reserved Instances The *Reserved Instance* pricing option enables customers to make capacity reservations for predictable workloads. By using Reserved Instances for these workloads, customers can save up to 75 percent over the on-demand hourly rate. When purchasing a reservation, the customer specifies the instance type and Availability Zone for that Reserved Instance and achieves a lower effective hourly price for that instance for the duration of the reservation. An additional benefit is that capacity in the AWS data centers is reserved for that customer. There are two factors that determine the cost of the reservation: the term commitment and the payment option.

The *term commitment* is the duration of the reservation and can be either one or three years. The longer the commitment, the bigger the discount.

There are three different payment options for Reserved Instances:

- ***All Upfront***—Pay for the entire reservation up front. There is no monthly charge for the customer during the term.
- ***Partial Upfront***—Pay a portion of the reservation charge up front and the rest in monthly installments for the duration of the term.
- ***No Upfront***—Pay the entire reservation charge in monthly installments for the duration of the term.

The amount of the discount is greater the more the customer pays up front.

For example, let's look at the effect of an all upfront, three-year reservation on the effective hourly cost of an m4.2xlarge instance. The cost of running one instance continuously for three years (or 26,280 hours) at both pricing options is shown in Table 3.5.

TABLE 3.5 Reserved Instance Pricing Example

Pricing Option	Effective Hourly Cost	Total Three-Year Cost
On-Demand	$0.479/hour	$0.479/hour* 26280 hours = $12588.12
Three-Year All Upfront Reservation	$4694/26280 hours = $0.1786/hour	$4694
Savings		63%

 This example uses the published prices at the time of this writing. AWS has lowered prices many times to date, so check the AWS website for current pricing information.

When your computing needs change, you can modify your Reserved Instances and continue to benefit from your capacity reservation. Modification does not change the remaining term of your Reserved Instances; their end dates remain the same. There is no fee, and you do not receive any new bills or invoices. Modification is separate from purchasing and does not affect how you use, purchase, or sell Reserved Instances. You can modify your whole reservation, or just a subset, in one or more of the following ways:

- Switch Availability Zones within the same region.
- Change between EC2-VPC and EC2-Classic.
- Change the instance type within the same instance family (Linux instances only).

Spot Instances For workloads that are not time critical and are tolerant of interruption, *Spot Instances* offer the greatest discount. With Spot Instances, customers specify the price they are willing to pay for a certain instance type. When the customer's bid price is above the current Spot price, the customer will receive the requested instance(s). These instances will operate like all other Amazon EC2 instances, and the customer will only pay the Spot price for the hours that instance(s) run. The instances will run until:

- The customer terminates them.
- The Spot price goes above the customer's bid price.
- There is not enough unused capacity to meet the demand for Spot Instances.

If Amazon EC2 needs to terminate a Spot Instance, the instance will receive a termination notice providing a two-minute warning prior to Amazon EC2 terminating the instance.

Because of the possibility of interruption, Spot Instances should only be used for workloads tolerant of interruption. This could include analytics, financial modeling, big data, media encoding, scientific computing, and testing.

Architectures with Different Pricing Models For the exam, it's important to know how to take advantage of the different pricing models to create a cost-efficient architecture. Such an architecture may include different pricing models within the same workload. For instance, a website that averages 5,000 visits a day, but ramps up to 20,000 visits a day during periodic peaks, may purchase two Reserved Instances to handle the average traffic, but depend on On-Demand Instances to fulfill compute needs during the peak times. Figure 3.2 shows such an architecture.

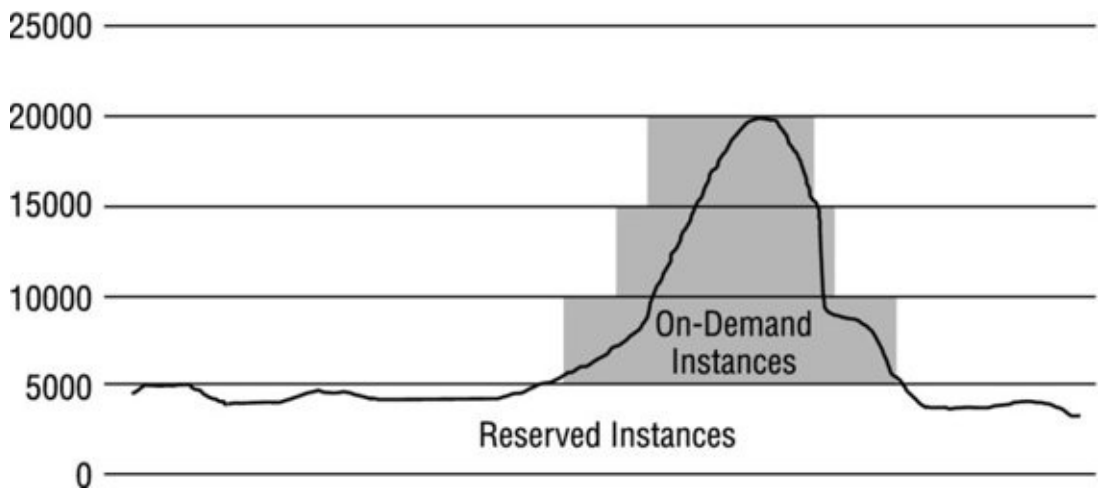


FIGURE 3.2 A workload using a mix of On-Demand and Reserved Instances

Tenancy Options

There are several *tenancy* options for Amazon EC2 instances that can help customers achieve security and compliance goals.

Shared Tenancy Shared tenancy is the default tenancy model for all Amazon EC2 instances, regardless of instance type, pricing model, and so forth. Shared tenancy means that a single host machine may house instances from different customers. As AWS does not use overprovisioning and fully isolates instances from other instances on the same host, this is a secure tenancy model.

Dedicated Instances Dedicated Instances run on hardware that's dedicated to a single customer. As a customer runs more Dedicated Instances, more underlying hardware may be dedicated to their account. Other instances in the account (those not designated as dedicated) will run on shared tenancy and will be isolated at the hardware level from the Dedicated Instances in the account.

Dedicated Host An Amazon EC2 Dedicated Host is a physical server with Amazon EC2 instance capacity fully dedicated to a single customer's use. Dedicated Hosts can help you address licensing requirements and reduce costs by allowing you to use your existing server-bound software licenses. The customer has complete control over which specific host runs an instance at launch. This differs from Dedicated Instances in that a Dedicated Instance can launch on any hardware that has been dedicated to the account.

Placement Groups

A *placement group* is a logical grouping of instances within a single Availability Zone. Placement groups enable applications to participate in a low-latency, 10 Gbps network. Placement groups are recommended for applications that benefit from low network latency, high network throughput, or both. Remember that this represents network connectivity between instances. To fully use this network performance for your placement group, choose an instance type that supports enhanced networking and 10 Gbps network performance.

Instance Stores

An *instance store* (sometimes referred to as *ephemeral storage*) provides temporary block-level storage for your instance. This storage is located on disks that are physically attached to the host computer. An instance store is ideal for temporary storage of information that changes frequently, such as buffers, caches, scratch data, and other temporary content, or for data that is replicated across a fleet of instances,

such as a load-balanced pool of web servers.

The size and type of instance stores available with an Amazon EC2 instance depend on the instance type. At this writing, storage available with various instance types ranges from no instance stores up to 24 2 TB instance stores. The instance type also determines the type of hardware for the instance store volumes. While some provide Hard Disk Drive (HDD) instance stores, other instance types use Solid State Drives (SSDs) to deliver very high random I/O performance.

Instance stores are included in the cost of an Amazon EC2 instance, so they are a very cost-effective solution for appropriate workloads. The key aspect of instance stores is that they are temporary. Data in the instance store is lost when:

- The underlying disk drive fails.
- The instance stops (the data will persist if an instance reboots).
- The instance terminates.

Therefore, do not rely on instance stores for valuable, long-term data. Instead, build a degree of redundancy via RAID or use a file system that supports redundancy and fault tolerance such as Hadoop's HDFS. Back up the data to more durable data storage solutions such as Amazon Simple Storage Service (Amazon S3) or Amazon EBS often enough to meet recovery point objectives.

Amazon Elastic Block Store (Amazon EBS)

While instance stores are an economical way to fulfill appropriate workloads, their limited persistence makes them ill-suited for many other workloads. For workloads requiring more durable block storage, Amazon provides Amazon EBS.

Elastic Block Store Basics

Amazon EBS provides persistent block-level storage volumes for use with Amazon EC2 instances. Each Amazon EBS volume is automatically replicated within its Availability Zone to protect you from component failure, offering high availability and durability. Amazon EBS volumes are available in a variety of types that differ in performance characteristics and price. Multiple Amazon EBS volumes can be attached to a single Amazon EC2 instance, although a volume can only be attached to a single instance at a time.

Types of Amazon EBS Volumes

Amazon EBS volumes are available in several different types. Types vary in areas such as underlying hardware, performance, and cost. It is important to know the properties of the different types so you can specify the most cost-efficient type that meets a workload's performance demands on the exam.

Magnetic Volumes

Magnetic volumes have the lowest performance characteristics of all Amazon EBS volume types. As such, they cost the lowest per gigabyte. They are an excellent, cost-effective solution for appropriate workloads.

A magnetic Amazon EBS volume can range in size from 1 GB to 1 TB and will average 100 *IOPS*, but has the ability to burst to hundreds of IOPS. They are best suited for:

- Workloads where data is accessed infrequently
- Sequential reads
- Situations where low-cost storage is a requirement

Magnetic volumes are billed based on the amount of data space provisioned, regardless of how much data you actually store on the volume.

General-Purpose SSD

General-purpose SSD volumes offer cost-effective storage that is ideal for a broad range of workloads. They deliver strong performance at a moderate price point that is suitable for a wide range of workloads.

A general-purpose SSD volume can range in size from 1 GB to 16 TB and provides a baseline performance of three IOPS per gigabyte provisioned, capping at 10,000 IOPS. For instance, if you provision a 1 TB volume, you can expect a baseline performance of 3,000 IOPS. A 5 TB volume will not provide a 15,000 IOPS baseline, as it would hit the cap at 10,000 IOPS.

General-purpose SSD volumes under 1 TB also feature the ability to burst to up to 3,000 IOPS for extended periods of time. For instance, if you have a 500 GB volume you can expect a baseline of 1,500

IOPS. Whenever you are not using these IOPS, they are accumulated as I/O credits. When your volume then has heavy traffic, it will use the I/O credits at a rate of up to 3,000 IOPS until they are depleted. At that point, your performance reverts to 1,500 IOPS. At 1 TB, the baseline performance of the volume is already at 3,000 IOPS, so bursting behavior does not apply.

General-purpose SSD volumes are billed based on the amount of data space provisioned, regardless of how much data you actually store on the volume. They are suited for a wide range of workloads where the very highest disk performance is not critical, such as:

- System boot volumes
- Small- to medium-sized databases
- Development and test environments

Provisioned IOPS SSD

Provisioned IOPS SSD volumes are designed to meet the needs of I/O-intensive workloads, particularly database workloads that are sensitive to storage performance and consistency in random access I/O throughput. While they are the most expensive Amazon EBS volume type per gigabyte, they provide the highest performance of any Amazon EBS volume type in a predictable manner.

A Provisioned IOPS SSD volume can range in size from 4 GB to 16 TB. When you provision a Provisioned IOPS SSD volume, you specify not just the size, but also the desired number of IOPS, up to the lower of the maximum of 30 times the number of GB of the volume, or 20,000 IOPS. You can stripe multiple volumes together in a RAID 0 configuration for larger size and greater performance. Amazon EBS delivers within 10 percent of the provisioned IOPS performance 99.9 percent of the time over a given year.

Pricing is based on the size of the volume and the amount of IOPS reserved. The cost per gigabyte is slightly more than that of general-purpose SSD volumes and is applied based on the size of the volume, not the amount of the volume used to store data. An additional monthly fee is applied based on the number of IOPS provisioned, whether they are consumed or not.

Provisioned IOPS SSD volumes provide predictable, high performance and are well suited for:

- Critical business applications that require sustained IOPS performance
- Large database workloads

Table 3.6 compares these Amazon EBS volume types.

TABLE 3.6 EBS Volume Type Comparison

Characteristic	General-Purpose SSD	Provisioned IOPS SSD	Magnetic
Use cases	• System boot volumes • Virtual desktops • Small-to-medium sized databases • Development and test environments	• Critical business applications that require sustained IOPS performance or more than 10,000 IOPS or 160MB of throughput per volume • Large database workloads	• Cold workloads where data is infrequently accessed • Scenarios where the lowest storage cost is important
Volume size	1 GiB–16TiB	4 GiB–16TiB	1 GiB–1TiB
Maximum throughput	160MB	320MB	40–90MB
IOPS performance	Baseline performance of 3 IOPS/GiB (up to 10,000 IOPS) with the ability to burst to 3,000 IOPS for volumes under 1,000 GiB	Consistently performs at provisioned level, up to 20,000 IOPS maximum	Averages 100 IOPS, with the ability to burst to hundreds of IOPS



At the time of this writing, AWS released two new HDD volume types: Throughput-Optimized HDD and Cold HDD. Over time, it is expected that these new types will eclipse the current magnetic volume type, fulfilling the needs of any workload requiring HDD performance.

Throughput-Optimized HDD volumes are low-cost HDD volumes designed for frequent-access, throughput-intensive workloads such as big data, data warehouses, and log processing. Volumes can be up to 16 TB with a maximum IOPS of 500 and maximum throughput of 500 MB/s. These volumes are significantly less expensive than general-purpose SSD volumes.

Cold HDD volumes are designed for less frequently accessed workloads, such as colder data requiring fewer scans per day. Volumes can be up to 16 TB with a maximum IOPS of 250 and maximum throughput of 250 MB/s. These volumes are significantly less expensive than Throughput-Optimized HDD volumes.

Amazon EBS-Optimized Instances

When using any volume type other than magnetic and Amazon EBS I/O is of consequence, it is important to use Amazon EBS-optimized instances to ensure that the Amazon EC2 instance is prepared to take advantage of the I/O of the Amazon EBS volume. An Amazon EBS-optimized instance uses an optimized configuration stack and provides additional, dedicated capacity for Amazon EBS I/O. This optimization provides the best performance for your Amazon EBS volumes by minimizing contention between Amazon

EBS I/O and other traffic from your instance. When you select Amazon EBS-optimized for an instance, you pay an additional hourly charge for that instance. Check the AWS documentation to confirm which instance types are available as Amazon EBS-optimized instance.

Protecting Data

Over the lifecycle of an Amazon EBS volume, there are several practices and services that you should know about when taking the exam.

Backup/Recovery (Snapshots)

You can back up the data on your Amazon EBS volumes, regardless of volume type, by taking point-in-time snapshots. Snapshots are incremental backups, which means that only the blocks on the device that have changed since your most recent snapshot are saved.

Taking Snapshots You can take snapshots in many ways:

- Through the AWS Management Console
- Through the CLI
- Through the API
- By setting up a schedule of regular snapshots

Data for the snapshot is stored using Amazon S3 technology. The action of taking a snapshot is free. You pay only the storage costs for the snapshot data.

When you request a snapshot, the point-in-time snapshot is created immediately and the volume may continue to be used, but the snapshot may remain in pending status until all the modified blocks have been transferred to Amazon S3.

It's important to know that while snapshots are stored using Amazon S3 technology, they are stored in AWS-controlled storage and not in your account's Amazon S3 buckets. This means you cannot manipulate them like other Amazon S3 objects. Rather, you must use the Amazon EBS snapshot features to manage them. Snapshots are constrained to the region in which they are created, meaning you can use them to create new volumes only in the same region. If you need to restore a snapshot in a different region, you can copy a snapshot to another region.

Creating a Volume from a Snapshot To use a snapshot, you create a new Amazon EBS volume from the snapshot. When you do this, the volume is created immediately but the data is loaded lazily. This means that the volume can be accessed upon creation, and if the data being requested has not yet been restored, it will be restored upon first request. Because of this, it is a best practice to initialize a volume created from a snapshot by accessing all the blocks in the volume.

Snapshots can also be used to increase the size of an Amazon EBS volume. To increase the size of an Amazon EBS volume, take a snapshot of the volume, then create a new volume of the desired size from the snapshot. Replace the original volume with the new volume.

Recovering Volumes

Because Amazon EBS volumes persist beyond the lifetime of an instance, it is possible to recover data if an instance fails. If an Amazon EBS-backed instance fails and there is data on the boot drive, it is relatively straightforward to detach the volume from the instance. Unless the `DeleteOnTermination` flag

for the volume has been set to false, the volume should be detached before the instance is terminated. The volume can then be attached as a data volume to another instance and the data read and recovered.

Encryption Options

Many workloads have requirements that data be encrypted at rest, either because of compliance regulations or internal corporate standards. Amazon EBS offers native encryption on all volume types.

When you launch an encrypted Amazon EBS volume, Amazon uses the *AWS Key Management Service (KMS)* to handle key management. A new master key will be created unless you select a master key that you created separately in the service. Your data and associated keys are encrypted using the industry-standard AES-256 algorithm. The encryption occurs on the servers that host Amazon EC2 instances, so the data is actually encrypted in transit between the host and the storage media and also on the media. (Consult the AWS documentation for a list of instance types that support Amazon EBS encryption.) Encryption is transparent, so all data access is the same as unencrypted volumes, and you can expect the same IOPS performance on encrypted volumes as you would with unencrypted volumes, with a minimal effect on latency. Snapshots that are taken from encrypted volumes are automatically encrypted, as are volumes that are created from encrypted snapshots.

Summary

Compute is the amount of computational power required to fulfill your workload. Amazon EC2 is the primary service for providing compute to customers.

The instance type defines the virtual hardware supporting the instance. Available instance types vary in vCPUs, memory, storage, and network performance to address nearly any workload.

An AMI defines the initial software state of the instance, both OS and applications. There are four sources of AMIs: AWS published generic OSs, partner-published AMIs in the AWS Marketplace with software packages preinstalled, customer-generated AMIs from existing Amazon EC2 instances, and uploaded AMIs from virtual servers.

Instances can be addressed by public DNS name, public IP address, or elastic IP address. To access a newly launched Linux instance, use the private half of the key pair to connect to the instance via SSH. To access a newly created Windows instance, use the private half of the key pair to decrypt the randomly initialized local administrator password.

Network traffic in and out of an instance can be controlled by a virtual firewall called a security group. A security group allows rules that block traffic based on direction, port, protocol, and source/destination address.

Bootstrapping allows you to run a script to initialize your instance with OS configurations and applications. This feature allows instances to configure themselves upon launch. Once an instance is launched, you can change its instance type or, for Amazon VPC instances, the security groups with which it is associated.

The three pricing options for instances are On-Demand, Reserved Instance, and Spot. On-Demand has the highest per hour cost, requiring no up-front commitment and giving you complete control over the lifetime of the instance. Reserved Instances require a commitment and provide a reduced overall cost over the lifetime of the reservation. Spot Instances are idle compute capacity that AWS makes available based on bid prices from customers. The savings on the per-hour cost can be significant, but instances can be shut down when the bid price exceeds the customer’s current bid.

Instance stores are block storage included with the hourly cost of the instance. The amount and type of storage available varies with the instance type. Instance stores terminate when the associated instance is stopped, so they should only be used for temporary data or in architectures providing redundancy such as Hadoop’s HDFS.

Amazon EBS provides durable block storage in several types. Magnetic has the lowest cost per gigabyte and delivers modest performance. General-purpose SSD is cost-effective storage that can provide up to 10,000 IOPS. Provisioned IOPS SSD has the highest cost per gigabyte and is well suited for I/O-intensive workloads sensitive to storage performance. Snapshots are incremental backups of Amazon EBS volumes stored in Amazon S3. Amazon EBS volumes can be encrypted.

Exam Essentials

Know the basics of launching an Amazon ec2 instance. To launch an instance, you must specify an AMI, which defines the software on the instance at launch, and an instance type, which defines the virtual hardware supporting the instance (memory, vCPUs, and so on).

Know what architectures are suited for what Amazon ec2 pricing options. Spot Instances are best suited for workloads that can accommodate interruption. Reserved Instances are best for consistent, long-term compute needs. On-Demand Instances provide flexible compute to respond to scaling needs.

Know how to combine multiple pricing options that result in cost optimization and scalability. On-Demand Instances can be used to scale up a web application running on Reserved Instances in response to a temporary traffic spike. For a workload with several Reserved Instances reading from a queue, it's possible to use Spot Instances to alleviate heavy traffic in a cost-effective way. These are just two of countless examples where a workload may use different pricing options.

Know the benefits of enhanced networking. Enhanced networking enables you to get significantly higher PPS performance, lower network jitter, and lower latencies.

Know the capabilities of vm import/export. VM Import/Export allows you to import existing VMs to AWS as Amazon EC2 instances or AMIs. Amazon EC2 instances that were imported through VM Import/Export can also be exported back to a virtual environment.

Know the methods for accessing an instance over the internet. You can access an Amazon EC2 instance over the web via public IP address, elastic IP address, or public DNS name. There are additional ways to access an instance within an Amazon VPC, including private IP addresses and ENIs.

Know the lifetime of an instance store. Data on an instance store is lost when the instance is stopped or terminated. Instance store data survives an OS reboot.

Know the properties of the Amazon EC2 pricing options. On-Demand Instances require no up-front commitment, can be launched any time, and are billed by the hour. Reserved Instances require an up-front commitment and vary in cost depending on whether they are paid all up front, partially up front, or not up front. Spot Instances are launched when your bid price exceeds the current spot price. Spot Instances will run until the spot price exceeds your bid price, in which case the instance will get a two-minute warning and terminate.

Know what determines network performance. Every instance type is rated for low, moderate, high, or 10 Gbps network performance, with larger instance types generally having higher ratings. Additionally, some instance types offer enhanced networking, which provides additional improvement in network performance.

Know what instance metadata is and how it's obtained. Metadata is information about an Amazon EC2 instance, such as instance ID, instance type, and security groups, that is available from within the instance. It can be obtained through an HTTP call to a specific IP address.

Know how security groups protect instances. Security groups are virtual firewalls controlling traffic in and out of your Amazon EC2 instances. They are deny by default, and you can allow traffic by adding rules specifying traffic direction, port, protocol, and destination address (via Classless Inter-Domain Routing [CIDR] block). They are applied at the instance level, meaning that traffic between instances in the same security group must adhere to the rules of that security group. They are stateful, meaning that an

outgoing rule will allow the response without a correlating incoming rule.

Know how to interpret the effect of security groups. When an instance is a member of multiple security groups, the effect is a union of all the rules in all the groups.

Know the different Amazon ebs volume types, their characteristics, and their appropriate workloads. Magnetic volumes provide an average performance of 100 IOPS and can be provisioned up to 1 TB. They are good for cold and infrequently accessed data. General-purpose SSD volumes provide three IOPS/GB up to 10,000 IOPS, with smaller volumes able to burst 3,000 IOPS. They can be provisioned up to 16 TB and are appropriate for dev/test environments, small databases, and so forth. Provisioned IOPS SSD can provide up to 20,000 consistent IOPS for volumes up to 16 TB. They are the best choice for workloads such as large databases executing many transactions.

Know how to encrypt an Amazon ebs volume. Any volume type can be encrypted at launch. Encryption is based on AWS KMS and is transparent to applications on the attached instances.

Understand the concept and process of snapshots. Snapshots provide a point-in-time backup of an Amazon EBS volume and are stored in Amazon S3. Subsequent snapshots are incremental—they only store deltas. When you request a snapshot, the point-in-time snapshot is created immediately and the volume may continue to be used, but the snapshot may remain in pending status until all the modified blocks have been transferred to Amazon S3. Snapshots may be copied between regions.

Know how Amazon ebs-optimized instances affect Amazon ebs performance. In addition to the IOPS that control the performance in and out of the Amazon EBS volume, use Amazon EBS-optimized instances to ensure additional, dedicated capacity for Amazon EBS I/O.

Exercises

For assistance in completing these exercises, refer to these user guides:

- ***Amazon EC2 (Linux)***—http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html
- ***Amazon EC2 (Windows)*** —http://docs.aws.amazon.com/AWSEC2/latest/WindowsGuide/concepts.html
- ***Amazon EBS***—http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AmazonEBS.html

EXERCISE 3.1

Launch and Connect to a Linux Instance

In this exercise, you will launch a new Linux instance, log in with SSH, and install any security updates.

1. Launch an instance in the Amazon EC2 console.
2. Choose the Amazon Linux AMI.
3. Choose the t2.medium instance type.
4. Launch the instance in either the default VPC or EC2-Classic.
5. Assign the instance a public IP address.
6. Add a tag to the instance of Key: Name, Value: **`Exercise 3.1`**.
7. Create a new security group called **`Cert Book`**.
8. Add a rule to Cert Book allowing SSH access from the IP address of your workstation (`www.WhatsMyIP.org` is a good way to determine your IP address).
9. Launch the instance.
10. When prompted for a key pair, choose a key pair you already have or create a new one and download the private portion.

    Amazon generates a `keyname.pem` file, and you will need a `keyname.ppk` file to connect to the instance via SSH. `Puttygen.exe` is one utility that will create a `.ppk` file from a `.pem` file.
11. SSH into the instance using the public IP address, the user name `ec2-user`, and the `keyname.ppk` file.
12. From the command-line prompt, run `sudo yum update–security -y`.
13. Close the SSH window and terminate the instance.

EXERCISE 3.2

Launch a Windows Instance with Bootstrapping

In this exercise, you will launch a Windows instance and specify a very simple bootstrap script. You will then confirm that the bootstrap script was executed on the instance.

1. Launch an instance in the Amazon EC2 console.
2. Choose the Microsoft Windows Server 2012 Base AMI.
3. Choose the t2.medium instance type.
4. Launch the instance in either the default VPC or EC2-Classic.
5. Assign the instance a public IP address.
6. In the Advanced Details section, enter the following text as UserData: `<script> md c:\temp </script>`
7. Add a tag to the instance of Key: Name, Value: ***Exercise 3.2***.
8. Use the Cert Book security group from Exercise 3.1.
9. Launch the instance.
10. Use the key pair from Exercise 3.1.
11. On the Connect Instance UI, decrypt the administrator password and then download the RDP file to attempt to connect to the instance. Your attempt should fail because the Cert Book security group does not allow RDP access.
12. Open the Cert Book security group and add a rule that allows RDP access from your IP address.
13. Attempt to access the instance via RDP again.
14. Once the RDP session is connected, open Windows Explorer and confirm that the `c:\temp` folder has been created.
15. End the RDP session and terminate the instance.

EXERCISE 3.3

Confirm That Instance Stores Are Lost When an Instance Is Stopped

In this exercise, you will observe that the data on an Amazon EC2 instance store is lost when the instance is stopped.

1. Launch an instance in the Amazon Management Console.
2. Choose the Microsoft Windows Server 2012 Base AMI.
3. Choose the m3.medium instance type.
4. Launch the instance in either the default VPC or EC2-Classic.
5. Assign the instance a public IP address.
6. Add a tag to the instance of Key: Name, Value: **`Exercise 3.3`**.
7. Use the Cert Book security group as updated in Exercise 3.2.
8. Launch the instance.
9. Use the key pair from Exercise 3.1.
10. Decrypt the administrator password login to the instance via RDP.
11. Once the RDP session is connected, open Windows Explorer.
12. Create a new folder named **`z:\temp`**.
13. Log out of the RDP session.
14. In the console, set the state of the instance to Stopped.
15. Once the instance is stopped, start it again.
16. Log back into the instance using RDP.
17. Open Windows Explorer and confirm that the `z:\temp` folder is gone.
18. End the RDP session and terminate the instance.

EXERCISE 3.4

Launch a Spot Instance

In this exercise, you will create a Spot Instance.

1. In the Amazon EC2 console, go to the Spot Request page.
2. Look at the pricing history for m3.medium, especially the recent price.
3. Make a note of the most recent price and Availability Zone.
4. Launch an instance in the Amazon EC2 console.
5. Choose the Amazon Linux AMI.
6. Choose the t2.medium instance type.
7. On the Configure Instance page, request a Spot Instance.
8. Launch the instance in either the Default VPC or EC2-Classic. (Note the Default VPC will define the Availability Zone for the instance.)
9. Assign the instance a public IP address.
10. Request a Spot Instance and enter a bid a few cents above the recorded Spot price.
11. Finish launching the instance.
12. Go back to the Spot Request page.

    Watch your request. If your bid was high enough, you should see it change to Active and an instance ID appear.
13. Find the instance on the instances page of the Amazon EC2 console.

    Note the Lifecycle field in the Description that says Spot.
14. Once the instance is running, terminate it.

EXERCISE 3.5

Access Metadata

In this exercise, you will access the instance metadata from the OS.

1. Launch an instance in the Amazon EC2 console.
2. Choose the Amazon Linux AMI.
3. Choose the t2.medium instance type.
4. Launch the instance in either the default VPC or EC2-Classic.
5. Assign the instance a public IP address.
6. Add a tag to the instance of Key: Name, Value: **`Exercise 3.5`**.
7. Use the Cert Book security group.
8. Launch the instance.
9. Use the key pair from Exercise 3.1.
10. Connect the instance via SSH using the public IP address, the user name `ec2-user`, and the `keyname.ppk` file.
11. At the Linux command prompt, retrieve a list of the available metadata by typing: curl http://169.254.169.254/latest/meta-data/
12. To see a value, add the name to the end of the URL. For example, to see the security groups, type: curl http://169.254.169.254/latest/meta-data/security-groups
13. Try other values as well. Names that end with a / indicate a longer list of sub-values.
14. Close the SSH window and terminate the instance.

EXERCISE 3.6

Create an Amazon EBS Volume and Show That It Remains After the Instance Is Terminated

In this exercise, you will see how an Amazon EBS volume persists beyond the life of an instance.

1. Launch an instance in the Amazon EC2 console.
2. Choose the Amazon Linux AMI.
3. Choose the t2.medium instance type.
4. Launch the instance in either the default VPC or EC2-Classic.
5. Assign the instance a public IP address.
6. Add a second Amazon EBS volume of size 50 GB. Note that the Root Volume is set to Delete on Termination.
7. Add a tag to the instance of Key: Name, Value: **`Exercise 3.6`**.
8. Use the Cert Book security group from earlier exercises.
9. Launch the instance.
10. Find the two Amazon EBS volumes on the Amazon EBS console. Name them both **`Exercise 3.6`**.
11. Terminate the instance.

Notice that the boot drive is destroyed, but the additional Amazon EBS volume remains and now says Available. Do not delete the Available volume.

EXERCISE 3.7

Take a Snapshot and Restore

This exercise guides you through taking a snapshot and restoring it in three different ways.

1. Find the volume you created in Exercise 3.6 in the Amazon EBS console.
2. Take a snapshot of that volume. Name the snapshot **`Exercise 3.7`**.
3. On the snapshot console, wait for the snapshot to be completed. (As the volume was empty, this should be very quick.)
4. On the snapshot page in the AWS Management Console, choose the new snapshot and select Create Volume.
5. Create the volume with all the defaults.
6. Locate the snapshot again and again choose Create Volume, setting the size of the new volume to 100 GB (taking a snapshot and restoring the snapshot to a new, larger volume is how you address the problem of increasing the size of an existing volume). Locate the snapshot again and choose Copy. Copy the snapshot to another region. Make the description **`Exercise 3.7`**.
7. Go to the other region and wait for the snapshot to become available.
8. Create a volume from the snapshot in the new region. This is how you share an Amazon EBS volume between regions; that is, by taking a snapshot and copying the snapshot.
9. Delete all four volumes.

EXERCISE 3.8

Launch an Encrypted Volume

In this exercise, you will launch an Amazon EC2 instance with an encrypted Amazon EBS volume and store some data on it to confirm that the encryption is transparent to the instance itself.

1. Launch an instance in the Amazon EC2 console.
2. Choose the Microsoft Windows Server 2012 Base AMI.
3. Choose the m3.medium instance type.
4. Launch the instance in either the default VPC or EC2-Classic.
5. Assign the instance a public IP address.
6. On the storage page, add a 50 GB encrypted Amazon EBS volume.
7. Add a tag to the instance of Key: Name, Value: **`Exercise 3.8`**.
8. Use the Cert Book security group as updated in Exercise 3.2.
9. Launch the instance.
10. Choose the key pair from Exercise 3.1.
11. Decrypt the administrator password and log in to the instance using RDP.
12. Once the RDP session is connected, open Notepad.
13. Type some random information into Notepad, save it at `d:\testfile.txt`, and then close Notepad.
14. Find `d:\testfile.txt` in Windows Explorer and open it with Notepad. Confirm that the data is not encrypted in Notepad.
15. Log out.
16. Terminate the instance.

EXERCISE 3.9

Detach a Boot Drive and Reattach to Another Instance

In this exercise, you will practice removing an Amazon EBS volume from a stopped drive and attaching to another instance to recover the data.

1. Launch an instance in the Amazon EC2 console.
2. Choose the Microsoft Windows Server 2012 Base AMI.
3. Choose the t2.medium instance type.
4. Launch the instance in either the default VPC or EC2-Classic.
5. Assign the instance a public IP address.
6. Add a tag to the instance of Key: Name, Value: **`Exercise 3.9 Source`**.
7. Use the Cert Book security group from earlier exercises.
8. Launch the instance with the key pair from Exercise 3.1.
9. Launch a second instance in the Amazon EC2 Console.
10. Choose the Microsoft Windows Server 2012 Base AMI.
11. Choose the t2.medium instance type.
12. Launch the instance in either the default VPC or EC2-Classic.
13. Assign the instance a public IP address.
14. Add a tag to the instance of Key: Name, Value: **`Exercise 3.9 Destination`**.
15. Use the Cert Book security group from earlier exercises.
16. Launch the instance with the key pair you used in Exercise 3.1.
17. Once both instances are running, stop the first instance (Source). Make a note of the instance ID.
18. Go to the Amazon EBS page in the Amazon EC2 console and find the volume attached to the Source instance via the instance ID. Detach the instance.
19. When the volume becomes Available, attach the instance to the second instance (Destination).
20. Log in to the Destination instance via RDP using the administrator account.
21. Open a command window (`cmd.exe`).
22. At the command prompt, type the following commands: `C:\Users\Administrator >diskpart DISKPART>select disk 1 DISKPART>online disk DISKPART>exit C:\Users\Administrator>dir e:`

    The volume removed from the stopped source drive can now be read as the E: drive on the destination instance, so its data can be retrieved.
23. Terminate all the instances and ensure the volumes are deleted in the process.

Review Questions

1. Your web application needs four instances to support steady traffic nearly all of the time. On the last day of each month, the traffic triples. What is a cost-effective way to handle this traffic pattern?
   A. Run 12 Reserved Instances all of the time.
   B. Run four On-Demand Instances constantly, then add eight more On-Demand Instances on the last day of each month.
   C. Run four Reserved Instances constantly, then add eight On-Demand Instances on the last day of each month.
   D. Run four On-Demand Instances constantly, then add eight Reserved Instances on the last day of each month.
2. Your order-processing application processes orders extracted from a queue with two Reserved Instances processing 10 orders/minute. If an order fails during processing, then it is returned to the queue without penalty. Due to a weekend sale, the queues have several hundred orders backed up. While the backup is not catastrophic, you would like to drain it so that customers get their confirmation emails faster. What is a cost-effective way to drain the queue for orders?
   A. Create more queues.
   B. Deploy additional Spot Instances to assist in processing the orders.
   C. Deploy additional Reserved Instances to assist in processing the orders.
   D. Deploy additional On-Demand Instances to assist in processing the orders.
3. Which of the following must be specified when launching a new Amazon Elastic Compute Cloud (Amazon EC2) Windows instance? (Choose 2 answers)
   A. The Amazon EC2 instance ID
   B. Password for the administrator account
   C. Amazon EC2 instance type
   D. Amazon Machine Image (AMI)
4. You have purchased an m3.xlarge Linux Reserved instance in us-east-1a. In which ways can you modify this reservation? (Choose 2 answers)
   A. Change it into two m3.large instances.
   B. Change it to a Windows instance.
   C. Move it to us-east-1b.
   D. Change it to an m4.xlarge.
5. Your instance is associated with two security groups. The first allows Remote Desktop Protocol (RDP) access over port 3389 from Classless Inter-Domain Routing (CIDR) block 72.14.0.0/16. The second allows HTTP access over port 80 from CIDR block 0.0.0.0/0. What traffic can reach your instance?

A. RDP and HTTP access from CIDR block 0.0.0.0/0

B. No traffic is allowed.

C. RDP and HTTP traffic from 72.14.0.0/16

D. RDP traffic over port 3389 from 72.14.0.0/16 and HTTP traffic over port 80 from 0.0.00/0

6. Which of the following are features of enhanced networking? (Choose 3 answers)

A. More Packets Per Second (PPS)

B. Lower latency

C. Multiple network interfaces

D. Border Gateway Protocol (BGP) routing

E. Less jitter

7. You are creating a High-Performance Computing (HPC) cluster and need very low latency and high bandwidth between instances. What combination of the following will allow this? (Choose 3 answers)

A. Use an instance type with 10 Gbps network performance.

B. Put the instances in a placement group.

C. Use Dedicated Instances.

D. Enable enhanced networking on the instances.

E. Use Reserved Instances.

8. Which Amazon Elastic Compute Cloud (Amazon EC2) feature ensures that your instances will not share a physical host with instances from any other AWS customer?

A. Amazon Virtual Private Cloud (VPC)

B. Placement groups

C. Dedicated Instances

D. Reserved Instances

9. Which of the following are true of instance stores? (Choose 2 answers)

A. Automatic backups

B. Data is lost when the instance stops.

C. Very high IOPS

D. Charge is based on the total amount of storage provisioned.

10. Which of the following are features of Amazon Elastic Block Store (Amazon EBS)? (Choose 2 answers)

A. Data stored on Amazon EBS is automatically replicated within an Availability Zone.

B. Amazon EBS data is automatically backed up to tape.

C. Amazon EBS volumes can be encrypted transparently to workloads on the attached instance.

D. Data on an Amazon EBS volume is lost when the attached instance is stopped.

11. You need to take a snapshot of an Amazon Elastic Block Store (Amazon EBS) volume. How long will the volume be unavailable?
    A. It depends on the provisioned size of the volume.
    B. The volume will be available immediately.
    C. It depends on the amount of data stored on the volume.
    D. It depends on whether the attached instance is an Amazon EBS-optimized instance.
12. You are restoring an Amazon Elastic Block Store (Amazon EBS) volume from a snapshot. How long will it be before the data is available?
    A. It depends on the provisioned size of the volume.
    B. The data will be available immediately.
    C. It depends on the amount of data stored on the volume.
    D. It depends on whether the attached instance is an Amazon EBS-optimized instance.
13. You have a workload that requires 15,000 consistent IOPS for data that must be durable. What combination of the following steps do you need? (Choose 2 answers)
    A. Use an Amazon Elastic Block Store (Amazon EBS)-optimized instance.
    B. Use an instance store.
    C. Use a Provisioned IOPS SSD volume.
    D. Use a magnetic volume.
14. Which of the following can be accomplished through bootstrapping?
    A. Install the most current security updates.
    B. Install the current version of the application.
    C. Configure Operating System (OS) services.
    D. All of the above.
15. How can you connect to a new Linux instance using SSH?
    A. Decrypt the root password.
    B. Using a certificate
    C. Using the private half of the instance's key pair
    D. Using Multi-Factor Authentication (MFA)
16. VM Import/Export can import existing virtual machines as: (Choose 2 answers)
    A. Amazon Elastic Block Store (Amazon EBS) volumes
    B. Amazon Elastic Compute Cloud (Amazon EC2) instances
    C. Amazon Machine Images (AMIs)
    D. Security groups

17. Which of the following can be used to address an Amazon Elastic Compute Cloud (Amazon EC2) instance over the web? (Choose 2 answers)

    A. Windows machine name

    B. Public DNS name

    C. Amazon EC2 instance ID

    D. Elastic IP address

18. Using the correctly decrypted Administrator password and RDP, you cannot log in to a Windows instance you just launched. Which of the following is a possible reason?

    A. There is no security group rule that allows RDP access over port 3389 from your IP address.

    B. The instance is a Reserved Instance.

    C. The instance is not using enhanced networking.

    D. The instance is not an Amazon EBS-optimized instance.

19. You have a workload that requires 1 TB of durable block storage at 1,500 IOPS during normal use. Every night there is an Extract, Transform, Load (ETL) task that requires 3,000 IOPS for 15 minutes. What is the most appropriate volume type for this workload?

    A. Use a Provisioned IOPS SSD volume at 3,000 IOPS.

    B. Use an instance store.

    C. Use a general-purpose SSD volume.

    D. Use a magnetic volume.

20. How are you billed for elastic IP addresses?

    A. Hourly when they are associated with an instance

    B. Hourly when they are not associated with an instance

    C. Based on the data that flows through them

    D. Based on the instance type to which they are attached

Chapter 4
Amazon Virtual Private Cloud (Amazon VPC)

THE AWS CERTIFIED SOLUTIONS ARCHITECT ASSOCIATE EXAM OBJECTIVES COVERED IN THIS CHAPTER MAY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

Domain 1.0: Designing highly available, cost-efficient, fault-tolerant, scalable systems

✓1.1 Identify and recognize cloud architecture considerations, such as fundamental components and effective designs.

Content may include the following:

- How to design cloud services
- Planning and design
- Familiarity with:
  - Best practices for AWS architecture
  - Architectural trade-off decisions (for example, high availability vs. cost, Amazon Relational Database Service [RDS] vs. installing your own database on Amazon Elastic Compute Cloud—EC2)
  - Hybrid IT architectures (for example, Direct Connect, Storage Gateway, VPC, Directory Services)

Domain 2.0: Implementation/Deployment

✓2.1 Identify the appropriate techniques and methods using Amazon EC2, Amazon S3, AWS Elastic Beanstalk, AWS CloudFormation, AWS OpsWorks, Amazon Virtual Private Cloud (VPC), and AWS Identity and Access Management (IAM) to code and implement a cloud solution.

Content may include the following:

- Operate and extend service management in a hybrid IT architecture
- Configure services to support compliance requirements in the cloud

Domain 3.0: Data Security

✓3.1 Recognize and implement secure practices for optimum cloud deployment and maintenance.

Content may include the following:

- AWS security attributes (customer workloads down to the physical layer)

- Amazon Virtual Private Cloud (VPC)
- Ingress vs. egress filtering, and which AWS services and features fit
- “Core” Amazon EC2 and S3 security feature sets
- Incorporating common conventional security products (Firewall and VPNs)
- Complex access controls (building sophisticated security groups, ACLs, and so on)



Introduction

The *Amazon Virtual Private Cloud (Amazon VPC)* is a custom-defined virtual network within the AWS Cloud. You can provision your own logically isolated section of AWS, similar to designing and implementing a separate independent network that would operate in an on-premises data center. This chapter explores the core components of Amazon VPC and, in the exercises, you learn how to build your own Amazon VPC in the cloud. A strong understanding of Amazon VPC topology and troubleshooting is required to pass the exam, and we highly recommend that you complete the exercises in this chapter.

Amazon Virtual Private Cloud (Amazon VPC)

Amazon VPC is the networking layer for Amazon Elastic Compute Cloud (Amazon EC2), and it allows you to build your own virtual network within AWS. You control various aspects of your Amazon VPC, including selecting your own IP address range; creating your own *subnets*; and configuring your own route tables, network gateways, and security settings. Within a region, you can create multiple Amazon VPCs, and each Amazon VPC is logically isolated even if it shares its IP address space.

When you create an Amazon VPC, you must specify the IPv4 address range by choosing a *Classless Inter-Domain Routing (CIDR)* block, such as 10.0.0.0/16. The address range of the Amazon VPC cannot be changed after the Amazon VPC is created. An Amazon VPC address range may be as large as /16 (65,536 available addresses) or as small as /28 (16 available addresses) and should not overlap any other network with which they are to be connected.

The Amazon VPC service was released after the Amazon EC2 service; because of this, there are two different networking platforms available within AWS: EC2-Classic and EC2-VPC. Amazon EC2 originally launched with a single, flat network shared with other AWS customers called EC2-Classic. As such, AWS accounts created prior to the arrival of the Amazon VPC service can launch instances into the EC2-Classic network and EC2-VPC. AWS accounts created after December 2013 only support launching instances using EC2-VPC. AWS accounts that support EC2-VPC will have a default VPC created in each region with a default subnet created in each Availability Zone. The assigned CIDR block of the VPC will be 172.31.0.0/16.

Figure 4.1 illustrates an Amazon VPC with an address space of 10.0.0.0/16, two subnets with different address ranges (10.0.0.0/24 and 10.0.1.0/24) placed in different Availability Zones, and a route table with the local route specified.

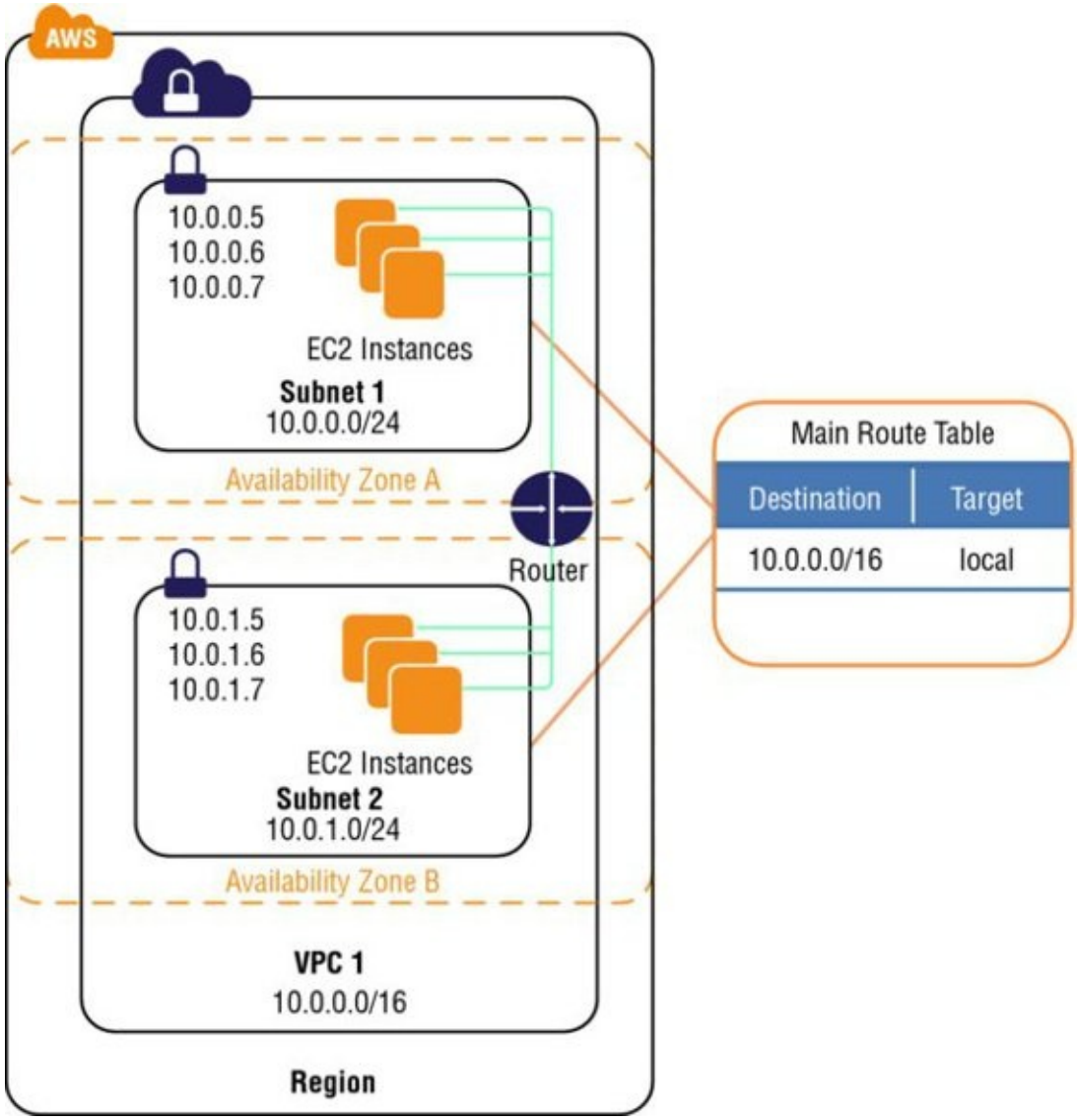


FIGURE 4.1 VPC, subnets, and a route table

An Amazon VPC consists of the following components:

- Subnets
- Route tables
- Dynamic Host Configuration Protocol (DHCP) option sets
- Security groups
- Network Access Control Lists (ACLs)

An Amazon VPC has the following optional components:

- Internet Gateways (IGWs)
- Elastic IP (EIP) addresses
- Elastic Network Interfaces (ENIs)
- Endpoints
- Peering
- Network Address Translation (NATs) instances and NAT gateways

- Virtual Private Gateway (VPG), Customer Gateways (CGWs), and Virtual Private Networks (VPNs)

Subnets

A *subnet* is a segment of an Amazon VPC's IP address range where you can launch Amazon EC2 instances, Amazon Relational Database Service (Amazon RDS) databases, and other AWS resources. CIDR blocks define subnets (for example, 10.0.1.0/24 and 192.168.0.0/24). The smallest subnet that you can create is a /28 (16 IP addresses). AWS reserves the first four IP addresses and the last IP address of every subnet for internal networking purposes. For example, a subnet defined as a /28 has 16 available IP addresses; subtract the 5 IPs needed by AWS to yield 11 IP addresses for your use within the subnet.

After creating an Amazon VPC, you can add one or more subnets in each Availability Zone. Subnets reside within one Availability Zone and cannot span zones. This is an important point that can come up in the exam, so remember that one subnet equals one Availability Zone. You can, however, have multiple subnets in one Availability Zone.

Subnets can be classified as public, private, or VPN-only. A public subnet is one in which the associated route table (discussed later) directs the subnet's traffic to the Amazon VPC's IGW (also discussed later). A private subnet is one in which the associated route table does not direct the subnet's traffic to the Amazon VPC's IGW. A VPN-only subnet is one in which the associated route table directs the subnet's traffic to the Amazon VPC's VPG (discussed later) and does not have a route to the IGW. Regardless of the type of subnet, the internal IP address range of the subnet is always private (that is, non-routable on the Internet).

Default Amazon VPCs contain one public subnet in every Availability Zone within the region, with a netmask of /20.

Route Tables

A *route table* is a logical construct within an Amazon VPC that contains a set of rules (called routes) that are applied to the subnet and used to determine where network traffic is directed. A route table's routes are what permit Amazon EC2 instances within different subnets within an Amazon VPC to communicate with each other. You can modify route tables and add your own custom routes. You can also use route tables to specify which subnets are public (by directing Internet traffic to the IGW) and which subnets are private (by not having a route that directs traffic to the IGW).

Each route table contains a default route called the local route, which enables communication within the Amazon VPC, and this route cannot be modified or removed. Additional routes can be added to direct traffic to exit the Amazon VPC via the IGW (discussed later), the VPG (discussed later), or the NAT instance (discussed later). In the exercises at the end of this chapter, you can practice how this is accomplished.

You should remember the following points about route tables:

- Your VPC has an implicit router.
- Your VPC automatically comes with a main route table that you can modify.
- You can create additional custom route tables for your VPC.
- Each subnet must be associated with a route table, which controls the routing for the subnet. If you don't explicitly associate a subnet with a particular route table, the subnet uses the main route table.
- You can replace the main route table with a custom table that you've created so that each new subnet is automatically associated with it.
- Each route in a table specifies a destination CIDR and a target; for example, traffic destined for 172.16.0.0/12 is targeted for the VPG. AWS uses the most specific route that matches the traffic to determine how to route the traffic.

Internet Gateways

An *Internet Gateway (IGW)* is a horizontally scaled, redundant, and highly available Amazon VPC component that allows communication between instances in your Amazon VPC and the Internet. An IGW provides a target in your Amazon VPC route tables for Internet-routable traffic, and it performs network address translation for instances that have been assigned public IP addresses.

Amazon EC2 instances within an Amazon VPC are only aware of their private IP addresses. When traffic is sent from the instance to the Internet, the IGW translates the reply address to the instance's public IP address (or EIP address, covered later) and maintains the one-to-one map of the instance private IP address and public IP address. When an instance receives traffic from the Internet, the IGW translates the destination address (public IP address) to the instance's private IP address and forwards the traffic to the Amazon VPC.

You must do the following to create a public subnet with Internet access:

- Attach an IGW to your Amazon VPC.
- Create a subnet route table rule to send all non-local traffic (0.0.0.0/0) to the IGW.
- Configure your network ACLs and security group rules to allow relevant traffic to flow to and from your instance.

You must do the following to enable an Amazon EC2 instance to send and receive traffic from the Internet:

- Assign a public IP address or EIP address.

You can scope the route to all destinations not explicitly known to the route table (0.0.0.0/0), or you can scope the route to a narrower range of IP addresses, such as the public IP addresses of your company's public endpoints outside of AWS or the EIP addresses of other Amazon EC2 instances outside your Amazon VPC.

Figure 4.2 illustrates an Amazon VPC with an address space of 10.0.0.0/16, one subnet with an address range of 10.0.0.0/24, a route table, an attached IGW, and a single Amazon EC2 instance with a private IP address and an EIP address. The route table contains two routes: the local route that permits inter-VPC communication and a route that sends all non-local traffic to the IGW (`igw-id`). Note that the Amazon EC2 instance has a public IP address (EIP = 198.51.100.2); this instance can be accessed from the Internet, and traffic may originate and return to this instance.

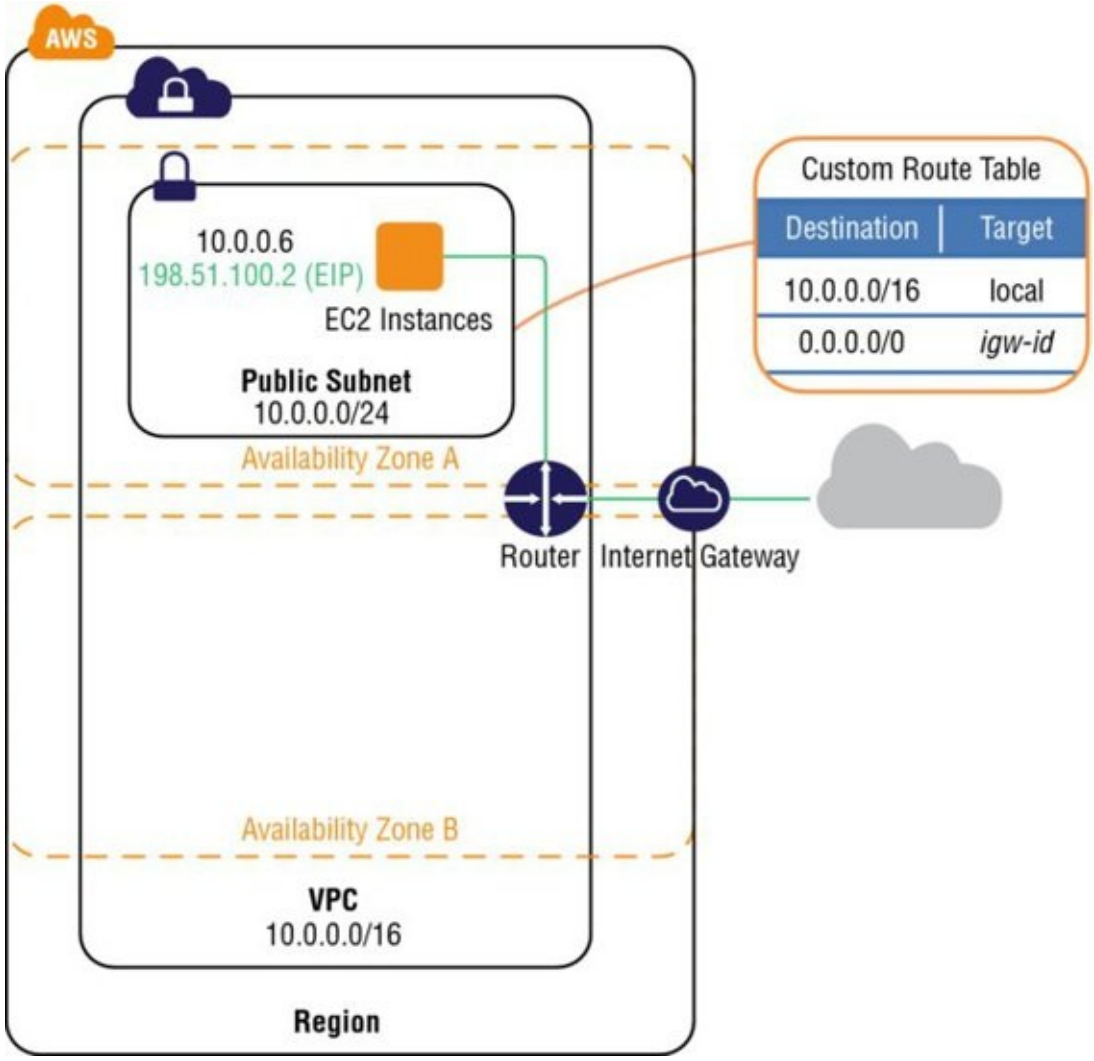


FIGURE 4.2 VPC, subnet, route table, and an Internet gateway

Dynamic Host Configuration Protocol (DHCP) Option Sets

Dynamic Host Configuration Protocol (DHCP) provides a standard for passing configuration information to hosts on a TCP/IP network. The options field of a DHCP message contains the configuration parameters. Some of those parameters are the domain name, domain name server, and the `netbios-node-type`.

AWS automatically creates and associates a DHCP option set for your Amazon VPC upon creation and sets two options: domain-name-servers (defaulted to AmazonProvidedDNS) and domain-name (defaulted to the domain name for your region). AmazonProvidedDNS is an Amazon Domain Name System (DNS) server, and this option enables DNS for instances that need to communicate over the Amazon VPC's IGW.

The DHCP option sets element of an Amazon VPC allows you to direct Amazon EC2 host name assignments to your own resources. To assign your own domain name to your instances, create a custom DHCP option set and assign it to your Amazon VPC. You can configure the following values within a DHCP option set:

- ***domain-name-servers***—The IP addresses of up to four domain name servers, separated by commas. The default is AmazonProvidedDNS.
- ***domain-name***—Specify the desired domain name here (for example, `mycompany.com`).
- ***ntp-servers***—The IP addresses of up to four Network Time Protocol (NTP) servers, separated by commas
- ***netbios-name-servers***—The IP addresses of up to four NetBIOS name servers, separated by commas
- ***netbios-node-type***—Set this value to 2.

Every Amazon VPC must have only one DHCP option set assigned to it.

Elastic IP Addresses (EIPs)

AWS maintains a pool of public IP addresses in each region and makes them available for you to associate to resources within your Amazon VPCs. An *Elastic IP Addresses (EIP)* is a static, public IP address in the pool for the region that you can allocate to your account (pull from the pool) and release (return to the pool). EIPs allow you to maintain a set of IP addresses that remain fixed while the underlying infrastructure may change over time. Here are the important points to understand about EIPs for the exam:

- You must first allocate an EIP for use within a VPC and then assign it to an instance.
- EIPs are specific to a region (that is, an EIP in one region cannot be assigned to an instance within an Amazon VPC in a different region).
- There is a one-to-one relationship between network interfaces and EIPs.
- You can move EIPs from one instance to another, either in the same Amazon VPC or a different Amazon VPC within the same region.
- EIPs remain associated with your AWS account until you explicitly release them.
- There are charges for EIPs allocated to your account, even when they are not associated with a resource.

Elastic Network Interfaces (ENIs)

An *Elastic Network Interface (ENI)* is a virtual network interface that you can attach to an instance in an Amazon VPC. ENIs are only available within an Amazon VPC, and they are associated with a subnet upon creation. They can have one public IP address and multiple private IP addresses. If there are multiple private IP addresses, one of them is primary. Assigning a second network interface to an instance via an ENI allows it to be dual-homed (have network presence in different subnets). An ENI created independently of a particular instance persists regardless of the lifetime of any instance to which it is attached; if an underlying instance fails, the IP address may be preserved by attaching the ENI to a replacement instance.

ENIs allow you to create a management network, use network and security appliances in your Amazon VPC, create dual-homed instances with workloads/roles on distinct subnets, or create a low-budget, high-availability solution.

Endpoints

An Amazon VPC *endpoint* enables you to create a private connection between your Amazon VPC and another AWS service without requiring access over the Internet or through a NAT instance, VPN connection, or AWS Direct Connect. You can create multiple endpoints for a single service, and you can use different route tables to enforce different access policies from different subnets to the same service.

Amazon VPC endpoints currently support communication with Amazon Simple Storage Service (Amazon S3), and other services are expected to be added in the future.

You must do the following to create an Amazon VPC endpoint:

- Specify the Amazon VPC.
- Specify the service. A service is identified by a prefix list of the form `com.amazonaws.<region>.<service>`.
- Specify the policy. You can allow full access or create a custom policy. This policy can be changed at any time.
- Specify the route tables. A route will be added to each specified route table, which will state the service as the destination and the endpoint as the target.

Table 4.1 is an example route table that has an existing route that directs all Internet traffic (0.0.0.0/0) to an IGW. Any traffic from the subnet that is destined for another AWS service (for example, Amazon S3 or Amazon DynamoDB) will be sent to the IGW in order to reach that service.

TABLE 4.1 Route Table with an IGW Routing Rule

Destination	Target
`10.0.0.0/16`	`Local`
`0.0.0.0/0`	`igw-1a2b3c4d`

Table 4.2 is an example route table that has existing routes directing all Internet traffic to an IGW and all Amazon S3 traffic to the Amazon VPC endpoint.

TABLE 4.2 Route Table with an IGW Routing Rule and VPC Endpoint Rule

Destination	Target
`10.0.0.0/16`	`Local`
`0.0.0.0/0`	`igw-1a2b3c4d`
`pl-1a2b3c4d`	`vpce-11bb22cc`

The route table depicted in Table 4.2 will direct any traffic from the subnet that's destined for Amazon S3 in the same region to the endpoint. All other Internet traffic goes to your IGW, including traffic that's destined for other services and for Amazon S3 in other regions.

Peering

An Amazon VPC *peering* connection is a networking connection between two Amazon VPCs that enables instances in either Amazon VPC to communicate with each other as if they are within the same network. You can create an Amazon VPC peering connection between your own Amazon VPCs or with an Amazon VPC in another AWS account within a single region. A peering connection is neither a gateway nor an Amazon VPN connection and does not introduce a single point of failure for communication.

Peering connections are created through a request/accept protocol. The owner of the requesting Amazon VPC sends a request to peer to the owner of the peer Amazon VPC. If the peer Amazon VPC is within the same account, it is identified by its VPC ID. If the peer VPC is within a different account, it is identified by Account ID and VPC ID. The owner of the peer Amazon VPC has one week to accept or reject the request to peer with the requesting Amazon VPC before the peering request expires.

An Amazon VPC may have multiple peering connections, and peering is a one-to-one relationship between Amazon VPCs, meaning two Amazon VPCs cannot have two peering agreements between them. Also, peering connections do not support transitive routing. Figure 4.3 depicts transitive routing.

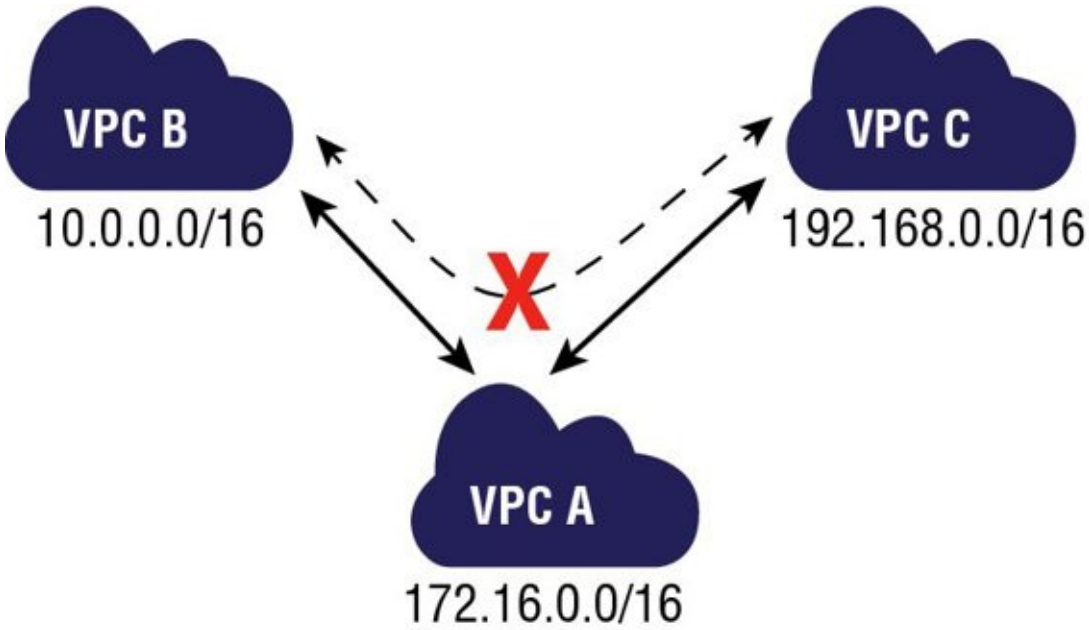


FIGURE 4.3 VPC peering connections do not support transitive routing

In Figure 4.3, VPC A has two peering connections with two different VPCs: VPC B and VPC C. Therefore, VPC A can communicate directly with VPCs B and C. Because peering connections do not support transitive routing, VPC A cannot be a transit point for traffic between VPCs B and C. In order for VPCs B and C to communicate with each other, a peering connection must be explicitly created between them.

Here are the important points to understand about peering for the exam:

- You cannot create a peering connection between Amazon VPCs that have matching or overlapping CIDR blocks.
- You cannot create a peering connection between Amazon VPCs in different regions.
- Amazon VPC peering connections do not support transitive routing.
- You cannot have more than one peering connection between the same two Amazon VPCs at the same

time.

Security Groups

A *security group* is a virtual stateful firewall that controls inbound and outbound network traffic to AWS resources and Amazon EC2 instances. All Amazon EC2 instances must be launched into a security group. If a security group is not specified at launch, then the instance will be launched into the default security group for the Amazon VPC. The default security group allows communication between all resources within the security group, allows all outbound traffic, and denies all other traffic. You may change the rules for the default security group, but you may not delete the default security group. Table 4.3 describes the settings of the default security group.

TABLE 4.3 Security Group Rules

Inbound			
Source	**Protocol**	**Port Range**	**Comments**
`sg-xxxxxxxx`	All	All	Allow inbound traffic from instances within the same security group.
Outbound			
Destination	**Protocol**	**Port Range**	**Comments**
`0.0.0.0/0`	All	All	Allow all outbound traffic.

For each security group, you add rules that control the inbound traffic to instances and a separate set of rules that control the outbound traffic. For example, Table 4.4 describes a security group for web servers.

TABLE 4.4 Security Group Rules for a Web Server

Inbound			
Source	**Protocol**	**Port Range**	**Comments**
`0.0.0.0/0`	TCP	80	Allow inbound traffic from the Internet to port 80.
Your network's public IP address range	TCP	22	Allow Secure Shell (SSH) traffic from your company network.
Your network's public IP address range	TCP	3389	Allow Remote Desktop Protocol (RDP) traffic from your company network.
Outbound			
Destination	**Protocol**	**Port Range**	**Comments**
The ID of the security group for your MySQL database servers	TCP	3306	Allow outbound MySQL access to instances in the specified security group.
The ID of the security group for your Microsoft SQL Server database servers	TCP	1433	Allow outbound Microsoft SQL Server access to instances in the specified security group.

Here are the important points to understand about security groups for the exam:

- You can create up to 500 security groups for each Amazon VPC.
- You can add up to 50 inbound and 50 outbound rules to each security group. If you need to apply more than 100 rules to an instance, you can associate up to five security groups with each network interface.
- You can specify allow rules, but not deny rules. This is an important difference between security groups and ACLs.
- You can specify separate rules for inbound and outbound traffic.
- By default, no inbound traffic is allowed until you add inbound rules to the security group.
- By default, new security groups have an outbound rule that allows all outbound traffic. You can remove the rule and add outbound rules that allow specific outbound traffic only.
- Security groups are stateful. This means that responses to allowed inbound traffic are allowed to flow outbound regardless of outbound rules and vice versa. This is an important difference between security groups and network ACLs.
- Instances associated with the same security group can't talk to each other unless you add rules allowing it (with the exception being the default security group).
- You can change the security groups with which an instance is associated after launch, and the changes will take effect immediately.

Network Access Control Lists (ACLs)

A network *access control list (ACL)* is another layer of security that acts as a stateless firewall on a subnet level. A network ACL is a numbered list of rules that AWS evaluates in order, starting with the lowest numbered rule, to determine whether traffic is allowed in or out of any subnet associated with the network ACL. Amazon VPCs are created with a modifiable default network ACL associated with every subnet that allows all inbound and outbound traffic. When you create a custom network ACL, its initial configuration will deny all inbound and outbound traffic until you create rules that allow otherwise. You may set up network ACLs with rules similar to your security groups in order to add a layer of security to your Amazon VPC, or you may choose to use the default network ACL that does not filter traffic traversing the subnet boundary. Overall, every subnet must be associated with a network ACL.

Table 4.5 explains the differences between a security group and a network ACL. You should remember the following differences between security groups and network ACLs for the exam.

TABLE 4.5 Comparison of Security Groups and Network ACLs

Security Group	Network ACL
Operates at the instance level (first layer of defense)	Operates at the subnet level (second layer of defense)
Supports allow rules only	Supports allow rules and deny rules
Stateful: Return traffic is automatically allowed, regardless of any rules	Stateless: Return traffic must be explicitly allowed by rules.
AWS evaluates all rules before deciding whether to allow traffic	AWS processes rules in number order when deciding whether to allow traffic.
Applied selectively to individual instances	Automatically applied to all instances in the associated subnets; this is a backup layer of defense, so you don’t have to rely on someone specifying the security group.

Network Address Translation (NAT) Instances and NAT Gateways

By default, any instance that you launch into a private subnet in an Amazon VPC is not able to communicate with the Internet through the IGW. This is problematic if the instances within private subnets need direct access to the Internet from the Amazon VPC in order to apply security updates, download patches, or update application software. AWS provides NAT instances and NAT gateways to allow instances deployed in private subnets to gain Internet access. For common use cases, we recommend that you use a NAT gateway instead of a NAT instance. The NAT gateway provides better availability and higher bandwidth, and requires less administrative effort than NAT instances.

NAT Instance

A *network address translation (NAT) instance* is an Amazon Linux Amazon Machine Image (AMI) that is designed to accept traffic from instances within a private subnet, translate the source IP address to the public IP address of the NAT instance, and forward the traffic to the IGW. In addition, the NAT instance maintains the state of the forwarded traffic in order to return response traffic from the Internet to the proper instance in the private subnet. These instances have the string `amzn-ami-vpc-nat` in their names, which is searchable in the Amazon EC2 console.

To allow instances within a private subnet to access Internet resources through the IGW via a NAT instance, you must do the following:

- Create a security group for the NAT with outbound rules that specify the needed Internet resources by port, protocol, and IP address.
- Launch an Amazon Linux NAT AMI as an instance in a public subnet and associate it with the NAT security group.
- Disable the Source/Destination Check attribute of the NAT.
- Configure the route table associated with a private subnet to direct Internet-bound traffic to the NAT instance (for example, `i-1a2b3c4d`).
- Allocate an EIP and associate it with the NAT instance.

This configuration allows instances in private subnets to send outbound Internet communication, but it prevents the instances from receiving inbound traffic initiated by someone on the Internet.

NAT Gateway

A *NAT gateway* is an Amazon managed resource that is designed to operate just like a NAT instance, but it is simpler to manage and highly available within an Availability Zone.

To allow instances within a private subnet to access Internet resources through the IGW via a NAT gateway, you must do the following:

- Configure the route table associated with the private subnet to direct Internet-bound traffic to the NAT gateway (for example, `nat-1a2b3c4d`).
- Allocate an EIP and associate it with the NAT gateway.

Like a NAT instance, this managed service allows outbound Internet communication and prevents the instances from receiving inbound traffic initiated by someone on the Internet.

 To create an Availability Zone-independent architecture, create a NAT gateway in each Availability Zone and configure your routing to ensure that resources use the NAT gateway in the same Availability Zone.

The exercises will demonstrate how a NAT gateway works.

Virtual Private Gateways (VPGs), Customer Gateways (CGWs), and Virtual Private Networks (VPNs)

You can connect an existing data center to Amazon VPC using either hardware or software VPN connections, which will make Amazon VPC an extension of the data center. Amazon VPC offers two ways to connect a corporate network to a VPC: VPG and CGW.

A *virtual private gateway (VPG)* is the *virtual private network (VPN)* concentrator on the AWS side of the VPN connection between the two networks. A *customer gateway (CGW)* represents a physical device or a software application on the customer's side of the VPN connection. After these two elements of an Amazon VPC have been created, the last step is to create a VPN tunnel. The VPN tunnel is established after traffic is generated from the customer's side of the VPN connection. Figure 4.4 illustrates a single VPN connection between a corporate network and an Amazon VPC.

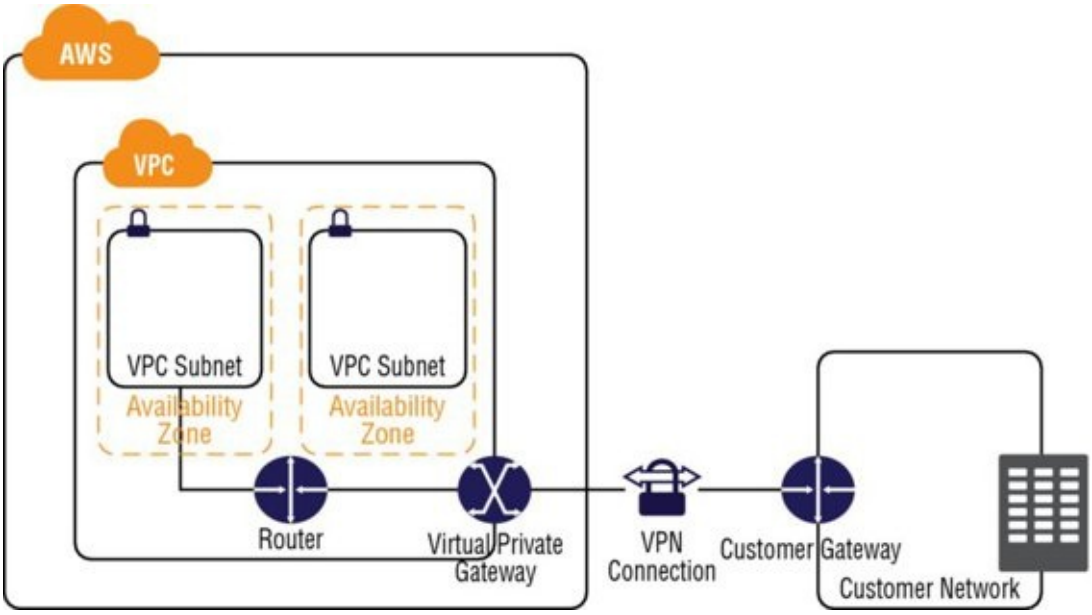


FIGURE 4.4 VPC with VPN connection to a customer network

You must specify the type of routing that you plan to use when you create a VPN connection. If the CGW supports Border Gateway Protocol (BGP), then configure the VPN connection for dynamic routing. Otherwise, configure the connections for static routing. If you will be using static routing, you must enter the routes for your network that should be communicated to the VPG. Routes will be propagated to the Amazon VPC to allow your resources to route network traffic back to the corporate network through the VGW and across the VPN tunnel.

Amazon VPC also supports multiple CGWs, each having a VPN connection to a single VPG (many-to-one design). In order to support this topology, the CGW IP addresses must be unique within the region.

Amazon VPC will provide the information needed by the network administrator to configure the CGW and establish the VPN connection with the VPG. The VPN connection consists of two Internet Protocol Security (IPSec) tunnels for higher availability to the Amazon VPC.

Following are the important points to understand about VPGs, CGWs, and VPNs for the exam:

- The VPG is the AWS end of the VPN tunnel.
- The CGW is a hardware or software application on the customer's side of the VPN tunnel.

- You must initiate the VPN tunnel from the CGW to the VPG.
- VPGs support both dynamic routing with BGP and static routing.
- The VPN connection consists of two tunnels for higher availability to the VPC.

Summary

In this chapter, you learned that Amazon VPC is the networking layer for Amazon EC2, and it allows you to create your own private virtual network within the cloud. You can provision your own logically isolated section of AWS similar to designing and implementing a separate independent network that you'd operate in a physical data center.

A VPC consists of the following components:

- Subnets
- Route tables
- DHCP option sets
- Security groups
- Network ACLs

A VPC has the following optional components:

- IGWs
- EIP addresses
- Endpoints
- Peering
- NAT instance and NAT gateway
- VPG, CGW, and VPN

Subnets can be public, private, or VPN-only. A public subnet is one in which the associated route table directs the subnet's traffic to the Amazon VPC's IGW. A private subnet is one in which the associated route table does not direct the subnet's traffic to the Amazon VPC's IGW. A VPN-only subnet is one in which the associated route table directs the subnet's traffic to the Amazon VPC's VPG and does not have a route to the IGW. Regardless of the type of subnet, the internal IP address range of the subnet is always private (non-routable on the Internet).

A route table is a logical construct within an Amazon VPC that contains a set of rules (called routes) that are applied to the subnet and used to determine where network traffic is directed. A route table's routes are what permit Amazon EC2 instances within different subnets within an Amazon VPC to communicate with each other. You can modify route tables and add your own custom routes. You can also use route tables to specify which subnets are public (by directing Internet traffic to the IGW) and which subnets are private (by not having a route that directs traffic to the IGW). An IGW is a horizontally scaled, redundant, and highly available Amazon VPC component that allows communication between instances in your Amazon VPC and the Internet. IGWs are fully redundant and have no bandwidth constraints. An IGW provides a target in your Amazon VPC route tables for Internet-routable traffic, and it performs network address translation for instances that have been assigned public IP addresses.

The DHCP option sets element of an Amazon VPC allows you to direct Amazon EC2 host name assignment to your own resources. In order for you to assign your own domain name to your instances, you create a custom DHCP option set and assign it to your Amazon VPC.

An EIP address is a static, public IP address in the pool for the region that you can allocate to your account (pull from the pool) and release (return to the pool). EIPs allow you to maintain a set of IP addresses that remain fixed while the underlying infrastructure may change over time.

An Amazon VPC endpoint enables you to create a private connection between your Amazon VPC and another AWS service without requiring access over the Internet or through a NAT instance, VPN connection, or AWS Direct Connect. You can create multiple endpoints for a single service, and you can use different route tables to enforce different access policies from different subnets to the same service.

An Amazon VPC peering connection is a networking connection between two Amazon VPCs that enables instances in either Amazon VPC to communicate with each other as if they were within the same network. You can create an Amazon VPC peering connection between your own Amazon VPCs or with an Amazon VPC in another AWS account within a single region. A peering connection is neither a gateway nor a VPN connection and does not introduce a single point of failure for communication.

A security group is a virtual stateful firewall that controls inbound and outbound traffic to Amazon EC2 instances. When you first launch an Amazon EC2 instance into an Amazon VPC, you must specify the security group with which it will be associated. AWS provides a default security group for your use, which has rules that allow all instances associated with the security group to communicate with each other and allow all outbound traffic. You may change the rules for the default security group, but you may not delete the default security group.

A network ACL is another layer of security that acts as a stateless firewall on a subnet level. Amazon VPCs are created with a modifiable default network ACL associated with every subnet that allows all inbound and outbound traffic. If you want to create a custom network ACL, its initial configuration will deny all inbound and outbound traffic until you create a rule that states otherwise.

A NAT instance is a customer-managed instance that is designed to accept traffic from instances within a private subnet, translate the source IP address to the public IP address of the NAT instance, and forward the traffic to the IGW. In addition, the NAT instance maintains the state of the forwarded traffic in order to return response traffic from the Internet to the proper instance in the private subnet.

A NAT gateway is an AWS-managed service that is designed to accept traffic from instances within a private subnet, translate the source IP address to the public IP address of the NAT gateway, and forward the traffic to the IGW. In addition, the NAT gateway maintains the state of the forwarded traffic in order to return response traffic from the Internet to the proper instance in the private subnet.

A VPG is the VPN concentrator on the AWS side of the VPN connection between the two networks. A CGW is a physical device or a software application on the customer’s side of the VPN connection. After these two elements of an Amazon VPC have been created, the last step is to create a VPN tunnel. The VPN tunnel is established after traffic is generated from the customer’s side of the VPN connection.

Exam Essentials

Understand what a VPC is and its core and optional components. An Amazon VPC is a logically isolated network in the AWS Cloud. An Amazon VPC is made up of the following core elements: subnets (public, private, and VPN-only), route tables, DHCP option sets, security groups, and network ACLs. Optional elements include an IGW, EIP addresses, endpoints, peering connections, NAT instances, VPGs, CGWs, and VPN connections.

Understand the purpose of a subnet. A subnet is a segment of an Amazon VPC's IP address range where you can place groups of isolated resources. Subnets are defined by CIDR blocks—for example, 10.0.1.0/24 and 10.0.2.0/24—and are contained within an Availability Zone.

Identify the difference between a public subnet, a private subnet, and a VPN-Only subnet. If a subnet's traffic is routed to an IGW, the subnet is known as a public subnet. If a subnet doesn't have a route to the IGW, the subnet is known as a private subnet. If a subnet doesn't have a route to the IGW, but has its traffic routed to a VPG, the subnet is known as a VPN-only subnet.

Understand the purpose of a route table. A route table is a set of rules (called routes) that are used to determine where network traffic is directed. A route table allows Amazon EC2 instances within different subnets to communicate with each other (within the same Amazon VPC). The Amazon VPC router also enables subnets, IGWs, and VPGs to communicate with each other.

Understand the purpose of an IGW. An IGW is a horizontally scaled, redundant, and highly available Amazon VPC component that allows communication between instances in your Amazon VPC and the Internet. IGWs are fully redundant and have no bandwidth constraints. An IGW provides a target in your Amazon VPC route tables for Internet-routable traffic and performs network address translation for instances that have been assigned public IP addresses.

Understand what DHCP option sets provide to an Amazon VPC. The DHCP option sets element of an Amazon VPC allows you to direct Amazon EC2 host name assignment to your own resources. You can specify the domain name for instances within an Amazon VPC and identify the IP addresses of custom DNS servers, NTP servers, and NetBIOS servers.

Know the difference between an Amazon VPC public IP address and an EIP address. A public IP address is an AWS-owned IP that can be automatically assigned to instances launched within a subnet. An EIP address is an AWS-owned public IP address that you allocate to your account and assign to instances or network interfaces on demand.

Understand what endpoints provide to an Amazon VPC. An Amazon VPC endpoint enables you to create a private connection between your Amazon VPC and another AWS service without requiring access over the Internet or through a NAT instance, a VPN connection, or AWS Direct Connect. Endpoints support services within the region only.

Understand Amazon VPC peering. An Amazon VPC peering connection is a networking connection between two Amazon VPCs that enables instances in either Amazon VPC to communicate with each other as if they are within the same network. Peering connections are created through a request/accept protocol. Transitive peering is not supported, and peering is only available between Amazon VPCs within the same region.

Know the difference between a security group and a network ACL. A security group applies at the instance level. You can have multiple instances in multiple subnets that are members of the same security

groups. Security groups are stateful, which means that return traffic is automatically allowed, regardless of any outbound rules. A network ACL is applied on a subnet level, and traffic is stateless. You need to allow both inbound and outbound traffic on the network ACL in order for Amazon EC2 instances in a subnet to be able to communicate over a particular protocol.

Understand what a NAT provides to an Amazon VPC. A NAT instance or NAT gateway enables instances in a private subnet to initiate outbound traffic to the Internet. This allows outbound Internet communication to download patches and updates, for example, but prevents the instances from receiving inbound traffic initiated by someone on the Internet.

Understand the components needed to establish a VPN connection from a network to an Amazon VPC. A VPG is the VPN concentrator on the AWS side of the VPN connection between the two networks. A CGW represents a physical device or a software application on the customer's side of the VPN connection. The VPN connection must be initiated from the CGW side, and the connection consists of two IPSec tunnels.

Exercises

The best way to become familiar with Amazon VPC is to build your own custom Amazon VPC and then deploy Amazon EC2 instances into it, which is what you'll be doing in this section. You should repeat these exercises until you can create and decommission Amazon VPCs with confidence.

For assistance completing these exercises, refer to the Amazon VPC User Guide located at `http://aws.amazon.com/documentation/vpc/`.

EXERCISE 4.1

Create a Custom Amazon VPC

1. Sign in to the AWS Management Console as an administrator or power user.
2. Select the Amazon VPC icon to launch the Amazon VPC Dashboard.
3. Create an Amazon VPC with a CIDR block equal to `192.168.0.0/16`, a name tag of **`My First VPC`**, and default tenancy.

You have created your first custom VPC.

EXERCISE 4.2

Create Two Subnets for Your Custom Amazon VPC

1. Create a subnet with a CIDR block equal to `192.168.1.0/24` and a name tag of **`My First Public Subnet`**. Create the subnet in the Amazon VPC from Exercise 4.1, and specify an Availability Zone for the subnet (for example, `US-East-1a`).
2. Create a subnet with a CIDR block equal to `192.168.2.0/24` and a name tag of **`My First Private Subnet`**. Create the subnet in the Amazon VPC from Exercise 4.1, and specify a different Availability Zone for the subnet than previously specified (for example, `US-East-1b`).

You have now created two new subnets, each in its own Availability Zone. It's important to remember that one subnet equals one Availability Zone. You cannot stretch a subnet across multiple Availability Zones.

EXERCISE 4.3

Connect Your Custom Amazon VPC to the Internet and Establish Routing

For assistance with this exercise, refer to the Amazon EC2 key pair documentation at:

http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-key-pairs.html

For additional assistance with this exercise, refer to the NAT instances documentation at: http://docs.aws.amazon.com/AmazonVPC/latest/UserGuide/VPC_NAT_Instance.html#NATInstance

1. Create an Amazon EC2 key pair in the same region as your custom Amazon VPC.
2. Create an IGW with a name tag of **`My First IGW`** and attach it to your custom Amazon VPC.
3. Add a route to the main route table for your custom Amazon VPC that directs Internet traffic (0.0.0.0/0) to the IGW.
4. Create a NAT gateway, place it in the public subnet of your custom Amazon VPC, and assign it an EIP.
5. Create a new route table with a name tag of **`My First Private Route Table`** and place it within your custom Amazon VPC. Add a route to it that directs Internet traffic (0.0.0.0/0) to the NAT gateway and associate it with the private subnet.

You have now created a connection to the Internet for resources within your Amazon VPC. You established routing rules that direct Internet traffic to the IGW regardless of the originating subnet.

EXERCISE 4.4

Launch an Amazon EC2 Instance and Test the Connection to the Internet

1. Launch a t2.micro Amazon Linux AMI as an Amazon EC2 instance into the public subnet of your custom Amazon VPC, give it a name tag of **`My First Public Instance`**, and select the newly-created key pair for secure access to the instance.
2. Securely access the Amazon EC2 instance in the public subnet via SSH with the newly-created key pair.
3. Execute an `update` to the operating system instance libraries by executing the following command: `# sudo yum update -y`
4. You should see output showing the instance downloading software from the Internet and installing it.

You have now provisioned an Amazon EC2 instance in a public subnet. You can apply patches to the Amazon EC2 instance in the public subnet, and you have demonstrated connectivity to the Internet.

Review Questions

1. What is the minimum size subnet that you can have in an Amazon VPC?
   A. /24
   B. /26
   C. /28
   D. /30
2. You are a solutions architect working for a large travel company that is migrating its existing server estate to AWS. You have recommended that they use a custom Amazon VPC, and they have agreed to proceed. They will need a public subnet for their web servers and a private subnet in which to place their databases. They also require that the web servers and database servers be highly available and that there be a minimum of two web servers and two database servers each. How many subnets should you have to maintain high availability?
   A. 2
   B. 3
   C. 4
   D. 1
3. Which of the following is an optional security control that can be applied at the subnet layer of a VPC?
   A. Network ACL
   B. Security Group
   C. Firewall
   D. Web application firewall
4. What is the maximum size IP address range that you can have in an Amazon VPC?
   A. /16
   B. /24
   C. /28
   D. /30
5. You create a new subnet and then add a route to your route table that routes traffic out from that subnet to the Internet using an IGW. What type of subnet have you created?
   A. An internal subnet
   B. A private subnet
   C. An external subnet
   D. A public subnet

6. What happens when you create a new Amazon VPC?
   A. A main route table is created by default.
   B. Three subnets are created by default—one for each Availability Zone.
   C. Three subnets are created by default in one Availability Zone.
   D. An IGW is created by default.
7. You create a new VPC in `US-East-1` and provision three subnets inside this Amazon VPC. Which of the following statements is true?
   A. By default, these subnets will not be able to communicate with each other; you will need to create routes.
   B. All subnets are public by default.
   C. All subnets will be able to communicate with each other by default.
   D. Each subnet will have identical CIDR blocks.
8. How many IGWs can you attach to an Amazon VPC at any one time?
   A. 1
   B. 2
   C. 3
   D. 4
9. What aspect of an Amazon VPC is stateful?
   A. Network ACLs
   B. Security groups
   C. Amazon DynamoDB
   D. Amazon S3
10. You have created a custom Amazon VPC with both private and public subnets. You have created a NAT instance and deployed this instance to a public subnet. You have attached an EIP address and added your NAT to the route table. Unfortunately, instances in your private subnet still cannot access the Internet. What may be the cause of this?
    A. Your NAT is in a public subnet, but it needs to be in a private subnet.
    B. Your NAT should be behind an Elastic Load Balancer.
    C. You should disable source/destination checks on the NAT.
    D. Your NAT has been deployed on a Windows instance, but your other instances are Linux. You should redeploy the NAT onto a Linux instance.
11. Which of the following will occur when an Amazon Elastic Block Store (Amazon EBS)-backed Amazon EC2 instance in an Amazon VPC with an associated EIP is stopped and started? (Choose 2 answers)
    A. The EIP will be dissociated from the instance.

B. All data on instance-store devices will be lost.

C. All data on Amazon EBS devices will be lost.

D. The ENI is detached.

E. The underlying host for the instance is changed.

12. How many VPC Peering connections are required for four VPCs located within the same AWS region to be able to send traffic to each of the others?

A. 3

B. 4

C. 5

D. 6

13. Which of the following AWS resources would you use in order for an EC2-VPC instance to resolve DNS names outside of AWS?

A. A VPC peering connection

B. A DHCP option set

C. A routing rule

D. An IGW

14. Which of the following is the Amazon side of an Amazon VPN connection?

A. An EIP

B. A CGW

C. An IGW

D. A VPG

15. What is the default limit for the number of Amazon VPCs that a customer may have in a region?

A. 5

B. 6

C. 7

D. There is no default maximum number of VPCs within a region.

16. You are responsible for your company's AWS resources, and you notice a significant amount of traffic from an IP address in a foreign country in which your company does not have customers. Further investigation of the traffic indicates the source of the traffic is scanning for open ports on your EC2-VPC instances. Which one of the following resources can deny the traffic from reaching the instances?

A. Security group

B. Network ACL

C. NAT instance

D. An Amazon VPC endpoint

17. Which of the following is the security protocol supported by Amazon VPC?
    A. SSH
    B. Advanced Encryption Standard (AES)
    C. Point-to-Point Tunneling Protocol (PPTP)
    D. IPsec
18. Which of the following Amazon VPC resources would you use in order for EC2-VPC instances to send traffic directly to Amazon S3?
    A. Amazon S3 gateway
    B. IGW
    C. CGW
    D. VPC endpoint
19. What properties of an Amazon VPC must be specified at the time of creation? (Choose 2 answers)
    A. The CIDR block representing the IP address range
    B. One or more subnets for the Amazon VPC
    C. The region for the Amazon VPC
    D. Amazon VPC Peering relationships
20. Which Amazon VPC feature allows you to create a dual-homed instance?
    A. EIP address
    B. ENI
    C. Security groups
    D. CGW

Chapter 5
Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling

THE AWS CERTIFIED SOLUTIONS ARCHITECT EXAM TOPICS COVERED IN THIS CHAPTER MAY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

Domain 1.0: Designing highly available, cost-effective, fault-tolerant, scalable systems

✓ **1.1 Identify and recognize cloud architecture considerations, such as fundamental components and effective designs.**

- Elasticity and scalability

Domain 2.0: Implementation/Deployment

✓ **2.1 Identify the appropriate techniques and methods using Amazon Elastic Compute Cloud (Amazon EC2), Amazon Simple Storage Service (Amazon S3), AWS Elastic Beanstalk, AWS CloudFormation, AWS OpsWorks, Amazon Virtual Private Cloud (Amazon VPC), and AWS Identity and Access Management (IAM) to code and implement a cloud solution.**

Content may include the following:

- Launch instances across the AWS global infrastructure

Domain 3.0: Data Security

✓ **3.1 Recognize and implement secure practices for optimum cloud deployment and maintenance.**

- CloudWatch Logs

Domain 4.0: Troubleshooting

Content may include the following:

- General troubleshooting information and questions



Introduction

In this chapter, you will learn how Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling work both independently and together to help you efficiently and cost-effectively deploy highly available and optimized workloads on AWS.

Elastic Load Balancing is a highly available service that distributes traffic across Amazon Elastic Compute Cloud (Amazon EC2) instances and includes options that provide flexibility and control of incoming requests to Amazon EC2 instances.

Amazon CloudWatch is a service that monitors AWS Cloud resources and applications running on AWS. It collects and tracks metrics, collects and monitors log files, and sets alarms. Amazon CloudWatch has a basic level of monitoring for no cost and a more detailed level of monitoring for an additional cost.

Auto Scaling is a service that allows you to maintain the availability of your applications by scaling Amazon EC2 capacity up or down in accordance with conditions you set.

This chapter covers all three services separately, but it also highlights how they can work together to build more robust and highly available architectures on AWS.

Elastic Load Balancing

An advantage of having access to a large number of servers in the cloud, such as Amazon EC2 instances on AWS, is the ability to provide a more consistent experience for the end user. One way to ensure consistency is to balance the request load across more than one server. A load balancer is a mechanism that automatically distributes traffic across multiple Amazon EC2 instances. You can either manage your own virtual load balancers on Amazon EC2 instances or leverage an AWS Cloud service called Elastic Load Balancing, which provides a managed load balancer for you.

The Elastic Load Balancing service allows you to distribute traffic across a group of Amazon EC2 instances in one or more *Availability Zones*, enabling you to achieve high availability in your applications. Elastic Load Balancing supports routing and load balancing of Hypertext Transfer Protocol (HTTP), Hypertext Transfer Protocol Secure (HTTPS), *Transmission Control Protocol (TCP)*, and *Secure Sockets Layer (SSL)* traffic to Amazon EC2 instances. Elastic Load Balancing provides a stable, single *Canonical Name record (CNAME)* entry point for *Domain Name System (DNS)* configuration and supports both Internet-facing and internal application-facing load balancers. Elastic Load Balancing supports health checks for Amazon EC2 instances to ensure traffic is not routed to unhealthy or failing instances. Also, Elastic Load Balancing can automatically scale based on collected metrics.

There are several advantages of using Elastic Load Balancing. Because Elastic Load Balancing is a managed service, it scales in and out automatically to meet the demands of increased application traffic and is highly available within a region itself as a service. Elastic Load Balancing helps you achieve high availability for your applications by distributing traffic across healthy instances in multiple Availability Zones. Additionally, Elastic Load Balancing seamlessly integrates with the Auto Scaling service to automatically scale the Amazon EC2 instances behind the load balancer. Finally, Elastic Load Balancing is secure, working with Amazon Virtual Private Cloud (Amazon VPC) to route traffic internally between application tiers, allowing you to expose only Internet-facing public IP addresses. Elastic Load Balancing also supports integrated certificate management and SSL termination.

 Elastic Load Balancing is a highly available service itself and can be used to help build highly available architectures.

Types of Load Balancers

Elastic Load Balancing provides several types of load balancers for handling different kinds of connections including Internet-facing, internal, and load balancers that support encrypted connections.

Internet-Facing Load Balancers

An *Internet-facing load balancer* is, as the name implies, a load balancer that takes requests from clients over the Internet and distributes them to Amazon EC2 instances that are registered with the load balancer.

When you configure a load balancer, it receives a public DNS name that clients can use to send requests to your application. The DNS servers resolve the DNS name to your load balancer’s public IP address, which can be visible to client applications.

 An AWS recommended best practice is always to reference a load balancer by its DNS name, instead of by the IP address of the load balancer, in order to provide a single, stable entry point.

Because Elastic Load Balancing scales in and out to meet traffic demand, it is not recommended to bind an application to an IP address that may no longer be part of a load balancer's pool of resources.

Elastic Load Balancing in Amazon VPC supports IPv4 addresses only. Elastic Load Balancing in EC2-Classic supports both IPv4 and IPv6 addresses.

Internal Load Balancers

In a multi-tier application, it is often useful to load balance between the tiers of the application. For example, an Internet-facing load balancer might receive and balance external traffic to the presentation or web tier whose Amazon EC2 instances then send its requests to a load balancer sitting in front of the application tier. You can use *internal load balancers* to route traffic to your Amazon EC2 instances in VPCs with private subnets.

HTTPS Load Balancers

You can create a load balancer that uses the SSL/Transport Layer Security (TLS) protocol for encrypted connections (also known as *SSL offload*). This feature enables traffic encryption between your load balancer and the clients that initiate HTTPS sessions, and for connections between your load balancer and your back-end instances. Elastic Load Balancing provides security policies that have predefined SSL negotiation configurations to use to negotiate connections between clients and the load balancer. In order to use SSL, you must install an SSL certificate on the load balancer that it uses to terminate the connection and then decrypt requests from clients before sending requests to the back-end Amazon EC2 instances. You can optionally choose to enable authentication on your back-end instances.

Elastic Load Balancing does not support *Server Name Indication* (SNI) on your load balancer. This means that if you want to host multiple websites on a fleet of Amazon EC2 instances behind Elastic Load Balancing with a single SSL certificate, you will need to add a *Subject Alternative Name (SAN)* for each website to the certificate to avoid site users seeing a warning message when the site is accessed.

Listeners

Every load balancer must have one or more *listeners* configured. A listener is a process that checks for connection requests—for example, a CNAME configured to the A record name of the load balancer. Every listener is configured with a protocol and a port (client to load balancer) for a front-end connection and a protocol and a port for the back-end (load balancer to Amazon EC2 instance) connection. Elastic Load Balancing supports the following protocols:

- HTTP
- HTTPS
- TCP
- SSL

Elastic Load Balancing supports protocols operating at two different *Open System Interconnection (OSI)* layers. In the OSI model, Layer 4 is the transport layer that describes the TCP connection between the client and your back-end instance through the load balancer. Layer 4 is the lowest level that is configurable for your load balancer. Layer 7 is the application layer that describes the use of HTTP and HTTPS connections from clients to the load balancer and from the load balancer to your back-end instance.

The SSL protocol is primarily used to encrypt confidential data over insecure networks such as the Internet. The SSL protocol establishes a secure connection between a client and the back-end server and ensures that all the data passed between your client and your server is private.

Configuring Elastic Load Balancing

Elastic Load Balancing allows you to configure many aspects of the load balancer, including *idle connection timeout*, *cross-zone load balancing*, *connection draining*, *proxy protocol*, *sticky sessions*, and *health checks*. Configuration settings can be modified using either the AWS Management Console or a Command Line Interface (CLI). Some of the options are described next.

Idle Connection Timeout

For each request that a client makes through a load balancer, the load balancer maintains two connections. One connection is with the client and the other connection is to the back-end instance. For each connection, the load balancer manages an idle timeout that is triggered when no data is sent over the connection for a specified time period. After the idle timeout period has elapsed, if no data has been sent or received, the load balancer closes the connection.

By default, Elastic Load Balancing sets the idle timeout to 60 seconds for both connections. If an HTTP request doesn't complete within the idle timeout period, the load balancer closes the connection, even if data is still being transferred. You can change the idle timeout setting for the connections to ensure that lengthy operations, such as file uploads, have time to complete.

If you use HTTP and HTTPS listeners, we recommend that you enable the *keep-alive* option for your Amazon EC2 instances. You can enable keep-alive in your web server settings or in the kernel settings for your Amazon EC2 instances. Keep-alive, when enabled, allows the load balancer to reuse connections to your back-end instance, which reduces CPU utilization.



To ensure that the load balancer is responsible for closing the connections to your back-end instance, make sure that the value you set for the keep-alive time is greater than the idle timeout setting on your load balancer.

Cross-Zone Load Balancing

To ensure that request traffic is routed evenly across all back-end instances for your load balancer, regardless of the Availability Zone in which they are located, you should enable cross-zone load balancing on your load balancer. Cross-zone load balancing reduces the need to maintain equivalent numbers of back-end instances in each Availability Zone and improves your application's ability to handle the loss of one or more back-end instances. However, it is still recommended that you maintain approximately equivalent numbers of instances in each Availability Zone for higher fault tolerance.

For environments where clients cache DNS lookups, incoming requests might favor one of the Availability Zones. Using cross-zone load balancing, this imbalance in the request load is spread across all available back-end instances in the region, reducing the impact of misconfigured clients.

Connection Draining

You should enable *connection draining* to ensure that the load balancer stops sending requests to instances that are deregistering or unhealthy, while keeping the existing connections open. This enables the load balancer to complete in-flight requests made to these instances.

When you enable connection draining, you can specify a maximum time for the load balancer to keep connections alive before reporting the instance as deregistered. The maximum timeout value can be set between 1 and 3,600 seconds (the default is 300 seconds). When the maximum time limit is reached, the load balancer forcibly closes connections to the deregistering instance.

Proxy Protocol

When you use TCP or SSL for both front-end and back-end connections, your load balancer forwards requests to the back-end instances without modifying the request headers. If you enable *Proxy Protocol*, a human-readable header is added to the request header with connection information such as the source IP address, destination IP address, and port numbers. The header is then sent to the back-end instance as part of the request.

Before using Proxy Protocol, verify that your load balancer is not behind a proxy server with Proxy Protocol enabled. If Proxy Protocol is enabled on both the proxy server and the load balancer, the load balancer adds another header to the request, which already has a header from the proxy server. Depending on how your back-end instance is configured, this duplication might result in errors.

Sticky Sessions

By default, a load balancer routes each request independently to the registered instance with the smallest load. However, you can use the *sticky session* feature (also known as *session affinity*), which enables the load balancer to bind a user's session to a specific instance. This ensures that all requests from the user during the session are sent to the same instance.

The key to managing sticky sessions is to determine how long your load balancer should consistently route the user's request to the same instance. If your application has its own session cookie, you can configure Elastic Load Balancing so that the session cookie follows the duration specified by the application's session cookie. If your application does not have its own session cookie, you can configure Elastic Load Balancing to create a session cookie by specifying your own stickiness duration. Elastic Load Balancing creates a cookie named AWSELB that is used to map the session to the instance.

Health Checks

Elastic Load Balancing supports health checks to test the status of the Amazon EC2 instances behind an Elastic Load Balancing load balancer. The status of the instances that are healthy at the time of the health check is `InService`. The status of any instances that are unhealthy at the time of the health check is `OutOfService`. The load balancer performs health checks on all registered instances to determine whether the instance is in a healthy state or an unhealthy state. A health check is a ping, a connection attempt, or a page that is checked periodically. You can set the time interval between health checks and also the amount of time to wait to respond in case the health check page includes a computational aspect.

Finally, you can set a threshold for the number of consecutive health check failures before an instance is marked as unhealthy.

Updates Behind an Elastic Load Balancing Load Balancer

Long-running applications will eventually need to be maintained and updated with a newer version of the application. When using Amazon EC2 instances running behind an Elastic Load Balancing load balancer, you may deregister these long-running Amazon EC2 instances associated with a load balancer manually and then register newly launched Amazon EC2 instances that you have started with the new updates installed.

Amazon CloudWatch

Amazon CloudWatch is a service that you can use to monitor your AWS resources and your applications in real time. With Amazon CloudWatch, you can collect and track metrics, create alarms that send notifications, and make changes to the resources being monitored based on rules you define.

For example, you might choose to monitor CPU utilization to decide when to add or remove Amazon EC2 instances in an application tier. Or, if a particular application-specific metric that is not visible to AWS is the best indicator for assessing your scaling needs, you can perform a `PUT` request to push that metric into Amazon CloudWatch. You can then use this custom metric to manage capacity.

You can specify parameters for a metric over a time period and configure alarms and automated actions when a threshold is reached. Amazon CloudWatch supports multiple types of actions such as sending a notification to an Amazon Simple Notification Service (Amazon SNS) topic or executing an Auto Scaling policy.

Amazon CloudWatch offers either basic or detailed monitoring for supported AWS products. *Basic monitoring* sends data points to Amazon CloudWatch every five minutes for a limited number of preselected metrics at no charge. *Detailed monitoring* sends data points to Amazon CloudWatch every minute and allows data aggregation for an additional charge. If you want to use detailed monitoring, you must enable it—basic is the default.

Amazon CloudWatch supports monitoring and specific metrics for most AWS Cloud services, including: Auto Scaling, Amazon CloudFront, Amazon CloudSearch, Amazon DynamoDB, Amazon EC2, Amazon EC2 Container Service (Amazon ECS), Amazon ElastiCache, Amazon Elastic Block Store (Amazon EBS), Elastic Load Balancing, Amazon Elastic MapReduce (Amazon EMR), Amazon Elasticsearch Service, Amazon Kinesis Streams, Amazon Kinesis Firehose, AWS Lambda, Amazon Machine Learning, AWS OpsWorks, Amazon Redshift, Amazon Relational Database Service (Amazon RDS), Amazon Route 53, Amazon SNS, Amazon Simple Queue Service (Amazon SQS), Amazon S3, AWS Simple Workflow Service (Amazon SWF), AWS Storage Gateway, AWS WAF, and Amazon WorkSpaces.

Read Alert

You may have an application that leverages Amazon DynamoDB, and you want to know when read requests reach a certain threshold and alert yourself with an email. You can do this by using `ProvisionedReadCapacityUnits` for the Amazon DynamoDB table for which you want to set an alarm. You simply set a threshold value during a number of consecutive periods and then specify email as the notification type. Now, when the threshold is sustained over the number of periods, your specified email will alert you to the read activity.

Amazon CloudWatch metrics can be retrieved by performing a `GET` request. When you use detailed monitoring, you can also aggregate metrics across a length of time you specify. Amazon CloudWatch does not aggregate data across regions but can aggregate across Availability Zones within a region.

AWS provides a rich set of metrics included with each service, but you can also define custom metrics to monitor resources and events AWS does not have visibility into—for example, Amazon EC2 instance memory consumption and disk metrics that are visible to the operating system of the Amazon EC2 instance

but not visible to AWS or application-specific thresholds running on instances that are not known to AWS. Amazon CloudWatch supports an Application Programming Interface (API) that allows programs and scripts to `PUT` metrics into Amazon CloudWatch as name-value pairs that can then be used to create events and trigger alarms in the same manner as the default Amazon CloudWatch metrics.

Amazon CloudWatch Logs can be used to monitor, store, and access log files from Amazon EC2 instances, AWS CloudTrail, and other sources. You can then retrieve the log data and monitor in real time for events—for example, you can track the number of errors in your application logs and send a notification if an error rate exceeds a threshold. Amazon CloudWatch Logs can also be used to store your logs in Amazon S3 or Amazon Glacier. Logs can be retained indefinitely or according to an aging policy that will delete older logs as no longer needed.

A *CloudWatch Logs agent* is available that provides an automated way to send log data to CloudWatch Logs for Amazon EC2 instances running Amazon Linux or Ubuntu. You can use the Amazon CloudWatch Logs agent installer on an existing Amazon EC2 instance to install and configure the CloudWatch Logs agent. After installation is complete, the agent confirms that it has started and it stays running until you disable it.

Amazon CloudWatch has some limits that you should keep in mind when using the service. Each AWS account is limited to 5,000 alarms per AWS account, and metrics data is retained for two weeks by default (at the time of this writing). If you want to keep the data longer, you will need to move the logs to a persistent store like Amazon S3 or Amazon Glacier. You should familiarize yourself with the limits for Amazon CloudWatch in the Amazon CloudWatch Developer Guide.

Auto Scaling

A distinct advantage of deploying applications to the cloud is the ability to launch and then release servers in response to variable workloads. Provisioning servers on demand and then releasing them when they are no longer needed can provide significant cost savings for workloads that are not steady state. Examples include a website for a specific sporting event, an end-of-month data-input system, a retail shopping site supporting flash sales, a music artist website during the release of new songs, a company website announcing successful earnings, or a nightly processing run to calculate daily activity.

Auto Scaling is a service that allows you to scale your Amazon EC2 capacity automatically by scaling out and scaling in according to criteria that you define. With Auto Scaling, you can ensure that the number of running Amazon EC2 instances increases during demand spikes or peak demand periods to maintain application performance and decreases automatically during demand lulls or troughs to minimize costs.

Embrace the Spike

Many web applications have unplanned load increases based on events outside of your control. For example, your company may get mentioned on a popular blog or television program driving many more people to visit your site than expected. Setting up Auto Scaling in advance will allow you to embrace and survive this kind of fast increase in the number of requests. Auto Scaling will scale up your site to meet the increased demand and then scale down when the event subsides.

Auto Scaling Plans

Auto Scaling has several schemes or plans that you can use to control how you want Auto Scaling to perform.

Maintain Current Instance Levels

You can configure your Auto Scaling group to maintain a minimum or specified number of running instances at all times. To maintain the current instance levels, Auto Scaling performs a periodic health check on running instances within an *Auto Scaling group*. When Auto Scaling finds an unhealthy instance, it terminates that instance and launches a new one.



Steady state workloads that need a consistent number of Amazon EC2 instances at all times can use Auto Scaling to monitor and keep that specific number of Amazon EC2 instances running.

Manual Scaling

Manual scaling is the most basic way to scale your resources. You only need to specify the change in the maximum, minimum, or desired capacity of your Auto Scaling group. Auto Scaling manages the process of creating or terminating instances to maintain the updated capacity.

 Manual scaling out can be very useful to increase resources for an infrequent event, such as the release of a new game version that will be available for download and require a user registration. For extremely large-scale events, even the Elastic Load Balancing load balancers can be pre-warmed by working with your local solutions architect or AWS Support.

Scheduled Scaling

Sometimes you know exactly when you will need to increase or decrease the number of instances in your group, simply because that need arises on a predictable schedule. Examples include periodic events such as end-of-month, end-of-quarter, or end-of-year processing, and also other predictable, recurring events. Scheduled scaling means that scaling actions are performed automatically as a function of time and date.

 Recurring events such as end-of-month, quarter, or year processing, or scheduled and recurring automated load and performance testing, can be anticipated and Auto Scaling can be ramped up appropriately at the time of the scheduled event.

Dynamic Scaling

Dynamic scaling lets you define parameters that control the Auto Scaling process in a scaling policy. For example, you might create a policy that adds more Amazon EC2 instances to the web tier when the network bandwidth, measured by Amazon CloudWatch, reaches a certain threshold.

Auto Scaling Components

Auto Scaling has several components that need to be configured to work properly: a *launch configuration*, an *Auto Scaling group*, and an optional *scaling policy*.

Launch Configuration

A *launch configuration* is the template that Auto Scaling uses to create new instances, and it is composed of the configuration name, *Amazon Machine Image (AMI)*, Amazon EC2 instance type, security group, and instance key pair. Each Auto Scaling group can have only one launch configuration at a time.

The CLI command that follows will create a launch configuration with the following attributes:

Name: `myLC`

AMI: `ami-0535d66c`

Instance type: `m3.medium`

Security groups: `sg-f57cde9d`

Instance key pair: `myKeyPair`

```
> aws autoscaling create-launch-configuration --launch-configuration-name myLC --
image-id ami-0535d66c --instance-type m3.medium --security-groups sg-f57cde9d --
key-name myKeyPair
```

Security groups for instances launched in EC2-Classic may be referenced by security group name such as “SSH” or “Web” if that is what they are named, or you can reference the security group IDs, such as `sg-f57cde9d`. If you launched the instances in Amazon VPC, which is recommended, you must use the security group IDs to reference the security groups you want associated with the instances in an Auto Scaling launch configuration.

The default limit for launch configurations is 100 per region. If you exceed this limit, the call to create-launch-configuration will fail. You may view and update this limit by running `describe-account-limits` at the command line, as shown here.

```
> aws autoscaling describe-account-limits
```

Auto Scaling may cause you to reach limits of other services, such as the default number of Amazon EC2 instances you can currently launch within a region, which is 20. When building more complex architectures with AWS, it is important to keep in mind the service limits for all AWS Cloud services you are using.



When you run a command using the CLI and it fails, check your syntax first. If that checks out, verify the limits for the command you are attempting, and check to see that you have not exceeded a limit. Some limits can be raised and usually defaulted to a reasonable value to limit a race condition, an errant script running in a loop, or other similar automation that might cause unintended high usage and billing of AWS resources. AWS service limits can be viewed in the AWS General Reference Guide under AWS Service Limits. You can raise your limits by creating a support case at the AWS Support Center online and then choosing Service Limit Increase under Regarding. Then fill in the appropriate service and limit to increase value in the online form.

Auto Scaling Group

An Auto Scaling group is a collection of Amazon EC2 instances managed by the Auto Scaling service. Each Auto Scaling group contains configuration options that control when Auto Scaling should launch new instances and terminate existing instances. An Auto Scaling group must contain a name and a minimum and maximum number of instances that can be in the group. You can optionally specify desired capacity, which is the number of instances that the group must have at all times. If you don’t specify a desired capacity, the default desired capacity is the minimum number of instances that you specify.

The CLI command that follows will create an Auto Scaling group that references the previous launch configuration and includes the following specifications:

Name: `myASG`

Launch configuration: `myLC`

Availability Zones: `us-east-1a` and `us-east-1c`

Minimum size: `1`

Desired capacity: `3`

Maximum capacity: `10`

Load balancers: `myELB`

```
> aws autoscaling create-auto-scaling-group --auto-scaling-group-name myASG --
launch-configuration-name myLC --availability-zones us-east-1a, us-east-1c --min-
size 1 --max-size 10 --desired-capacity 3 --load-balancer-names myELB
```

Figure 5.1 depicts deployed AWS resources after a load balancer named `myELB` is created and the launch configuration `myLC` and Auto Scaling Group `myASG` are set up.

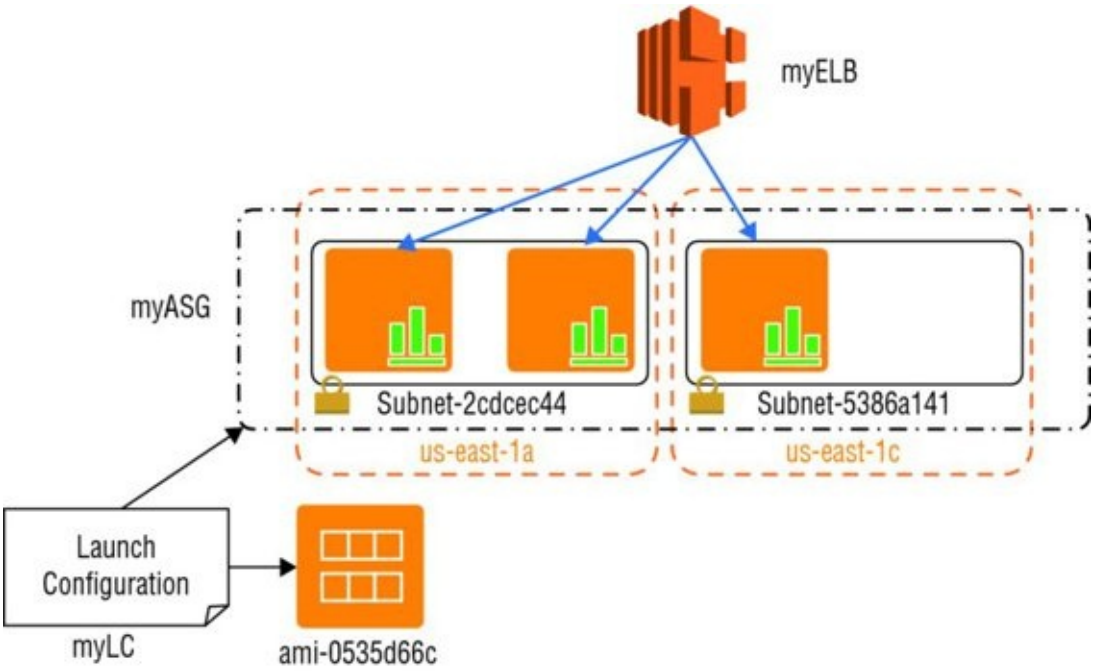


FIGURE 5.1 Auto Scaling group behind an Elastic Load Balancing load balancer

An Auto Scaling group can use either On-Demand or Spot Instances as the Amazon EC2 instances it manages. On-Demand is the default, but Spot Instances can be used by referencing a maximum bid price in the launch configuration (`–spot-price "0.15"`) associated with the Auto Scaling group. You may change the bid price by creating a new launch configuration with the new bid price and then associating it with your Auto Scaling group. If instances are available at or below your bid price, they will be launched in your Auto Scaling group. Spot Instances in an Auto Scaling group follow the same guidelines as Spot Instances outside an Auto Scaling group and require applications that are flexible and can tolerate Amazon EC2 instances that are terminated with short notice, for example, when the Spot price rises above the bid price you set in the launch configuration. A launch configuration can reference On-Demand Instances or Spot Instances, but not both.

Spot On!

Auto Scaling supports using cost-effective Spot Instances. This can be very useful when you are hosting sites where you want to provide additional compute capacity but are price constrained. An example is a “freemium” site model where you may offer some basic functionality to users for free and additional functionality for premium users who pay for use. Spot Instances can be used for providing the basic functionality when available by referencing a maximum bid price in the launch configuration (`–spot-price "0.15"`) associated with the Auto Scaling group.

Scaling Policy

You can associate Amazon CloudWatch alarms and *scaling policies* with an Auto Scaling group to adjust

Auto Scaling dynamically. When a threshold is crossed, Amazon CloudWatch sends alarms to trigger changes (scaling in or out) to the number of Amazon EC2 instances currently receiving traffic behind a load balancer. After the Amazon CloudWatch alarm sends a message to the Auto Scaling group, Auto Scaling executes the associated policy to scale your group. The policy is a set of instructions that tells Auto Scaling whether to scale out, launching new Amazon EC2 instances referenced in the associated launch configuration, or to scale in and terminate instances.

There are several ways to configure a scaling policy: You can increase or decrease by a specific number of instances, such as adding two instances; you can target a specific number of instances, such as a maximum of five total Amazon EC2 instances; or you can adjust based on a percentage. You can also scale by steps and increase or decrease the current capacity of the group based on a set of scaling adjustments that vary based on the size of the alarm threshold trigger.

You can associate more than one scaling policy with an Auto Scaling group. For example, you can create a policy using the trigger for CPU utilization, called *CPULoad*, and the CloudWatch metric *CPUUtilization* to specify scaling out if CPU utilization is greater than 75 percent for two minutes. You could attach another policy to the same Auto Scaling group to scale in if CPU utilization is less than 40 percent for 20 minutes.

The following CLI commands will create the scaling policy just described.

```
> aws autoscaling put-scaling-policy --auto-scaling-group-name myASG --policy-name
CPULoadScaleOut --scaling-adjustment 1 --adjustment-type ChangeInCapacity --
cooldown 30  > aws autoscaling put-scaling-policy --auto-scaling-group-name myASG -
-policy-name CPULoadScaleIn --scaling-adjustment -1 --adjustment-type
ChangeInCapacity --cooldown 600
```

The following CLI commands will associate Amazon CloudWatch alarms for scaling out and scaling in with the scaling policy, as shown in Figure 5.2. In this example, the Amazon CloudWatch alarms reference the scaling policy by Amazon Resource Name (ARN).

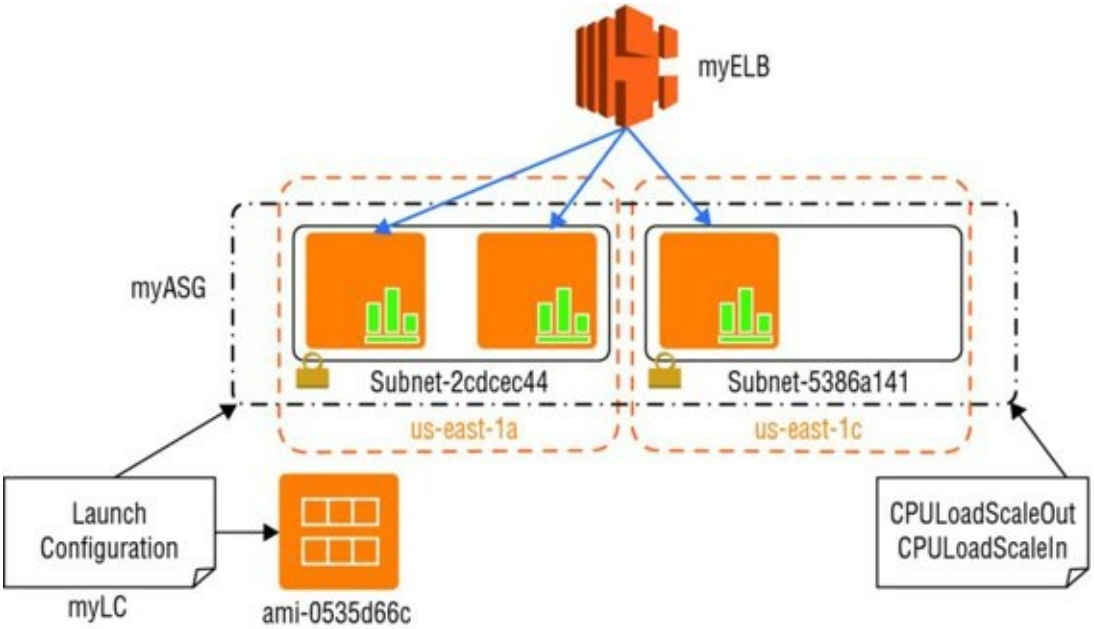


FIGURE 5.2 Auto Scaling group with policy

```
 > aws cloudwatch put-metric-alarm --alarm name capacityAdd --metric-name
CPUUtilization --namespace AWS/EC2 --statistic Average –-period 300 --threshold 75
--comparison-operator GreaterThanOrEqualToThreshold --dimensions
"Name=AutoScalingGroupName, Value=myASG" --evaluation-periods 1 --alarm-actions
```

```
arn:aws:autoscaling:us-east-1:123456789012:scalingPolicy:12345678-90ab-cdef-
1234567890ab:autoScalingGroupName/myASG:policyName/CPULoadScaleOut --unit Percent
> aws cloudwatch put-metric-alarm --alarm name capacityReduce --metric-name
CPUUtilization --namespace AWS/EC2 --statistic Average --period 1200 --threshold 40
--comparison-operator GreaterThanOrEqualToThreshold --dimensions
"Name=AutoScalingGroupName, Value=myASG" --evaluation-periods 1 --alarm-actions
arn:aws:autoscaling:us-east-1:123456789011:scalingPolicy:11345678-90ab-cdef-
1234567890ab:autoScalingGroupName/myASG:policyName/CPULoadScaleIn --unit Percent
```

If the scaling policy defined in the previous paragraph is associated with the Auto Scaling group named `myASG`, and the CPU utilization is over 75 percent for more than five minutes, as shown in Figure 5.3, a new Amazon EC2 instance will be launched and attached to the load balancer named `myELB`.

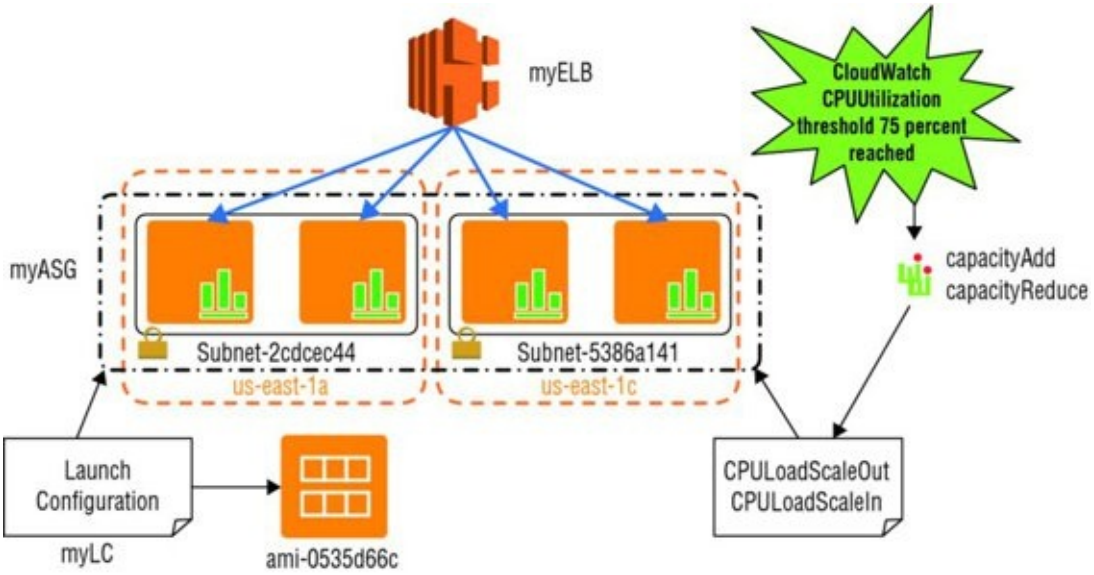


FIGURE 5.3 Amazon CloudWatch alarm triggering scaling out

A recommended best practice is to scale out quickly and scale in slowly so you can respond to bursts or spikes but avoid inadvertently terminating Amazon EC2 instances too quickly, only having to launch more Amazon EC2 instances if the burst is sustained. Auto Scaling also supports a *cooldown period*, which is a configurable setting that determines when to suspend scaling activities for a short time for an Auto Scaling group.

If you start an Amazon EC2 instance, you will be billed for one full hour of running time. Partial instance hours consumed are billed as full hours. This means that if you have a permissive scaling policy that launches, terminates, and relaunches many instances an hour, you are billing a full hour for each and every instance you launch, even if you terminate some of those instances in less than hour. A recommended best practice for cost effectiveness is to scale out quickly when needed but scale in more slowly to avoid having to relaunch new and separate Amazon EC2 instances for a spike in workload demand that fluctuates up and down within minutes but generally continues to need more resources within an hour.



Scale out quickly; scale in slowly.

It is important to consider bootstrapping for Amazon EC2 instances launched using Auto Scaling. It takes time to configure each newly launched Amazon EC2 instance before the instance is healthy and capable of accepting traffic. Instances that start and are available for load faster can join the capacity pool more quickly. Furthermore, instances that are more stateless instead of stateful will more gracefully enter and

exit an Auto Scaling group.

Rolling Out a Patch at Scale

In large deployments of Amazon EC2 instances, Auto Scaling can be used to make rolling out a patch to your instances easy. The launch configuration associated with the Auto Scaling group may be modified to reference a new AMI and even a new Amazon EC2 instance if needed. Then you can deregister or terminate instances one at a time or in small groups, and the new Amazon EC2 instances will reference the new patched AMI.

Summary

This chapter introduced three services:

- Elastic Load Balancing, which is used to distribute traffic across a group of Amazon EC2 instances in one or more Availability Zones to achieve greater levels of fault tolerance for your applications.
- Amazon CloudWatch, which monitors resources and applications. Amazon CloudWatch is used to collect and track metrics, create alarms that send notifications, and make changes to resources being monitored based on rules you define.
- Auto Scaling, which allows you to automatically scale your Amazon EC2 capacity out and in using criteria that you define.

These three services can be used very effectively together to create a highly available application with a resilient architecture on AWS.

Exam Essentials

Understand what the Elastic Load Balancing service provides. Elastic Load Balancing is a highly available service that distributes traffic across Amazon EC2 instances and includes options that provide flexibility and control of incoming requests to Amazon EC2 instances.

Know the types of load balancers the Elastic Load Balancing service provides and when to use each one. An Internet-facing load balancer is, as the name implies, a load balancer that takes requests from clients over the Internet and distributes them to Amazon EC2 instances that are registered with the load balancer.

An internal load balancer is used to route traffic to your Amazon EC2 instances in VPCs with private subnets.

An HTTPS load balancer is used when you want to encrypt data between your load balancer and the clients that initiate HTTPS sessions and for connections between your load balancer and your back-end instances.

Know the types of listeners the Elastic Load Balancing service provides and the use case and requirements for using each one. A listener is a process that checks for connection requests. It is configured with a protocol and a port for front-end (client to load balancer) connections and a protocol and a port for back-end (load balancer to back-end instance) connections.

Understand the configuration options for Elastic Load Balancing. Elastic Load Balancing allows you to configure many aspects of the load balancer, including idle connection timeout, cross-zone load balancing, connection draining, proxy protocol, sticky sessions, and health checks.

Know what an Elastic Load Balancing health check is and why it is important. Elastic Load Balancing supports health checks to test the status of the Amazon EC2 instances behind an Elastic Load Balancing load balancer.

Understand what the amazon CloudWatch service provides and what use cases there are for using it. Amazon CloudWatch is a service that you can use to monitor your AWS resources and your applications in real time. With Amazon CloudWatch, you can collect and track metrics, create alarms that send notifications, and make changes to the resources being monitored based on rules you define.

For example, you might choose to monitor CPU utilization to decide when to add or remove Amazon EC2 instances in an application tier. Or, if a particular application-specific metric that is not visible to AWS is the best indicator for assessing your scaling needs, you can perform a `PUT` request to push that metric into Amazon CloudWatch. You can then use this custom metric to manage capacity.

Know the differences between the two types of monitoring—basic and detailed—for Amazon CloudWatch. Amazon CloudWatch offers basic or detailed monitoring for supported AWS products. Basic monitoring sends data points to Amazon CloudWatch every five minutes for a limited number of preselected metrics at no charge. Detailed monitoring sends data points to Amazon CloudWatch every minute and allows data aggregation for an additional charge. If you want to use detailed monitoring, you must enable it—basic is the default.

Understand Auto Scaling and why it is an important advantage of the AWS Cloud. A distinct advantage of deploying applications to the cloud is the ability to launch and then release servers in response to variable workloads. Provisioning servers on demand and then releasing them when they are

no longer needed can provide significant cost savings for workloads that are not steady state.

Know when and why to use Auto Scaling. Auto Scaling is a service that allows you to scale your Amazon EC2 capacity automatically by scaling out and scaling in according to criteria that you define. With Auto Scaling, you can ensure that the number of running Amazon EC2 instances increases during demand spikes or peak demand periods to maintain application performance and decreases automatically during demand lulls or troughs to minimize costs.

Know the supported Auto Scaling plans. Auto Scaling has several schemes or plans that you can use to control how you want Auto Scaling to perform. The Auto Scaling plans are named Maintain Current Instant Levels, Manual Scaling, Scheduled Scaling, and Dynamic Scaling.

Understand how to build an Auto Scaling launch configuration and an Auto Scaling group and what each is used for. A launch configuration is the template that Auto Scaling uses to create new instances and is composed of the configuration name, AMI, Amazon EC2 instance type, security group, and instance key pair.

Know what a scaling policy is and what use cases to use it for. A scaling policy is used by Auto Scaling with CloudWatch alarms to determine when your Auto Scaling group should scale out or scale in. Each CloudWatch alarm watches a single metric and sends messages to Auto Scaling when the metric breaches a threshold that you specify in your policy.

Understand how Elastic Load Balancing, amazon CloudWatch, and Auto Scaling are used together to provide dynamic scaling. Elastic Load Balancing, Amazon CloudWatch, and Auto Scaling can be used together to create a highly available application with a resilient architecture on AWS.

Exercises

For assistance in completing the following exercises, refer to the Elastic Load Balancing Developer Guide located at http://docs.aws.amazon.com/ElasticLoadBalancing/latest/DeveloperGuide/elastic-load-balancing.html, the Amazon CloudWatch Developer Guide at http://docs.aws.amazon.com/AmazonCloudWatch/latest/DeveloperGuide/WhatIsCloudWatch.h and the Auto Scaling User Guide at http://docs.aws.amazon.com/autoscaling/latest/userguide/WhatIsAutoScaling.html.

EXERCISE 5.1

Create an Elastic Load Balancing Load Balancer

In this exercise, you will use the AWS Management Console to create an Elastic Load Balancing load balancer.

1. Launch an Amazon EC2 instance using an AMI with a web server on it, or install and configure a web server.
2. Create a static page to display and a health check page that returns HTTP 200. Configure the Amazon EC2 instance to accept traffic over port 80.
3. Register the Amazon EC2 instance with the Elastic Load Balancing load balancer, and configure it to use the health check page to evaluate the health of the instance.

EXERCISE 5.2

Use an Amazon CloudWatch Metric

1. Launch an Amazon EC2 instance.
2. Use an existing Amazon CloudWatch metric to monitor a value.

EXERCISE 5.3

Create a Custom Amazon CloudWatch Metric

1. Create a custom Amazon CloudWatch metric for memory consumption.
2. Use the CLI to `PUT` values into the metric.

EXERCISE 5.4

Create a Launch Configuration and Auto Scaling Group

1. Using the AWS Management Console, create a launch configuration using an existing AMI.
2. Create an Auto Scaling group using this launch configuration with a group size of four and spanning two Availability Zones. Do not use a scaling policy. Keep the group at its initial size.
3. Manually terminate an Amazon EC2 instance, and observe Auto Scaling launch a new Amazon EC2 instance.

EXERCISE 5.5

Create a Scaling Policy

1. Create an Amazon Cloud Watch metric and alarm for CPU utilization using the AWS Management Console.
2. Using the Auto Scaling group from Exercise 5.4, edit the Auto Scaling group to include a policy that uses the CPU utilization alarm.
3. Drive CPU utilization on the monitored Amazon EC2 instance(s) up to observe Auto Scaling.

EXERCISE 5.6

Create a Web Application That Scales

1. Create a small web application architected with an Elastic Load Balancing load balancer, an Auto Scaling group spanning two Availability Zones that uses an Amazon CloudWatch metric, and an alarm attached to a scaling policy used by the Auto Scaling group.
2. Verify that Auto Scaling is operating correctly by removing instances and driving the metric up and down to force Auto Scaling.

Review Questions

1. Which of the following are required elements of an Auto Scaling group? (Choose 2 answers)
   A. Minimum size
   B. Health checks
   C. Desired capacity
   D. Launch configuration
2. You have created an Elastic Load Balancing load balancer listening on port 80, and you registered it with a single Amazon Elastic Compute Cloud (Amazon EC2) instance also listening on port 80. A client makes a request to the load balancer with the correct protocol and port for the load balancer. In this scenario, how many connections does the balancer maintain?
   A. 1
   B. 2
   C. 3
   D. 4
3. How long does Amazon CloudWatch keep metric data?
   A. 1 day
   B. 2 days
   C. 1 week
   D. 2 weeks
4. Which of the following are the minimum required elements to create an Auto Scaling launch configuration?
   A. Launch configuration name, Amazon Machine Image (AMI), and instance type
   B. Launch configuration name, AMI, instance type, and key pair
   C. Launch configuration name, AMI, instance type, key pair, and security group
   D. Launch configuration name, AMI, instance type, key pair, security group, and block device mapping
5. You are responsible for the application logging solution for your company's existing applications running on multiple Amazon EC2 instances. Which of the following is the best approach for aggregating the application logs within AWS?
   A. Amazon CloudWatch custom metrics
   B. Amazon CloudWatch Logs Agent
   C. An Elastic Load Balancing listener
   D. An internal Elastic Load Balancing load balancer

6. Which of the following must be configured on an Elastic Load Balancing load balancer to accept incoming traffic?
   A. A port
   B. A network interface
   C. A listener
   D. An instance
7. You create an Auto Scaling group in a new region that is configured with a minimum size value of 10, a maximum size value of 100, and a desired capacity value of 50. However, you notice that 30 of the Amazon Elastic Compute Cloud (Amazon EC2) instances within the Auto Scaling group fail to launch. Which of the following is the cause of this behavior?
   A. You cannot define an Auto Scaling group larger than 20.
   B. The Auto Scaling group maximum value cannot be more than 20.
   C. You did not attach an Elastic Load Balancing load balancer to the Auto Scaling group.
   D. You have not raised your default Amazon EC2 capacity (20) for the new region.
8. You want to host multiple Hypertext Transfer Protocol Secure (HTTPS) websites on a fleet of Amazon EC2 instances behind an Elastic Load Balancing load balancer with a single X.509 certificate. How must you configure the Secure Sockets Layer (SSL) certificate so that clients connecting to the load balancer are not presented with a warning when they connect?
   A. Create one SSL certificate with a Subject Alternative Name (SAN) value for each website name.
   B. Create one SSL certificate with the Server Name Indication (SNI) value checked.
   C. Create multiple SSL certificates with a SAN value for each website name.
   D. Create SSL certificates for each Availability Zone with a SAN value for each website name.
9. Your web application front end consists of multiple Amazon Compute Cloud (Amazon EC2) instances behind an Elastic Load Balancing load balancer. You have configured the load balancer to perform health checks on these Amazon EC2 instances. If an instance fails to pass health checks, which statement will be true?
   A. The instance is replaced automatically by the load balancer.
   B. The instance is terminated automatically by the load balancer.
   C. The load balancer stops sending traffic to the instance that failed its health check.
   D. The instance is quarantined by the load balancer for root cause analysis.
10. In the basic monitoring package for Amazon Elastic Compute Cloud (Amazon EC2), what Amazon CloudWatch metrics are available?
    A. Web server visible metrics such as number of failed transaction requests
    B. Operating system visible metrics such as memory utilization
    C. Database visible metrics such as number of connections

D. Hypervisor visible metrics such as CPU utilization

11. A cell phone company is running dynamic-content television commercials for a contest. They want their website to handle traffic spikes that come after a commercial airs. The website is interactive, offering personalized content to each visitor based on location, purchase history, and the current commercial airing. Which architecture will configure Auto Scaling to scale out to respond to spikes of demand, while minimizing costs during quiet periods?

    A. Set the minimum size of the Auto Scaling group so that it can handle high traffic volumes without needing to scale out.

    B. Create an Auto Scaling group large enough to handle peak traffic loads, and then stop some instances. Configure Auto Scaling to scale out when traffic increases using the stopped instances, so new capacity will come online quickly.

    C. Configure Auto Scaling to scale out as traffic increases. Configure the launch configuration to start new instances from a preconfigured Amazon Machine Image (AMI).

    D. Use Amazon CloudFront and Amazon Simple Storage Service (Amazon S3) to cache changing content, with the Auto Scaling group set as the origin. Configure Auto Scaling to have sufficient instances necessary to initially populate CloudFront and Amazon ElastiCache, and then scale in after the cache is fully populated.

12. For an application running in the ap-northeast-1 region with three Availability Zones (ap-northeast-1a, ap-northeast-1b, and ap-northeast-1c), which instance deployment provides high availability for the application that normally requires nine running Amazon Elastic Compute Cloud (Amazon EC2) instances but can run on a minimum of 65 percent capacity while Auto Scaling launches replacement instances in the remaining Availability Zones?

    A. Deploy the application on four servers in ap-northeast-1a and five servers in ap-northeast-1b, and keep five stopped instances in ap-northeast-1a as reserve.

    B. Deploy the application on three servers in ap-northeast-1a, three servers in ap-northeast-1b, and three servers in ap-northeast-1c.

    C. Deploy the application on six servers in ap-northeast-1b and three servers in ap-northeast-1c.

    D. Deploy the application on nine servers in ap-northeast-1b, and keep nine stopped instances in ap-northeast-1a as reserve.

13. Which of the following are characteristics of the Auto Scaling service on AWS? (Choose 3 answers)

    A. Sends traffic to healthy instances

    B. Responds to changing conditions by adding or terminating Amazon Elastic Compute Cloud (Amazon EC2) instances

    C. Collects and tracks metrics and sets alarms

    D. Delivers push notifications

    E. Launches instances from a specified Amazon Machine Image (AMI)

    F. Enforces a minimum number of running Amazon EC2 instances

14. Why is the launch configuration referenced by the Auto Scaling group instead of being part of the Auto Scaling group?

A. It allows you to change the Amazon Elastic Compute Cloud (Amazon EC2) instance type and Amazon Machine Image (AMI) without disrupting the Auto Scaling group.

B. It facilitates rolling out a patch to an existing set of instances managed by an Auto Scaling group.

C. It allows you to change security groups associated with the instances launched without having to make changes to the Auto Scaling group.

D. All of the above

E. None of the above

15. An Auto Scaling group may use: (Choose 2 answers)

A. On-Demand Instances

B. Stopped instances

C. Spot Instances

D. On-premises instances

E. Already running instances if they use the same Amazon Machine Image (AMI) as the Auto Scaling group's launch configuration and are not already part of another Auto Scaling group

16. Amazon CloudWatch supports which types of monitoring plans? (Choose 2 answers)

A. Basic monitoring, which is free

B. Basic monitoring, which has an additional cost

C. Ad hoc monitoring, which is free

D. Ad hoc monitoring, which has an additional cost

E. Detailed monitoring, which is free

F. Detailed monitoring, which has an additional cost

17. Elastic Load Balancing health checks may be: (Choose 3 answers)

A. A ping

B. A key pair verification

C. A connection attempt

D. A page request

E. An Amazon Elastic Compute Cloud (Amazon EC2) instance status check

18. When an Amazon Elastic Compute Cloud (Amazon EC2) instance registered with an Elastic Load Balancing load balancer using connection draining is deregistered or unhealthy, which of the following will happen? (Choose 2 answers)

A. Immediately close all existing connections to that instance.

B. Keep the connections open to that instance, and attempt to complete in-flight requests.

C. Redirect the requests to a user-defined error page like "Oops this is embarrassing" or "Under Construction."

D. Forcibly close all connections to that instance after a timeout period.

E. Leave the connections open as long as the load balancer is running.

19. Elastic Load Balancing supports which of the following types of load balancers? (Choose 3 answers)

A. Cross-region

B. Internet-facing

C. Interim

D. Itinerant

E. Internal

F. Hypertext Transfer Protocol Secure (HTTPS) using Secure Sockets Layer (SSL)

20. Auto Scaling supports which of the following plans for Auto Scaling groups? (Choose 3 answers)

A. Predictive

B. Manual

C. Preemptive

D. Scheduled

E. Dynamic

F. End-user request driven

G. Optimistic

Chapter 6
AWS Identity and Access Management (IAM)

THE AWS CERTIFIED SOLUTIONS ARCHITECT ASSOCIATE EXAM OBJECTIVES COVERED IN THIS CHAPTER MAY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

Domain 2.0: Implementation/Deployment

✓ **2.1 Identify the appropriate techniques and methods using Amazon EC2, Amazon S3, Elastic Beanstalk, CloudFormation, Amazon Virtual Private Cloud (VPC), and AWS Identity and Access Management (IAM) to code and implement a cloud solution.**

Content may include the following:

- Configure IAM policies and best practices

Domain 3.0: Data Security

✓ **3.1 Recognize and implement secure practices for optimum cloud deployment and maintenance.**

Content may include the following:

- AWS Identity and Access Management (IAM)



Introduction

In this chapter, you will learn how *AWS Identity and Access Management (IAM)* secures interactions with the AWS resources in your account, including:

- Which principals interact with AWS through the AWS Management Console, Command Line Interface (CLI), and Software Development Kits (SDKs)
- How each principal is authenticated
- How IAM policies are written to specify the access privileges of principals
- How IAM policies are associated with principals
- How to secure your infrastructure further through Multi-Factor Authentication (MFA) and key rotation

- How IAM roles can be used to delegate permissions and federate users
- How to resolve multiple, possibly conflicting IAM permissions

IAM is a powerful service that allows you to control how people and programs are allowed to manipulate your AWS infrastructure. IAM uses traditional identity concepts such as users, groups, and access control policies to control who can use your AWS account, what services and resources they can use, and how they can use them. The control provided by IAM is granular enough to limit a single user to the ability to perform a single action on a specific resource from a specific IP address during a specific time window. Applications can be granted access to AWS resources whether they are running on-premises or in the cloud. This flexibility creates a very powerful system that will give you all the power you need to ensure that your AWS account users have the ability to meet your business needs while addressing all of the security concerns of your organization.

This chapter will cover the different principals that can interact with AWS and how they are authenticated. It will then discuss how to write policies that define permitted access to services, actions, and resources and associate these policies with authenticated principals. Finally, it will cover additional features of IAM that will help you secure your infrastructure, including MFA, rotating keys, federation, resolving multiple permissions, and using IAM roles.

As important as it is to know what IAM is exactly, it is equally important to understand what it is not:

- First, IAM is not an identity store/authorization system for your applications. The permissions that you assign are permissions to manipulate AWS infrastructure, not permissions within your application. If you are migrating an existing on-premises application that already has its own user repository and authentication/authorization mechanism, then that should continue to work when you deploy on AWS and is probably the right choice. If your application identities are based on Active Directory, your on-premises Active Directory can be extended into the cloud to continue to fill that need. A great solution for using Active Directory in the cloud is AWS Directory Service, which is an Active Directory-compatible directory service that can work on its own or integrate with your on-premises Active Directory. Finally, if you are working with a mobile app, consider *Amazon Cognito* for identity management for mobile applications.
- Second, IAM is not operating system identity management. Remember that under the shared responsibility model, you are in control of your operating system console and configuration. Whatever mechanism you currently use to control access to your server infrastructure will continue to work on Amazon Elastic Compute Cloud (Amazon EC2) instances, whether that is managing individual machine login accounts or a directory service such as Active Directory or Lightweight Directory Access Protocol (LDAP). You can run an Active Directory or LDAP server on Amazon EC2, or you can extend your on-premises system into the cloud. AWS Directory Service will also work well to provide Active Directory functionality in the cloud as a service, whether standalone or integrated with your existing Active Directory.

Table 6.1 summarizes the role that different authentication systems can play in your AWS environment.

TABLE 6.1 Authentication Technologies

Use Case	Technology Solutions
Operating System Access	Active Directory LDAP Machine-specific accounts
Application Access	Active Directory Application User Repositories Amazon Cognito
AWS Resources	IAM

IAM is controlled like most other AWS Cloud services:

- Through the *AWS Management Console*—Like other services, the AWS Management Console is the easiest way to start learning about and manipulating a service.
- With the *CLI*—As you learn the system, you can start scripting repeated tasks using the CLI.
- Via the *AWS SDKs*—Eventually you may start writing your own tools and complex processes by manipulating IAM directly through the REST API via one of several SDKs.

All of these methods work to control IAM just as they work with other services. In addition, the AWS Partner Network (APN) includes a rich ecosystem of tools to manage and extend IAM.

Principals

The first IAM concept to understand is principals. A *principal* is an IAM entity that is allowed to interact with AWS resources. A principal can be permanent or temporary, and it can represent a human or an application. There are three types of principals: root users, IAM users, and roles/temporary security tokens.

Root User

When you first create an AWS account, you begin with only a single sign-in principal that has complete access to all AWS Cloud services and resources in the account. This principal is called the *root user*. As long as you have an open account with AWS, the root user for that relationship will persist. The root user can be used for both console and programmatic access to AWS resources.

The root user is similar in concept to the UNIX root or Windows Administrator account—it has full privileges to do anything in the account, including closing the account. It is strongly recommended that you do not use the root user for your everyday tasks, even the administrative ones. Instead, adhere to the best practice of using the root user only to create your first IAM user and then securely locking away the root user credentials.

IAM Users

Users are persistent identities set up through the IAM service to represent individual people or applications. You may create separate IAM users for each member of your operations team so they can interact with the console and use the CLI. You might also create dev, test, and production users for applications that need to access AWS Cloud services (although you will see later in this chapter that IAM roles may be a better solution for that use case).

IAM users can be created by principals with IAM administrative privileges at any time through the AWS Management Console, CLI, or SDKs. Users are persistent in that there is no expiration period; they are permanent entities that exist until an *IAM administrator* takes an action to delete them.



Users are an excellent way to enforce the principle of least privilege; that is, the concept of allowing a person or process interacting with your AWS resources to perform exactly the tasks they need but nothing else. Users can be associated with very granular policies that define these permissions. Policies will be covered in a later section.

Roles/Temporary Security Tokens

Roles and temporary security tokens are very important for advanced IAM usage, but many AWS users find them confusing. Roles are used to grant specific privileges to specific actors for a set duration of time. These actors can be authenticated by AWS or some trusted external system. When one of these actors assumes a role, AWS provides the actor with a temporary security token from the *AWS Security Token Service (STS)* that the actor can use to access AWS Cloud services. Requesting a temporary security token requires specifying how long the token will exist before it expires. The range of a

temporary security token lifetime is 15 minutes to 36 hours.

Roles and temporary security tokens enable a number of use cases:

- ***Amazon EC2 Roles***—Granting permissions to applications running on an Amazon EC2 instance.
- ***Cross-Account Access***—Granting permissions to users from other AWS accounts, whether you control those accounts or not.
- ***Federation***—Granting permissions to users authenticated by a trusted external system.

Amazon EC2 Roles

Granting permissions to an application is always tricky, as it usually requires configuring the application with some sort of credential upon installation. This leads to issues around securely storing the credential prior to use, how to access it safely during installation, and how to secure it in the configuration. Suppose that an application running on an Amazon EC2 instance needs to access an Amazon Simple Storage Service (Amazon S3) bucket. A policy granting permission to read and write that bucket can be created and assigned to an IAM user, and the application can use the access key for that IAM user to access the Amazon S3 bucket. The problem with this approach is that the access key for the user must be accessible to the application, probably by storing it in some sort of configuration file. The process for obtaining the access key and storing it encrypted in the configuration is usually complicated and a hindrance to agile development. Additionally, the access key is at risk when being passed around. Finally, when the time comes to rotate the access key, the rotation involves performing that whole process again.



Using IAM roles for Amazon EC2 removes the need to store AWS credentials in a configuration file.

An alternative is to create an IAM role that grants the required access to the Amazon S3 bucket. When the Amazon EC2 instance is launched, the role is assigned to the instance. When the application running on the instance uses the Application Programming Interface (API) to access the Amazon S3 bucket, it assumes the role assigned to the instance and obtains a temporary token that it sends to the API. The process of obtaining the temporary token and passing it to the API is handled automatically by most of the AWS SDKs, allowing the application to make a call to access the Amazon S3 bucket without worrying about authentication. In addition to being easy for the developer, this removes any need to store an access key in a configuration file. Also, because the API access uses a temporary token, there is no fixed access key that must be rotated.

Cross-Account Access

Another common use case for IAM roles is to grant access to AWS resources to IAM users in other AWS accounts. These accounts may be other AWS accounts controlled by your company or outside agents like customers or suppliers. You can set up an IAM role with the permissions you want to grant to users in the other account, then users in the other account can assume that role to access your resources. This is highly recommended as a best practice, as opposed to distributing access keys outside your organization.

Federation

Many organizations already have an identity repository outside of AWS and would rather leverage that repository than create a new and largely duplicate repository of IAM users. Similarly, web-based

applications may want to leverage web-based identities such as Facebook, Google, or Login with Amazon. *IAM Identity Providers* provide the ability to federate these outside identities with IAM and assign privileges to those users authenticated outside of IAM.

IAM can integrate with two different types of outside *Identity Providers (IdP)*. For federating web identities such as Facebook, Google, or Login with Amazon, IAM supports integration via OpenID Connect (OIDC). This allows IAM to grant privileges to users authenticated with some of the major web-based IdPs. For federating internal identities, such as Active Directory or LDAP, IAM supports integration via Security Assertion Markup Language 2.0 (SAML). A SAML-compliant IdP such as Active Directory Federation Services (ADFS) is used to federate the internal directory to IAM. (Instructions for configuring many compatible products can be found on the AWS website.) In each case, federation works by returning a temporary token associated with a role to the IdP for the authenticated identity to use for calls to the AWS API. The actual role returned is determined via information received from the IdP, either attributes of the user in the on-premises identity store or the user name and authenticating service of the web identity store.

The three types of principals and their general traits are listed in Table 6.2.

TABLE 6.2 Traits of AWS Principals

Principal	**Traits**
Root User	Cannot be limited Permanent
IAM Users	Access controlled by policy Durable Can be removed by IAM administrator
Roles/Temporary Security Tokens	Access controlled by policy Temporary Expire after specific time interval

Authentication

There are three ways that IAM *authenticates* a principal:

- ***User Name/Password***—When a principal represents a human interacting with the console, the human will provide a user name/password pair to verify their identity. IAM allows you to create a password policy enforcing password complexity and expiration.
- ***Access Key***—An access key is a combination of an access key ID (20 characters) and an access secret key (40 characters). When a program is manipulating the AWS infrastructure via the API, it will use these values to sign the underlying REST calls to the services. The AWS SDKs and tools handle all the intricacies of signing the REST calls, so using an access key will almost always be a matter of providing the values to the SDK or tool.
- ***Access Key/Session Token***—When a process operates under an assumed role, the temporary security token provides an access key for authentication. In addition to the access key (remember that it consists of two parts), the token also includes a *session token*. Calls to AWS must include both the two-part access key and the session token to authenticate.

It is important to note that when an IAM user is created, it has neither an access key nor a password, and the IAM administrator can set up either or both. This adds an extra layer of security in that console users cannot use their credentials to run a program that accesses your AWS infrastructure.

Figure 6.1 shows a summary of the different authentication methods.

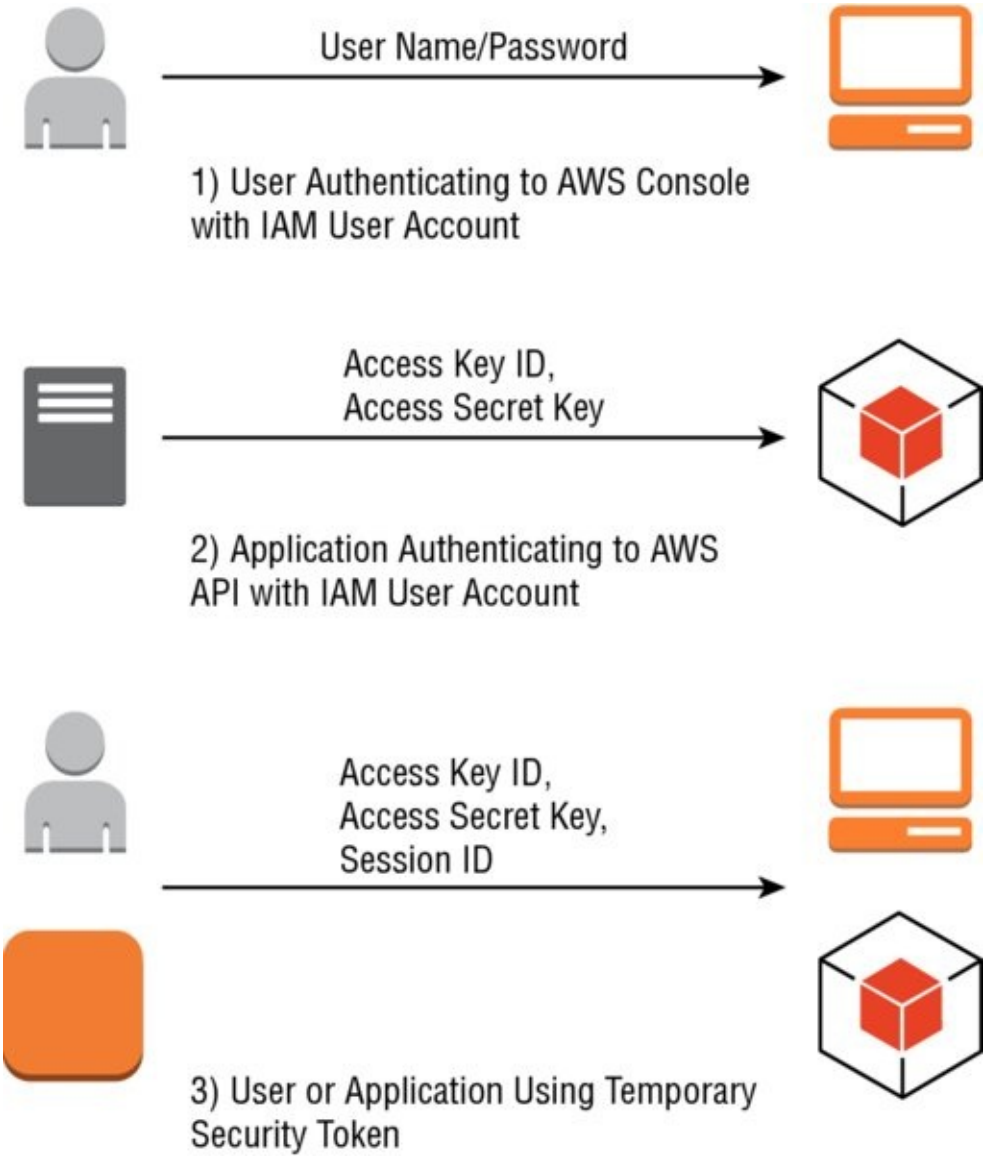


FIGURE 6.1 Different identities authenticating with AWS

Authorization

After IAM has authenticated a principal, it must then manage the access of that principal to protect your AWS infrastructure. The process of specifying exactly what actions a principal can and cannot perform is called *authorization*. Authorization is handled in IAM by defining specific privileges in *policies* and associating those policies with principals.

Policies

Understanding how access management works under IAM begins with understanding policies. A *policy* is a JSON document that fully defines a set of permissions to access and manipulate AWS resources. Policy documents contain one or more permissions, with each permission defining:

- ***Effect***—A single word: Allow or Deny.
- ***Service***—For what service does this permission apply? Most AWS Cloud services support granting access through IAM, including IAM itself.
- ***Resource***—The resource value specifies the specific AWS infrastructure for which this permission applies. This is specified as an *Amazon Resource Name (ARN)*. The format for an ARN varies slightly between services, but the basic format is:

  ```
  "arn:aws:service:region:account-id:[resourcetype:]resource"
  ```

For some services, wildcard values are allowed; for instance, an Amazon S3 ARN could have a resource of `foldername\*` to indicate all objects in the specified folder. Table 6.3 displays some sample ARNs.

TABLE 6.3 Sample ARNs

Resource	ARN Format
Amazon S3 Bucket	`arn:aws:s3:us-east-1:123456789012:my_corporate_bucket/*`
IAM User	`arn:aws:iam:us-east-1:123456789012:user/David`
Amazon DynamoDB Table	`arn:aws:dynamodb:us-east-1:123456789012:table/tablename`

- ***Action***—The action value specifies the subset of actions within a service that the permission allows or denies. For instance, a permission may grant access to any read-based action for Amazon S3. A set of actions can be specified with an enumerated list or by using wildcards (`Read*`).
- ***Condition***—The condition value optionally defines one or more additional restrictions that limit the actions allowed by the permission. For instance, the permission might contain a condition that limits the ability to access a resource to calls that come from a specific IP address range. Another condition could restrict the permission only to apply during a specific time interval. There are many types of permissions that allow a rich variety of functionality that varies between services. See the IAM documentation for lists of supported conditions for each service.

A sample policy is shown in the following listing. This policy allows a principal to list the objects in a specific bucket and to retrieve those objects, but only if the call comes from a specific IP address.

```
{
    "Version": "2012-10-17",
```

```
    "Statement": [
        {
            "Sid": "Stmt1441716043000",
            "Effect": "Allow",  <-  This policy grants access
            "Action": [ <-  Allows identities to list
                "s3:GetObject", <-  and get objects in
                "s3:ListBucket" <-  the S3 bucket
            ],
            "Condition": {
                "IpAddress": {                          <-  Only from a specific
                    "aws:SourceIp": "192.168.0.1"       <-  IP Address
                }
            },
            "Resource": [
                "arn:aws:s3:::my_public_bucket/*"       <-  Only this bucket
            ]
        }
    ]
}
```

Associating Policies with Principals

There are several ways to associate a policy with an IAM user; this section will only cover the most common.

A policy can be associated directly with an IAM user in one of two ways:

- ***User Policy***—These policies exist only in the context of the user to which they are attached. In the console, a user policy is entered into the user interface on the IAM user page.
- ***Managed Policies***—These policies are created in the Policies tab on the IAM page (or through the CLI, and so forth) and exist independently of any individual user. In this way, the same policy can be associated with many users or groups of users. There are a large number of predefined managed policies that you can review on the Policies tab of the IAM page in the AWS Management Console. In addition, you can write your own policies specific to your use cases.


Using predefined managed policies ensures that when new permissions are added for new features, your users will still have the correct access.

The other common method for associating policies with users is with the IAM groups feature. Groups simplify managing permissions for large numbers of users. After a policy is assigned to a group, any user who is a member of that group assumes those permissions. This makes it simpler to assign policies to an entire team in your organization. For instance, if you create an “Operations” group with every IAM user for your operations team assigned to that group, then it is a simple matter to associate the needed permissions to the group, and all of the team’s IAM users will assume those permissions. New IAM users can then be assigned directly to the group.

This is a much simpler management process than having to review what policies a new IAM user for the operations team should receive and manually adding those policies to the user. There are two ways a policy can be associated with an IAM group:

- ***Group Policy***—These policies exist only in the context of the group to which they are attached. In

the AWS Management Console, a group policy is entered into the user interface on the IAM Group page.

- ***Managed Policies***—In the same way that managed policies (discussed in the "Authorization" section) can be associated with IAM users, they can also be associated with IAM groups.

Figure 6.2 shows the different ways that polices can be associated with an IAM User.

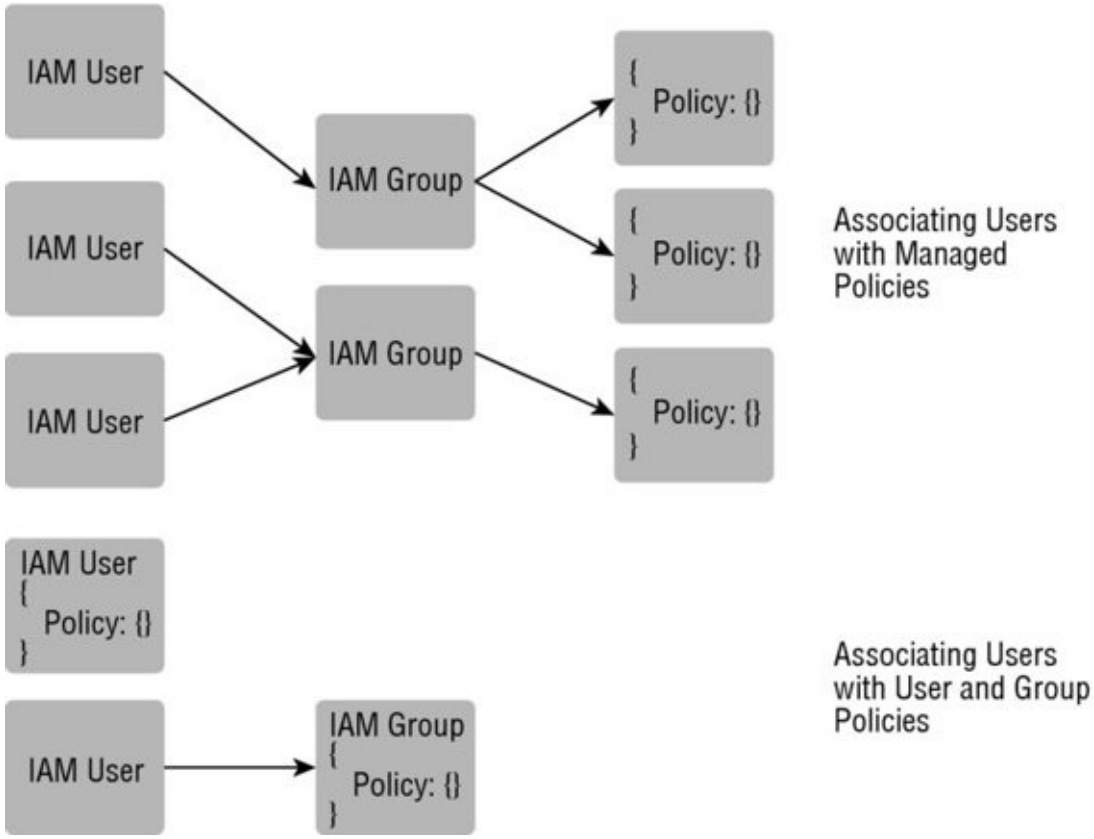


FIGURE 6.2 Associating IAM users with policies



A good first step is to use the root user to create a new IAM group called "IAM Administrators" and assign the managed policy, "IAMFullAccess." Then create a new IAM user called "Administrator," assign a password, and add it to the IAM Administrators group. At this point, you can log off as the root user and perform all further administration with the IAM user account.

The final way an actor can be associated with a policy is by assuming a role. In this case, the actor can be:

- An authenticated IAM user (person or process). In this case, the IAM user must have the rights to assume the role.
- A person or process authenticated by a trusted service outside of AWS, such as an on-premises LDAP directory or a web authentication service. In this situation, an AWS Cloud service will assume the role on the actor's behalf and return a token to the actor.

After an actor has assumed a role, it is provided with a temporary security token associated with the policies of that role. The token contains all the information required to authenticate API calls. This information includes a standard access key plus an additional session token required for authenticating

calls under an assumed role.

Other Key Features

Beyond the critical concepts of principals, authentication, and authorization, there are several other features of the IAM service that are important to understand to realize the full benefits of IAM.

Multi-Factor Authentication (MFA)

Multi-Factor Authentication (MFA) can add an extra layer of security to your infrastructure by adding a second method of authentication beyond just a password or access key. With MFA, authentication also requires entering a One-Time Password (OTP) from a small device. The MFA device can be either a small hardware device you carry with you or a virtual device via an app on your smart phone (for example, the AWS Virtual MFA app).

 MFA requires you to verify your identity with both something you *know* and something you *have*.

MFA can be assigned to any IAM user account, whether the account represents a person or application. When a person using an IAM user configured with MFA attempts to access the AWS Management Console, after providing their password they will be prompted to enter the current code displayed on their MFA device before being granted access. An application using an IAM user configured with MFA must query the application user to provide the current code, which the application will then pass to the API.

It is strongly recommended that AWS customers add MFA protection to their root user.

Rotating Keys

The security risk of any credential increases with the age of the credential. To this end, it is a security best practice to *rotate access keys* associated with your IAM users. IAM facilitates this process by allowing two active access keys at a time. The process to rotate keys can be conducted via the console, CLI, or SDKs:

1. Create a new access key for the user.
2. Reconfigure all applications to use the new access key.
3. Disable the original access key (disabling instead of deleting at this stage is critical, as it allows rollback to the original key if there are issues with the rotation).
4. Verify the operation of all applications.
5. Delete the original access key.

 Access keys should be rotated on a regular schedule.

Resolving Multiple Permissions

Occasionally, multiple permissions will be applicable when determining whether a principal has the privilege to perform some action. These permissions may come from multiple policies associated with a principal or resource policies attached to the AWS resource in question. It is important to know how conflicts between these permissions are resolved:

1. Initially the request is denied by default.
2. All the appropriate policies are evaluated; if there is an explicit “deny” found in any policy, the request is denied and evaluation stops.
3. If no explicit “deny” is found and an explicit “allow” is found in any policy, the request is allowed.
4. If there are no explicit “allow” or “deny” permissions found, then the default “deny” is maintained and the request is denied.

The only exception to this rule is if an `AssumeRole` call includes a role and a policy, the policy cannot expand the privileges of the role (for example, the policy cannot override any permission that is denied by default in the role).

Summary

IAM is a powerful service that gives you the ability to control which people and applications can access your AWS account at a very granular level. Because the root user in an AWS account cannot be limited, you should set up IAM users and temporary security tokens for your people and processes to interact with AWS.

Policies define what actions can and cannot be taken. Policies are associated with IAM users either directly or through group membership. A temporary security token is associated with a policy by assuming an IAM role. You can write your own policies or use one of the managed policies provided by AWS.

Common use cases for IAM roles include federating identities from external IdPs, assigning privileges to an Amazon EC2 instance where they can be assumed by applications running on the instance, and cross-account access.

IAM user accounts can be further secured by rotating keys, implementing MFA, and adding conditions to policies. MFA ensures that authentication is based on something you have in addition to something you know, and conditions can add further restrictions such as limiting client IP address ranges or setting a particular time interval.

Exam Essentials

Know the different principals in IAM. The three principals that can authenticate and interact with AWS resources are the root user, IAM users, and roles. The root user is associated with the actual AWS account and cannot be restricted in any way. IAM users are persistent identities that can be controlled through IAM. Roles allow people or processes the ability to operate temporarily with a different identity. People or processes assume a role by being granted a temporary security token that will expire after a specified period of time.

Know how principals are authenticated in IAM. When you log in to the AWS Management Console as an IAM user or root user, you use a user name/password combination. A program that accesses the API with an IAM user or root user uses a two-part access key. A temporary security token authenticates with an access key plus an additional session token unique to that temporary security token.

Know the parts of a policy. A policy is a JSON document that defines one or more permissions to interact with AWS resources. Each permission includes the effect, service, action, and resource. It may also include one or more conditions. AWS makes many predefined policies available as managed policies.

Know how a policy is associated with a principal. An authenticated principal is associated with zero to many policies. For an IAM user, these policies may be attached directly to the user account or attached to an IAM group of which the user account is a member. A temporary security token is associated with policies by assuming an IAM role.

Understand MFA. MFA increases the security of an AWS account by augmenting the password (something you know) with a rotating OTP from a small device (something you have), ensuring that anyone authenticating the account has both knowledge of the password and possession of the device. AWS supports both Gemalto hardware MFA devices and a number of virtual MFA apps.

Understand key rotation. To protect your AWS infrastructure, access keys should be rotated regularly. AWS allows two access keys to be valid simultaneously to make the rotation process straightforward: Generate a new access key, configure your application to use the new access key, test, disable the original access key, test, delete the original access key, and test again.

Understand IAM roles and federation. IAM roles are prepackaged sets of permissions that have no credentials. Principals can assume a role and then use the associated permissions. When a temporary security token is created, it assumes a role that defines the permissions assigned to the token. When an Amazon EC2 instance is associated with an IAM role, SDK calls acquire a temporary security token based on the role associated with the instance and use that token to access AWS resources.

Roles are the basis for federating external IdPs with AWS. You configure an IAM IdP to interact with the external IdP, the authenticated identity from the IdP is mapped to a role, and a temporary security token is returned that has assumed that role. AWS supports both SAML and OIDC IdPs.

Know how to resolve conflicting permissions. Resolving multiple permissions is relatively straightforward. If an action on a resource has not been explicitly allowed by a policy, it is denied. If two policies contradict each other; that is, if one policy allows an action on a resource and another policy denies that action, the action is denied. While this sounds improbable, it may occur due to scope differences in a policy. One policy may expose an entire fleet of Amazon EC2 instances, and a second policy may explicitly lock down one particular instance.

Exercises

For assistance in completing the following exercises, refer to the IAM User Guide at http://docs.aws.amazon.com/IAM/latest/UserGuide/.

EXERCISE 6.1

Create an IAM Group

In this exercise, you will create a group for all IAM administrator users and assign the proper permissions to the new group. This will allow you to avoid assigning policies directly to a user later in these exercises.

1. Log in as the root user.
2. Create an IAM group called `Administrators`.
3. Attach the managed policy, `IAMFullAccess`, to the `Administrators` group.

EXERCISE 6.2

Create a Customized Sign-In Link and Password Policy

In this exercise, you will set up your account with some basic IAM safeguards. The password policy is a recommended security practice, and the sign-in link makes it easier for your users to log in to the AWS Management Console.

1. Customize a sign-in link, and write down the new link name in full.
2. Create a password policy for your account.

EXERCISE 6.3

Create an IAM User

In this exercise, you will create an IAM user who can perform all administrative IAM functions. Then you will log in as that user so that you no longer need to use the root user login. Using the root user login only when explicitly required is a recommended security practice (along with adding MFA to your root user).

1. While logged in as the root user, create a new IAM user called `Administrator`.
2. Add your new user to the `Administrators` group.
3. On the Details page for the administrator user, create a password.
4. Log out as the root user.
5. Use the customized sign-in link to sign in as `Administrator`.

EXERCISE 6.4

Create and Use an IAM Role

In this exercise, you will create an IAM role, associate it with a new instance, and verify that applications running on the instance assume the permissions of the role. IAM roles allow you to avoid storing access keys on your Amazon EC2 instances.

1. While signed in as administrator, create an Amazon EC2-type role named `S3Client`.
2. Attach the managed policy, `AmazonS3ReadOnlyAccess`, to `S3Client`.
3. Launch an Amazon Linux EC2 instance with the new role attached (Amazon Linux AMIs come with CLI installed).
4. SSH into the new instance, and use the CLI to list the contents of an Amazon S3 bucket.

EXERCISE 6.5

Rotate Keys

In this exercise, you will go through the process of rotating access keys, a recommended security practice.

1. Select the administrator, and create a two-part access key.
2. Download the access key.
3. Download and install the CLI to your desktop.
4. Configure the CLI to use the access key with the AWS Configure command.
5. Use the CLI to list the contents of an Amazon S3 bucket.
6. Return to the console, and create a new access key for the administrator account.
7. Download the access key, and reconfigure the CLI to use the new access key.
8. In the console, make the original access key inactive.
9. Confirm that you are using the new access key by once again listing the contents of the Amazon S3 bucket.
10. Delete the original access key.

EXERCISE 6.6

Set Up MFA

In this exercise, you will add MFA to your IAM administrator. You will use a virtual MFA application for your phone. MFA is a security recommendation on powerful accounts such as IAM administrators.

1. Download the AWS Virtual MFA app to your phone.
2. Select the administrator user, and manage the MFA device.
3. Go through the steps to activate a Virtual MFA device.
4. Log off as administrator.
5. Log in as administrator, and enter the MFA value to complete the authentication process.

EXERCISE 6.7

Resolve Conflicting Permissions

In this exercise, you will add a policy to your IAM administrator user with a conflicting permission. You will then attempt actions that verify how IAM resolves conflicting permissions.

1. Use the policy generator to create a new policy.
2. Create the policy with Effect: Deny; AWS Service: Amazon S3; Actions: *; and ARN: *.
3. Attach the new policy to the `Administrators` group.
4. Use the CLI to attempt to list the contents of an Amazon S3 bucket. The policy that allows access and the policy that denies access should resolve to deny access.

Review Questions

1. Which of the following methods will allow an application using an AWS SDK to be authenticated as a principal to access AWS Cloud services? (Choose 2 answers)
   A. Create an IAM user and store the user name and password for the user in the application's configuration.
   B. Create an IAM user and store both parts of the access key for the user in the application's configuration.
   C. Run the application on an Amazon EC2 instance with an assigned IAM role.
   D. Make all the API calls over an SSL connection.
2. Which of the following are found in an IAM policy? (Choose 2 answers)
   A. Service Name
   B. Region
   C. Action
   D. Password
3. Your AWS account administrator left your company today. The administrator had access to the root user and a personal IAM administrator account. With these accounts, he generated other IAM accounts and keys. Which of the following should you do today to protect your AWS infrastructure? (Choose 4 answers)
   A. Change the password and add MFA to the root user.
   B. Put an IP restriction on the root user.
   C. Rotate keys and change passwords for IAM accounts.
   D. Delete all IAM accounts.
   E. Delete the administrator's personal IAM account.
   F. Relaunch all Amazon EC2 instances with new roles.
4. Which of the following actions can be authorized by IAM? (Choose 2 answers)
   A. Installing ASP.NET on a Windows Server
   B. Launching an Amazon Linux EC2 instance
   C. Querying an Oracle database
   D. Adding a message to an Amazon Simple Queue Service (Amazon SQS) queue
5. Which of the following are IAM security features? (Choose 2 answers)
   A. Password policies
   B. Amazon DynamoDB global secondary indexes
   C. MFA

D. Consolidated Billing

6. Which of the following are benefits of using Amazon EC2 roles? (Choose 2 answers)
   A. No policies are required.
   B. Credentials do not need to be stored on the Amazon EC2 instance.
   C. Key rotation is not necessary.
   D. Integration with Active Directory is automatic.
7. Which of the following are based on temporary security tokens? (Choose 2 answers)
   A. Amazon EC2 roles
   B. MFA
   C. Root user
   D. Federation
8. Your security team is very concerned about the vulnerability of the IAM administrator user accounts (the accounts used to configure all IAM features and accounts). What steps can be taken to lock down these accounts? (Choose 3 answers)
   A. Add multi-factor authentication (MFA) to the accounts.
   B. Limit logins to a particular U.S. state.
   C. Implement a password policy on the AWS account.
   D. Apply a source IP address condition to the policy that only grants permissions when the user is on the corporate network.
   E. Add a CAPTCHA test to the accounts.
9. You want to grant the individuals on your network team the ability to fully manipulate Amazon EC2 instances. Which of the following accomplish this goal? (Choose 2 answers)
   A. Create a new policy allowing EC2:* actions, and name the policy NetworkTeam.
   B. Assign the managed policy, EC2FullAccess, to a group named NetworkTeam, and assign all the team members' IAM user accounts to that group.
   C. Create a new policy that grants EC2:* actions on all resources, and assign that policy to each individual's IAM user account on the network team.
   D. Create a NetworkTeam IAM group, and have each team member log in to the AWS Management Console using the user name/password for the group.
10. What is the format of an IAM policy?
    A. XML
    B. Key/value pairs
    C. JSON
    D. Tab-delimited text

Chapter 7
Databases and AWS

THE AWS CERTIFIED SOLUTIONS ARCHITECT ASSOCIATE EXAM OBJECTIVES COVERED IN THIS CHAPTER MAY INCLUDE, BUT ARE NOT LIMITED TO, THE FOLLOWING:

Domain 1.0: Designing highly available, cost-efficient, fault-tolerant, and scalable systems

✓ **1.1 Identify and recognize cloud architecture considerations, such as fundamental components and effective designs.**

Content may include the following:

- Planning and design
- Architectural trade-off decisions (Amazon Relational Database Service [Amazon RDS] vs. installing on Amazon Elastic Compute Cloud [Amazon EC2])
- Best practices for AWS architecture
- Recovery Time Objective (RTO) and Recovery Point Objective (RPO) Disaster Recovery (DR) design
- Elasticity and scalability

Domain 3.0: Data Security

✓ **3.1 Recognize and implement secure practices for optimum cloud deployment and maintenance.**

Content may include the following:

- AWS administration and security services
- Design patterns

✓ **3.2 Recognize critical disaster recovery techniques and their implementation.**



This chapter will cover essential database concepts and introduce three of Amazon's managed database services: Amazon Relational Database Service (Amazon RDS), Amazon DynamoDB, and Amazon Redshift. These managed services simplify the setup and operation of relational databases, NoSQL databases, and data warehouses.

This chapter focuses on key topics you need to understand for the exam, including:

- The differences among a relational database, a NoSQL database, and a data warehouse
- The benefits and tradeoffs between running a database on Amazon EC2 or on Amazon RDS
- How to deploy database engines into the cloud
- How to back up and recover your database and meet your Recovery Point Objective (RPO) and Recovery Time Objective (RTO) requirements
- How to build highly available database architectures
- How to scale your database compute and storage vertically
- How to select the right type of storage volume
- How to use read replicas to scale horizontally
- How to design and scale an Amazon DynamoDB table
- How to read and write from an Amazon DynamoDB table
- How to use secondary indexes to speed queries
- How to design an Amazon Redshift table
- How to load and query an Amazon Redshift data warehouse
- How to secure your databases, tables, and clusters

Database Primer

Almost every application relies on a database to store important data and records for its users. A database engine allows your application to access, manage, and search large volumes of data records. In a well-architected application, the database will need to meet the performance demands, the availability needs, and the recoverability characteristics of the system.

Database systems and engines can be grouped into two broad categories: Relational Database Management Systems (RDBMS) and NoSQL (or non-relational) databases. It is not uncommon to build an application using a combination of RDBMS and NoSQL databases. A strong understanding of essential database concepts, Amazon RDS, and Amazon DynamoDB are required to pass this exam.

Relational Databases

The most common type of database in use today is the *relational database*. The relational database has roots going back to the 1970s when Edgar F. Codd, working for IBM, developed the concepts of the relational model. Today, relational databases power all types of applications from social media apps, e-commerce websites, and blogs to complex enterprise applications. Commonly used relational database software packages include MySQL, PostgreSQL, Microsoft SQL Server, and Oracle.

Relational databases provide a common interface that lets users read and write from the database using commands or queries written using *Structured Query Language (SQL)*. A relational database consists of one or more tables, and a table consists of columns and rows similar to a spreadsheet. A database column contains a specific attribute of the record, such as a person's name, address, and telephone number. Each attribute is assigned a data type such as text, number, or date, and the database engine will reject invalid inputs.

A database row comprises an individual record, such as the details about a student who attends a school. Consider the example in Table 7.1.

TABLE 7.1 Students Table

StudentID	FirstName	LastName	Gender	Age
1001	Joe	Dusty	M	29
1002	Andrea	Romanov	F	20
1003	Ben	Johnson	M	30
1004	Beth	Roberts	F	30

This is an example of a basic table that would sit in a relational database. There are five fields with different data types:

`StudentID` = Number or integer

`FirstName` = String

`LastName` = String

`Gender` = String (Character Length = 1)

`Age` = Integer

This sample table has four records, with each record representing an individual student. Each student has a `StudentID` field, which is usually a unique number per student. A unique number that identifies each student can be called a *primary key*.

One record in a table can relate to a record in another table by referencing the primary key of a record. This pointer or reference is called a foreign key. For example, the Grades table that records scores for each student would have its own primary key and an additional column known as a foreign key that refers to the primary key of the student record. By referencing the primary keys of other tables, relational databases minimize duplication of data in associated tables. With relational databases, it is important to note that the structure of the table (such as the number of columns and data type of each column) must be defined prior to data being added to the table.

A relational database can be categorized as either an *Online Transaction Processing (OLTP)* or *Online Analytical Processing (OLAP)* database system, depending on how the tables are organized and how the application uses the relational database. OLTP refers to transaction-oriented applications that are frequently writing and changing data (for example, data entry and e-commerce). OLAP is typically the domain of data warehouses and refers to reporting or analyzing large data sets. Large applications often have a mix of both OLTP and OLAP databases.

Amazon Relational Database Service (Amazon RDS) significantly simplifies the setup and maintenance of OLTP and OLAP databases. Amazon RDS provides support for six popular relational database engines: MySQL, Oracle, PostgreSQL, Microsoft SQL Server, MariaDB, and Amazon Aurora. You can also choose to run nearly any database engine using Windows or Linux Amazon Elastic Compute Cloud (Amazon EC2) instances and manage the installation and administration yourself.

Data Warehouses

A *data warehouse* is a central repository for data that can come from one or more sources. This data repository is often a specialized type of relational database that can be used for reporting and analysis via OLAP. Organizations typically use data warehouses to compile reports and search the database using highly complex queries.

Data warehouses are also typically updated on a batch schedule multiple times per day or per hour, compared to an OLTP relational database that can be updated thousands of times per second. Many organizations split their relational databases into two different databases: one database as their main production database for OLTP transactions, and the other database as their data warehouse for OLAP. OLTP transactions occur frequently and are relatively simple. OLAP transactions occur much less frequently but are much more complex.

Amazon RDS is often used for OLTP workloads, but it can also be used for OLAP. *Amazon Redshift* is a high-performance data warehouse designed specifically for OLAP use cases. It is also common to combine Amazon RDS with Amazon Redshift in the same application and periodically extract recent transactions and load them into a reporting database.

NoSQL Databases

NoSQL databases have gained significant popularity in recent years because they are often simpler to use, more flexible, and can achieve performance levels that are difficult or impossible with traditional relational databases. Traditional relational databases are difficult to scale beyond a single server without significant engineering and cost, but a NoSQL architecture allows for horizontal scalability on commodity

hardware.

NoSQL databases are non-relational and do not have the same table and column semantics of a relational database. NoSQL databases are instead often key/value stores or document stores with flexible schemas that can evolve over time or vary. Contrast that to a relational database, which requires a very rigid schema.

Many of the concepts of NoSQL architectures trace their foundational concepts back to whitepapers published in 2006 and 2007 that described distributed systems like Dynamo at Amazon. Today, many application teams use Hbase, MongoDB, Cassandra, CouchDB, Riak, and *Amazon DynamoDB* to store large volumes of data with high transaction rates. Many of these database engines support clustering and scale horizontally across many machines for performance and fault tolerance. A common use case for NoSQL is managing user session state, user profiles, shopping cart data, or time-series data.

You can run any type of NoSQL database on AWS using Amazon EC2, or you can choose a managed service like Amazon DynamoDB to deal with the heavy lifting involved with building a distributed cluster spanning multiple data centers.

Amazon Relational Database Service (Amazon RDS)

Amazon RDS is a service that simplifies the setup, operations, and scaling of a relational database on AWS. With Amazon RDS, you can spend more time focusing on the application and the schema and let Amazon RDS offload common tasks like backups, patching, scaling, and replication.

Amazon RDS helps you to streamline the installation of the database software and also the provisioning of infrastructure capacity. Within a few minutes, Amazon RDS can launch one of many popular database engines that is ready to start taking SQL transactions. After the initial launch, Amazon RDS simplifies ongoing maintenance by automating common administrative tasks on a recurring basis.

With Amazon RDS, you can accelerate your development timelines and establish a consistent operating model for managing relational databases. For example, Amazon RDS makes it easy to replicate your data to increase availability, improve durability, or scale up or beyond a single database instance for read-heavy database workloads.

Amazon RDS exposes a database endpoint to which client software can connect and execute SQL. Amazon RDS does not provide shell access to Database (DB) Instances, and it restricts access to certain system procedures and tables that require advanced privileges. With Amazon RDS, you can typically use the same tools to query, analyze, modify, and administer the database. For example, current Extract, Transform, Load (ETL) tools and reporting tools can connect to Amazon RDS databases in the same way with the same drivers, and often all it takes to reconfigure is changing the hostname in the connection string.

Database (DB) Instances

The Amazon RDS service itself provides an Application Programming Interface (API) that lets you create and manage one or more *DB Instances*. A DB Instance is an isolated database environment deployed in your private network segments in the cloud. Each DB Instance runs and manages a popular commercial or open source database engine on your behalf. Amazon RDS currently supports the following database engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora.

You can launch a new DB Instance by calling the `CreateDBInstance` API or by using the AWS Management Console. Existing DB Instances can be changed or resized using the `ModifyDBInstance` API. A DB Instance can contain multiple different databases, all of which you create and manage within the DB Instance itself by executing SQL commands with the Amazon RDS endpoint. The different databases can be created, accessed, and managed using the same SQL client tools and applications that you use today.

The compute and memory resources of a DB Instance are determined by its DB Instance class. You can select the DB Instance class that best meets your needs for compute and memory. The range of DB Instance classes extends from a db.t2.micro with 1 virtual CPU (vCPU) and 1 GB of memory, up to a db.r3.8xlarge with 32 vCPUs and 244 GB of memory. As your needs change over time, you can change the instance class and the balance of compute of memory, and Amazon RDS will migrate your data to a larger or smaller instance class. Independent from the DB Instance class that you select, you can also control the size and performance characteristics of the storage used.

Amazon RDS supports a large variety of engines, versions, and feature combinations. Check the Amazon RDS documentation to determine support for specific features. Many features and common configuration

settings are exposed and managed using *DB parameter groups* and *DB option groups*. A DB parameter group acts as a container for engine configuration values that can be applied to one or more DB Instances. You may change the DB parameter group for an existing instance, but a reboot is required. A DB option group acts as a container for engine features, which is empty by default. In order to enable specific features of a DB engine (for example, Oracle Statspack, Microsoft SQL Server Mirroring), you create a new DB option group and configure the settings accordingly.



Existing databases can be migrated to Amazon RDS using native tools and techniques that vary depending on the engine. For example with MySQL, you can export a backup using mysqldump and import the file into Amazon RDS MySQL. You can also use the AWS Database Migration Service, which gives you a graphical interface that simplifies the migration of both schema and data between databases. AWS Database Migration Service also helps convert databases from one database engine to another.

Operational Benefits

Amazon RDS increases the operational reliability of your databases by applying a very consistent deployment and operational model. This level of consistency is achieved in part by limiting the types of changes that can be made to the underlying infrastructure and through the extensive use of automation. For example with Amazon RDS, you cannot use Secure Shell (SSH) to log in to the host instance and install a custom piece of software. You can, however, connect using SQL administrator tools or use DB option groups and DB parameter groups to change the behavior or feature configuration for a DB Instance. If you want full control of the Operating System (OS) or require elevated permissions to run, then consider installing your database on Amazon EC2 instead of Amazon RDS.

Amazon RDS is designed to simplify the common tasks required to operate a relational database in a reliable manner. It's useful to compare the responsibilities of an administrator when operating a relational database in your data center, on Amazon EC2, or with Amazon RDS (see Table 7.2).

TABLE 7.2 Comparison of Operational Responsibilities

Responsibility	Database On-Premise	Database on Amazon EC2	Database on Amazon RDS
App Optimization	You	You	You
Scaling	You	You	AWS
High Availability	You	You	AWS
Backups	You	You	AWS
DB Engine Patches	You	You	AWS
Software Installation	You	You	AWS
OS Patches	You	You	AWS
OS Installation	You	AWS	AWS
Server Maintenance	You	AWS	AWS
Rack and Stack	You	AWS	AWS
Power and Cooling	You	AWS	AWS

Database Engines

Amazon RDS supports six database engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora. Features and capabilities vary slightly depending on the engine that you select.

MySQL

MySQL is one of the most popular open source databases in the world, and it is used to power a wide range of applications, from small personal blogs to some of the largest websites in the world. As of the time of this writing, Amazon RDS for MySQL currently supports MySQL 5.7, 5.6, 5.5, and 5.1. The engine is running the open source Community Edition with InnoDB as the default and recommended database storage engine. Amazon RDS MySQL allows you to connect using standard MySQL tools such as MySQL Workbench or SQL Workbench/J. Amazon RDS MySQL supports *Multi-AZ* deployments for high availability and *read replicas* for horizontal scaling.

PostgreSQL

PostgreSQL is a widely used open source database engine with a very rich set of features and advanced functionality. Amazon RDS supports DB Instances running several versions of PostgreSQL. As of the time of this writing, Amazon RDS supports multiple releases of PostgreSQL, including 9.5.x, 9.4.x, and 9.3.x. Amazon RDS PostgreSQL can be managed using standard tools like pgAdmin and supports standard JDBC/ODBC drivers. Amazon RDS PostgreSQL also supports Multi-AZ deployment for high availability and read replicas for horizontal scaling.

MariaDB

Amazon RDS recently added support for DB Instances running MariaDB. MariaDB is a popular open source database engine built by the creators of MySQL and enhanced with enterprise tools and functionality. MariaDB adds features that enhance the performance, availability, and scalability of MySQL. As of the time of this writing, AWS supports MariaDB version 10.0.17. Amazon RDS fully supports the XtraDB storage engine for MariaDB DB Instances and, like Amazon RDS MySQL and PostgreSQL, has support for Multi-AZ deployment and read replicas.

Oracle

Oracle is one of the most popular relational databases used in the enterprise and is fully supported by Amazon RDS. As of the time of this writing, Amazon RDS supports DB Instances running several editions of Oracle 11g and Oracle 12c. Amazon RDS supports access to schemas on a DB Instance using any standard SQL client application, such as Oracle SQL Plus.

Amazon RDS Oracle supports three different editions of the popular database engine: Standard Edition One, Standard Edition, and Enterprise Edition. Table 7.3 outlines some of the major differences between editions:

TABLE 7.3 Amazon RDS Oracle Editions Compared

Edition	Performance	Multi-AZ	Encryption
Standard One	++++	Yes	KMS
Standard	++++++++	Yes	KMS
Enterprise	++++++++	Yes	KMS and TDE

Microsoft SQL Server

Microsoft SQL Server is another very popular relational database used in the enterprise. Amazon RDS allows Database Administrators (DBAs) to connect to their SQL Server DB Instance in the cloud using native tools like SQL Server Management Studio. As of the time of this writing, Amazon RDS provides support for several versions of Microsoft SQL Server, including SQL Server 2008 R2, SQL Server 2012, and SQL Server 2014.

Amazon RDS SQL Server also supports four different editions of SQL Server: Express Edition, Web Edition, Standard Edition, and Enterprise Edition. Table 7.4 highlights the relative performance, availability, and encryption differences among these editions.

TABLE 7.4 Amazon RDS SQL Server Editions Compared

Edition	Performance	Multi-AZ	Encryption
Express	+	No	KMS
Web	++++	No	KMS
Standard	++++	Yes	KMS
Enterprise	++++++++	Yes	KMS and TDE

Licensing

Amazon RDS Oracle and Microsoft SQL Server are commercial software products that require appropriate licenses to operate in the cloud. AWS offers two licensing models: *License Included* and *Bring Your Own License (BYOL)*.

License Included In the License Included model, the license is held by AWS and is included in the Amazon RDS instance price. For Oracle, License Included provides licensing for Standard Edition One. For SQL Server, License Included provides licensing for SQL Server Express Edition, Web Edition, and Standard Edition.

Bring Your Own License (BYOL) In the BYOL model, you provide your own license. For Oracle, you must have the appropriate Oracle Database license for the DB Instance class and Oracle Database edition you want to run. You can bring over Standard Edition One, Standard Edition, and Enterprise Edition.

For SQL Server, you provide your own license under the Microsoft License Mobility program. You can bring over Microsoft SQL Standard Edition and also Enterprise Edition. You are responsible for tracking and managing how licenses are allocated.

Amazon Aurora

Amazon Aurora offers enterprise-grade commercial database technology while offering the simplicity and cost effectiveness of an open source database. This is achieved by redesigning the internal components of MySQL to take a more service-oriented approach.

Like other Amazon RDS engines, Amazon Aurora is a fully managed service, is MySQL-compatible out of the box, and provides for increased reliability and performance over standard MySQL deployments. Amazon Aurora can deliver up to five times the performance of MySQL without requiring changes to most of your existing web applications. You can use the same code, tools, and applications that you use with your existing MySQL databases with Amazon Aurora.

When you first create an Amazon Aurora instance, you create a DB cluster. A DB cluster has one or more

instances and includes a cluster volume that manages the data for those instances. An Amazon Aurora cluster volume is a virtual database storage volume that spans multiple Availability Zones, with each Availability Zone having a copy of the cluster data. An Amazon Aurora DB cluster consists of two different types of instances:

Primary Instance This is the main instance, which supports both read and write workloads. When you modify your data, you are modifying the primary instance. Each Amazon Aurora DB cluster has one primary instance.

Amazon Aurora Replica This is a secondary instance that supports only read operations. Each DB cluster can have up to 15 Amazon Aurora Replicas in addition to the primary instance. By using multiple Amazon Aurora Replicas, you can distribute the read workload among various instances, increasing performance. You can also locate your Amazon Aurora Replicas in multiple Availability Zones to increase your database availability.

Storage Options

Amazon RDS is built using Amazon Elastic Block Store (Amazon EBS) and allows you to select the right storage option based on your performance and cost requirements. Depending on the database engine and workload, you can scale up to 4 to 6TB in provisioned storage and up to 30,000 IOPS. Amazon RDS supports three storage types: Magnetic, General Purpose (Solid State Drive [SSD]), and Provisioned IOPS (SSD). Table 7.5 highlights the relative size, performance, and cost differences between types.

TABLE 7.5 Amazon RDS Storage Types

	Magnetic	General Purpose (SSD)	Provisioned IOPS (SSD)
Size	+++	+++++	+++++
Performance	+	+++	+++++
Cost	++	+++	+++++

Magnetic Magnetic storage, also called standard storage, offers cost-effective storage that is ideal for applications with light I/O requirements.

General Purpose (SSD) General purpose (SSD)-backed storage, also called gp2, can provide faster access than magnetic storage. This storage type can provide burst performance to meet spikes and is excellent for small- to medium-sized databases.

Provisioned IOPS (SSD) Provisioned IOPS (SSD) storage is designed to meet the needs of I/O-intensive workloads, particularly database workloads, that are sensitive to storage performance and consistency in random access I/O throughput.



For most applications, General Purpose (SSD) is the best option and provides a good mix of lower-cost and higher-performance characteristics.

Backup and Recovery

Amazon RDS provides a consistent operational model for backup and recovery procedures across the

different database engines. Amazon RDS provides two mechanisms for backing up the database: automated backups and manual snapshots. By using a combination of both techniques, you can design a backup recovery model to protect your application data.

Each organization typically will define a *Recovery Point Objective (RPO)* and *Recovery Time Objective (RTO)* for important applications based on the criticality of the application and the expectations of the users. It's common for enterprise systems to have an RPO measured in minutes and an RTO measured in hours or even days, while some critical applications may have much lower tolerances.

RPO is defined as the maximum period of data loss that is acceptable in the event of a failure or incident. For example, many systems back up transaction logs every 15 minutes to allow them to minimize data loss in the event of an accidental deletion or hardware failure.

RTO is defined as the maximum amount of downtime that is permitted to recover from backup and to resume processing. For large databases in particular, it can take hours to restore from a full backup. In the event of a hardware failure, you can reduce your RTO to minutes by failing over to a secondary node. You should create a recovery plan that, at a minimum, lets you recover from a recent backup.

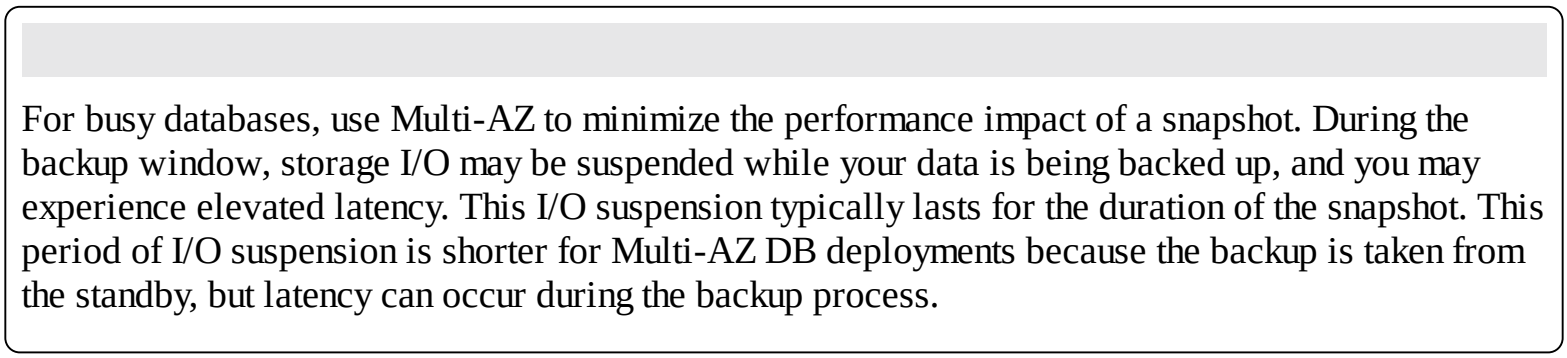
Automated Backups

An *automated backup* is an Amazon RDS feature that continuously tracks changes and backs up your database. Amazon RDS creates a storage volume snapshot of your DB Instance, backing up the entire DB Instance and not just individual databases. You can set the backup retention period when you create a DB Instance. One day of backups will be retained by default, but you can modify the retention period up to a maximum of 35 days. Keep in mind that when you delete a DB Instance, all automated backup snapshots are deleted and cannot be recovered. Manual snapshots, however, are not deleted.

Automated backups will occur daily during a configurable 30-minute maintenance window called the backup window. Automated backups are kept for a configurable number of days, called the *backup retention period*. You can restore your DB Instance to any specific time during this retention period, creating a new DB Instance.

Manual DB Snapshots

In addition to automated backups, you can perform manual *DB snapshots* at any time. A DB snapshot is initiated by you and can be created as frequently as you want. You can then restore the DB Instance to the specific state in the DB snapshot at any time. DB snapshots can be created with the Amazon RDS console or the `CreateDBSnapshot` action. Unlike automated snapshots that are deleted after the retention period, manual DB snapshots are kept until you explicitly delete them with the Amazon RDS console or the `DeleteDBSnapshot` action.



For busy databases, use Multi-AZ to minimize the performance impact of a snapshot. During the backup window, storage I/O may be suspended while your data is being backed up, and you may experience elevated latency. This I/O suspension typically lasts for the duration of the snapshot. This period of I/O suspension is shorter for Multi-AZ DB deployments because the backup is taken from the standby, but latency can occur during the backup process.

Recovery

Amazon RDS allows you to recover your database quickly whether you are performing automated backups or manual DB snapshots. You cannot restore from a DB snapshot to an existing DB Instance; a new DB Instance is created when you restore. When you restore a DB Instance, only the default DB parameter and security groups are associated with the restored instance. As soon as the restore is complete, you should associate any custom DB parameter or security groups used by the instance from which you restored. When using automated backups, Amazon RDS combines the daily backups performed during your predefined maintenance window in conjunction with transaction logs to enable you to restore your DB Instance to any point during your retention period, typically up to the last five minutes.

High Availability with Multi-AZ

One of the most powerful features of Amazon RDS is Multi-AZ deployments, which allows you to create a database cluster across multiple Availability Zones. Setting up a relational database to run in a highly available and fault-tolerant fashion is a challenging task. With Amazon RDS Multi-AZ, you can reduce the complexity involved with this common administrative task; with a single option, Amazon RDS can increase the availability of your database using replication. Multi-AZ lets you meet the most demanding RPO and RTO targets by using synchronous replication to minimize RPO and fast failover to minimize RTO to minutes.

Multi-AZ allows you to place a secondary copy of your database in another Availability Zone for disaster recovery purposes. Multi-AZ deployments are available for all types of Amazon RDS database engines. When you create a Multi-AZ DB Instance, a primary instance is created in one Availability Zone and a secondary instance is created in another Availability Zone. You are assigned a database instance endpoint such as the following:

```
my_app_db.ch6fe7ykq1zd.us-west-2.rds.amazonaws.com
```

This endpoint is a Domain Name System (DNS) name that AWS takes responsibility for resolving to a specific IP address. You use this DNS name when creating the connection to your database. Figure 7.1 illustrates a typical Multi-AZ deployment spanning two Availability Zones.

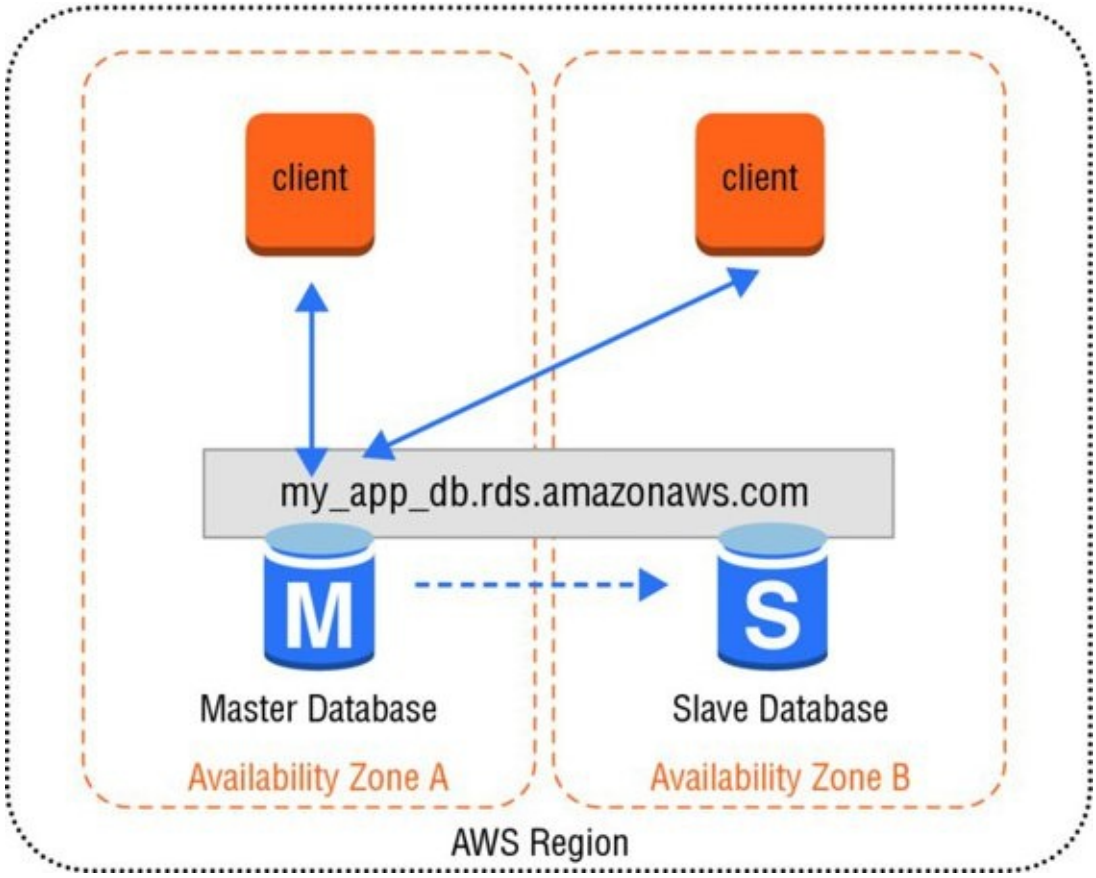


FIGURE 7.1 Multi-AZ Amazon RDS architecture

Amazon RDS automatically replicates the data from the master database or primary instance to the slave database or secondary instance using synchronous replication. Each Availability Zone runs on its own physically distinct, independent infrastructure and is engineered to be highly reliable. Amazon RDS detects and automatically recovers from the most common failure scenarios for Multi-AZ deployments so that you can resume database operations as quickly as possible without administrative intervention. Amazon RDS automatically performs a failover in the event of any of the following:

- Loss of availability in primary Availability Zone
- Loss of network connectivity to primary database
- Compute unit failure on primary database
- Storage failure on primary database

Amazon RDS will automatically fail over to the standby instance without user intervention. The DNS name remains the same, but the Amazon RDS service changes the CNAME to point to the standby. The primary DB Instance switches over automatically to the standby replica if there was an Availability Zone service disruption, if the primary DB Instance fails, or if the instance type is changed. You can also perform a manual failover of the DB Instance. Failover between the primary and the secondary instance is fast, and the time automatic failover takes to complete is typically one to two minutes.



It is important to remember that Multi-AZ deployments are for disaster recovery only; they are not meant to enhance database performance. The standby DB Instance is not available to offline queries from the primary master DB Instance. To improve database performance using multiple DB Instances, use read replicas or other DB caching technologies such as Amazon ElastiCache.

Scaling Up and Out

As the number of transactions increase to a relational database, scaling up, or vertically, by getting a larger machine allows you to process more reads and writes. Scaling out, or horizontally, is also possible, but it is often more difficult. Amazon RDS allows you to scale compute and storage vertically, and for some DB engines, you can scale horizontally.

Vertical Scalability

Adding additional compute, memory, or storage resources to your database allows you to process more transactions, run more queries, and store more data. Amazon RDS makes it easy to scale up or down your database tier to meet the demands of your application. Changes can be scheduled to occur during the next maintenance window or to begin immediately using the `ModifyDBInstance` action.

To change the amount of compute and memory, you can select a different DB Instance class of the database. After you select a larger or smaller DB Instance class, Amazon RDS automates the migration process to a new class with only a short disruption and minimal effort.

You can also increase the amount of storage, the storage class, and the storage performance for an Amazon RDS Instance. Each database instance can scale from 5GB up to 6TB in provisioned storage depending on the storage type and engine. Storage for Amazon RDS can be increased over time as needs grow with minimal impact to the running database. Storage expansion is supported for all of the database engines except for SQL Server.

Horizontal Scalability with Partitioning

A relational database can be scaled vertically only so much before you reach the maximum instance size. Partitioning a large relational database into multiple instances or shards is a common technique for handling more requests beyond the capabilities of a single instance.

Partitioning, or *sharding*, allows you to scale horizontally to handle more users and requests but requires additional logic in the application layer. The application needs to decide how to route database requests to the correct shard and becomes limited in the types of queries that can be performed across server boundaries. NoSQL databases like Amazon DynamoDB or Cassandra are designed to scale horizontally.

Horizontal Scalability with Read Replicas

Another important scaling technique is to use *read replicas* to offload read transactions from the primary database and increase the overall number of transactions. Amazon RDS supports read replicas that allow you to scale out elastically beyond the capacity constraints of a single DB Instance for read-heavy database workloads.

There are a variety of use cases where deploying one or more read replica DB Instances is helpful. Some common scenarios include:

- Scale beyond the capacity of a single DB Instance for read-heavy workloads.
- Handle read traffic while the source DB Instance is unavailable. For example, due to I/O suspension for backups or scheduled maintenance, you can direct read traffic to a replica.
- Offload reporting or data warehousing scenarios against a replica instead of the primary DB Instance.

For example, a blogging website may have very little write activity except for the occasional comment, and the vast majority of database activity will be read-only. By offloading some or all of the read activity to one or more read replicas, the primary database instance can focus on handling the writes and replicating the data out to the replicas.

Read replicas are currently supported in Amazon RDS for MySQL, PostgreSQL, MariaDB, and Amazon Aurora. Amazon RDS uses the MySQL, MariaDB, and PostgreSQL DB engines' built-in replication functionality to create a special type of DB Instance, called a read replica, from a source DB Instance. Updates made to the source DB Instance are asynchronously copied to the read replica. You can reduce the load on your source DB Instance by routing read queries from your applications to the read replica.



You can create one or more replicas of a database within a single AWS Region or across multiple AWS Regions. To enhance your disaster recovery capabilities or reduce global latencies, you can use cross-region read replicas to serve read traffic from a region closest to your global users or migrate your databases across AWS Regions.

Security

Securing your Amazon RDS DB Instances and relational databases requires a comprehensive plan that addresses the many layers commonly found in database-driven systems. This includes the infrastructure resources, the database, and the network.

Protect access to your infrastructure resources using AWS Identity and Access Management (IAM) policies that limit which actions AWS administrators can perform. For example, some key administrator actions that can be controlled in IAM include `CreateDBInstance` and `DeleteDBInstance`.

Another security best practice is to deploy your Amazon RDS DB Instances into a private subnet within an Amazon Virtual Private Cloud (Amazon VPC) that limits network access to the DB Instance. Before you can deploy into an Amazon VPC, you must first create a *DB subnet group* that predefines which subnets are available for Amazon RDS deployments. Further, restrict network access using network Access Control Lists (ACLs) and security groups to limit inbound traffic to a short list of source IP addresses.

At the database level, you will also need to create users and grant them permissions to read and write to your databases. Access to the database is controlled using the database engine-specific access control and user management mechanisms. Create users at the database level with strong passwords that you rotate frequently.

Finally, protect the confidentiality of your data in transit and at rest with multiple encryption capabilities provided with Amazon RDS. Security features vary slightly from one engine to another, but all engines support some form of in-transit encryption and also at-rest encryption. You can securely connect a client to a running DB Instance using Secure Sockets Layer (SSL) to protect data in transit. Encryption at rest is possible for all engines using the Amazon Key Management Service (KMS) or *Transparent Data Encryption (TDE)*. All logs, backups, and snapshots are encrypted for an encrypted Amazon RDS instance.

Amazon Redshift

Amazon Redshift is a fast, powerful, fully managed, petabyte-scale data warehouse service in the cloud. Amazon Redshift is a relational database designed for OLAP scenarios and optimized for high-performance analysis and reporting of very large datasets. Traditional data warehouses are difficult and expensive to manage, especially for large datasets. Amazon Redshift not only significantly lowers the cost of a data warehouse, but it also makes it easy to analyze large amounts of data very quickly.

Amazon Redshift gives you fast querying capabilities over structured data using standard SQL commands to support interactive querying over large datasets. With connectivity via ODBC or JDBC, Amazon Redshift integrates well with various data loading, reporting, data mining, and analytics tools. Amazon Redshift is based on industry-standard PostgreSQL, so most existing SQL client applications will work with only minimal changes.

Amazon Redshift manages the work needed to set up, operate, and scale a data warehouse, from provisioning the infrastructure capacity to automating ongoing administrative tasks such as backups and patching. Amazon Redshift automatically monitors your nodes and drives to help you recover from failures.

Clusters and Nodes

The key component of an Amazon Redshift data warehouse is a *cluster*. A cluster is composed of a leader node and one or more compute nodes. The client application interacts directly only with the leader node, and the compute nodes are transparent to external applications.

Amazon Redshift currently has support for six different node types and each has a different mix of CPU, memory, and storage. The six node types are grouped into two categories: Dense Compute and Dense Storage. The Dense Compute node types support clusters up to 326TB using fast SSDs, while the Dense Storage nodes support clusters up to 2PB using large magnetic disks. Each cluster consists of one leader node and one or more compute nodes. Figure 7.2 shows the internal components of an Amazon Redshift data warehouse cluster.

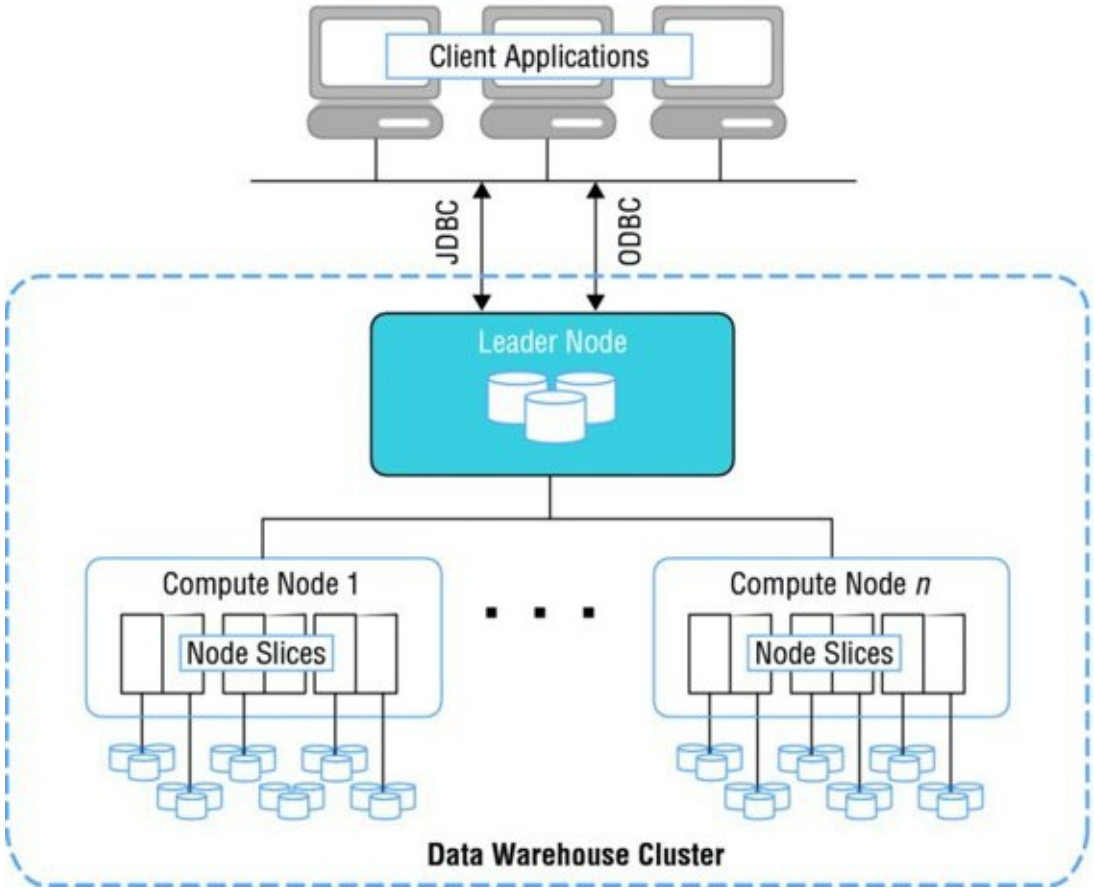


FIGURE 7.2 Amazon Redshift cluster architecture

Each cluster contains one or more databases. User data for each table is distributed across the compute nodes. Your application or SQL client communicates with Amazon Redshift using standard JDBC or ODBC connections with the leader node, which in turn coordinates query execution with the compute nodes. Your application does not interact directly with the compute nodes.

The disk storage for a compute node is divided into a number of slices. The number of slices per node depends on the node size of the cluster and typically varies between 2 and 16. The nodes all participate in parallel query execution, working on data that is distributed as evenly as possible across the slices.

You can increase query performance by adding multiple nodes to a cluster. When you submit a query, Amazon Redshift distributes and executes the query in parallel across all of a cluster's compute nodes. Amazon Redshift also spreads your table data across all compute nodes in a cluster based on a *distribution strategy* that you specify. This partitioning of data across multiple compute resources allows you to achieve high levels of performance.

Amazon Redshift allows you to resize a cluster to add storage and compute capacity over time as your needs evolve. You can also change the node type of a cluster and keep the overall size the same. Whenever you perform a resize operation, Amazon Redshift will create a new cluster and migrate data from the old cluster to the new one. During a resize operation, the database will become read-only until the operation is finished.

Table Design

Each Amazon Redshift cluster can support one or more databases, and each database can contain many tables. Like most SQL-based databases, you can create a table using the `CREATE TABLE` command. This command specifies the name of the table, the columns, and their data types. In addition to columns and

data types, the Amazon Redshift `CREATE TABLE` command also supports specifying compression encodings, distribution strategy, and sort keys.

Data Types

Amazon Redshift columns support a wide range of data types. This includes common numeric data types like INTEGER, DECIMAL, and DOUBLE, text data types like CHAR and VARCHAR, and date data types like DATE and TIMESTAMP. Additional columns can be added to a table using the `ALTER TABLE` command; however, existing columns cannot be modified.

Compression Encoding

One of the key performance optimizations used by Amazon Redshift is data compression. When loading data for the first time into an empty table, Amazon Redshift will automatically sample your data and select the best compression scheme for each column. Alternatively, you can specify compression encoding on a per-column basis as part of the `CREATE TABLE` command.

Distribution Strategy

One of the primary decisions when creating a table in Amazon Redshift is how to distribute the records across the nodes and slices in a cluster. You can configure the distribution style of a table to give Amazon Redshift hints as to how the data should be partitioned to best meet your query patterns. When you run a query, the optimizer shifts the rows to the compute nodes as needed to perform any joins and aggregates. The goal in selecting a table distribution style is to minimize the impact of the redistribution step by putting the data where it needs to be before the query is performed.

The data distribution style that you select for your database has a big impact on query performance, storage requirements, data loading, and maintenance. By choosing the best distribution strategy for each table, you can balance your data distribution and significantly improve overall system performance. When creating a table, you can choose between one of three distribution styles: EVEN, KEY, or ALL.

EVEN distribution This is the default option and results in the data being distributed across the slices in a uniform fashion regardless of the data.

KEY distribution With KEY distribution, the rows are distributed according to the values in one column. The leader node will store matching values close together and increase query performance for joins.

ALL distribution With ALL, a full copy of the entire table is distributed to every node. This is useful for lookup tables and other large tables that are not updated frequently.

Sort Keys

Another important decision to make during the creation of a table is whether to specify one or more columns as sort keys. Sorting enables efficient handling of range-restricted predicates. If a query uses a range-restricted predicate, the query processor can rapidly skip over large numbers of blocks during table scans.

The sort keys for a table can be either compound or interleaved. A compound sort key is more efficient when query predicates use a prefix, which is a subset of the sort key columns in order. An interleaved sort key gives equal weight to each column in the sort key, so query predicates can use any subset of the columns that make up the sort key, in any order.

Loading Data

Amazon Redshift supports standard SQL commands like `INSERT` and `UPDATE` to create and modify records in a table. For bulk operations, however, Amazon Redshift provides the `COPY` command as a much more efficient alternative than repeatedly calling `INSERT`.



A `COPY` command can load data into a table in the most efficient manner, and it supports multiple types of input data sources. The fastest way to load data into Amazon Redshift is doing bulk data loads from flat files stored in an Amazon Simple Storage Service (Amazon S3) bucket or from an Amazon DynamoDB table.

When loading data from Amazon S3, the `COPY` command can read from multiple files at the same time. Amazon Redshift can distribute the workload to the nodes and perform the load process in parallel. Instead of having one single large file with your data, you can enable parallel processing by having a cluster with multiple nodes and multiple input files.

After each bulk data load that modifies a significant amount of data, you will need to perform a `VACUUM` command to reorganize your data and reclaim space after deletes. It is also recommended to run an `ANALYZE` command to update table statistics.

Data can also be exported out of Amazon Redshift using the `UNLOAD` command. This command can be used to generate delimited text files and store them in Amazon S3.

Querying Data

Amazon Redshift allows you to write standard SQL commands to query your tables. By supporting commands like `SELECT` to query and join tables, analysts can quickly become productive using Amazon Redshift or integrate it easily. For complex queries, you can analyze the query plan to better optimize your access pattern. You can monitor the performance of the cluster and specific queries using Amazon CloudWatch and the Amazon Redshift web console.

For large Amazon Redshift clusters supporting many users, you can configure Workload Management (WLM) to queue and prioritize queries. WLM allows you define multiple queues and set the concurrency level for each queue. For example, you might want to have one queue set up for long-running queries and limit the concurrency and another queue for short-running queries and allow higher levels of concurrency.

Snapshots

Similar to Amazon RDS, you can create point-in-time snapshots of your Amazon Redshift cluster. A snapshot can then be used to restore a copy or create a clone of your original Amazon Redshift cluster. Snapshots are durably stored internally in Amazon S3 by Amazon Redshift.

Amazon Redshift supports both automated snapshots and manual snapshots. With automated snapshots, Amazon Redshift will periodically take snapshots of your cluster and keep a copy for a configurable retention period. You can also perform manual snapshots and share them across regions or even with other AWS accounts. Manual snapshots are retained until you explicitly delete them.

Security

Securing your Amazon Redshift cluster is similar to securing other databases running in the cloud. Your security plan should include controls to protect the infrastructure resources, the database schema, the records in the table, and network access. By addressing security at every level, you can securely operate an Amazon Redshift data warehouse in the cloud.

The first layer of security comes at the infrastructure level using IAM policies that limit the actions AWS administrators can perform. With IAM, you can create policies that grant other AWS users the permission to create and manage the lifecycle of a cluster, including scaling, backup, and recovery operations.

At the network level, Amazon Redshift clusters can be deployed within the private IP address space of your Amazon VPC to restrict overall network connectivity. Fine-grained network access can be further restricted using security groups and network ACLs at the subnet level.

In addition to controlling infrastructure access at the infrastructure level, you must protect access at the database level. When you initially create an Amazon Redshift cluster, you will create a master user account and password. The master account can be used to log in to the Amazon Redshift database and to create more users and groups. Each database user can be granted permission to schemas, tables, and other database objects. These permissions are independent from the IAM policies used to control access to the infrastructure resources and the Amazon Redshift cluster configuration.

Protecting the data stored in Amazon Redshift is another important aspect of your security design. Amazon Redshift supports encryption of data in transit using SSL-encrypted connections, and also encryption of data at rest using multiple techniques. To encrypt data at rest, Amazon Redshift integrates with KMS and AWS CloudHSM for encryption key management services. Encryption at rest and in transit assists in meeting compliance requirements, such as for the Health Insurance Portability and Accountability Act (HIPAA) or the Payment Card Industry Data Security Standard (PCI DSS), and provides additional protections for your data.

Amazon DynamoDB

Amazon DynamoDB is a fully managed NoSQL database service that provides fast and low-latency performance that scales with ease. Amazon DynamoDB lets you offload the administrative burdens of operating a distributed NoSQL database and focus on the application. Amazon DynamoDB significantly simplifies the hardware provisioning, setup and configuration, replication, software patching, and cluster scaling of NoSQL databases.

Amazon DynamoDB is designed to simplify database and cluster management, provide consistently high levels of performance, simplify scalability tasks, and improve reliability with automatic replication. Developers can create a table in Amazon DynamoDB and write an unlimited number of items with consistent latency.

Amazon DynamoDB can provide consistent performance levels by automatically distributing the data and traffic for a table over multiple partitions. After you configure a certain read or write capacity, Amazon DynamoDB will automatically add enough infrastructure capacity to support the requested throughput levels. As your demand changes over time, you can adjust the read or write capacity after a table has been created, and Amazon DynamoDB will add or remove infrastructure and adjust the internal partitioning accordingly.

To help maintain consistent, fast performance levels, all table data is stored on high-performance SSD disk drives. Performance metrics, including transactions rates, can be monitored using Amazon CloudWatch. In addition to providing high-performance levels, Amazon DynamoDB also provides automatic high-availability and durability protections by replicating data across multiple Availability Zones within an AWS Region.

Data Model

The basic components of the Amazon DynamoDB data model include *tables*, items, and attributes. As depicted in Figure 7.3, a table is a collection of items and each item is a collection of one or more attributes. Each item also has a primary key that uniquely identifies the item.

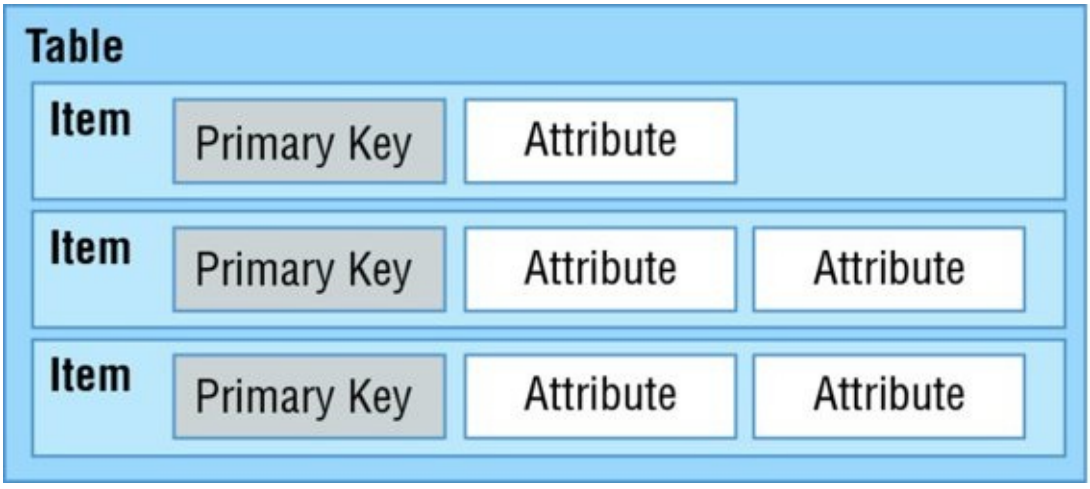


FIGURE 7.3 Table, items, attributes relationship

In a relational database, a table has a predefined schema such as the table name, primary key, list of its column names, and their data types. All records stored in the table must have the same set of columns. In contrast, Amazon DynamoDB only requires that a table have a primary key, but it does not require you to

define all of the attribute names and data types in advance. Individual items in an Amazon DynamoDB table can have any number of attributes, although there is a limit of 400KB on the item size.

Each attribute in an item is a name/value pair. An attribute can be a single-valued or multi-valued set. For example, a book item can have title and authors attributes. Each book has one title but can have many authors. The multi-valued attribute is a set; duplicate values are not allowed. Data is stored in Amazon DynamoDB in key/value pairs such as the following:

```
{
   Id = 101
   ProductName = "Book 101 Title"
   ISBN = "123-1234567890"
   Authors = [ "Author 1", "Author 2" ]
   Price = 2.88
   Dimensions = "8.5 x 11.0 x 0.5"
   PageCount = 500
   InPublication = 1
   ProductCategory = "Book"
}
```

Applications can connect to the Amazon DynamoDB service endpoint and submit requests over HTTP/S to read and write items to a table or even to create and delete tables. DynamoDB provides a web service API that accepts requests in JSON format. While you could program directly against the web service API endpoints, most developers choose to use the AWS Software Development Kit (SDK) to interact with their items and tables. The AWS SDK is available in many different languages and provides a simplified, high-level programming interface.

Data Types

Amazon DynamoDB gives you a lot of flexibility with your database schema. Unlike a traditional relational database that requires you to define your column types ahead of time, DynamoDB only requires a primary key attribute. Each item that is added to the table can then add additional attributes. This gives you flexibility over time to expand your schema without having to rebuild the entire table and deal with record version differences with application logic.

When you create a table or a secondary index, you must specify the names and data types of each primary key attribute (partition key and sort key). Amazon DynamoDB supports a wide range of data types for attributes. Data types fall into three major categories: Scalar, Set, or Document.

Scalar Data Types A scalar type represents exactly one value. Amazon DynamoDB supports the following five scalar types:

String Text and variable length characters up to 400KB. Supports Unicode with UTF8 encoding

Number Positive or negative number with up to 38 digits of precision

Binary Binary data, images, compressed objects up to 400KB in size

Boolean Binary flag representing a true or false value

Null Represents a blank, empty, or unknown state. String, Number, Binary, Boolean cannot be empty.

Set Data Types Sets are useful to represent a unique list of one or more scalar values. Each value in a set needs to be unique and must be the same data type. Sets do not guarantee order. Amazon DynamoDB supports three set types: String Set, Number Set, and Binary Set.

String Set Unique list of String attributes

Number Set Unique list of Number attributes

Binary Set Unique list of Binary attributes

Document Data Types Document type is useful to represent multiple nested attributes, similar to the structure of a JSON file. Amazon DynamoDB supports two document types: List and Map. Multiple Lists and Maps can be combined and nested to create complex structures.

List Each List can be used to store an ordered list of attributes of different data types.

Map Each Map can be used to store an unordered list of key/value pairs. Maps can be used to represent the structure of any JSON object.

Primary Key

When you create a table, you must specify the primary key of the table in addition to the table name. Like a relational database, the primary key uniquely identifies each item in the table. A primary key will point to exactly one item. Amazon DynamoDB supports two types of primary keys, and this configuration cannot be changed after a table has been created:

Partition Key The primary key is made of one attribute, a partition (or hash) key. Amazon DynamoDB builds an unordered hash index on this primary key attribute.

Partition and Sort Key The primary key is made of two attributes. The first attribute is the partition key and the second one is the sort (or range) key. Each item in the table is uniquely identified by the combination of its partition and sort key values. It is possible for two items to have the same partition key value, but those two items must have different sort key values.

Furthermore, each primary key attribute must be defined as type string, number, or binary. Amazon DynamoDB uses the partition key to distribute the request to the right partition.

 If you are performing many reads or writes per second on the same primary key, you will not be able to fully use the compute capacity of the Amazon DynamoDB cluster. A best practice is to maximize your throughput by distributing requests across the full range of partition keys.

Provisioned Capacity

When you create an Amazon DynamoDB table, you are required to provision a certain amount of read and write capacity to handle your expected workloads. Based on your configuration settings, DynamoDB will then provision the right amount of infrastructure capacity to meet your requirements with sustained, low-latency response times. Overall capacity is measured in read and write capacity units. These values can later be scaled up or down by using an `UpdateTable` action.

Each operation against an Amazon DynamoDB table will consume some of the provisioned capacity units. The specific amount of capacity units consumed depends largely on the size of the item, but also on other factors. For read operations, the amount of capacity consumed also depends on the read consistency selected in the request. Read more about eventual and strong consistency later in this chapter.

For example, given a table without a local secondary index, you will consume 1 capacity unit if you read an item that is 4KB or smaller. Similarly, for write operations you will consume 1 capacity unit if you

write an item that is 1KB or smaller. This means that if you read an item that is 110KB, you will consume 28 capacity units, or 110 / 4 = 27.5 rounded up to 28. For read operations that are strongly consistent, they will use twice the number of capacity units, or 56 in this example.

You can use Amazon CloudWatch to monitor your Amazon DynamoDB capacity and make scaling decisions. There is a rich set of metrics, including `ConsumedReadCapacityUnits` and `ConsumedWriteCapacityUnits`. If you do exceed your provisioned capacity for a period of time, requests will be throttled and can be retried later. You can monitor and alert on the `ThrottledRequests` metric using Amazon CloudWatch to notify you of changing usage patterns.

Secondary Indexes

When you create a table with a partition and sort key (formerly known as a hash and range key), you can optionally define one or more secondary indexes on that table. A secondary index lets you query the data in the table using an alternate key, in addition to queries against the primary key. Amazon DynamoDB supports two different kinds of indexes:

Global Secondary Index The *global secondary index* is an index with a partition and sort key that can be different from those on the table. You can create or delete a global secondary index on a table at any time.

Local Secondary Index The *local secondary index* is an index that has the same partition key attribute as the primary key of the table, but a different sort key. You can only create a local secondary index when you create a table.

Secondary indexes allow you to search a large table efficiently and avoid an expensive scan operation to find items with specific attributes. These indexes allow you to support different query access patterns and use cases beyond what is possible with only a primary key. While a table can only have one local secondary index, you can have multiple global secondary indexes.

Amazon DynamoDB updates each secondary index when an item is modified. These updates consume write capacity units. For a local secondary index, item updates will consume write capacity units from the main table, while global secondary indexes maintain their own provisioned throughput settings separate from the table.

Writing and Reading Data

After you create a table with a primary key and indexes, you can begin writing and reading items to the table. Amazon DynamoDB provides multiple operations that let you create, update, and delete individual items. Amazon DynamoDB also provides multiple querying options that let you search a table or an index or retrieve back a specific item or a batch of items.

Writing Items

Amazon DynamoDB provides three primary API actions to create, update, and delete items: `PutItem`, `UpdateItem`, and `DeleteItem`. Using the `PutItem` action, you can create a new item with one or more attributes. Calls to `PutItem` will update an existing item if the primary key already exists. `PutItem` only requires a table name and a primary key; any additional attributes are optional.

The `UpdateItem` action will find existing items based on the primary key and replace the attributes. This operation can be useful to only update a single attribute and leave the other attributes unchanged. `UpdateItem` can also be used to create items if they don’t already exist. Finally, you can remove an item from a table by using `DeleteItem` and specifying a specific primary key.

The `UpdateItem` action also provides support for atomic counters. *Atomic counters* allow you to increment and decrement a value and are guaranteed to be consistent across multiple concurrent requests. For example, a counter attribute used to track the overall score of a mobile game can be updated by many clients at the same time.

These three actions also support conditional expressions that allow you to perform validation before an action is applied. For example, you can apply a conditional expression on `PutItem` that checks that certain conditions are met before the item is created. This can be useful to prevent accidental overwrites or to enforce some type of business logic checks.

Reading Items

After an item has been created, it can be retrieved through a direct lookup by calling the `GetItem` action or through a search using the Query or Scan action. `GetItem` allows you to retrieve an item based on its primary key. All of the item's attributes are returned by default, and you have the option to select individual attributes to filter down the results.

If a primary key is composed of a partition key, the entire partition key needs to be specified to retrieve the item. If the primary key is a composite of a partition key and a sort key, `GetItem` will require both the partition and sort key as well. Each call to `GetItem` consumes read capacity units based on the size of the item and the consistency option selected.

By default, a `GetItem` operation performs an eventually consistent read. You can optionally request a strongly consistent read instead; this will consume additional read capacity units, but it will return the most up-to-date version of the item.

Eventual Consistency

When reading items from Amazon DynamoDB, the operation can be either eventually consistent or strongly consistent. Amazon DynamoDB is a distributed system that stores multiple copies of an item across an AWS Region to provide high availability and increased durability. When an item is updated in Amazon DynamoDB, it starts replicating across multiple servers. Because Amazon DynamoDB is a distributed system, the replication can take some time to complete. Because of this we refer to the data as being eventually consistent, meaning that a read request immediately after a write operation might not show the latest change. In some cases, the application needs to guarantee that the data is the latest and Amazon DynamoDB offers an option for strongly consistent reads.

Eventually Consistent Reads When you read data, the response might not reflect the results of a recently completed write operation. The response might include some stale data. Consistency across all copies of the data is usually reached within a second; if you repeat your read request after a short time, the response returns the latest data.

Strongly Consistent Reads When you issue a strongly consistent read request, Amazon DynamoDB returns a response with the most up-to-date data that reflects updates by all prior related write operations to which Amazon DynamoDB returned a successful response. A strongly consistent read might be less available in the case of a network delay or outage. You can request a strongly consistent read result by specifying optional parameters in your request.

Batch Operations

Amazon DynamoDB also provides several operations designed for working with large batches of items, including `BatchGetItem` and `BatchWriteItem`. Using the `BatchWriteItem` action, you can perform up

to 25 item creates or updates with a single operation. This allows you to minimize the overhead of each individual call when processing large numbers of items.

Searching Items

Amazon DynamoDB also gives you two operations, Query and Scan, that can be used to search a table or an index. A Query operation is the primary search operation you can use to find items in a table or a secondary index using only primary key attribute values. Each Query requires a partition key attribute name and a distinct value to search. You can optionally provide a sort key value and use a comparison operator to refine the search results. Results are automatically sorted by the primary key and are limited to 1MB.

In contrast to a Query, a Scan operation will read every item in a table or a secondary index. By default, a Scan operation returns all of the data attributes for every item in the table or index. Each request can return up to 1MB of data. Items can be filtered out using expressions, but this can be a resource-intensive operation. If the result set for a Query or a Scan exceeds 1MB, you can page through the results in 1MB increments.



For most operations, performing a Query operation instead of a Scan operation will be the most efficient option. Performing a Scan operation will result in a full scan of the entire table or secondary index, then it filters out values to provide the desired result. Use a Query operation when possible and avoid a Scan on a large table or index for only a small number of items.

Scaling and Partitioning

Amazon DynamoDB is a fully managed service that abstracts away most of the complexity involved in building and scaling a NoSQL cluster. You can create tables that can scale up to hold a virtually unlimited number of items with consistent low-latency performance. An Amazon DynamoDB table can scale horizontally through the use of partitions to meet the storage and performance requirements of your application. Each individual partition represents a unit of compute and storage capacity. A well-designed application will take the partition structure of a table into account to distribute read and write transactions evenly and achieve high transaction rates at low latencies.

Amazon DynamoDB stores items for a single table across multiple partitions, as represented in Figure 7.4. Amazon DynamoDB decides which partition to store the item in based on the partition key. The partition key is used to distribute the new item among all of the available partitions, and items with the same partition key will be stored on the same partition.

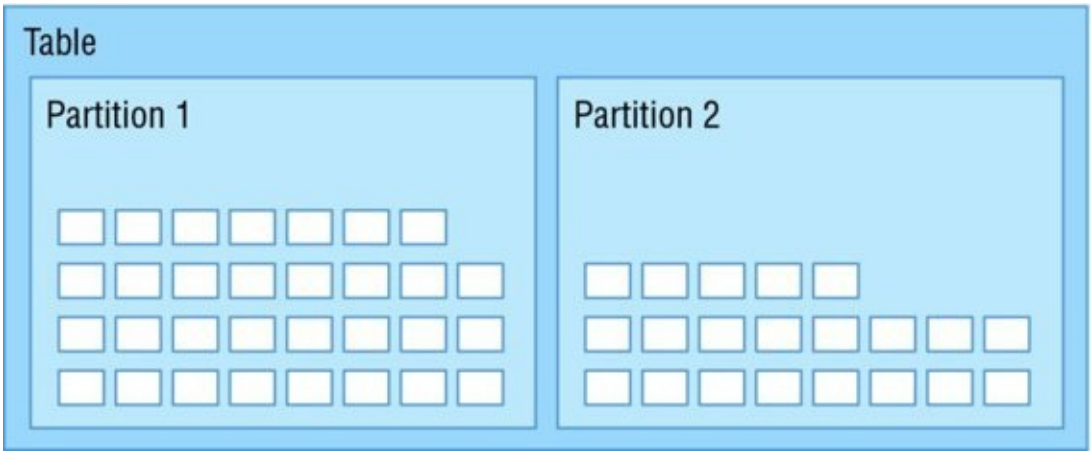


FIGURE 7.4 Table partitioning

As the number of items in a table grows, additional partitions can be added by splitting an existing partition. The provisioned throughput configured for a table is also divided evenly among the partitions. Provisioned throughput allocated to a partition is entirely dedicated to that partition, and there is no sharing of provisioned throughput across partitions.

When a table is created, Amazon DynamoDB configures the table's partitions based on the desired read and write capacity. One single partition can hold about 10GB of data and supports a maximum of 3,000 read capacity units or 1,000 write capacity units. For partitions that are not fully using their provisioned capacity, Amazon DynamoDB provides some burst capacity to handle spikes in traffic. A portion of your unused capacity will be reserved to handle bursts for short periods.

As storage or capacity requirements change, Amazon DynamoDB can split a partition to accommodate more data or higher provisioned request rates. After a partition is split, however, it cannot be merged back together. Keep this in mind when planning to increase provisioned capacity temporarily and then lower it again. With each additional partition added, its share of the provisioned capacity is reduced.

To achieve the full amount of request throughput provisioned for a table, keep your workload spread evenly across the partition key values. Distributing requests across partition key values distributes the requests across partitions. For example, if a table has 10,000 read capacity units configured but all of the traffic is hitting one partition key, you will not be able to get more than the 3,000 maximum read capacity units that one partition can support.



To maximize Amazon DynamoDB throughput, create tables with a partition key that has a large number of distinct values and ensure that the values are requested fairly uniformly. Adding a random element that can be calculated or hashed is one common technique to improve partition distribution.

Security

Amazon DynamoDB gives you granular control over the access rights and permissions for users and administrators. Amazon DynamoDB integrates with the IAM service to provide strong control over permissions using policies. You can create one or more policies that allow or deny specific operations on specific tables. You can also use conditions to restrict access to individual items or attributes.

All operations must first be authenticated as a valid user or user session. Applications that need to read

and write from Amazon DynamoDB need to obtain a set of temporary or permanent access control keys. While these keys could be stored in a configuration file, a best practice is for applications running on AWS to use IAM Amazon EC2 instance profiles to manage credentials. IAM Amazon EC2 instance profiles or roles allow you to avoid storing sensitive keys in configuration files that must then be secured.

 For mobile applications, a best practice is to use a combination of web identity federation with the AWS Security Token Service (AWS STS) to issue temporary keys that expire after a short period.

Amazon DynamoDB also provides support for fine-grained access control that can restrict access to specific items within a table or even specific attributes within an item. For example, you may want to limit a user to only access his or her items within a table and prevent access to items associated with a different user. Using conditions in an IAM policy allows you to restrict which actions a user can perform, on which tables, and to which attributes a user can read or write.

Amazon DynamoDB Streams

A common requirement for many applications is to keep track of recent changes and then perform some kind of processing on the changed records. Amazon DynamoDB Streams makes it easy to get a list of item modifications for the last 24-hour period. For example, you might need to calculate metrics on a rolling basis and update a dashboard, or maybe synchronize two tables or log activity and changes to an audit trail. With Amazon DynamoDB Streams, these types of applications become easier to build.

Amazon DynamoDB Streams allows you to extend application functionality without modifying the original application. By reading the log of activity changes from the stream, you can build new integrations or support new reporting requirements that weren't part of the original design.

Each item change is buffered in a time-ordered sequence or stream that can be read by other applications. Changes are logged to the stream in near real-time and allow you to respond quickly or chain together a sequence of events based on a modification.

Streams can be enabled or disabled for an Amazon DynamoDB table using the AWS Management Console, Command Line Interface (CLI), or SDK. A stream consists of stream records. Each stream record represents a single data modification in the Amazon DynamoDB table to which the stream belongs. Each stream record is assigned a sequence number, reflecting the order in which the record was published to the stream.

Stream records are organized into groups, also referred to as *shards*. Each shard acts as a container for multiple stream records and contains information on accessing and iterating through the records. Shards live for a maximum of 24 hours and, with fluctuating load levels, could be split one or more times before they are eventually closed.

 To build an application that reads from a shard, it is recommended to use the Amazon DynamoDB Streams Kinesis Adapter. The Kinesis Client Library (KCL) simplifies the application logic required to process reading records from streams and shards.

Summary

In this chapter, you learned the basic concepts of relational databases, data warehouses, and NoSQL databases. You also learned about the benefits and features of AWS managed database services Amazon RDS, Amazon Redshift, and Amazon DynamoDB.

Amazon RDS manages the heavy lifting involved in administering a database infrastructure and software and lets you focus on building the relational schemas that best fit your use case and the performance tuning to optimize your queries.

Amazon RDS supports popular open-source and commercial database engines and provides a consistent operational model for common administrative tasks. Increase your availability by running a master-slave configuration across Availability Zones using Multi-AZ deployment. Scale your application and increase your database read performance using read replicas.

Amazon Redshift allows you to deploy a data warehouse cluster that is optimized for analytics and reporting workloads within minutes. Amazon Redshift distributes your records using columnar storage and parallelizes your query execution across multiple compute nodes to deliver fast query performance. Amazon Redshift clusters can be scaled up or down to support large, petabyte-scale databases using SSD or magnetic disk storage.

Connect to Amazon Redshift clusters using standard SQL clients with JDBC/ODBC drivers and execute SQL queries using many of the same analytics and ETL tools that you use today. Load data into your Amazon Redshift clusters using the `COPY` command to bulk import flat files stored in Amazon S3, then run standard `SELECT` commands to search and query the table.

Back up both your Amazon RDS databases and Amazon Redshift clusters using automated and manual snapshots to allow for point-in-time recovery. Secure your Amazon RDS and Amazon Redshift databases using a combination of IAM, database-level access control, network-level access control, and data encryption techniques.

Amazon DynamoDB simplifies the administration and operations of a NoSQL database in the cloud. Amazon DynamoDB allows you to create tables quickly that can scale to an unlimited number of items and configure very high levels of provisioned read and write capacity.

Amazon DynamoDB tables provide a flexible data storage mechanism that only requires a primary key and allows for one or more attributes. Amazon DynamoDB supports both simple scalar data types like String and Number, and also more complex structures using List and Map. Secure your Amazon DynamoDB tables using IAM and restrict access to items and attributes using fine-grained access control.

Amazon DynamoDB will handle the difficult task of cluster and partition management and provide you with a highly available database table that replicates data across Availability Zones for increased durability. Track and process recent changes by tapping into Amazon DynamoDB Streams.

Exam Essentials

Know what a relational database is. A relational database consists of one or more tables. Communication to and from relational databases usually involves simple SQL queries, such as "Add a new record," or "What is the cost of product *x*?" These simple queries are often referred to as OLTP.

Understand which databases are supported by Amazon RDS. Amazon RDS currently supports six relational database engines:

- Microsoft SQL Server
- MySQL Server
- Oracle
- PostgreSQL
- MariaDB
- Amazon Aurora

Understand the operational benefits of using Amazon RDS. Amazon RDS is a managed service provided by AWS. AWS is responsible for patching, antivirus, and management of the underlying guest OS for Amazon RDS. Amazon RDS greatly simplifies the process of setting a secondary slave with replication for failover and setting up read replicas to offload queries.

Remember that you cannot access the underlying OS for Amazon RDS DB instances. You cannot use Remote Desktop Protocol (RDP) or SSH to connect to the underlying OS. If you need to access the OS, install custom software or agents, or want to use a database engine not supported by Amazon RDS, consider running your database on Amazon EC2 instead.

Know that you can increase availability using Amazon RDS Multi-AZ deployment. Add fault tolerance to your Amazon RDS database using Multi-AZ deployment. You can quickly set up a secondary DB Instance in another Availability Zone with Multi-AZ for rapid failover.

Understand the importance of RPO and RTO. Each application should set RPO and RTO targets to define the amount of acceptable data loss and also the amount of time required to recover from an incident. Amazon RDS can be used to meet a wide range of RPO and RTO requirements.

Understand that Amazon RDS handles Multi-AZ failover for you. If your primary Amazon RDS Instance becomes unavailable, AWS fails over to your secondary instance in another Availability Zone automatically. This failover is done by pointing your existing database endpoint to a new IP address. You do not have to change the connection string manually; AWS handles the DNS change automatically.

Remember that Amazon RDS read replicas are used for scaling out and increased performance. This replication feature makes it easy to scale out your read-intensive databases. Read replicas are currently supported in Amazon RDS for MySQL, PostgreSQL, and Amazon Aurora. You can create one or more replicas of a database within a single AWS Region or across multiple AWS Regions. Amazon RDS uses native replication to propagate changes made to a source DB Instance to any associated read replicas. Amazon RDS also supports cross-region read replicas to replicate changes asynchronously to another geography or AWS Region.

Know what a NoSQL database is. NoSQL databases are non-relational databases, meaning that you do

not have to have an existing table created in which to store your data. NoSQL databases come in the following formats:

- Document databases
- Graph stores
- Key/value stores
- Wide-column stores

Remember that Amazon DynamoDB is AWS NoSQL service. You should remember that for NoSQL databases, AWS provides a fully managed service called Amazon DynamoDB. Amazon DynamoDB is an extremely fast NoSQL database with predictable performance and high scalability. You can use Amazon DynamoDB to create a table that can store and retrieve any amount of data and serve any level of request traffic. Amazon DynamoDB automatically spreads the data and traffic for the table over a sufficient number of partitions to handle the request capacity specified by the customer and the amount of data stored, while maintaining consistent and fast performance.

Know what a data warehouse is. A data warehouse is a central repository for data that can come from one or more sources. This data repository would be used for query and analysis using OLAP. An organization's management typically uses a data warehouse to compile reports on specific data. Data warehouses are usually queried with highly complex queries.

Remember that Amazon Redshift is AWS data warehouse service. You should remember that Amazon Redshift is Amazon's data warehouse service. Amazon Redshift organizes the data by column instead of storing data as a series of rows. Because only the columns involved in the queries are processed and columnar data is stored sequentially on the storage media, column-based systems require far fewer I/Os, which greatly improves query performance. Another advantage of columnar data storage is the increased compression, which can further reduce overall I/O.

Exercises

In order to pass the exam, you should practice deploying databases and creating tables using Amazon RDS, Amazon DynamoDB, and Amazon Redshift. Remember to delete any resources you provision to minimize any charges.

EXERCISE 7.1

Create a MySQL Amazon RDS Instance

1. Log in to the AWS Management Console, and navigate to the Amazon RDS Console.
2. Launch a new Amazon RDS DB Instance, and select MySQL Community Edition instance as the database engine.
3. Configure the DB Instance to use Multi-AZ and General Purpose (SSD) storage.

   Warning: This is not eligible for AWS Free Tier; you will incur a small charge by provisioning this instance.
4. Set the DB Instance identifier and database name to **`MySQL123`**, and configure the master username and password.
5. Validate the configuration settings, and launch the DB Instance.
6. Return to the list of the Amazon RDS instances. You will see the status of your Amazon RDS database as `Creating`. It may take up to 20 minutes to create your new Amazon RDS instance.

You have provisioned your first Amazon RDS instance using Multi-AZ.

EXERCISE 7.2

Simulate a Failover from One AZ to Another

In this exercise, you will use Multi-AZ failover to simulate a failover from one Availability Zone to another.

1. In the Amazon RDS Console, view the list of DB Instances.
2. Find your DB Instance called MySQL123, and check its status. When its status is `Available`, proceed to the next step.
3. Select the instance, and issue a Reboot command from the actions menu.
4. Confirm the reboot.

You have now simulated a failover from one Availability Zone to another using Multi-AZ failover. The failover should take approximately two or three minutes.

EXERCISE 7.3

Create a Read Replica

In this exercise, you will create a read replica of your existing MySQL123 DB server.

1. In the Amazon RDS Console, view the list of DB Instances.
2. Find your DB Instance called `MySQL123`, and check its status. When its status is `Available`, proceed to the next step.
3. Select the instance, and issue a Create Read Replica command from the list of actions.
4. Configure the name of the read replica and any other settings. Create the replica.
5. Wait for the replica to be created, which can typically take several minutes. When it is complete, delete both the MySQL123 and MySQLReadReplica databases by clicking the checkboxes next to them, clicking the Instance Actions drop-down box, and then clicking Delete.

In the preceding exercises, you created a new Amazon RDS MySQL instance with Multi-AZ enabled. You then simulated a failover from one Availability Zone to another by rebooting the primary instance. After that, you scaled your Amazon RDS instance out by creating a read replica of the primary database. Delete the DB Instance.

EXERCISE 7.4

Read and Write from a DynamoDB Table

In this exercise, you will create an Amazon DynamoDB table and then read and write to it using the AWS Management Console.

1. Log in to the AWS Management Console, and view the Amazon DynamoDB console.
2. Create a new table named **`UserProfile`** with a partition key of userID of type String.
3. After the table has been created, view the list of items in the table.
4. Using the Amazon DynamoDB console, create and save a new item in the table. Set the userID to U01, and append another String attribute called **name** with a value of **`Joe`**.
5. Perform a scan on the table to retrieve the new item.

You have now created a simple Amazon DynamoDB table, put a new item, and retrieved it using Scan. Delete the DynamoDB table.

EXERCISE 7.5

Launch a Redshift Cluster

In this exercise, you will create a data warehouse using Amazon Redshift and then read and write to it using the AWS Management Console.

1. Log in to the AWS Management Console, and view the Amazon Redshift Console.
2. Create a new cluster, configuring the database name, username, and password.
3. Configure the cluster to be single node using one SSD-backed storage node.
4. Launch the cluster into an Amazon VPC using the appropriate security group.
5. Install and configure SQL Workbench on your local computer, and connect to the new cluster.
6. Create a new table and load data using the `COPY` command.

You have now created an Amazon Redshift cluster and connected to it using a standard SQL client. Delete the cluster when you have completed the exercise.

Review Questions

1. Which AWS database service is best suited for traditional Online Transaction Processing (OLTP)?
   A. Amazon Redshift
   B. Amazon Relational Database Service (Amazon RDS)
   C. Amazon Glacier
   D. Elastic Database
2. Which AWS database service is best suited for non-relational databases?
   A. Amazon Redshift
   B. Amazon Relational Database Service (Amazon RDS)
   C. Amazon Glacier
   D. Amazon DynamoDB
3. You are a solutions architect working for a media company that hosts its website on AWS. Currently, there is a single Amazon Elastic Compute Cloud (Amazon EC2) Instance on AWS with MySQL installed locally to that Amazon EC2 Instance. You have been asked to make the company's production environment more resilient and to increase performance. You suggest that the company split out the MySQL database onto an Amazon RDS Instance with Multi-AZ enabled. This addresses the company's increased resiliency requirements. Now you need to suggest how you can increase performance. Ninety-nine percent of the company's end users are magazine subscribers who will be reading additional articles on the website, so only one percent of end users will need to write data to the site. What should you suggest to increase performance?
   A. Alter the connection string so that if a user is going to write data, it is written to the secondary copy of the Multi-AZ database.
   B. Alter the connection string so that if a user is going to write data, it is written to the primary copy of the Multi-AZ database.
   C. Recommend that the company use read replicas, and distribute the traffic across multiple read replicas.
   D. Migrate the MySQL database to Amazon Redshift to take advantage of columnar storage and maximize performance.
4. Which AWS Cloud service is best suited for Online Analytics Processing (OLAP)?
   A. Amazon Redshift
   B. Amazon Relational Database Service (Amazon RDS)
   C. Amazon Glacier
   D. Amazon DynamoDB
5. You have been using Amazon Relational Database Service (Amazon RDS) for the last year to run an important application with automated backups enabled. One of your team members is performing routine maintenance and accidentally drops an important table, causing an outage. How can you

recover the missing data while minimizing the duration of the outage?

A. Perform an undo operation and recover the table.

B. Restore the database from a recent automated DB snapshot.

C. Restore only the dropped table from the DB snapshot.

D. The data cannot be recovered.

6. Which Amazon Relational Database Service (Amazon RDS) database engines support Multi-AZ?

A. All of them

B. Microsoft SQL Server, MySQL, and Oracle

C. Oracle, Amazon Aurora, and PostgreSQL

D. MySQL

7. Which Amazon Relational Database Service (Amazon RDS) database engines support read replicas?

A. Microsoft SQL Server and Oracle

B. MySQL, MariaDB, PostgreSQL, and Aurora

C. Aurora, Microsoft SQL Server, and Oracle

D. MySQL and PostgreSQL

8. Your team is building an order processing system that will span multiple Availability Zones. During testing, the team wanted to test how the application will react to a database failover. How can you enable this type of test?

A. Force a Multi-AZ failover from one Availability Zone to another by rebooting the primary instance using the Amazon RDS console.

B. Terminate the DB instance, and create a new one. Update the connection string.

C. Create a support case asking for a failover.

D. It is not possible to test a failover.

9. You are a system administrator whose company has moved its production database to AWS. Your company monitors its estate using Amazon CloudWatch, which sends alarms using Amazon Simple Notification Service (Amazon SNS) to your mobile phone. One night, you get an alert that your primary Amazon Relational Database Service (Amazon RDS) Instance has gone down. You have Multi-AZ enabled on this instance. What should you do to ensure the failover happens quickly?

A. Update your Domain Name System (DNS) to point to the secondary instance's new IP address, forcing your application to fail over to the secondary instance.

B. Connect to your server using Secure Shell (SSH) and update your connection strings so that your application can communicate to the secondary instance instead of the failed primary instance.

C. Take a snapshot of the secondary instance and create a new instance using this snapshot, then update your connection string to point to the new instance.

D. No action is necessary. Your connection string points to the database endpoint, and AWS automatically updates this endpoint to point to your secondary instance.

10. You are working for a small organization without a dedicated database administrator on staff. You need to install Microsoft SQL Server Enterprise edition quickly to support an accounting back office application on Amazon Relational Database Service (Amazon RDS). What should you do?

    A. Launch an Amazon RDS DB Instance, and select Microsoft SQL Server Enterprise Edition under the Bring Your Own License (BYOL) model.

    B. Provision SQL Server Enterprise Edition using the License Included option from the Amazon RDS Console.

    C. SQL Server Enterprise edition is only available via the Command Line Interface (CLI). Install the command-line tools on your laptop, and then provision your new Amazon RDS Instance using the CLI.

    D. You cannot use SQL Server Enterprise edition on Amazon RDS. You should install this on to a dedicated Amazon Elastic Compute Cloud (Amazon EC2) Instance.

11. You are building the database tier for an enterprise application that gets occasional activity throughout the day. Which storage type should you select as your default option?

    A. Magnetic storage

    B. General Purpose Solid State Drive (SSD)

    C. Provisioned IOPS (SSD)

    D. Storage Area Network (SAN)-attached

12. You are designing an e-commerce web application that will scale to potentially hundreds of thousands of concurrent users. Which database technology is best suited to hold the session state for large numbers of concurrent users?

    A. Relational database using Amazon Relational Database Service (Amazon RDS)

    B. NoSQL database table using Amazon DynamoDB

    C. Data warehouse using Amazon Redshift

    D. Amazon Simple Storage Service (Amazon S3)

13. Which of the following techniques can you use to help you meet Recovery Point Objective (RPO) and Recovery Time Objective (RTO) requirements? (Choose 3 answers)

    A. DB snapshots

    B. DB option groups

    C. Read replica

    D. Multi-AZ deployment

14. When using Amazon Relational Database Service (Amazon RDS) Multi-AZ, how can you offload read requests from the primary? (Choose 2 answers)

    A. Configure the connection string of the clients to connect to the secondary node and perform reads while the primary is used for writes.

B. Amazon RDS automatically sends writes to the primary and sends reads to the secondary.

C. Add a read replica DB instance, and configure the client's application logic to use a read-replica.

D. Create a caching environment using ElastiCache to cache frequently used data. Update the application logic to read/write from the cache.

15. You are building a large order processing system and are responsible for securing the database. Which actions will you take to protect the data? (Choose 3 answers)

A. Adjust AWS Identity and Access Management (IAM) permissions for administrators.

B. Configure security groups and network Access Control Lists (ACLs) to limit network access.

C. Configure database users, and grant permissions to database objects.

D. Install anti-virus software on the Amazon RDS DB Instance.

16. Your team manages a popular website running Amazon Relational Database Service (Amazon RDS) MySQL back end. The Marketing department has just informed you about an upcoming television commercial that will drive thousands of new visitors to the website. How can you prepare your database to handle the load? (Choose 3 answers)

A. Vertically scale the DB Instance by selecting a more powerful instance class.

B. Create read replicas to offload read requests and update your application.

C. Upgrade the storage from Magnetic volumes to General Purpose Solid State Drive (SSD) volumes.

D. Upgrade to Amazon Redshift for faster columnar storage.

17. You are building a photo management application that maintains metadata on millions of images in an Amazon DynamoDB table. When a photo is retrieved, you want to display the metadata next to the image. Which Amazon DynamoDB operation will you use to retrieve the metadata attributes from the table?

A. Scan operation

B. Search operation

C. Query operation

D. Find operation

18. You are creating an Amazon DynamoDB table that will contain messages for a social chat application. This table will have the following attributes: Username (String), Timestamp (Number), Message (String). Which attribute should you use as the partition key? The sort key?

A. Username, Timestamp

B. Username, Message

C. Timestamp, Message

D. Message, Timestamp

19. Which of the following statements about Amazon DynamoDB tables are true? (Choose 2 answers)

A. Global secondary indexes can only be created when the table is being created.

B. Local secondary indexes can only be created when the table is being created.

C. You can only have one global secondary index.

D. You can only have one local secondary index.

20. Which of the following workloads are a good fit for running on Amazon Redshift? (Choose 2 answers)

A. Transactional database supporting a busy e-commerce order processing website

B. Reporting database supporting back-office analytics

C. Data warehouse used to aggregate multiple disparate data sources

D. Manage session state and user profile data for thousands of concurrent users